THE JAVASCRIPT MASTERY SERIES

VOLUME I

JavaScript Fundamentals

Understand the language.
Trace the behavior.
Build with confidence.

Values · Functions · Objects · Collections · Errors · Modules

Sumit

Contents

Volume I: JavaScript Fundamentals

Preface

Introduction to JavaScript

Variables and Data Types

Operators and Expressions

Type Conversion and Coercion

Strings, Numbers, and Dates

Control Flow

Functions and Callbacks

Objects and Data Ownership

Arrays and Collections

Errors and Debugging

Modules and Execution Modes

JavaScript Execution Model: A Companion

Each chapter includes theory, internals, production examples, interviews, exercises, MCQs, revision, debugging, performance and security, and a professional field guide.

Volume I: JavaScript Fundamentals

Volume I builds the language foundation used in the later volumes: values and bindings, expressions, validation, control flow, functions, ownership, collections, errors, and modules. Eleven chapters each contain eleven authored sections, including worked exercises, MCQs, production examples, debugging notes, and references.

Chapters

1. Introduction to JavaScript
2. Variables and Data Types
3. Operators and Expressions
4. Type Conversion and Coercion
5. Strings, Numbers, and Dates
6. Control Flow
7. Functions and Callbacks
8. Objects and Data Ownership
9. Arrays and Collections
10. Errors and Debugging
11. Modules and Execution Modes

Reading Path

Read the preface, then follow the chapters in order. Chapters 1–6 introduce the language and common data boundaries. Chapter 7 establishes function and callback contracts; Chapters 8–9 explain the objects and collections those functions exchange. Chapters 10–11 make failure handling and execution mode explicit.

The execution-model companion can be read after Chapter 7, then revisited after object ownership. It connects calls, bindings, retained state, and defensive snapshots. After Chapter 11, continue with Lexical Scope and Closures in Volume 2.

Use the complete section summary for targeted review. Attempt each exercise before reading its solution and explain an MCQ result before checking the answer.

Running and Checking Examples

Use Node.js 20 or later for the documented language baseline. Choose a currently supported runtime for deployment. Standalone assertion programs live in `code/volume-1`; the execution-model companion uses `code/chapter-01`. Chapter 11 uses explicit ES module and CommonJS files and keeps its library dependencies in the same directory.

```
npm run examples:test
npm run examples:validator:test
npm run docs:examples:test
```

The first command runs every companion, the second checks the validator's failure handling, and the third executes Volume 1's documented JavaScript blocks in fresh processes and verifies their output, local link targets, and embedded diagram sources. Expected-output comments use `(none)` for a library that intentionally prints nothing.

Publishing

All chapter sections, the preface, and the execution-model companion are included in both site navigation systems and the standalone Volume One books. Build the website with `npm run docs:build`.

Run `npm run volume1:build` to generate `pdf/javascript-mastery-volume-1.pdf` and `epub/javascript-mastery-volume-1.epub`. This build uses an installed Chrome, Chromium, or Edge browser; set `BOOK_BROWSER_PATH` when it is not in a standard location. It renders all 24 diagrams into offline images, preserves internal links, and generates a table of contents. Pandoc and LaTeX are not needed for these standalone exports.

With Python 3, install the PDF validation dependency using `python -m pip install -r scripts/requirements-export-validation.txt`, then run `npm run volume1:exports:test` to check the generated books' structure, EPUB source coverage, images, and links. The export includes 124 source documents; its generated contents replace `SUMMARY.md`. The complete repository check is `npm run build:all`.

Preface

This volume is written for engineers who want JavaScript mastery rather than surface familiarity. It assumes curiosity, discipline, and a willingness to trace behavior carefully.

The teaching style is practical: define the rule, explain why it exists, connect it to engine or host behavior, show production code, then rehearse how to communicate the idea in interviews. The same habit helps in code reviews, debugging sessions, and design discussions.

JavaScript has historical quirks, but it is not chaos. Most confusing behavior becomes predictable when you separate values from references, language rules from host APIs, and syntax from runtime execution.

How To Study This Volume

Do not read this volume like a list of syntax features. Read it like a map of runtime behavior. For each chapter, start with the introduction and theory, then pause before reading the internal-working section. Try to predict what the engine, memory model, or host environment must do for the feature to work. After that, compare your prediction with the diagrams and execution steps.

When you reach a production example, run the corresponding file under `code/volume-1`. Change one input at a time. Replace a valid value with `null`, `undefined`, an empty string, a zero, a negative number, a sparse array, or an object with an unexpected shape. The point is not to break the code for entertainment. The point is to train the instinct that production bugs often begin as ordinary values crossing a boundary without enough ceremony.

For interview preparation, use active recall. Close the book and answer the beginner, intermediate, senior, and FAANG prompts out loud. A strong answer has four parts: the visible behavior, the specification-level rule, the internal model, and the production consequence. If you can explain only the visible behavior, you know the syntax but not the system. If you can explain only the internals, you may sound theoretical. The best interview answers join both.

Use the MCQs as diagnostic tools, not as a memorization contest. If you miss a question, write down the rule in your own words and create a tiny example that demonstrates it. Then add one sentence explaining when that rule matters in a real codebase. This turns trivia into judgment.

Finally, keep a personal revision sheet. For each chapter, record one edge case, one performance note, one security concern, and one phrase you would use on a whiteboard. By the end of Volume I, you should be able to describe JavaScript fundamentals without panic, mysticism, or hand-waving. That is the real target: calm precision under pressure.

Notation

Code examples use modern JavaScript and strict-mode-friendly habits. Diagrams are Mermaid blocks so they can render in the website and remain readable in plain Markdown. Complexity notes use `n` for input size, `k` for copied properties, and `m` for retained output items. When the text says browser or Node.js, it is naming the host environment; when it says JavaScript or ECMAScript, it is naming the language rules.

CHAPTER 1

Introduction to JavaScript

JavaScript is a programming language used inside many hosts: browser pages, browser workers, Node.js processes, and other runtimes. Its core rules stay recognizable across those environments, but the available capabilities differ. A calculation can run unchanged in a page and a server; code that reads `document` needs a host that supplies a document.

This chapter establishes three layers you should keep separate:

Layer	Responsibility	Example
Language	Meaning of declarations, values, expressions, and function calls	`const total = 1200 + 300`
Engine	Implements execution, representation, compilation, and garbage collection	V8, SpiderMonkey, JavaScriptCore
Host	Supplies loading, scheduling, I/O, and environment-specific capabilities	Browser document APIs; Node file APIs

A runtime usually combines an engine with host facilities. A package manager, bundler, or framework is another part of the ecosystem; none changes the meaning of a standard addition expression by itself.

Learning Objectives

After working through the examples, you should be able to:

- Run a small JavaScript program and predict its output.
- Separate a language rule from an engine implementation and a host API.
- Explain why standardization, deployment support, and proposal maturity are different questions.
- Describe a conceptual source-to-execution pipeline without claiming every engine uses one design.
- Detect the capability an operation needs and design a testable boundary around it.
- Share a validated calculation between hosts while keeping server-side authority.
- Diagnose an unavailable host API before changing the calculation.

Prerequisites and Running the Examples

No prior JavaScript knowledge is required for the first example. Later sections use short named functions: a function receives inputs between parentheses and returns an output with `return`. An object groups named values between braces. Those constructs are introduced as needed; the later fundamentals chapters develop them systematically.

Use a text editor and Node.js 20 or later. The examples deliberately use features available in Node 20; this is a reproducibility baseline, not a recommendation to deploy an unsupported runtime. Save an example in a `.js` file in this repository and run it from a terminal:

```
node code/volume-1/chapter-01/example-01-runtime-detection.js
node code/volume-1/chapter-01/example-02-shared-validation.js
```

Every JavaScript block in this chapter runs independently. Expected-output comments show the exact printed text. `console.log` asks the host to print a value; it is not part of the ECMAScript language specification.

```
'use strict';

const bookTitle = 'JavaScript';
const copies = 3;
const centsPerCopy = 1500;
console.log(bookTitle);
console.log(copies * centsPerCopy);

// Expected output:
// JavaScript
// 4500
```

Read the first line as a strict-mode directive, the three declarations as names for values, and the two final statements as printing requests. Prices are integer cents here; validation and safe ranges are introduced in the production section.

Reading Path

Start with theory, then follow the execution trace and diagrams. Apply the model in production examples and the six exercises. Use the revision sheet after attempting the MCQs.

The separate execution-model companion develops function calls and lexical environments after you have studied variable and function basics. It preserves a stable reading destination for later closure material; it is not a competing Chapter 1.

Theory

A Language, an Engine, and a Host

A language defines what a program means. For example, multiplication of two Number values produces another Number according to ECMAScript rules. An engine implements those rules using its own data structures and machine code. A host decides how source arrives, which external operations are available, and how work is scheduled.

This separation explains a common failure: moving a function that calls `document.querySelector` from a browser page into Node does not make the selector expression invalid JavaScript. The program parses, but resolving `document` fails because that host does not supply it by default. Conversely, `Array.isArray` is a language built-in available across conforming hosts.

```
'use strict';

const values = [1, 2, 3];
console.log(Array.isArray(values), values.length);
console.log(typeof missingHostApi);

// Expected output:
// true 3
// undefined
```

The final line demonstrates a safe `typeof` check for an undeclared name. It does not establish that every present value is callable, permitted, or functioning. Temporal-dead-zone bindings have a different rule, covered in Chapter 2.

How JavaScript Reached Its Current Role

JavaScript began at Netscape in 1995 to make web pages programmable. Standardization produced the first ECMAScript edition in 1997. The language subsequently accumulated more structured facilities while retaining compatibility with existing programs: ES5 introduced strict mode, and ECMAScript 2015 added features including lexical declarations, classes, promises, and modules. The historical details and design constraints are documented in the co-designers' JavaScript history paper.

The practical consequence is coexistence. You will encounter `var` and `let`, CommonJS and ECMAScript modules, callback APIs and promises. Older syntax is not automatically broken, and newer syntax does not automatically fit every deployment target. Learn the behavior and the compatibility requirement before migrating code.

ECMAScript and TC39

TC39 develops ECMAScript through a proposal process. Ecma publishes the resulting standards. A proposal is a design being evaluated; a shipped engine implementation and a finalized standard are related but separate milestones.

The current process uses stages 0, 1, 2, 2.7, 3, and 4:

Stage	Meaning for a reader
0	An idea under exploration
1	A problem and possible solution under committee consideration

Stage	Meaning for a reader
2	A preferred design being refined
2.7	Complete design undergoing testing and validation
3	Implementation experience is being gathered
4	Completed feature ready for integration into the standard

Check the TC39 process when evaluating a proposal. A stage number alone cannot tell you whether a particular production browser supports a feature. Check the actual runtime targets and test the deployed artifact.

Tools Do Different Jobs

Tool	What it changes	What it cannot guarantee
Package manager	Installs declared dependencies	That dependencies are safe or compatible
Bundler	Resolves and packages an application's code	That a host API exists at runtime
Transpiler	Rewrites supported source syntax	That every missing runtime behavior is supplied
Polyfill	Supplies some missing API behavior	That unsupported syntax will parse
Linter	Flags selected patterns and mistakes	Full program correctness
Test runner	Executes checks you provide	Correctness outside those checks

A transpiler can rewrite a syntax construct before delivery. A polyfill is itself executable code, so a file containing syntax the engine cannot parse fails before that file can install its own fallback. Do not confuse build-time acceptance with runtime support.

Capabilities Are More Useful Than Runtime Labels

A browser worker runs JavaScript but has no page DOM. A Node process may offer familiar web APIs. An embedded runtime may expose a carefully restricted subset of host features. Therefore, `window exists` is not a universal test for "JavaScript running in a browser," and `fetch exists` does not prove a browser page.

Prefer asking the narrow question that the operation needs. For an injected adapter, check that its operation is function-valued:

```
'use strict';

function hasWriter(adapter) {
  return adapter !== null &&
    (typeof adapter === 'object' || typeof adapter === 'function') &&
    typeof adapter.writeText === 'function';
}

console.log(hasWriter({ writeText() {} }));
console.log(hasWriter({ writeText: true }), hasWriter(null));

// Expected output:
// true
// false false
```

This is a capability-shape check on trusted adapters, not a permission probe or a guarantee that a later write will succeed. Even a class has `typeof` result `'function'`, but calling it without `new` throws. The trusted adapter contract must supply an ordinary write operation. An adapter can also throw because a file is unavailable, a permission is denied, or its implementation fails. Handle operation outcomes as well as presence. See the typeof reference for the function/class distinction.

Browser Pages, Workers, and Node

A page can update its document and respond to UI events. A worker runs separately from that page's UI context and communicates through supported messaging APIs; it cannot directly manipulate the page's DOM. Node provides process and server-oriented APIs and supports both CommonJS and ECMAScript modules. Consult the browser worker guide and Node globals reference for the actual facilities.

Keep business rules independent of these capabilities when practical. Let a page adapter read a form and let a server adapter read a request. Both can call the same amount validator. The server must still validate its own input: a browser-side check improves feedback but cannot authorize a payment.

Predictability and Dynamic Behavior

JavaScript is dynamically typed: operations inspect the values present at execution time. It is not "untyped," and an object, a string, and a number do not obey identical operations. Its flexibility makes explicit contracts valuable. If a function accepts integer cents, state that it requires a Number within a defined range; do not silently accept every value that can be converted to a number.

Use a class, framework, or build system only when its responsibilities are useful. A small shared validator can be a plain function. Adding a framework does not remove the need to understand its input values, effects, and host dependencies.

Internal Working

From Source Text to Observable Behavior

A host obtains source and asks an engine to evaluate it in a particular mode. Parsing checks syntax. Declaration instantiation prepares the relevant bindings. Evaluation performs operations, calls functions, and interacts with the host. These are language-level distinctions; an engine can combine or optimize implementation steps while preserving their observable result.

V8's Ignition documentation describes its bytecode interpreter. Other tiers and optimizations can act on executed code. Do not draw "every line is first interpreted and then compiled" as a universal language rule. Engines can defer work, compile at different times, or remove work that cannot affect observations.

A Concrete Execution Trace

```
'use strict';

const priceCents = 1500;
function totalFor(copies) {
  const result = copies * priceCents;
  return result;
}
const invoiceCents = totalFor(3);
console.log(invoiceCents);

// Expected output:
// 4500
```

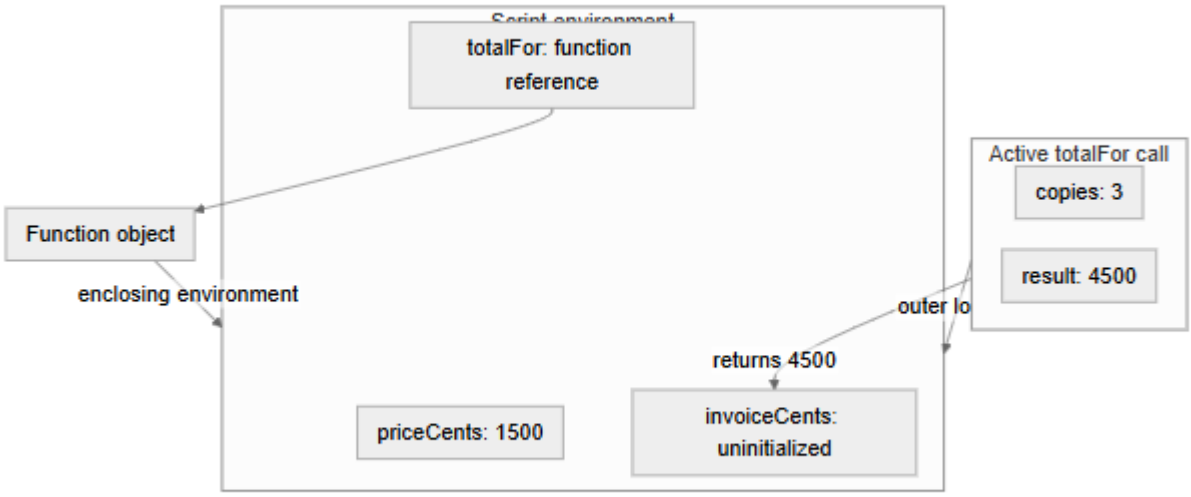
The important steps are:

1. Parse the script and prepare its declarations. The function declaration is initialized with its function object before top-level statement evaluation.
2. Evaluate the `priceCents` initializer, initializing its lexical binding to 1500.
3. Evaluate the call expression `totalFor(3)`. The function object is obtained and the argument value is supplied.
4. Enter the function call with `copies` equal to 3. Resolve `priceCents` from the function's enclosing environment.
5. Multiply, initialize `result`, and return 4500.
6. Initialize `invoiceCents` with that returned value and ask the host's console to print it.

A function declaration's source position is not the moment at which its function object must first become callable. Chapter 2 develops declaration initialization and the temporal dead zone.

Semantic Memory Diagram

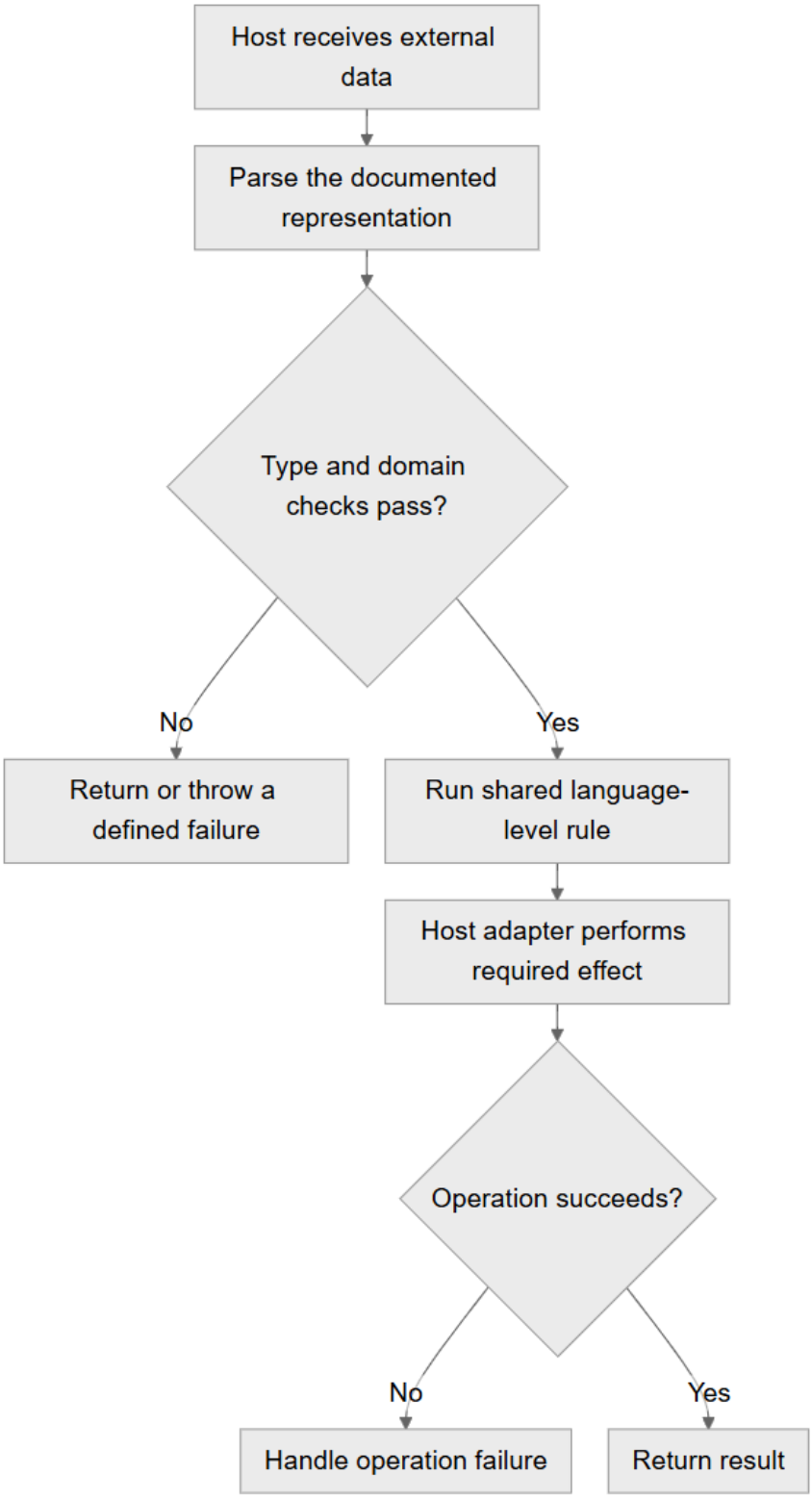
The diagram represents bindings and references while `totalFor` is executing. It does not assert physical stack addresses, object sizes, or that the engine must allocate every drawn box.



Source: `diagrams/volume-1-chapter-01-memory.mmd`.

The active call finishes before `invoiceCents` receives its value. The returned number does not require the call's local binding to remain accessible. A returned function could retain access to an enclosing binding; that is developed in the execution-model companion.

Flow Through an Application Boundary



Source: `diagrams/volume-1-chapter-01-boundary-flow.mmd`.

This is separate from the engine pipeline. A runtime can successfully execute code whose application input is invalid; parsing JavaScript source and parsing application data solve different problems.

What Engines May Optimize

An engine may represent a small integer differently from a large number, avoid an unobservable allocation, inline a call, or change compiled code after observed assumptions stop holding. Those are implementation choices. "There is a function execution context" is a specification model; it does not promise a corresponding heap object or a physical machine-stack frame in every optimized execution.

Use the model to predict values, scope, and ordering. Measure actual execution when latency or memory matters. A guessed engine layout is not a reason to replace clear validated code with opaque tricks.

The Host Still Matters

A CPU-bound loop occupies its executing thread. In a browser page that can delay UI work; in a Node service it can delay unrelated callbacks on the same event loop. Host APIs can initiate asynchronous operations, but wrapping expensive synchronous work in a function does not make it asynchronous. The later asynchronous volume explains scheduling; this chapter's responsibility is to keep the calculation and the host operation distinct.

Production Examples

A Checkout Rule Shared by Page and Server

A checkout accepts a positive amount represented as a Number of cents, with a maximum of 100,000,000 cents. This explicit business limit is below the safe-integer boundary. The helper neither parses strings nor performs payment I/O.

```
'use strict';

function validateCheckoutAmount(amountInCents) {
  if (!Number.isSafeInteger(amountInCents)) {
    throw new TypeError('amount must be a safe integer Number of cents');
  }
  if (amountInCents < 1 || amountInCents > 100_000_000) {
    throw new RangeError('amount is outside the checkout range');
  }
  return { amountInCents, valid: true };
}

console.log(JSON.stringify(validateCheckoutAmount(2599)));
for (const input of ['2599', 0, Number.MAX_SAFE_INTEGER + 1]) {
  try {
    validateCheckoutAmount(input);
  } catch (error) {
    console.log(error.name);
  }
}

// Expected output:
```

```
// {"amountInCents":2599,"valid":true}
// TypeError
// RangeError
// TypeError
```

The rule rejects unsupported representations before comparing the domain range. A form adapter can parse a documented decimal format before calling it; a request adapter validates the parsed request independently. This boundary does not prove that a price is authorized for the customer's cart. The server must compute or verify the price from trusted product and order data.

The companion `code/volume-1/chapter-01/example-02-shared-validation.js` checks both boundaries, invalid numbers, and exact output. Validation has constant-size numeric bookkeeping. A safe-integer check cannot recover precision already lost while parsing an overlarge decimal input; constrain representation before conversion when exact larger values matter. See Number.isSafeInteger.

Report Required Capabilities, Not Permission Guesses

Suppose an export workflow receives a trusted adapter with optional `writeText` and `showStatus` operations. Report whether those properties are function-valued. The application constructs the adapter and supplies ordinary operations rather than class constructors; it is not an arbitrary proxy or an untrusted object containing getters.

```
'use strict';

function getRuntimeCapabilities(adapter) {
  if (adapter === null || typeof adapter !== 'object') {
    throw new TypeError('adapter must be an object');
  }
  return {
    canWriteText: typeof adapter.writeText === 'function',
    canShowStatus: typeof adapter.showStatus === 'function'
  };
}

const workerAdapter = { writeText: undefined, showStatus: undefined };
const fileAdapter = { writeText() {} };
console.log(JSON.stringify(getRuntimeCapabilities(workerAdapter)));
console.log(JSON.stringify(getRuntimeCapabilities(fileAdapter)));

// Expected output:
// {"canWriteText":false,"canShowStatus":false}
// {"canWriteText":true,"canShowStatus":false}
```

This helper intentionally does not infer filesystem permission from `process`, or DOM access from an environment label. A browser page can supply an adapter that downloads a file; Node can supply one that writes an approved path. Both must report the outcome of the actual operation.

The existing companion path `code/volume-1/chapter-01/example-01-runtime-detection.js` now demonstrates this narrower capability contract and asserts that noncallable properties are rejected as capabilities.

Inject an Effect and Keep the Result Explicit

```
'use strict';

function publishSummary(summary, writeText) {
  if (typeof summary !== 'string' || summary.trim() === '') {
    throw new TypeError('summary must be non-empty text');
  }
  if (typeof writeText !== 'function') throw new TypeError('writer must be callable');
  const text = summary.trim();
  writeText(text);
  return text.length;
}

const written = [];
console.log(publishSummary('  Ready for review  ', text => written.push(text)));
console.log(written.join('|'));
try {
  publishSummary('Ready', () => { throw new Error('storage unavailable'); });
} catch (error) {
  console.log(error.message);
}

// Expected output:
// 16
// Ready for review
// storage unavailable
```

The writer contract is synchronous; the return means the call returned successfully, not that an asynchronous operation later completed. The example propagates the same operation failure to its caller. A real asynchronous writer needs an asynchronous API with an awaited outcome, covered later.

Normalization takes time proportional to input text length. The injected writer's own cost is additional. A successful capability check does not eliminate this failure path. Avoid swallowing a write error and returning a success-shaped result.

Review the Boundary

Keep parsing, domain validation, and authorization distinct. List accepted representations and numeric limits. Inject host operations so their failures can be tested. Return small explicit results instead of publishing mutable global state. These decisions make the shared code portable while preserving the authority of the environment that owns the operation.

Interview Perspective

Explain the Three Layers

Question: Is `console.log` part of JavaScript?

Answer: Calling a function, reading a property, and evaluating its arguments use ECMAScript rules. The console object is supplied by the host. A conforming language implementation need not include a terminal or browser console. Separate the language operation from the host service it invokes.

Follow-up: Why does a valid program fail with `document is not defined`? The source parsed, but identifier resolution failed during evaluation. The current host did not provide that binding. Changing a multiplication expression will not repair a missing DOM dependency.

Predict the Output and Explain Portability

```
'use strict';

function invoiceLabel(cents) {
  return `Invoice: ${cents} cents`;
}
const write = message => console.log(message);
write(invoiceLabel(1250));
console.log(typeof Array.isArray);
console.log(typeof unavailablePageApi);

// Expected output:
// Invoice: 1250 cents
// function
// undefined
```

The label calculation uses language features. `write` is an adapter that performs output through the host. `Array.isArray` is a callable language built-in. `typeof` on this undeclared identifier returns a string; it does not attempt to call a missing API. The lexical-binding exception to this last rule is developed in Chapter 2.

Compilation and Interpretation

Question: Is JavaScript compiled or interpreted?

Answer: ECMAScript specifies observable semantics, not one execution technology. Engines may interpret bytecode and compile selected code to machine code. V8 documents its Ignition interpreter. Saying all JavaScript runs through exactly the same pipeline confuses an implementation with a language requirement.

Follow-up: Does a successful build prove an application will run? No. Syntax transformation, dependency resolution, runtime built-ins, host capabilities, and permissions are separate concerns. Test the built artifact in the actual target environment.

Capability Versus Environment Name

Question: A helper sees `process` and returns `canWriteFiles: true`. What is wrong?

Answer: A process-like object neither proves that a writer exists nor that its destination is writable. Node can run under restricted permissions. Accept an explicit writer dependency, check its callable shape, and handle its operation result.

Follow-up: Does `typeof writer === 'function'` guarantee success? It establishes only a function-valued shape. A class also passes this check but throws when called without `new`. A trusted writer operation can still throw, reject if asynchronous, or report an application failure.

Design Question: Share a Checkout Rule

Question: A page and a server both validate an amount. Should the server trust the page's result?

Answer: Put representation-independent rules in a shared function, such as requiring a positive safe integer number of cents within the product's documented limit. The page uses it for immediate feedback. The server independently validates the submitted representation and verifies the authoritative cart price. Sharing code reduces drift; it does not transfer authority to the client.

Test zero, the maximum, one above the maximum, fractional values, strings, `NaN`, infinity, and unsafe integers. Keep payment I/O outside the validator so its failure contract can be tested independently.

Standards Question

Question: Does a Stage 3 proposal work in every browser?

Answer: No. Proposal maturity describes standardization progress. Deployed support depends on specific versions and implementation status. The TC39 process and the application's runtime support matrix answer different questions.

Strong Explanation Checklist

A useful answer identifies the language rule, the required host capability, accepted inputs, and observable failure. Draw only the boundary needed for the question. Avoid substituting phrases such as "the engine optimizes it" for a value trace or a measured performance result.

Exercises and Coding Challenges

Each solution runs independently in Node.js 20 or later. `assert` is Node's assertion utility: an incorrect expectation throws and makes the check fail. Attempt the requirements before reading the solution. These tasks practice language/host boundaries rather than depending on a browser document.

Exercise 1: Trace a Portable Calculation

Requirements: For three reports costing 250 cents each, compute a label without reading any host state. Print the label separately. Identify which line actually performs output.

Hint: Separate the returned text from the statement that prints it.

Solution: Multiplication and string interpolation are language operations; the final console call requests host output.

```
'use strict';
const assert = require('node:assert/strict');

function reportLabel(copies, centsPerCopy) {
  return `${copies} reports: ${copies * centsPerCopy} cents`;
}
const label = reportLabel(3, 250);
assert.equal(label, '3 reports: 750 cents');
console.log(label);

// Expected output:
// 3 reports: 750 cents
```

Inputs in this trace are trusted small positive integers. The later tasks establish a validation boundary. The calculation uses constant-size arithmetic; producing text also costs time and space proportional to its length.

Exercise 2: Check the Required Capability

Requirements: Given a trusted ordinary adapter, return whether it has a callable `writeText`. Missing, boolean, and null values must report false. The adapter itself may be null or undefined. Do not infer permission or identify a runtime.

Hint: Guard absent containers before reading a property.

Solution: Guard the container before reading the operation, then inspect its type.

```
'use strict';
const assert = require('node:assert/strict');

function canAttemptWrite(adapter) {
  return adapter !== null && adapter !== undefined &&
    typeof adapter.writeText === 'function';
}
assert.equal(canAttemptWrite({ writeText() {} }), true);
for (const adapter of [null, undefined, {}, { writeText: true }]) {
  assert.equal(canAttemptWrite(adapter), false);
}
console.log(canAttemptWrite({ writeText() {} }), canAttemptWrite({}));
```

```
// Expected output:
// true false
```

The check is constant-sized for ordinary data properties. A true result means an operation can be attempted; the operation may still fail. Getters and proxies are outside this trusted-adapter contract.

Exercise 3: Define the Checkout Boundary

Requirements: Accept a Number of cents from 1 through 100,000,000 inclusive. Reject fractional, unsafe, or nonnumeric input with `TypeError`; reject a safe integer outside the domain with `RangeError`. Return the accepted amount without converting it.

Hint: Check safe integer representation before the domain limits.

Solution: Validate representation before the business range.

```
'use strict';
const assert = require('node:assert/strict');

function checkoutCents(value) {
  if (!Number.isSafeInteger(value)) throw new TypeError('safe integer cents required');
  if (value < 1 || value > 100_000_000) throw new RangeError('checkout range exceeded');
  return value;
}
assert.equal(checkoutCents(1), 1);
assert.equal(checkoutCents(100_000_000), 100_000_000);
for (const value of ['25', null, NaN, Infinity, 1.5, Number.MAX_SAFE_INTEGER + 1]) {
  assert.throws(() => checkoutCents(value), TypeError);
}
for (const value of [0, -1, 100_000_001]) {
  assert.throws(() => checkoutCents(value), RangeError);
}
console.log(checkoutCents(2599));

// Expected output:
// 2599
```

Time and auxiliary space are constant-sized. Do not silently widen this function to parse arbitrary strings: a form parser has a different accepted-input contract.

Exercise 4: Select a Fallback Without Hiding Failure

Requirements: `formatStatus(text, formatter)` accepts a string and an optional synchronous function. When the formatter is undefined, return `Status:` followed by the text. Reject an explicitly supplied nonfunction. A formatter's thrown error must reach the caller unchanged.

Hint: Use undefined to distinguish an omitted dependency from an invalid supplied value.

Solution: Distinguish an absent dependency from a failing dependency.

```
'use strict';
const assert = require('node:assert/strict');

function formatStatus(text, formatter) {
  if (typeof text !== 'string') throw new TypeError('text required');
  if (formatter === undefined) return `Status: ${text}`;
  if (typeof formatter !== 'function') throw new TypeError('formatter must be callable');
  return formatter(text);
}
const failure = new Error('format failed');
assert.throws(() => formatStatus('ready', null), TypeError);
assert.throws(() => formatStatus('ready', () => { throw failure; }), error => error === failure);
console.log(formatStatus('ready'));
console.log(formatStatus('ready', text => text.toUpperCase()));

// Expected output:
// Status: ready
// READY
```

The fallback produces text proportional to the input length. An injected formatter has its own cost and output contract. Avoid converting operational failure into a success-shaped fallback unless the API explicitly promises that policy.

Exercise 5: Make a Host Effect Testable

Requirements: `exportTitle(title, writeText)` trims a nonempty string, calls an injected synchronous writer exactly once, and returns the written text length. Invalid text must cause no write. A writer error must propagate. Use an array as the test writer's private log.

Hint: Make all validation happen before the writer call.

Solution: Complete validation before performing the effect.

```
'use strict';
const assert = require('node:assert/strict');

function exportTitle(title, writeText) {
  if (typeof title !== 'string' || title.trim() === '') throw new TypeError('title required');
  if (typeof writeText !== 'function') throw new TypeError('writer required');
  const normalized = title.trim();
  writeText(normalized);
  return normalized.length;
}
const writes = [];
const writer = text => writes.push(text);
assert.throws(() => exportTitle('   ', writer), TypeError);
assert.equal(writes.length, 0);
assert.equal(exportTitle('  Report  ', writer), 6);
assert.deepEqual(writes, ['Report']);
assert.throws(() => exportTitle('Report', () => { throw new Error('offline'); }), /offline/);
console.log(writes.join('|'));
```

```
// Expected output:
// Report
```

Normalization is linear in text length; the writer adds its own work. The contract is synchronous. An asynchronous writer requires an awaited API, not this immediate success result.

Exercise 6: Summarize Rejected Inputs

Requirements: For an array of unknown amounts, count valid checkout amounts and rejected inputs using the range from Exercise 3. Return only counts, with no host I/O and no mutation. Missing or non-array input must throw. The caller decides where to display the summary.

Hint: Keep two counters and classify each entry once.

Solution: Classification stays inside the calculation; console output stays outside it.

```
'use strict';
const assert = require('node:assert/strict');

function summarizeAmounts(values) {
  if (!Array.isArray(values)) throw new TypeError('array required');
  let accepted = 0;
  let rejected = 0;
  for (const value of values) {
    if (Number.isSafeInteger(value) && value >= 1 && value <= 100_000_000) accepted += 1;
    else rejected += 1;
  }
  return { accepted, rejected };
}
const input = Object.freeze([1, '1', 0, 100_000_000, Infinity]);
assert.deepEqual(summarizeAmounts(input), { accepted: 2, rejected: 3 });
assert.deepEqual(summarizeAmounts([]), { accepted: 0, rejected: 0 });
assert.throws(() => summarizeAmounts(null), TypeError);
console.log(JSON.stringify(summarizeAmounts(input)));

// Expected output:
// {"accepted":2,"rejected":3}
```

For `n` entries, time is O(n) and auxiliary space is O(1). Each entry is classified once. Neither a count nor a passed type check authorizes a purchase; that is a separate server decision.

Companion Checks

`code/volume-1/chapter-01/example-03-boundary-challenges.js` runs these six solutions and assertions. The first two companion scripts add capability failure and checkout boundary regressions.

Multiple-Choice Questions

Choose one answer per question. Each explanation names the rule; the distractors represent common boundary mistakes.

1. Which operation is a language built-in?

- A. Reading document.title
- B. Opening a local file through a Node API
- C. Calling Array.isArray(value)
- D. Writing to the developer console

Answer: C. Array.isArray is specified by ECMAScript. The other operations rely on host facilities.

2. A worker cannot read the page document. What follows?

- A. The worker needs a message or adapter boundary for page-owned data
- B. The engine cannot parse property access
- C. The worker uses a different Number multiplication rule
- D. A transpiler must insert the entire page DOM

Answer: A. Hosts can expose different capabilities while sharing the same core language rules.

3. What does a callable writeText property establish?

- A. The target path is writable
- B. The call will finish asynchronously
- C. The host is Node
- D. The operation can be attempted, but it can still fail

Answer: D. Callability alone proves neither permission, completion mode, nor host identity.

4. What does TC39 proposal maturity describe?

- A. Installed browser market share
- B. The speed of an engine
- C. Progress through standardization
- D. Whether npm can install a package

Answer: C. A stage describes proposal progress. Deployment support must be checked separately.

5. Why can a polyfill fail to solve unsupported syntax in the same file?

- A. Polyfills require a DOM
- B. Parsing can fail before any fallback code executes
- C. Only server code can use polyfills
- D. Polyfills can change values but cannot define functions

Answer: B. A runtime fallback cannot execute in a source unit the engine cannot parse.

6. Which boundary belongs on the server for a payment?

- A. Trust the client runtime label
- B. Trust the displayed amount alone
- C. Trust a client valid flag
- D. Validate the request and verify the authoritative price

Answer: D. Client checks improve feedback but can be bypassed by the caller.

7. Why use Number.isSafeInteger for Number cents?

- A. It applies the product price limit automatically
- B. It proves the amount is positive
- C. It checks integral values within the reliably distinguishable integer range
- D. It accepts any decimal string

Answer: C. Safe range and integrality are representation checks. Positivity and business maximum are additional checks.

8. A file parses, prints a line, then reads an undeclared identifier. What can happen?

- A. Evaluation throws ReferenceError after the earlier effect
- B. All prior output is automatically rolled back
- C. The identifier becomes a zero-valued local
- D. The parser repairs the name

Answer: A. Runtime failures can follow already completed effects; parsing success does not imply successful evaluation.

9. Which description of an engine is accurate?

- A. It is a package repository
- B. It decides every application authorization rule
- C. It implements the language semantics
- D. It is the browser document

Answer: C. An engine evaluates language operations. Host APIs and application policies are separate responsibilities.

10. What does a successful bundler build guarantee?

- A. The build step completed under its configured rules
- B. Every input is valid
- C. Every network request will succeed
- D. Every deployed host API exists

Answer: A. Build success alone cannot establish runtime compatibility, valid application data, or external service availability.

11. What is the cleanest way to test a report calculation that must save text?

- A. Inject a writer and assert its calls and failure behavior
- B. Catch every error and return saved
- C. Infer write permission from process
- D. Hard-code a user filesystem path

Answer: A. Injection makes effects observable and failures testable without assuming a particular host or destination.

12. Which statement about JavaScript execution is portable?

- A. All engines have the same optimization tiers
- B. Every object must occupy one physical heap allocation
- C. Every function first runs exactly once in an interpreter
- D. Implementations must preserve observable language behavior

Answer: D. The specification constrains semantics, while representation and compilation strategy are implementation choices.

13. Why does a long synchronous calculation delay unrelated callbacks on its thread?

- A. The thread is occupied until the calculation yields or completes
- B. Callbacks stop being functions
- C. The host always copies the calculation to another thread
- D. Numbers disable the event loop

Answer: A. Synchronous work occupies the thread. A wrapper function does not make CPU work asynchronous.

14. What does typeof aMissingName return when there is no binding for that name?

- A. The string undefined
- B. It always throws TypeError
- C. null
- D. The value undefined rather than a string

Answer: A. typeof has a special rule for unresolvable identifiers. An existing uninitialized lexical binding is a different case.

15. A trusted writer throws quota exceeded. Which result is honest for a caller?

- A. Return saved because the writer was callable
- B. Propagate or explicitly report the operation failure
- C. Relabel the host as unknown
- D. Repeat forever immediately

Answer: B. Capability presence does not establish outcome. Recovery needs a defined policy.

16. What belongs in a shared checkout validator?

- A. Filesystem path discovery
- B. Representation and domain checks independent of host I/O
- C. DOM selection and user prompting
- D. The assumption that all client values are trusted

Answer: B. Keeping the rule independent of host effects makes it reusable; authorization remains a distinct responsibility.

Revision Sheet and Summary

The Rules to Remember

Question	Working answer
What is ECMAScript?	The standard defining core language syntax and semantics.
What is an engine?	An implementation that parses and evaluates the language.
What is a host?	The environment supplying loading, external APIs, and scheduling facilities.
Is `Array.isArray` host-specific?	No; it is a language built-in.
Is `document` universally available?	No; browser pages provide a DOM, while workers and ordinary Node code do not provide that page document.
Does a capability-shaped object imply permission?	No; the operation still has to succeed.
Does transpiling install every missing API?	No; syntax transformation and runtime support are different.
Does integer-valued mean safely exact?	No; Number inputs need safe-integer and domain bounds when exact integer arithmetic is required.
Does client validation authorize a transaction?	No; the server owns its authorization and trusted price checks.

Trace an Operation

Source is parsed; declarations are instantiated; statements evaluate; function calls introduce call-local bindings; host adapters perform effects. This is a semantic account, not a promise that every engine allocates these records literally.

For `totalFor(3)` with a price of 1500 cents, the parameter receives 3, outer lookup finds 1500, multiplication produces 4500, and the caller receives that value. Logging is a separate host request.

Decisions Worth Practicing

Use shared pure calculations when rules are portable. Inject a host adapter when an operation needs output, storage, or network access. Check a proposal's status and the actual target runtime separately. Define a supported input representation before converting external data. Let callers observe an operation failure rather than returning a misleading success result.

Summary

JavaScript's core behavior and its surrounding environment are separate layers. This explains why portable calculations can be reused while DOM, filesystem, and permission assumptions require explicit adapters. A clear input contract and independently tested effects make that separation practical.

Continue with Variables and Data Types. After learning functions, use the execution-model companion to revisit calls and lexical environments in more depth.

References and Further Reading

- TC39 process: proposal maturity and integration into the standard.
- ECMAScript hosts and implementations: the language/host boundary.
- Node globals: actual runtime facilities, including their version and stability information.
- Using Web Workers: a browser context with a different capability set from a page.
- V8 Ignition: one engine's interpreter, rather than a universal JavaScript pipeline.

Edge Cases and Debugging

Distinguish Syntax Failure From Operation Failure

A parser error prevents evaluation of the affected script. A missing identifier can fail after earlier statements have already run. A writer can exist and still throw while performing its operation. These failures call for different repairs: fix the source, fix the dependency, or handle the operation outcome.

```
'use strict';

console.log('before lookup');
try {
  unavailableDocument.querySelector('main');
} catch (error) {
  console.log(error.name);
}
console.log('after handled failure');

// Expected output:
// before lookup
// ReferenceError
// after handled failure
```

The unavailable root name fails before property lookup or method invocation. Read the first relevant application frame in the error stack, then identify whether the missing name was supposed to be a local binding, an import, or a host capability.

Presence Is Not Callability

```
'use strict';

const adapter = { writeText: true };
console.log('writeText' in adapter);
console.log(typeof adapter.writeText === 'function');
try {
  adapter.writeText('report');
} catch (error) {
  console.log(error.name);
}

// Expected output:
// true
// false
// TypeError
```

Checking a property name alone accepts unusable values. Even the improved check assumes a trusted adapter: property access can invoke a getter or proxy trap. Do not present capability inspection as a sandbox for arbitrary JavaScript objects.

A Working Adapter Can Fail on One Request

```
'use strict';

function writeReport(writeText, text) {
  if (typeof writeText !== 'function') throw new TypeError('writer required');
  writeText(text);
  return 'saved';
}
try {
  console.log(writeReport(() => { throw new Error('quota exceeded'); }, 'report'));
} catch (error) {
  console.log(error.message);
}

// Expected output:
// quota exceeded
```

The success value is produced only after the synchronous writer returns. Retrying every failure immediately may worsen a full disk or exhausted quota; callers need an operation-specific recovery policy.

Other Boundaries to Reproduce

Symptom	Check first	Appropriate evidence
Works in page, fails in worker	Direct DOM dependency	Run the adapter test in the worker context.

Symptom	Check first	Appropriate evidence
Works locally, fails after deployment	Runtime version, transformed output, asset loading	Test the actual built artifact.
Number looks rounded unexpectedly	Source text, conversion, safe range	Compare documented input with the parsed value before calculations.
Works in console, fails as a file	Script/module mode and injected console bindings	Reproduce in the intended file mode.
Error message differs across engines	Test relies on exact message wording	Assert error category and application-owned messages where appropriate.

Keep a minimal failing input and an assertion for the promised outcome. A log of one successful checkout would not detect accepting an unsafe amount or mistaking a process label for permission.

Performance and Security

Measure the Work at the Correct Layer

A constant-sized amount check is unlikely to explain a slow network checkout. Separate validation time, serialization time, host I/O, and downstream latency. Logging inside a benchmark can dominate the operation being measured. Warm-up, runtime version, input distribution, and the observation method also affect measurements.

A calculation that loops over `n` cart rows performs work proportional to `n`; moving it into a helper function does not remove that work. Repeating a full calculation after every row can turn a linear task into quadratic work. Establish an algorithmic cost before investigating engine tiers.

Main-Thread Work Remains Work

A long synchronous loop occupies its executing thread. Browser page rendering and Node callbacks can be delayed while that thread remains busy. A timer schedules later work; it does not automatically move a CPU-heavy calculation onto another thread. Consider bounded input, incremental processing, or workers when measurements justify them. Scheduling is developed in the asynchronous volume.

Do Not Treat a Runtime Label as Authority

The presence of `process`, `window`, or `fetch` proves neither the caller's identity nor permission to write a destination. Authorization belongs to the service or resource owner. Test the actual operation and handle its documented failure. Restrict a supplied adapter to the operations the calculation needs instead of passing an entire application container.

Parse Data as Data

Text received from an API is input to a documented parser and validator. Do not use `eval` or a constructed function to interpret a price, feature flag, or configuration value. Code execution is a much larger capability than data parsing. Parsed data still needs domain checks: a JSON number can be negative, fractional, or already rounded beyond the safe-integer range.

Shared Validation Does Not Eliminate Server Validation

A browser user can bypass application code and submit a request directly. The server must enforce the type, amount bounds, cart ownership, and authoritative product price. Do not log complete payment payloads just to diagnose a failed type check; record the field name, error class, request identifier, and an appropriate redacted description.

Review Criteria

Accept a change when its allowed representations and failure behavior are explicit, its expensive work is bounded, and its effect dependencies are testable. Reject an optimization justified solely by "JavaScript is interpreted" or an authorization decision justified by "the client already checked it." Both claims skip the layer that determines the outcome.

Professional Field Guide

Case Study: Export Works in a Page but Fails in a Worker

A report calculator queries the document for a status label before returning its result. A team moves the calculation into a worker to keep the page responsive. The first report now fails before calculation begins because the worker has no page document.

The repair separates responsibilities. The calculator receives parsed data and returns a report value. A page adapter reads form controls and updates status. A worker adapter receives messages and posts results. The language-level rule has no reason to discover a DOM or infer its host from global names.

Verify the calculation with plain inputs, the page adapter with a document-capable test environment, and the worker adapter with its message contract. A passing calculator test alone does not verify either host integration.

Case Study: Checkout Rejects an Unexpected Amount

A server accepts integer cents, but a form submits the string `"2599"`. Silently widening the shared validator to convert any input would also accept representations the API never promised, such as whitespace or booleans under some conversions. Define the form parser's decimal grammar and bounds, then pass a Number to the shared validator. The server applies its own request contract and price verification.

Record the accepted representation, minimum, maximum, error category, and the component that owns authorization. Boundary tests should include the maximum plus one and malformed text, not just a typical purchase.

Choosing Where Code Belongs

Responsibility	Good home	Reason
Validate integer-cent amount	Shared domain function	Same rule can run in multiple hosts.
Read an input element	Page adapter	Depends on a particular document.
Write a report file	Explicit storage adapter	Permission and failure belong to the operation.
Verify cart ownership	Authoritative server service	Client behavior is controlled by the user.
Decide runtime support	Deployment policy and target tests	A proposal stage is insufficient evidence.

A Review Conversation

Ask what happens before the calculation, what it guarantees, and what can fail after it returns. A successful amount check does not imply that a writer succeeded. A successful write does not prove that the amount was authorized. Keeping these claims separate produces useful result types and logs.

When a bug appears, reproduce it in the intended execution mode using the smallest real input. State the violated contract in an assertion. Place the repair in the layer that owns that contract so the next person can understand the fix.

Readiness Check

Explain why `Array.isArray` is portable while a page DOM is not; trace a function call; distinguish a callable capability from permission; name an input that violates the checkout range; and explain why the server validates independently. Then continue to Variables and Data Types.

CHAPTER 2

Variables and Data Types

A variable name identifies a binding; a binding holds a value. That value can be a primitive, such as a string, or an object identity shared with other bindings. Keeping these three ideas separate explains why `const` prevents reassignment while an object property can still change.

This chapter follows one record through declaration, initialization, aliasing, mutation, and replacement. It also develops the checks needed when unknown input reaches a function: a `typeof` result is evidence about a value, not a complete business contract.

Learning Objectives

- Compare the scope, initialization, reassignment, and redeclaration behavior of `var`, `let`, and `const`.
- Identify the seven primitive types and distinguish them from objects.
- Predict a temporal-dead-zone read, including the `typeof` exception.
- Explain dynamic typing without calling JavaScript untyped.
- Trace shared object identity, parameter reassignment, and mutation.
- Build a boundary normalizer that returns a new, explicitly shaped record.
- Explain reachability without claiming that scope exit forces immediate collection.

Prerequisites and Running Guide

Read Introduction to JavaScript. Know that a function receives argument values and returns a result, and that an object groups named properties between braces. This chapter explains the value behavior used by those constructs; Functions and Callbacks and Objects and Data Ownership develop their syntax and API design further.

Run each example independently in Node.js 20 or later. Companion programs contain assertions and expected-output comments:

```
node code/volume-1/chapter-02/example-01-const-object-mutation.js
node code/volume-1/chapter-02/example-02-type-guards.js
node code/volume-1/chapter-02/example-03-binding-challenges.js
```

The baseline is for reproducible examples, not a deployment lifecycle recommendation. Files here are CommonJS-compatible and use strict mode explicitly. Do not assume a browser console's top-level behavior is identical to a Node file or an ECMAScript module.

First Observation

```
'use strict';

const shipment = { status: 'queued' };
shipment.status = 'ready';
console.log(shipment.status);
try {
  shipment = { status: 'sent' };
} catch (error) {
  console.log(error.name);
}

// Expected output:
// ready
// TypeError
```

The first assignment targets an object property. The second targets the immutable binding. Read the theory, then use the diagrams to draw the difference before attempting the exercises.

Theory

Binding, Value, and Object Identity

A binding associates a name with a value in an environment. Evaluating an identifier reads that binding. Assignment to an identifier changes a mutable binding's value; assignment to a property changes the target object if that property permits it. The operations are different even when they use the same `=` token.

```
'use strict';

let selected = { id: 'A', status: 'queued' };
const original = selected;
selected.status = 'ready';
selected = { id: 'B', status: 'queued' };
console.log(original.id, original.status);
console.log(selected.id, selected === original);

// Expected output:
// A ready
// B false
```

The first two bindings initially designate one object. Mutation changes that object's status. Replacement changes only `selected`; `original` still designates the first object. This model avoids the misleading idea that every assignment duplicates a whole object graph.

Declaration Forms

Rule	`var`	`let`	`const`
Ordinary scope	Nearest function, or top-level script/module scope	Enclosing block or top-level scope	Enclosing block or top-level scope
Initial state before its declaration executes	Initialized to `undefined`	Uninitialized	Uninitialized
Ordinary declaration without initializer	Allowed	Allowed; initializes to `undefined` when reached	Not allowed
Reassignment after initialization	Allowed	Allowed	Throws `TypeError`
Repeating the same declaration name in one scope	Often permitted for `var`; conflicts with lexical declarations are errors	Duplicate lexical declaration is an early error	Duplicate lexical declaration is an early error

`for...of` and `for...in` loop declarations are special forms that can initialize a fresh `const` binding from each iterated value without a source `=` initializer. Use `const` for a binding that will keep its value, `let` for deliberate reassignment, and understand `var` when reading existing code. Choosing `const` does not promise deep immutability or greater execution speed.

```
'use strict';

function inspectDeclarations() {
  console.log(legacy);
  var legacy = 'ready';
  let pending;
  console.log(legacy, pending);
  if (true) {
    var visibleAfterBlock = 3;
    const onlyInside = 7;
    console.log(onlyInside);
  }
  console.log(visibleAfterBlock);
}
inspectDeclarations();

// Expected output:
// undefined
// ready undefined
// 7
// 3
```

Function calls have their own declaration instances. A block alone does not create a separate `var` scope. Formal lexical-declaration rules are specified in ECMAScript let and const declarations.

Initialization and the Temporal Dead Zone

An uninitialized binding is not a binding whose value happens to be `undefined`. Reading an uninitialized lexical binding throws `ReferenceError`. The interval from scope entry until that binding's initialization is called its temporal dead zone, or TDZ.

```
'use strict';

const status = 'outer';
try {
  console.log(status);
  let status = 'inner';
} catch (error) {
  console.log(error.name);
}
let ready;
console.log(ready);
console.log(status);

// Expected output:
// ReferenceError
// undefined
// outer
```

The block's `status` shadows the outer name throughout that block, including before initialization. Lookup finds the inner binding and fails; it does not skip that binding to use the outer value. By contrast, `let ready;` actually executes and initializes `ready` to `undefined`.

"Hoisting" is informal terminology for these declaration effects. It does not mean the engine physically moves source statements. Explain what binding exists and when it initializes instead of saying all declarations behave identically. The MDN let reference provides TDZ and shadowing examples.

Primitive Values and Objects

JavaScript has seven primitive types: Undefined, Null, Boolean, String, Symbol, Number, and BigInt. Primitives are immutable values. Object is the remaining language type; arrays and functions are objects with additional behavior. A function's special `typeof` result does not make it an eighth primitive type.

Type	Example	Practical distinction
Undefined	`undefined`	Common result of an absent property or an initialized declaration without a value.
Null	`null`	Explicit absence chosen by the application contract.
Boolean	`false`	One of two logical values.
String	`'ready'`	Immutable text value.

Type	Example	Practical distinction
Symbol	`Symbol('id')`	A unique symbol for each direct call; often used as a property key.
Number	`42`, `1.5`, `NaN`	Floating-point numeric values, including special values.
BigInt	`42n`	Arbitrary-precision integers, subject to available resources.
Object	`{ id: 'A' }`	Identity-bearing value with properties; may be mutable.

The ECMAScript data types overview gives the language categories. A business type such as "positive safe integer cents" is a narrower contract built from these types.

```
'use strict';

let title = 'report';
const previousTitle = title;
title = title.toUpperCase();
console.log(previousTitle, title);
console.log(Symbol('id') === Symbol('id'));
console.log(42n === 42);

// Expected output:
// report REPORT
// false
// false
```

Calling a string method produces a result; it does not mutate the original primitive string. Equal symbol descriptions do not mean equal symbols. BigInt and Number are distinct types; converting or comparing them has additional rules developed in later chapters.

Dynamic Typing

A mutable binding can successively hold values of different types. Operations use the values currently present, so the same expression can behave differently after reassignment.

```
'use strict';

let quantity = 4;
console.log(typeof quantity, quantity + 1);
quantity = '4';
console.log(typeof quantity, quantity + 1);

// Expected output:
// number 5
// string 41
```

Dynamic typing does not mean values lack types. It means a binding does not permanently constrain its values to one declared runtime type. For production state, keep the representation stable when practical: parse a form string once, validate it, and pass a Number to numeric code.

What typeof Can and Cannot Tell You

```
'use strict';

console.log(typeof null, typeof [], typeof (() => {}));
console.log(typeof NaN, Number.isNaN(NaN));
console.log(Array.isArray([]), Array.isArray({}));
console.log(typeof neverDeclaredHere);
try {
  console.log(typeof pending);
  let pending = 1;
} catch (error) {
  console.log(error.name);
}

// Expected output:
// object object function
// number true
// true false
// undefined
// ReferenceError
```

Use `value === null` for null, `Array.isArray` for arrays, and `Number.isFinite` or `Number.isSafeInteger` for appropriate numeric contracts. `typeof value === 'object'` also accepts null and arrays. `typeof NaN` is `number`, because NaN belongs to the Number type. The undeclared-name exception does not bypass the TDZ of a binding that exists.

Do not generalize browser legacy behavior such as `document.all` into a validation strategy. The examples use ordinary language values. See MDN typeof for the compatibility exception and complete operator behavior.

Passing Values and Sharing Objects

Arguments are passed by value. When that value is an object, the caller and parameter can designate the same object. Mutating its property can be visible to the caller; reassigning the parameter cannot reassign the caller's binding.

```
'use strict';

function review(record) {
  record.reviewed = true;
  record = { reviewed: false };
  return record;
}
const original = { reviewed: false };
const replacement = review(original);
console.log(original.reviewed, replacement.reviewed, original === replacement);
```

```
// Expected output:
// true false false
```

Calling this "objects are passed by reference" often causes the wrong prediction about the parameter reassignment. Say that the object identity is shared, then trace the mutation and rebinding separately.

Reachability and Representation

A value can remain reachable through another binding, a property, a collection, or a retained function even when one local scope ends. An unreachable object becomes eligible for collection; collection time is not a language-level promise. Cycles alone do not necessarily make a leak; a reachable retention path matters.

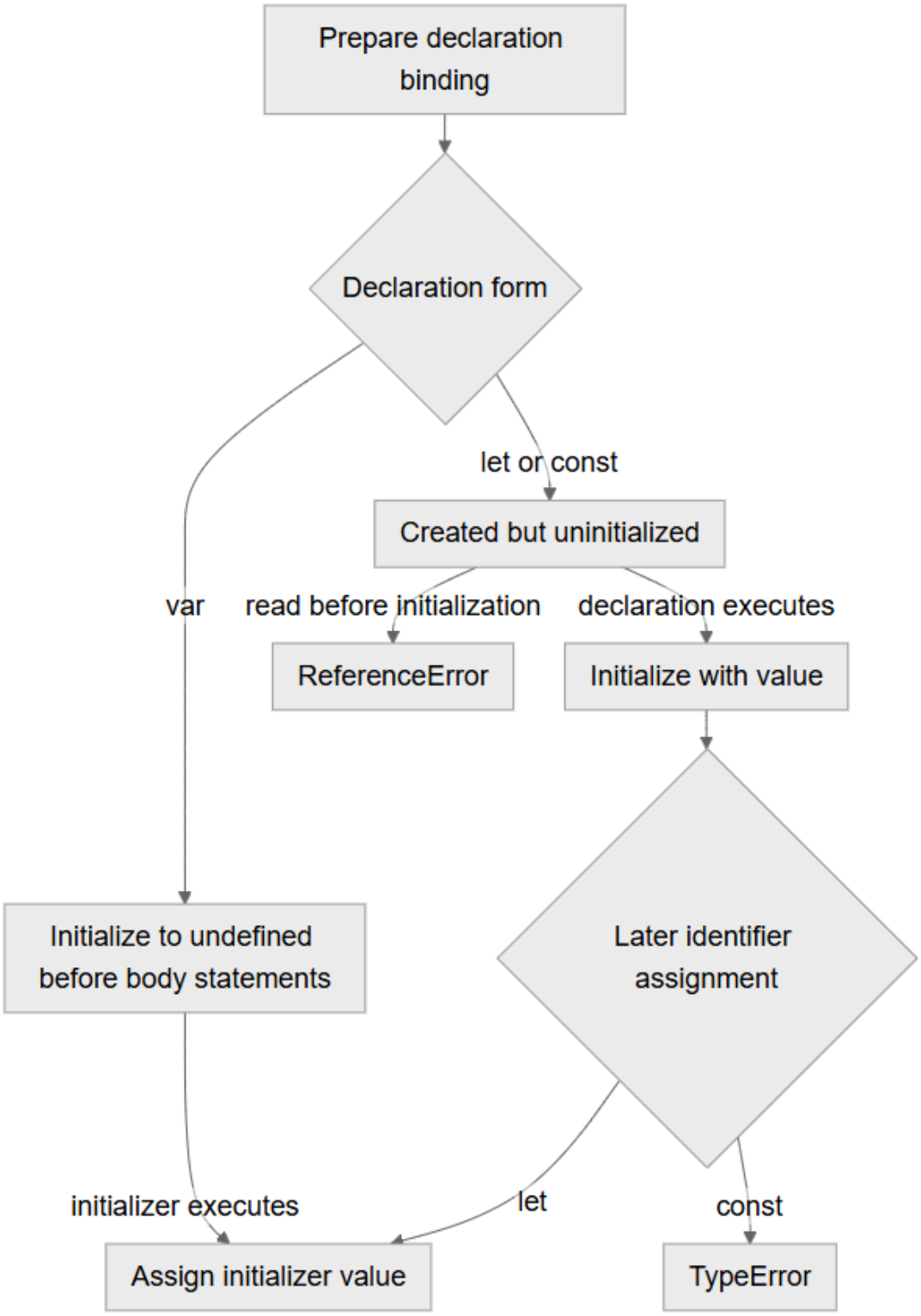
Stack/heap drawings are conceptual aids, not universal allocation rules. Engines can optimize away bindings or allocations and use different physical representations. Use semantic diagrams to predict aliases; use profiling to investigate actual memory. MDN memory management explains reachability and collection limitations.

Internal Working

Trace the Binding Lifecycle

Consider a function containing `var legacy = 1`, `let current = 2`, and `const fixed = 3`. Before evaluating those body statements, the ordinary `var` binding is initialized to `undefined`. The lexical declarations are present but uninitialized. Evaluating their declarations initializes them in statement order. A later assignment can change `legacy` or `current`; changing `fixed` throws.

The lexical environment determines which binding an identifier denotes. The declaration form and lifecycle determine which operations are allowed on that binding. A value of `undefined` is an initialized state, not a synonym for "no binding."



Source: `diagrams/volume-1-chapter-02-binding-lifecycle.mmd`. This diagram describes ordinary declarations; imports and loop declarations have additional rules.

Trace a Shared Record

```
'use strict';

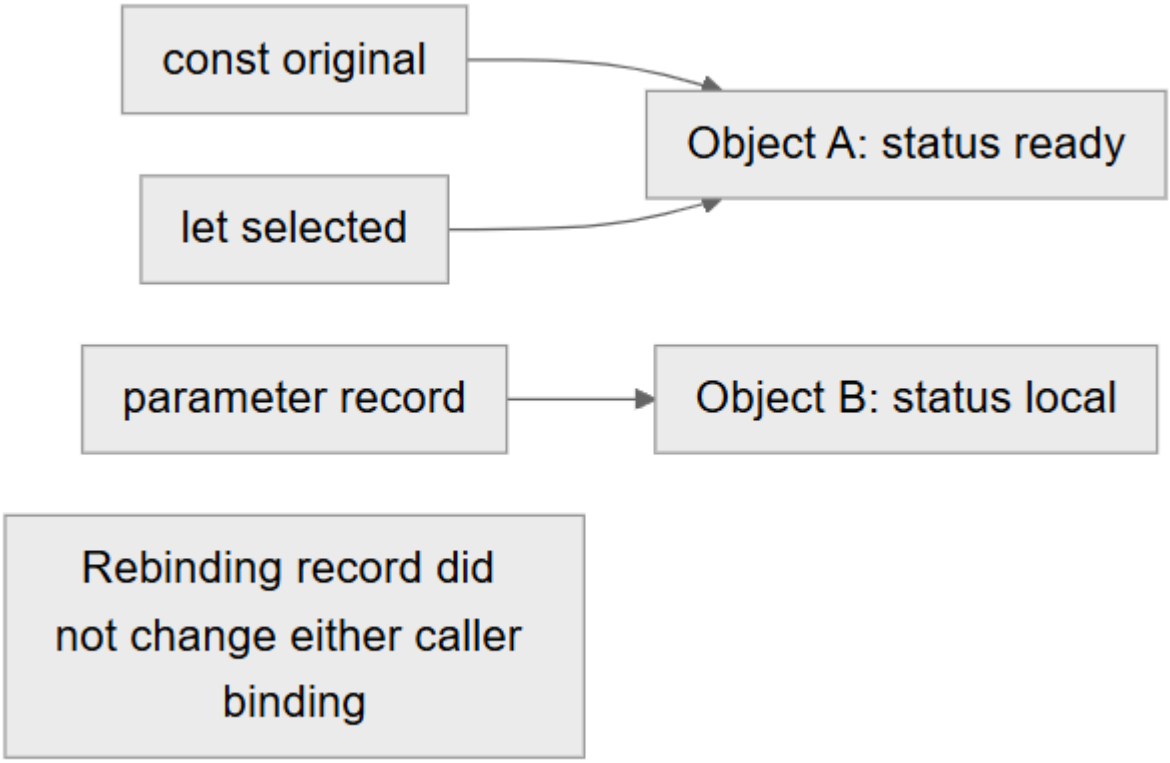
const original = { status: 'queued' };
let selected = original;
function mark(record) {
  record.status = 'ready';
  record = { status: 'local' };
  return record.status;
}
console.log(mark(selected));
console.log(original.status, selected === original);
selected = { status: 'replacement' };
console.log(original.status, selected.status);

// Expected output:
// local
// ready true
// ready replacement
```

1. Evaluate the first object literal, creating object A; initialize `original` with its identity.
2. Read `original` and initialize `selected` with the same identity. No object clone occurs.
3. Call `mark` with that value; initialize parameter `record` with object A.
4. Assign `record.status`, changing object A to contain `ready`.
5. Create object B and reassign the parameter to B. The caller's bindings still designate A.
6. Return B's `status` value, `local`. The parameter's reassignment does not escape as a binding change.
7. Create object C and reassign `selected`. `original` still designates A.

Semantic Memory Diagram

The snapshot is taken inside `mark`, after step 5 and before the return:



Source: `diagrams/volume-1-chapter-02-object-aliases.mmd`. An arrow means that a binding designates an object identity; it does not promise an exposed memory address. After the final replacement, `selected` would point to a third object.

Primitive Assignment Has No Mutable Object to Share

```
'use strict';

let pending = 4;
const snapshot = pending;
pending = 5;
console.log(snapshot, pending);

// Expected output:
// 4 5
```

The second binding receives the Number value 4. Assigning 5 to `pending` does not change that earlier primitive value. No distinction between "deep" and "shallow" copying is necessary for these immutable Number values.

Engine Internals: Semantics Before Storage

Environment records are specification machinery for resolving and updating bindings. Engines may represent locals in registers, stack slots, retained environment structures, or optimized forms. A compiler can remove a binding that has no observable role. Neither `const` nor `let` forces a specific physical storage location.

Likewise, the diagram's object B may become unreachable once the call finishes, but the language does not require an immediate collection. The retained reference from `original` keeps A available independently of whether `selected` changes. For a leak investigation, follow the retaining path rather than counting how many functions have returned.

Read ECMAScript environment records for the formal binding model. The execution-model companion extends this trace to returned functions after the function fundamentals chapter.

Production Examples

Make Mutation Ownership Visible

A trusted in-memory account record can be updated in place when the caller deliberately owns that state. A `const` declaration communicates that the binding stays attached to this account, not that its properties are immutable.

```
'use strict';

const account = { id: 'acct_100', status: 'active' };
const observer = account;
account.status = 'suspended';
console.log(account.status, observer.status, observer === account);
try {
  account = { id: 'acct_200', status: 'active' };
} catch (error) {
  console.log(error.name);
}

// Expected output:
// suspended suspended true
// TypeError
```

An observer holding the same identity sees the change. Use this design when sharing live state is intentional. When a caller needs a prior snapshot, create a record with the required owned values instead of assuming `const` protects the old state. The assertions in `code/volume-1/chapter-02/example-01-const-object-mutation.js` cover both mutation and binding replacement.

Normalize a User Display Record

A boundary accepts an ordinary parsed JSON object containing `id` and `email` strings. The application's display policy trims both fields and lowercases email text; this is not a general mailbox identity or deliverability validator. The output contains only those two fields. It excludes arbitrary input properties such as `role`.

```
'use strict';

function normalizeUser(input) {
  if (input === null || typeof input !== 'object' || Array.isArray(input)) {
    throw new TypeError('user object required');
  }
  const { id, email } = input;
  if (typeof id !== 'string' || typeof email !== 'string') {
    throw new TypeError('id and email must be strings');
  }
  const cleanId = id.trim();
  const cleanEmail = email.trim().toLowerCase();
  if (cleanId === '' || cleanEmail === '') throw new RangeError('fields must be nonempty');
  return { id: cleanId, email: cleanEmail };
}
const raw = { id: ' U-1 ', email: ' LEA@EXAMPLE.COM ', role: 'admin' };
const normalized = normalizeUser(raw);
console.log(JSON.stringify(normalized));
console.log(raw.id === ' U-1 ', normalized === raw);
for (const input of [null, [], { id: 'U-2', email: 3 }, { id: ' ', email: 'a@b.test' }]) {
  try { normalizeUser(input); } catch (error) { console.log(error.name); }
}

// Expected output:
// {"id":"U-1","email":"lea@example.com"}
// true false
// TypeError
// TypeError
// TypeError
// RangeError
```

The outer type check rejects null and arrays before property access. String checks precede string methods. Domain checks reject empty normalized text. The new output record stores immutable string values, so changing either record's field later does not mutate the other record.

This is a contract for parsed data, not arbitrary objects that can execute getters or proxy traps. Schema validation and application authorization are separate responsibilities. `code/volume-1/chapter-02/example-02-type-guards.js` tests these accepted and rejected values plus output ownership. If total field length is `L`, normalization costs O(L) time and creates output text proportional to the normalized length.

Use a New Binding for Each Representation

```
'use strict';

function validateStockCount(value) {
```

```
  if (!Number.isSafeInteger(value)) throw new TypeError('safe integer count required');
  if (value < 0 || value > 10_000) throw new RangeError('stock count outside range');
  return value;
}
const incoming = { count: 12 };
const stockCount = validateStockCount(incoming.count);
const message = `${stockCount} units available`;
console.log(message);
try {
  validateStockCount('12');
} catch (error) {
  console.log(error.name);
}

// Expected output:
// 12 units available
// TypeError
```

`incoming`, `stockCount`, and `message` describe distinct roles. Reusing a mutable `value` binding for the parsed object, then a number, then text would be legal, but makes subsequent operations harder to inspect. Representation changes deserve names and explicit boundaries. Numeric guards are constant-sized; formatting adds the cost of producing the message.

Production Review

State whether a function mutates its argument, returns a new record, or intentionally retains a shared reference. Validate language type and domain separately. Do not treat a `typeof` result, a `const` declaration, or a freshly allocated outer object as proof of a deeper ownership or authorization claim.

Interview Perspective

What Does const Guarantee?

A `const` binding cannot be reassigned after initialization. It does not recursively freeze the value it holds. Property assignment and identifier assignment target different things. Show the difference with a record rather than saying "const means constant object."

```
'use strict';

const invoice = { cents: 500 };
const alias = invoice;
alias.cents = 700;
console.log(invoice.cents);
try { invoice = { cents: 900 }; } catch (error) { console.log(error.name); }

// Expected output:
// 700
// TypeError
```

The alias and original designate the same object. The property mutation succeeds; replacing the immutable binding fails. Freezing an object is a separate operation and is shallow, as the later object chapter explains.

Why Does This Shadowing Fail?

```
'use strict';

let count = 10;
try {
  const count = count + 1;
  console.log(count);
} catch (error) {
  console.log(error.name);
}
console.log(count);

// Expected output:
// ReferenceError
// 10
```

The initializer's read resolves to the inner `count`, which is uninitialized until its initializer completes. The outer value is not a fallback. Use a different name when an initializer deliberately derives a new value from the outer binding.

Is JavaScript Pass-by-Reference?

Argument values initialize parameter bindings. For an object, the copied value designates the same object identity. A property mutation can affect the caller's observation; assigning another object to the parameter does not replace the caller's binding. That second fact is why "pass-by-reference" is usually a misleading answer here.

Draw two bindings pointing to object A. Then draw a parameter reassignment as moving only the parameter's arrow. If the interviewer asks about primitive arguments, there is no mutable primitive object whose contents can be altered through the parameter.

What Is Wrong With typeof x === 'object' as a Record Guard?

It accepts null and arrays as well as ordinary records. A non-null object check plus `!Array.isArray(x)` is a first container check for an ordinary JSON-object contract, not proof that every required field is valid. Inspect field types and business constraints next. Arbitrary JavaScript objects may contain getters, proxies, or unusual prototypes; do not silently promise to handle all of them.

What Does Hoisting Mean?

It is informal vocabulary for declaration behavior observable before a declaration's textual position. A `var` binding ordinarily starts as undefined; lexical bindings exist but cannot be read until initialization. A function declaration can already be callable before its statement position. State the declaration form and its timing instead of claiming every declaration is simply moved to the top.

Why Can typeof Throw?

`typeof missingName` returns the string `undefined` when there is no binding for the name. A lexical binding in its TDZ exists and throws when read, including through `typeof`. This distinction is useful when diagnosing a capability check accidentally shadowed by a local declaration.

Does Leaving a Function Free Its Objects?

Not necessarily. Another binding, object property, collection, or returned function can retain access to data. Unreachable data becomes eligible for collection, with no promised immediate collection time. A local variable and the object it designates can have different useful lifetimes.

Senior Design Question

A normalizer returns `{ ...input }` and claims callers cannot affect its state. Ask whether any properties contain objects and whether those objects are shared. A new outer record is insufficient to establish nested ownership. For a narrow `{ id, email }` string schema, construct a new record from validated primitive fields. For a nested schema, document copying or immutability at every mutable boundary.

Exercises and Coding Challenges

Each solution is independent and includes assertions. Node's `assert` utility throws if an expectation fails. Draw bindings and object identities before running mutation examples. The companion program is `code/volume-1/chapter-02/example-03-binding-challenges.js`.

Exercise 1: Separate Property Mutation From Rebinding

Requirements: Begin with a queued record and a second binding to that record. Mutate through the second binding, then replace only that mutable binding. Prove that the original retains the first mutation and that the two final records differ.

Hint: Draw two arrows first; move only one arrow on identifier assignment.

```
'use strict';
const assert = require('node:assert/strict');
```

```
const original = { status: 'queued' };
let selected = original;
selected.status = 'ready';
selected = { status: 'sent' };
assert.equal(original.status, 'ready');
assert.equal(selected.status, 'sent');
assert.notEqual(original, selected);
console.log(original.status, selected.status);

// Expected output:
// ready sent
```

The property assignment changes the shared object. The later identifier assignment replaces only `selected`. This fixed-sized trace uses O(1) work and object storage.

Exercise 2: Repair a TDZ Initializer

Requirements: An outer quantity is 8. In a nested block derive a preview quantity one greater without changing the outer binding. Avoid `const quantity = quantity + 1`, which would read its own uninitialized binding. Verify both values.

Hint: Give the derived representation a distinct name.

```
'use strict';
const assert = require('node:assert/strict');

const quantity = 8;
{
  const previewQuantity = quantity + 1;
  assert.equal(previewQuantity, 9);
  console.log(previewQuantity);
}
assert.equal(quantity, 8);
console.log(quantity);

// Expected output:
// 9
// 8
```

The initializer now resolves the outer quantity. A new block-local binding names the derived result. Work and storage are O(1); this repair changes naming, not the surrounding quantity's value.

Exercise 3: Classify Values for Diagnostics

Requirements: Return `null`, `array`, or `nan` for those special categories; otherwise return the ordinary `typeof` result. Do not mistake a number-like string for a Number or a function for a primitive. This is a diagnostic label, not a schema validator.

Hint: Check the narrower cases before using the broad operator result.

```
'use strict';
const assert = require('node:assert/strict');

function diagnosticKind(value) {
  if (value === null) return 'null';
  if (Array.isArray(value)) return 'array';
  if (Number.isNaN(value)) return 'nan';
  return typeof value;
}
const pairs = [
  [null, 'null'], [[], 'array'], [NaN, 'nan'], [undefined, 'undefined'],
  [false, 'boolean'], ['4', 'string'], [4, 'number'], [4n, 'bigint'],
  [Symbol('id'), 'symbol'], [{}, 'object'], [() => {}, 'function']
];
for (const [value, kind] of pairs) assert.equal(diagnosticKind(value), kind);
console.log(pairs.map(([value]) => diagnosticKind(value)).join('|'));

// Expected output:
// null|array|nan|undefined|boolean|string|number|bigint|symbol|object|function
```

Each classification uses fixed-sized checks. Labeling a list of `n` values takes O(n) checks plus output construction. Infinity remains `number`; if the caller needs finite values, add a separate numeric contract.

Exercise 4: Validate a Mutable Counter's Next Value

Requirements: Accept a Number stock count between 0 and 10,000 inclusive and a Number delta of exactly 1 or -1. Return the adjusted count when it remains in range. Reject unsupported representations with `TypeError` and range violations with `RangeError`. The function must not silently convert strings.

Hint: Validate both inputs before adding them; arithmetic is exact within these small integer bounds.

```
'use strict';
const assert = require('node:assert/strict');

function adjustStock(count, delta) {
  if (!Number.isSafeInteger(count) || !Number.isSafeInteger(delta)) {
    throw new TypeError('safe integer inputs required');
  }
  if (count < 0 || count > 10_000 || (delta !== 1 && delta !== -1)) {
    throw new RangeError('invalid stock adjustment');
  }
  const nextCount = count + delta;
  if (nextCount < 0 || nextCount > 10_000) throw new RangeError('stock range exceeded');
  return nextCount;
}
assert.equal(adjustStock(0, 1), 1);
assert.equal(adjustStock(10_000, -1), 9999);
for (const args of [[0, -1], [10_000, 1], [-1, 1], [3, 2]]) {
  assert.throws(() => adjustStock(...args), RangeError);
}
for (const args of [['3', 1], [NaN, 1], [3, '1'], [3.5, 1]]) {
  assert.throws(() => adjustStock(...args), TypeError);
}
console.log(adjustStock(3, 1));
```

```
// Expected output:
// 4
```

Fixed-sized numeric guards and arithmetic use O(1) time and auxiliary space. The returned value is independent of the caller's binding; the caller decides whether to assign it.

Exercise 5: Take a Narrow Shipment Snapshot

Requirements: Accept an ordinary record with nonempty string `id` and `status`. Return a new record containing trimmed values for just those fields. Changes to input or output fields must not change the other record. Reject null, arrays, wrong field types, and blank text. Do not promise to clone nested properties that the schema excludes.

Hint: Construct the output from validated primitive fields rather than spreading the entire input.

```
'use strict';
const assert = require('node:assert/strict');

function shipmentSnapshot(input) {
  if (input === null || typeof input !== 'object' || Array.isArray(input)) {
    throw new TypeError('shipment object required');
  }
  const { id, status } = input;
  if (typeof id !== 'string' || typeof status !== 'string') {
    throw new TypeError('string fields required');
  }
  const cleanId = id.trim();
  const cleanStatus = status.trim();
  if (cleanId === '' || cleanStatus === '') throw new RangeError('nonempty fields required');
  return { id: cleanId, status: cleanStatus };
}
const source = { id: ' S-1 ', status: 'queued', secret: 'excluded' };
const snapshot = shipmentSnapshot(source);
source.status = 'sent';
snapshot.id = 'local';
assert.equal(source.id, ' S-1 ');
assert.equal(snapshot.status, 'queued');
assert.equal(Object.hasOwn(snapshot, 'secret'), false);
for (const invalid of [null, [], {}, { id: 3, status: 'ready' }]) {
  assert.throws(() => shipmentSnapshot(invalid), TypeError);
}
assert.throws(() => shipmentSnapshot({ id: ' ', status: 'ready' }), RangeError);
console.log(snapshot.status, source.status);

// Expected output:
// queued sent
```

Normalization costs O(L) for total text length L. The fixed output schema owns its property slots and stores immutable strings. Adding nested object fields later requires a new ownership decision.

Exercise 6: Prove Parameter Reassignment Is Local

Requirements: Write a function that marks the supplied record reviewed, then reassigns its parameter to a distinct unreviewed record and returns it. Prove that the caller still holds the original reviewed record. Explain both identities rather than using the phrase "pass-by-reference."

Hint: The parameter is a new binding initialized with the argument value.

```
'use strict';
const assert = require('node:assert/strict');

function reviewAndReplaceLocal(record) {
  record.reviewed = true;
  record = { reviewed: false };
  return record;
}
const original = { reviewed: false };
const returned = reviewAndReplaceLocal(original);
assert.equal(original.reviewed, true);
assert.equal(returned.reviewed, false);
assert.notEqual(original, returned);
console.log(original.reviewed, returned.reviewed, original === returned);

// Expected output:
// true false false
```

The function intentionally mutates a trusted ordinary record; it is a value-semantics trace, not a general input validator. It performs O(1) property work and creates one additional object. Reassigning the parameter does not assign to the caller's binding.

Multiple-Choice Questions

Choose one answer for each question. Predict the binding state and value type before reading the explanation.

1. What does const prevent after initialization?

- A. Passing its value to a function
- B. Aliases from referring to its value
- C. Assignment of another value to its binding
- D. Property writes to any object it holds

Answer: C. The binding cannot be reassigned. Object mutability and aliasing are separate properties.

2. What does an ordinary var binding contain before its initializer executes?

- A. undefined
- B. The initializer result
- C. null
- D. It always throws ReferenceError

Answer: A. Ordinary var bindings are initialized to undefined before body statement evaluation.

3. What happens when reading an inner lexical binding before initialization?

- A. It returns null
- B. It throws ReferenceError
- C. Lookup uses an outer name instead
- D. It copies the outer value

Answer: B. Shadowing selects the inner binding throughout its scope; its TDZ does not trigger an outer fallback.

4. Which value has primitive Null type?

- A. null
- B. {}
- C. []
- D. undefined

Answer: A. Null is a primitive type even though typeof null returns object.

5. What does typeof NaN return?

- A. undefined
- B. nan
- C. object
- D. number

Answer: D. NaN is a special Number value. Number.isNaN distinguishes it from other numeric values.

6. Which test specifically detects arrays?

- A. typeof value === object

- B. Number.isFinite(value)
- C. value !== null
- D. Array.isArray(value)

Answer: D. Arrays are objects, so the broad object test does not distinguish them.

7. What does assigning an object value to a second variable do?

- A. Freezes the object
- B. Shares the same object identity
- C. Links the variable bindings so later reassignment affects both
- D. Recursively copies all properties

Answer: B. The bindings are independent, but initially designate the same object. Rebinding one does not rebind the other.

8. A function reassigns its object parameter without mutating the object. What happens to the caller binding?

- A. It receives the new object
- B. It remains unchanged
- C. It becomes immutable
- D. It becomes undefined

Answer: B. The parameter is a separate binding initialized with the argument value.

9. What happens at an executed let result; declaration?

- A. result remains forever in the TDZ
- B. The declaration throws without an initializer
- C. result initializes to null
- D. result initializes to undefined

Answer: D. An ordinary let declaration permits omission of the initializer; reaching it initializes the binding.

10. Which list contains only primitive types?

- A. String, Symbol, BigInt
- B. String, Array, Number
- C. Function, Boolean, Null

- D. Object, Symbol, BigInt

Answer: A. Arrays and functions are objects. String, Symbol, and BigInt are primitive types.

11. What can typeof do for an existing uninitialized lexical binding?

- A. Return the string uninitialized
- B. Throw ReferenceError
- C. Always return undefined
- D. Skip to an outer binding

Answer: B. The undeclared-identifier exception does not bypass a lexical binding's TDZ.

12. What does dynamic typing permit?

- A. Mutating a primitive string in place
- B. Ignoring value types during operations
- C. A const binding to be reassigned
- D. A mutable binding to hold values of different types over time

Answer: D. Operations still depend on types; mutable bindings are not permanently restricted to one runtime type.

13. Why is Number.isSafeInteger alone insufficient for a positive stock count?

- A. It also accepts zero and negative safe integers
- B. It accepts numeric strings
- C. It only works for BigInt
- D. It rejects every integer greater than one

Answer: A. Safe representation and integrality are separate from business range restrictions.

14. Which statement about collection is accurate?

- A. Unreachable data is eligible for collection without a guaranteed immediate time
- B. A cycle is necessarily a leak
- C. Every local object is collected when its function returns
- D. const objects cannot be collected

Answer: A. Reachability, not the declaration keyword or scope exit alone, determines whether data can still be used.

15. What is true of Symbol("id") === Symbol("id")?

- A. It is false because each direct call creates a distinct symbol
- B. It depends on the enclosing binding keyword
- C. It throws because symbols cannot be compared
- D. It is true because the descriptions match

Answer: A. Symbol descriptions help diagnostics; they do not merge distinct directly created symbols.

16. Why construct a new `{id, email}` record from validated strings?

- A. To guarantee that the input has no getters in every possible JavaScript object
- B. To authorize the user automatically
- C. To define a narrow output shape with independent property slots
- D. To make all possible input graphs deeply immutable

Answer: C. A schema-specific result establishes the promised ownership for those primitive fields without making broader claims.

Revision Sheet and Summary

Binding Rules

Situation	Result
Read ordinary function-scoped `var` before its initializer	`undefined`
Read `let` or `const` before initialization	`ReferenceError`
Execute `let name;`	Initializes the binding to `undefined`
Reassign initialized `const`	`TypeError`
Mutate an ordinary mutable object held by `const`	Allowed by the binding rule
Read an inner lexical name before its initializer	The inner TDZ applies; outer value is not substituted
Reassign a function parameter	Caller binding is unchanged

Situation	Result
Mutate a shared argument object	Other holders of that identity can observe it

Type Rules

The seven primitive types are Undefined, Null, Boolean, String, Symbol, Number, and BigInt. Objects have identity; arrays and functions are objects. `typeof null` is `object`, `typeof []` is `object`, `typeof NaN` is `number`, and ordinary callable functions report `function`.

Check null with equality, arrays with `Array.isArray`, finite Number values with `Number.isFinite`, and safe integers with `Number.isSafeInteger`. A language type check does not impose application constraints such as positivity, nonempty text, or an allowed status.

Ownership and Lifetime

Assignment shares an object identity; it does not clone the object's graph. Primitive values cannot be mutated. A normalizer returning validated string fields in a new record gives independent field storage for that schema. A shallow copy containing nested objects still shares those nested identities.

Scope and reachability are different: a call can finish while returned functions or other references retain its data. Semantic diagrams do not promise literal stack/heap allocation or collection timing.

Summary

A correct explanation names the binding being accessed, its initialization state, the current value's type, and any shared object identity. Those four questions resolve most declaration and mutation surprises without relying on vague hoisting or memory slogans.

Continue to Operators and Expressions. Return to the execution-model companion after learning functions.

References and Further Reading

- ECMAScript lexical declarations: creation and initialization rules.
- MDN const: immutable bindings and mutable values.
- MDN typeof: type results and the TDZ exception.
- ECMAScript environment records: formal binding machinery.
- MDN memory management: reachability and collection.

Edge Cases and Debugging

Undefined Can Have Several Causes

```
'use strict';

let declared;
const record = { present: undefined };
console.log(declared, record.present, record.missing);
console.log(Object.hasOwn(record, 'present'), Object.hasOwn(record, 'missing'));
console.log(typeof absentIdentifier);

// Expected output:
// undefined undefined undefined
// true false
// undefined
```

The displayed value alone does not tell you whether an object property exists. An initialized binding, an explicitly present undefined-valued property, and a missing property can produce the same value. Check ownership when the distinction belongs to the API contract. Chapter 8 develops property access in detail.

A Name Can Be Known but Unreadable

```
'use strict';

function inspect() {
  try { console.log(typeof configured); }
  catch (error) { console.log(error.name); }
  let configured;
  console.log(typeof configured);
}
inspect();

// Expected output:
// ReferenceError
// undefined
```

The first read occurs in the TDZ; the second reads an initialized undefined value. Do not fix this by catching every ReferenceError and treating it as a missing optional setting. Initialize the intended binding before use.

Block Scope Versus Function Scope

```
'use strict';

function deliveryState() {
  if (true) {
```

```
    var legacy = 'sent';
    let local = 'queued';
    console.log(local);
  }
  console.log(legacy);
  try { console.log(local); } catch (error) { console.log(error.name); }
}
deliveryState();

// Expected output:
// queued
// sent
// ReferenceError
```

The outer read of `local` has no matching binding in this example; it is different from a TDZ read even though both produce ReferenceError. Draw the block's boundary before changing declaration keywords.

Numeric Type Is Too Broad for Many Contracts

```
'use strict';

for (const value of [NaN, Infinity, 2.5, 4]) {
  console.log(typeof value, Number.isFinite(value), Number.isSafeInteger(value));
}

// Expected output:
// number false false
// number false false
// number true false
// number true true
```

A stock count needs more than `typeof number`: integrality, safe representation, and domain bounds matter. If an input decimal was rounded during parsing, a later check cannot reconstruct the original text. Preserve the original representation until the parser's contract has been enforced.

Debug the Target of Assignment

When a const-related bug appears, identify whether the left side is an identifier or a property expression. When an object changes unexpectedly, locate every alias, including arguments, array entries, and returned records. When a value changes type, inspect the last assignment and the input boundary that allowed it.

Avoid testing engine-owned error message text exactly; wording can differ. Assert `ReferenceError`, `TypeError`, or the application-owned message that the contract actually promises.

Performance and Security

Clear Bindings Are Not a Benchmark Result

Use `const` to express no reassignment and `let` for changing bindings. Neither keyword guarantees a particular physical allocation or a universal speed advantage. Engines optimize according to their implementation and workload. Do not replace a clear local binding with property access on a global container based on guessed stack/heap rules.

Copying Has a Cost and a Contract

Copying a fixed two-string user record creates one new object and normalized strings. Copying `n` records costs work proportional to the records and text processed. Copying every accumulated prefix after every insertion creates quadratic cumulative record work. Choose snapshots when callers need independent state; choose explicit shared state when live mutation is the intended interface.

A shallow copy is cheaper than traversing an entire graph, but it preserves nested aliases. Avoid claiming full isolation merely because a new outer object exists. A schema-specific copy can be both cheaper and more precise than an unspecified general clone.

Retention Matters More Than Scope Slogans

A growing cache, global array, or retained callback can keep records reachable indefinitely. Ending a local scope does not impose a retention limit. Inspect heap retainers when memory grows, bound collections when the domain permits, and remove references when their owner is finished. Do not add forced-collection assumptions to application logic.

Type Checks Are Not Authorization

A string-valued `role: 'admin'` is still client-supplied data. Passing a type guard does not grant the role. Construct an allowed output schema and obtain privileged fields from an authoritative source. The production normalizer intentionally returns only `id` and `email`; it does not copy the input's arbitrary properties into trusted state.

The chapter's boundary functions accept ordinary parsed data. Reading arbitrary JavaScript properties can invoke accessors or proxy traps. If an API accepts objects from third-party code rather than parsed JSON, document that additional trust boundary instead of claiming the guard is effect-free for every possible object.

Avoid Accidental Shared Secrets

Returning an internal mutable record can expose or alter sensitive state even when the binding that holds it is const. Return only the fields the caller needs, with the ownership promised by the schema. Avoid logging full records just to identify a type mismatch; record the field name and a safe description of the unexpected type.

Professional Field Guide

Case Study: A Snapshot Changes After Review

An order page stores `const snapshot = order` and later changes `order.status`. The earlier snapshot now displays the new status. The bug is an ownership error: the second binding received the same identity. No historical snapshot was created.

For a view containing only string `id` and `status`, build `{ id: order.id, status: order.status }` from validated fields. Each view then owns its two property slots and immutable primitive values. If the view later includes `items`, decide whether it owns copies of those records or intentionally displays a live collection. A future nested field can invalidate an earlier isolation claim.

Case Study: A Type Guard Accepts Null

A handler checks `typeof payload === 'object'` and then reads `payload.id`. A null payload passes the guard and throws at the property access. Repair the boundary: reject null and arrays for an ordinary-record contract, then validate required fields before calling their methods.

Add null, empty object, array, wrong field types, blank normalized strings, and a valid record to the regression suite. Also assert that the output is a new record with only allowed fields. The tests now describe the contract that the handler actually depends on.

Choosing the Binding and Ownership Policy

Need	Choice	Review question
Stable reference to current mutable state	`const` binding with deliberate property mutation	Who else observes this identity?
Replace the selected value	`let` binding	Which events permit replacement?
Preserve a narrow historical view	New record containing owned schema values	Are any nested objects still shared?
Convert boundary input	New named normalized value	Which representations are accepted?

Need	Choice	Review question
Retain records between calls	Explicitly owned collection or closure	What bounds retention and releases it?

Explain an Incident With Four Questions

First locate the binding resolved by the identifier. Then determine whether it had initialized before the read. Next identify the value's actual type. Finally identify any aliases to its object identity. This sequence distinguishes a TDZ failure from a missing property, and a parameter reassignment from a shared-object mutation.

For an unexpected memory increase, extend the final question to retaining paths. A name leaving scope does not prove that its previous value became unreachable.

Readiness Check

Predict a `typeof` TDZ read; explain why const permits property mutation; distinguish null from an ordinary record; show why reassigning a parameter does not replace the caller's binding; and design a normalizer with an explicit output schema. Use the exercises to verify these explanations, then continue to Operators and Expressions.

CHAPTER 3

Operators and Expressions

An expression computes a value. It may also call a function, read a getter, or change state. Operators determine how those actions combine. A short expression such as `options.retries ?? 3` therefore encodes a policy: use the supplied value unless it is `null` or `undefined`.

This chapter develops a precise reading method: group the expression, evaluate the required operands, apply the operator's conversion rules, and account for effects. It distinguishes arithmetic from concatenation, precedence from evaluation order, and absence from falsiness.

Learning Objectives

- Predict arithmetic, comparison, assignment, and bitwise results.
- Trace operand evaluation separately from operator grouping.
- Explain which operand `&&`, `||`, and `??` return and when they skip work.
- Place optional chaining where absence is actually allowed.
- Recognize assignment effects and prefix/postfix update results.
- Build defaults and feature gates with explicit validation contracts.

Prerequisites and Running Guide

Read Variables and Data Types. Recognize primitive values, object references, property access, and `let` versus `const`. Examples use small functions and arrays: a function groups reusable operations, `return` supplies its result, and an array is an ordered collection. Later foundation chapters develop those constructs systematically.

Each JavaScript block is an independent program with exact output. Run a copied block with Node.js 20 or newer; assertion examples use its built-in `node:assert/strict` module.

```
const supplied = { retries: 0, enabled: false };
console.log(supplied.retries ?? 3);
console.log(supplied.retries || 3);
console.log(supplied.enabled ?? true);

// Expected output:
// 0
// 3
// false
```

The first expression preserves zero; the second treats it as falsy. Neither validates the domain: `-1 ?? 3` still produces `-1`. Defaults and validation solve different problems.

Study theory, follow the execution traces, then compare the production contracts. Complete the six exercises before using the MCQs.

Companion programs live in `code/volume-1/chapter-03/`:

```
node code/volume-1/chapter-03/example-00-operator-model.js
node code/volume-1/chapter-03/example-01-nullish-config.js
node code/volume-1/chapter-03/example-02-feature-gate.js
node code/volume-1/chapter-03/example-03-operator-challenges.js
node code/volume-1/chapter-03/example-04-edge-cases.js
```

When an expression controls access or resource use, a reviewer should be able to state which values pass and which operations run.

Theory

Values, References, and Effects

An expression can produce a value, such as `price * count`, while also causing effects. A property read may invoke a getter; a function call may modify state; assignment writes to a binding or property. A reference identifies a destination such as `cart.count`. The value read from that destination is not itself a writable destination.

This distinction explains why `count++` changes a variable but its result cannot itself be incremented again. It also explains why compound assignment must remember the destination it evaluated.

Arithmetic and Numeric Domains

Operator	Meaning and boundary
`+`	Addition or string concatenation after primitive conversion.
`-`, `*`, `/`	Numeric operations; strings may undergo numeric conversion.
`%`	Remainder, whose nonzero sign follows the dividend.
`**`	Exponentiation, grouped from the right.
Unary `+`	Number conversion; rejects BigInt.
Unary `-`	Numeric negation for Number or BigInt.

Number uses floating-point arithmetic. Division by zero can produce infinity, and invalid numeric results can produce `NaN`. BigInt arithmetic uses integers: division truncates toward zero and division by zero throws. Arithmetic generally cannot mix a Number with a BigInt. Compare these domains before trying to fix a result by adding parentheses.

```
console.log(12 + '3');
console.log(12 - '3');
console.log(-8 % 3);
console.log(7 / 2);
console.log(String(7n / 2n));
try { console.log(7n + 2); } catch (error) { console.log(error.name); }

// Expected output:
// 123
// 9
// -2
// 3.5
// 3
// TypeError
```

The next chapter traces the conversion rules. For now, an API expecting numbers should validate numbers at its boundary; subtraction's permissive conversion does not make a numeric-looking string a trustworthy quantity.

Precedence Groups; Evaluation Executes

Precedence decides the expression tree. Multiplication binds more tightly than addition, so `a + b * c` groups as `a + (b * c)`. Associativity resolves grouping at the same level: subtraction groups left, exponentiation groups right. Neither rule means that the engine may arbitrarily reorder calls that produce the operands. MDN: precedence and evaluation.

```
const calls = [];
function read(name, value) { calls.push(name); return value; }
console.log(read('left', 3) + read('middle', 4) * read('right', 5));
console.log(calls.join(','));
console.log(20 - 6 - 2);
console.log(2 ** 3 ** 2);

// Expected output:
// 23
// left,middle,right
// 12
// 512
```

The calls happen left, middle, right. The resulting multiplication is applied before the addition. Parentheses around the multiplication make that grouping visible without changing the call order.

A useful partial precedence ladder, from tighter to looser, is: property access/calls; updates; unary operations; exponentiation; multiplicative; additive; shifts; relational; equality; bitwise AND/XOR/OR; logical AND; logical OR/nullish coalescing; conditional; assignment; comma. This is a reading aid, not

permission to mix every group. Unary negation directly to the left of exponentiation requires parentheses, and unparenthesized mixtures of `??` with `&&` or `||` are syntax errors.

Logical Operators Return Operands

Expression	When the right operand runs	Result
`left && right`	Left is truthy	Left if falsy; otherwise right.
`left		right`		
`left ?? right`	Left is null or undefined	Left if present; otherwise right.

These operators select values. They do not automatically return booleans. `!` does return a boolean, so `!!value` and `Boolean(value)` express truthiness conversion.

```
let fallbackCalls = 0;
function fallback() { fallbackCalls += 1; return 'fallback'; }
console.log(0 && fallback());
console.log('ready' || fallback());
console.log(false ?? fallback());
console.log(null ?? fallback());
console.log(fallbackCalls);

// Expected output:
// 0
// ready
// false
// fallback
// 1
```

Use `??` when zero, false, or an empty string is meaningful. Use `||` when every falsy value should select a fallback. Neither is universal. An empty display label may intentionally fall back; an explicit zero retry count usually must survive. See nullish coalescing.

Optional Chaining Protects a Specified Boundary

`object?.field` returns undefined when `object` is nullish. `object?.[key]` also skips evaluation of the key in that case. `method?.()` skips a call only when the method is nullish; an existing nonfunction still throws.

```
let keyReads = 0;
const missing = null;
console.log(missing?.[keyReads++]);
console.log(keyReads);
console.log(missing?.profile.name);
try { console.log((missing?.profile).name); } catch (error) { console.log(error.name); }
try { ({ run: 3 }).run?.(); } catch (error) { console.log(error.name); }

// Expected output:
```

```
// undefined
// 0
// undefined
// TypeError
// TypeError
```

The continuous chain short-circuits when its guarded base is nullish. Parentheses end that chain before the following ordinary access. Also, if an existing object lacks `profile`, `object?.profile.name` still throws; write `object?.profile?.name` only if both absences are allowed. An undeclared identifier is not rescued by optional chaining. MDN: optional chaining.

Do not hide a required dependency behind `?.`. If a payment handler must exist, validate and fail explicitly rather than silently skipping it.

Assignment and Updates

Assignment returns the assigned value. `a = b = 4` groups as `a = (b = 4)`. A compound assignment such as `target[key()] += 2` evaluates its destination once, reads the old value, computes, then writes. Expanding it carelessly into two property expressions may call `key()` twice.

Logical assignments `&&=`, `||=`, and `??=` additionally skip the write when their condition does not select the right side. This matters for setters and even for `const`: a skipped assignment does not attempt to change the binding.

```
let count = 5;
console.log(count++);
console.log(++count);
const fixed = 0;
console.log(fixed ??= 9);
let reads = 0;
const limits = { retry: 0 };
function key() { reads += 1; return 'retry'; }
limits[key()] += 2;
console.log(limits.retry, reads);

// Expected output:
// 5
// 7
// 0
// 2 1
```

Prefer standalone updates in business logic. Combining several increments, assignments, and calls in one expression increases the number of intermediate states a reviewer must reconstruct.

Comparisons and Conditional Expressions

Strict equality `===` does not coerce differing operand types; object equality compares identity. Loose equality `==` follows a conversion algorithm. Relational operators may compare strings lexicographically or convert operands numerically. A chained comparison is not a mathematical range test: `0 < count < 10`

compares a boolean to 10 after the first comparison.

```
const count = 50;
console.log(0 < count < 10);
console.log(0 < count && count < 10);
console.log('12' < '3');
console.log('12' < 3);
console.log(count < 10 ? 'small' : 'large');

// Expected output:
// true
// false
// true
// false
// large
```

The conditional operator evaluates its condition and exactly one branch. Use it for a small value choice. Several dependent actions, exceptions, or state updates deserve statements with named intermediate results.

Bitwise Operations Have a Narrow Numeric Model

For Number operands, bitwise operations work on 32-bit integer representations. They discard fractional parts and wrap large values into that range. Unsigned right shift >>> produces an unsigned 32-bit Number; signed right shift >> preserves a sign bit. Shift counts use their low five bits for Number shifts.

```
const READ = 1;
const WRITE = 2;
const EXECUTE = 4;
const permissions = READ | WRITE;
console.log((permissions & READ) !== 0);
console.log((permissions & EXECUTE) !== 0);
console.log(permissions & ~WRITE);
console.log(-1 >>> 0);
console.log(2 ** 32 | 0);
console.log(1 << 32);

// Expected output:
// true
// false
// 1
// 4294967295
// 0
// 1
```

Bit masks suit a deliberately bounded flag schema. They are a poor replacement for safe-integer validation or general truncation. BigInt supports its own bitwise operations without Number's 32-bit truncation, but it has no unsigned right shift. The ECMAScript expression algorithms define these operations separately.

Internal Working

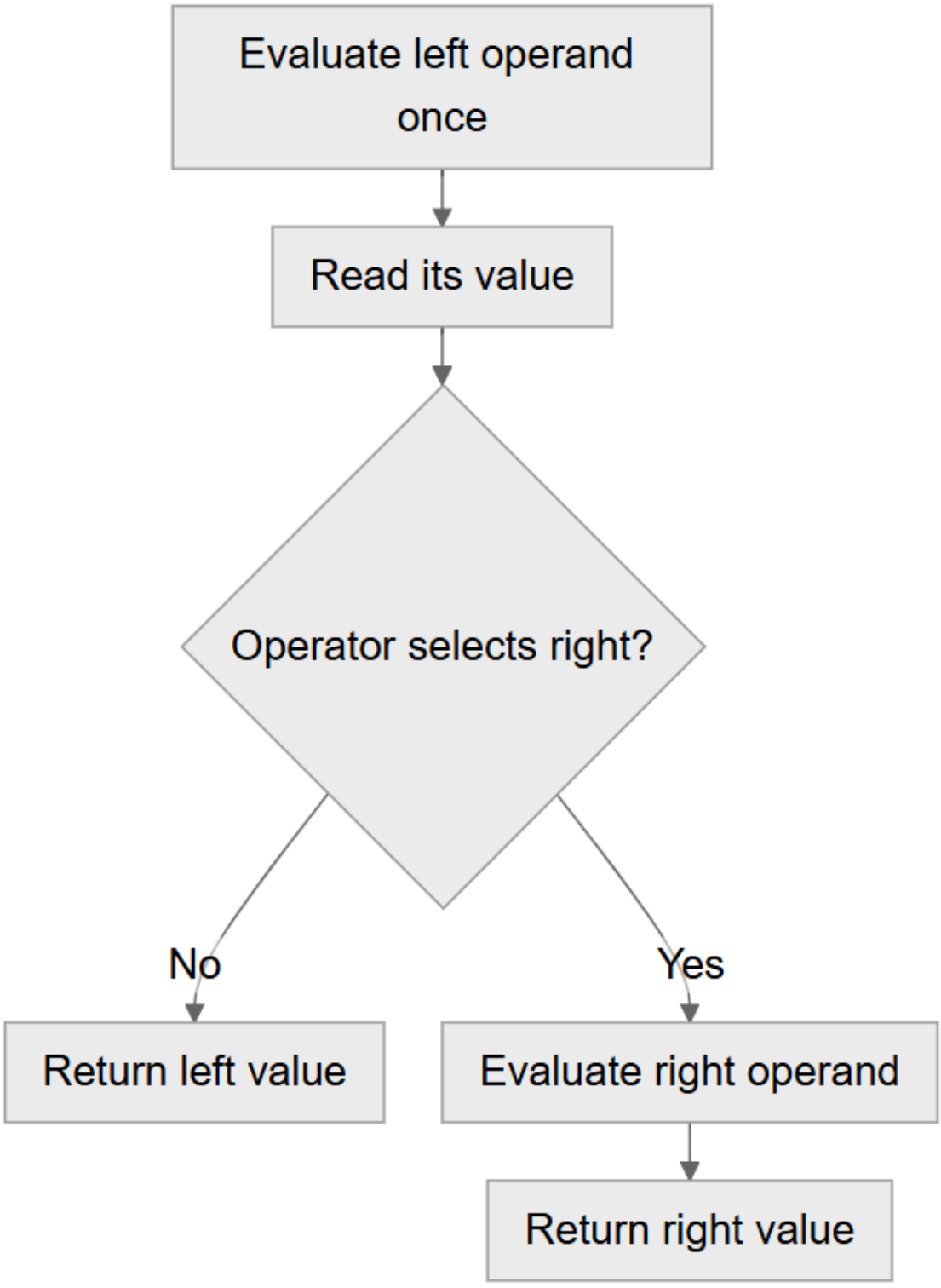
From Syntax to Observable Operations

The parser builds a representation of grouping; it does not run application calls while deciding precedence. Execution then visits the required operands, obtains their values, and applies the relevant operation. An engine may compile or optimize this work, but it must preserve observable calls, reads, writes, and errors.

Consider `left() + middle() * right()`. The observable trace is: call left; call middle; call right; multiply the last two returned values; add the first. If middle throws, right is never called. If the outer operator were a short-circuiting operator, an entire right subtree could be skipped.

Short-Circuit Flow

Stored source: `diagrams/volume-1-chapter-03-short-circuit.mmd`.



The decision depends on the operator: truthiness for AND/OR, nullishness for coalescing. A skipped subtree contributes neither its return value nor its effects. This is why a fallback function can allocate only when needed.

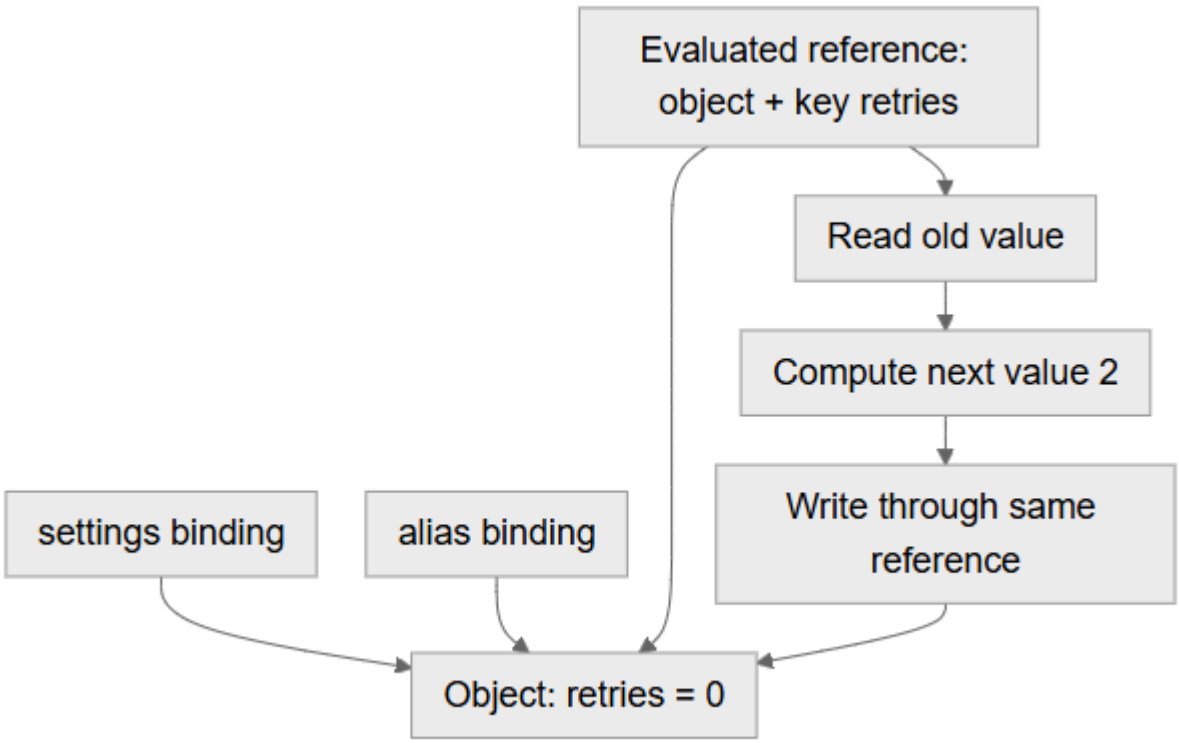
```
const events = [];
function step(label, value) { events.push(label); return value; }
```

```
const result = step('configured', 0) ?? step('default', 3);
console.log(result);
console.log(events.join(','));
events.length = 0;
const access = step('enabled', false) && step('permission', true);
console.log(access);
console.log(events.join(','));

// Expected output:
// 0
// configured
// false
// enabled
```

Assignment References and Memory

Stored source: `diagrams/volume-1-chapter-03-assignment-memory.mmd`.



A property assignment changes the shared object, so every alias observes the new property. It does not rebind each alias. A local variable that previously copied a primitive property value keeps that old primitive.

```
const settings = { retries: 0 };
const alias = settings;
const previous = settings.retries;
settings.retries += 2;
console.log(alias.retries);
```

```
console.log(previous);
console.log(alias === settings);

// Expected output:
// 2
// 0
// true
```

These are semantic references, not a promise about physical stack or heap addresses. Engines may optimize temporary values away while preserving the same behavior.

Getters and Setters Make the Trace Visible

```
const trace = [];
let stored = 0;
const config = {
  get retries() { trace.push('get'); return stored; },
  set retries(value) { trace.push('set ' + value); stored = value; }
};
config.retries ??= 3;
console.log(trace.join(','));
trace.length = 0;
config.retries ||= 3;
console.log(trace.join(','));
console.log(stored);

// Expected output:
// get
// get,set 3
// 3
```

The first operation reads zero, judges it present, and performs no write. The second reads zero, judges it falsy, evaluates three, and invokes the setter. Rewriting `config.retries ??= 3` as `config.retries = config.retries ?? 3` would always invoke the setter after the read. Similar-looking source can have different effects.

Engine and Performance Perspective

Fast numeric paths depend on the actual operand values. A reusable addition site receiving both strings and numbers must still concatenate on one call and add on another. Optimization does not change the language into a statically typed system.

Do not infer a specific optimization tier, machine instruction, or object allocation from a small example. Measure a realistic workload with its input distribution. Expensive work is often inside an operand: a search, getter, allocation, or network adapter. Short-circuiting can avoid that work, but changing operand order is valid only if required effects and errors stay correct.

Execution Checklist

For a difficult expression, write four columns: grouped expression, next operand, value/type, and effect. Mark skipped operands explicitly. For an assignment, additionally record the evaluated destination. For an exception, stop the trace at the throwing operation. This method works in browsers and Node because these operator rules belong to ECMAScript; host-specific functions called by the operands may differ.

Consult the specification's expression evaluation algorithms when an observable ordering question remains unclear. The edge cases apply this trace method to common failures.

Production Examples

Retry Defaults With Validation

A worker accepts a retry count, a timeout in milliseconds, and an enabled flag. Zero retries and `false` are valid explicit choices. Missing or null fields receive defaults. Other supplied values must satisfy the schema; a string `"0"` is not silently converted into a count.

The function accepts an ordinary application data record, such as parsed JSON. It reads only the documented fields, tolerates unrelated fields, returns a new frozen record, and does not mutate its input. It rejects a null root; passing no argument selects an empty record. This distinction is intentional: absent configuration and a malformed root are separate cases.

```
'use strict';
const assert = require('node:assert/strict');

function resolveRetryPolicy(config = {}) {
  if (config === null || typeof config !== 'object' || Array.isArray(config)) {
    throw new TypeError('config must be a data record');
  }
  const retries = config.retries ?? 3;
  const timeoutMs = config.timeoutMs ?? 1500;
  const enabled = config.enabled ?? true;
  if (!Number.isSafeInteger(retries) || retries < 0 || retries > 10) {
    throw new RangeError('retries must be an integer from 0 through 10');
  }
  if (!Number.isSafeInteger(timeoutMs) || timeoutMs < 1 || timeoutMs > 60000) {
    throw new RangeError('timeoutMs must be an integer from 1 through 60000');
  }
  if (typeof enabled !== 'boolean') throw new TypeError('enabled must be boolean');
  return Object.freeze({ retries, timeoutMs, enabled });
}

const supplied = { retries: 0, enabled: false };
const result = resolveRetryPolicy(supplied);
assert.deepEqual(result, { retries: 0, timeoutMs: 1500, enabled: false });
assert.equal(Object.isFrozen(result), true);
assert.notEqual(result, supplied);
assert.deepEqual(supplied, { retries: 0, enabled: false });
assert.throws(() => resolveRetryPolicy({ retries: -1 }), RangeError);
```

```
assert.throws(() => resolveRetryPolicy({ enabled: 'false' }), TypeError);
console.log(JSON.stringify(result));
console.log(JSON.stringify(resolveRetryPolicy({ retries: null })));

// Expected output:
// {"retries":0,"timeoutMs":1500,"enabled":false}
// {"retries":3,"timeoutMs":1500,"enabled":true}

// O(1) time and new storage for the fixed three-field schema.
```

The implementation first resolves three candidate values with `??`, then validates them, then creates the result. Replacing `??` with `||` would overwrite valid zero and false values. Moving validation before default selection would require separate missing-value handling. Returning the input after modifying it would introduce an ownership problem for other readers.

The output has only primitive fields, so freezing that one record protects its entire schema. There is no nested graph to freeze. This local policy does not implement retry scheduling, exponential backoff, request cancellation, or idempotency; those belong to the worker using it.

Companion: `code/volume-1/chapter-03/example-01-nullish-config.js`. Additional assertions cover boundary counts, timeout limits, invalid root values, and output ownership.

A Feature Gate That Denies Malformed Data

A feature is available when the account is explicitly active, the feature is explicitly enabled, and the user is staff or appears in its allowed-ID list. Missing or null lists mean no listed users. A malformed list denies access, including for staff, because this service treats corrupt feature configuration as disabled.

Input case	Decision
Missing user or feature	Deny.
`active` or `enabled` is any value other than boolean true	Deny.
Missing/empty/oversized user ID or missing role string	Deny.
Missing/null allowlist	Use an empty list; staff may still pass.
Nonarray list, more than 1000 IDs, or invalid list entry	Deny.
Valid staff user and valid enabled feature	Allow.
Valid nonstaff user	Allow only an exact, case-sensitive ID match.

```
'use strict';
const assert = require('node:assert/strict');

function canAccessBeta(user, feature) {
  if (user === null || typeof user !== 'object' || Array.isArray(user) ||
      feature === null || typeof feature !== 'object' || Array.isArray(feature)) return false;
```

```
  if (user.active !== true || feature.enabled !== true) return false;
  if (typeof user.id !== 'string' || user.id.length < 1 || user.id.length > 64) return false;
  if (typeof user.role !== 'string') return false;
  const allowed = feature.allowedUserIds ?? [];
  if (!Array.isArray(allowed) || allowed.length > 1000) return false;
  for (const id of allowed) {
    if (typeof id !== 'string' || id.length < 1 || id.length > 64) return false;
  }
  return user.role === 'staff' || allowed.includes(user.id);
}

const member = { id: 'u1', active: true, role: 'member' };
const enabled = { enabled: true, allowedUserIds: ['u1'] };
assert.equal(canAccessBeta(member, enabled), true);
assert.equal(canAccessBeta(member, { enabled: true }), false);
assert.equal(canAccessBeta(member, { enabled: true, allowedUserIds: 'u1' }), false);
assert.equal(canAccessBeta({ ...member, active: 'false' }, enabled), false);
assert.equal(canAccessBeta(null, enabled), false);
assert.equal(canAccessBeta({ ...member, role: 'staff' }, { enabled: true }), true);
assert.equal(canAccessBeta({ ...member, role: 'staff' }, { enabled: true, allowedUserIds: [null] }),
false);
console.log(canAccessBeta(member, enabled));
console.log(canAccessBeta(member, { enabled: true }));
console.log(canAccessBeta(member, { enabled: 'true', allowedUserIds: ['u1'] }));

// Expected output:
// true
// false
// false

// O(n) time for at most 1000 IDs; O(1) additional space.
// A missing list allocates an empty array of constant size.
```

The guards reject unusable roots before any property access. Exact boolean comparisons avoid enabling a feature from the truthy string `"false"`. The list is checked before `includes` is called, so a missing list no longer throws. Returning a boolean is part of the contract; this expression uses comparisons and `includes`, which already return booleans.

Short-circuiting skips later checks after an early denial. Validating the list before the staff branch deliberately spends O(n) work even for staff, enforcing the corrupt-configuration policy. If an application instead wants staff to bypass list validation, that is a policy change requiring different tests.

The list bound prevents this helper from searching an unbounded configuration array. It does not replace request-size limits before parsing. IDs are opaque strings with bounded length; this helper does not normalize case or trim them because that could merge identities.

Trust and Ownership Boundaries

These examples accept parsed, application-owned data, not arbitrary objects with getters or proxy traps. Reading a JavaScript property can run code. Validation of data shapes is not a sandbox for hostile JavaScript objects.

A feature rollout gate is also not a complete authorization system. The caller must obtain user identity and staff status from a trusted server-side source. A browser-supplied `role: "staff"` is not evidence of privilege. Keep authentication and authorization policy at the service boundary.

Both helpers leave input objects intact. The gate does not retain or freeze the caller's list; it makes a decision from the data observed during the synchronous call. If configuration changes later, subsequent calls may decide differently. Cache results only with a defined invalidation policy.

Companion: `code/volume-1/chapter-03/example-02-feature-gate.js`. It covers missing data, malformed list entries, strict booleans, independent users, staff behavior, and both list-size boundaries.

Review Questions

Can zero survive defaulting? Can a malformed permission list throw? Which branch runs when the feature is disabled? Is an untrusted field being mistaken for authority? State these answers before discussing whether an expression is compact.

The exercise set applies the same operator rules to different contracts rather than repeating these functions.

Interview Perspective

Explain a Rule, Then Trace It

A strong answer names the operator, states its selection or conversion rule, and accounts for observable effects. Avoid answering only with a memorized output. The interviewer may change an operand from zero to null or insert a getter.

Does Precedence Mean Multiplication's Operands Run First?

No. Precedence groups the expression. In `readA() + readB() * readC()`, the calls occur A, B, C if none throws; multiplication combines B/C before addition combines that result with A. Mention skipped operands separately for logical operators.

Why Does a Logical Expression Return Zero?

`&&` returns the first falsy operand it encounters, or the final operand if all preceding operands are truthy. It does not convert that returned value into false. A feature API promising booleans should either use boolean-producing comparisons or explicitly normalize its final result.

```
console.log('user' && 0);
console.log(Boolean('user' && 0));
console.log(false ?? 'fallback');
console.log(false || 'fallback');
```

```
// Expected output:
// 0
// false
// false
// fallback
```

When Is Nullish Coalescing the Wrong Default?

When the domain intentionally treats an empty string as missing. Conversely, `||` is wrong when zero or false is meaningful. Neither validates a negative count, an oversized timeout, or a string boolean. State the accepted input policy before selecting an operator.

Does Optional Chaining Make an Entire Expression Safe?

It protects only the documented nullish boundary along a continuous chain. It does not catch exceptions from getters or callbacks, make nonfunctions callable, or resolve undeclared variables. Parentheses may end the chain before a later ordinary property access.

Can Logical Assignment Invoke a Setter Less Often?

Yes. `object.count ??= 1` skips the write when count is present. `object.count = object.count ?? 1` writes even then. A getter runs for both forms; a setter can distinguish them. This is an observable semantic difference, not merely an optimization.

Is a Chained Comparison a Range Check?

No. `0 < count < 10` first produces a boolean, then compares that boolean with 10 after conversion. Use two comparisons joined with `&&`. Ask whether the bounds are inclusive, then choose `<=` or `<` deliberately.

Why Is a Bitwise Shortcut Dangerous for Counts?

Number bitwise operators use a 32-bit representation. A valid larger integer may wrap and a fraction may be truncated. Use explicit numeric validation. Bit masks remain appropriate for a documented small flag domain.

Worked Senior Prompt

A configuration helper uses `retries: config.retries || 3` and a feature check uses `feature.allowedUserIds.includes(user.id)`. Product requirements say zero retries is valid and a missing allowlist denies ordinary members.

A complete answer has four parts:

1. Replace the default with `??`, then validate the selected count.
2. Define the list's absence and malformed-data policies before accessing it.
3. Require actual boolean flags rather than trusting strings such as "false".
4. Test zero, false, missing values, malformed lists, and unchanged input ownership.

The production examples implement that contract. Optional chaining alone would still leave unclear whether an absent list means deny, error, or unrestricted access.

Whiteboard Trace

Trace `cache.value ??= build()` under three cases: value is zero; value is undefined; build throws. Draw one property reference, the read, the branch, and the possible write. Expected answers: return zero without build/write; build and store; or propagate the error without the assignment occurring. If build mutates other state before throwing, the operator does not roll that back.

The follow-up is whether this is a correct asynchronous cache. It is not sufficient by itself: promise rejection, concurrent requests, and retry policy need explicit design.

Exercises and Coding Challenges

Solve each contract before reading its solution. All six programs run independently; assertions cover the boundary that motivates the operator choice. The companion `code/volume-1/chapter-03/example-03-operator-challenges.js` runs the same six solutions.

1. Preserve an Explicit Empty Label

Problem: Return the supplied label when it is a string, including an empty string. Only null or undefined select 'Untitled'. Reject other types. Do not trim the label.

Hint: Default by nullishness, then validate the selected type.

Solution

```
const assert = require('node:assert/strict');
function displayLabel(value) {
  const label = value ?? 'Untitled';
  if (typeof label !== 'string') throw new TypeError('label must be a string');
  return label;
}
assert.equal(displayLabel(''), '');
assert.equal(displayLabel(null), 'Untitled');
assert.equal(displayLabel(undefined), 'Untitled');
assert.throws(() => displayLabel(0), TypeError);
console.log(JSON.stringify([displayLabel(''), displayLabel('Report'), displayLabel(null)]));

// Expected output:
// ["","Report","Untitled"]

// O(1) time and additional storage; existing strings are returned unchanged.
```

Using || would replace the valid empty label. Converting every input with String would instead accept values the contract rejects.

2. Cache a Lazy Default With One Read

Problem: Read a record's named value once. If it is nullish, call compute exactly once, store its result, and return it. Preserve false, zero, and empty strings. A thrown computation must leave the property unwritten.

Hint: Use logical nullish assignment and observe getter, computation, and setter order.

Solution

```
const assert = require('node:assert/strict');
function getOrCompute(record, key, compute) {
  return record[key] ??= compute();
}
let reads = 0;
let computes = 0;
let writes = 0;
let stored;
const record = {
  get value() { reads += 1; return stored; },
  set value(next) { writes += 1; stored = next; }
};
const compute = () => { computes += 1; return 0; };
assert.equal(getOrCompute(record, 'value', compute), 0);
assert.equal(getOrCompute(record, 'value', compute), 0);
assert.deepEqual([reads, computes, writes], [2, 1, 1]);
const fresh = {};
assert.throws(() => getOrCompute(fresh, 'value', () => { throw new Error('failed'); }), /failed/);
assert.equal(Object.hasOwn(fresh, 'value'), false);
console.log(reads, computes, writes);

// Expected output:
// 2 1 1

// O(1) operator bookkeeping, excluding accessor and compute work.
```

The record and callback are trusted collaborators. If compute returns undefined, the next call computes again; nullish values are deliberately not cache hits. This is not a concurrent or asynchronous cache.

3. Check All Required Permission Bits

Problem: Use three flags with values 1, 2, and 4. Accept only integer masks from 0 through 7. Return whether every required bit is present. An empty requirement succeeds.

Hint: A bitwise intersection must equal the required mask; mere nonzero overlap means something else.

Solution

```
const assert = require('node:assert/strict');
function hasAllFlags(available, required) {
  for (const mask of [available, required]) {
    if (!Number.isInteger(mask) || mask < 0 || mask > 7) throw new RangeError('invalid mask');
  }
  return (available & required) === required;
}
assert.equal(hasAllFlags(3, 3), true);
assert.equal(hasAllFlags(1, 3), false);
assert.equal(hasAllFlags(0, 0), true);
assert.throws(() => hasAllFlags(2 ** 32 + 1, 1), RangeError);
console.log(hasAllFlags(7, 5), hasAllFlags(2, 5));

// Expected output:
// true false

// O(1) time and space for the fixed three-bit schema.
```

Validate before the bitwise operation. Otherwise large numbers can wrap into an apparently valid 32-bit mask. A set of permission names is clearer when the schema is not intentionally bit-based.

4. Allocate a Ticket Without Advancing on Failure

Problem: A mutable counter contains next, a nonnegative safe integer. Return the current ticket and advance next by one. Reject invalid or exhausted counters before mutation.

Hint: Postfix increment returns the old numeric value; range validation makes that operation safe.

Solution

```
const assert = require('node:assert/strict');
function allocateTicket(counter) {
  if (!Number.isSafeInteger(counter.next) || counter.next < 0 ||
      counter.next === Number.MAX_SAFE_INTEGER) throw new RangeError('counter exhausted or invalid');
  return counter.next++;
}
const counter = { next: 4 };
assert.equal(allocateTicket(counter), 4);
assert.equal(counter.next, 5);
const full = { next: Number.MAX_SAFE_INTEGER };
assert.throws(() => allocateTicket(full), RangeError);
assert.equal(full.next, Number.MAX_SAFE_INTEGER);
console.log(allocateTicket(counter), counter.next);

// Expected output:
// 5 6

// O(1) time and space; one documented mutation of a trusted data record.
```

Returning ++counter.next would skip the current ticket. This local counter does not supply durable or cross-process uniqueness. An accessor-bearing object is outside the data-record contract.

5. Make Discount Grouping Explicit

Problem: Compute an integer-cent subtotal from unitCents (0–1,000,000), quantity (0–1,000), and discountPercent (0–100), all integers. Apply the discount to the full subtotal and round once to the nearest cent.

Hint: Separate subtotal from its percentage multiplier; validate numbers before multiplication.

Solution

```
const assert = require('node:assert/strict');
function discountedTotal(unitCents, quantity, discountPercent) {
  for (const [value, max] of [[unitCents, 1000000], [quantity, 1000], [discountPercent, 100]]) {
    if (!Number.isInteger(value) || value < 0 || value > max) throw new RangeError('invalid input');
  }
  const subtotal = unitCents * quantity;
  return Math.round(subtotal * (100 - discountPercent) / 100);
}
assert.equal(discountedTotal(199, 2, 10), 358);
assert.equal(discountedTotal(199, 2, 100), 0);
assert.equal(discountedTotal(199, 0, 10), 0);
assert.throws(() => discountedTotal('199', 2, 10), RangeError);
console.log(discountedTotal(199, 2, 10));

// Expected output:
// 358

// O(1) time and space within the stated bounded arithmetic domain.
```

The accepted bounds keep intermediate integer products exactly representable. The rounding policy is part of this example's contract; real billing rules may require different tax/discount ordering or decimal arithmetic.

6. Call an Optional Transformer and Preserve Its Receiver

Problem: A plugin may be nullish, or its transform method may be nullish. Return the original payload in those cases and when the method returns null/undefined. Preserve other results, including zero and false. Call a present method with the plugin as its receiver; a nonfunction property should throw.

Hint: Keep optional invocation on the property reference, then coalesce its result.

Solution

```
const assert = require('node:assert/strict');
function transformOrOriginal(plugin, payload) {
  return plugin?.transform?.(payload) ?? payload;
```

```
}
const plugin = { prefix: 'ok:', transform(value) { return this.prefix + value; } };
assert.equal(transformOrOriginal(plugin, 'item'), 'ok:item');
assert.equal(transformOrOriginal(null, 'item'), 'item');
assert.equal(transformOrOriginal({}, 'item'), 'item');
assert.equal(transformOrOriginal({ transform: () => 0 }, 'item'), 0);
assert.equal(transformOrOriginal({ transform: () => false }, 'item'), false);
assert.throws(() => transformOrOriginal({ transform: 3 }, 'item'), TypeError);
console.log(transformOrOriginal(plugin, 'item'));

// Expected output:
// ok:item

// O(1) dispatch overhead, excluding the transformer's work and returned data.
```

Extracting transform into a standalone variable would lose the property-call receiver. The fallback policy intentionally treats nullish method results as absence; use an explicit presence test if undefined must be a meaningful result.

MCQs

Choose an answer before reading the explanation. Assume modern JavaScript and ordinary data values unless the question states otherwise.

1. What does `0 ?? 8` produce?

A. 8 B. 0 C. false D. undefined

Answer: B. Zero is present. Only null and undefined choose the coalescing fallback.

2. What is the result of `'ready' && 0`?

A. true B. false C. 'ready' D. 0

Answer: D. The truthy left operand selects the right operand, which is returned unchanged.

3. In `a() + b() * c()`, which calls execute first when none throws?

A. a, then b, then c B. b, then c, then a C. c, then b, then a D. The engine may freely reorder observable calls

Answer: A. Multiplication controls grouping, not the order of those operand calls.

4. How does `2 ** 3 ** 2` group?

A. (2 ** 3) ** 2 B. 2 ** (3 * 2) C. 2 ** (3 ** 2) D. It is a syntax error

Answer: C. Exponentiation is right-associative, so the result is 512.

5. Given `let n = 4`, what value does `n++` return?

A. 5 B. 4 C. undefined D. A reference to n

Answer: B. Postfix update returns the old numeric value while storing the incremented value.

6. With `const x = 0`, what happens at `x ??= 9`?

A. x becomes 9 B. SyntaxError C. TypeError D. It returns 0 without writing

Answer: D. The nullish condition is false, so no assignment to the const binding is attempted.

7. For `const a = null`, what happens at `(a?.b).c`?

A. TypeError B. undefined C. null D. ReferenceError

Answer: A. Grouping ends the optional chain; ordinary access on undefined throws.

8. What happens at `({ run: 3 }).run?.()`?

A. Returns 3 B. Returns undefined C. TypeError D. Skips run because it is not callable

Answer: C. Optional invocation skips nullish values, not arbitrary nonfunctions.

9. What does `-8 % 3` produce?

A. 1 B. -2 C. 2 D. -1

Answer: B. Remainder is not a nonnegative modulo operation; its nonzero sign follows the dividend.

10. What is `1 << 32` for Number operands?

A. 0 B. 4294967296 C. RangeError D. 1

Answer: D. The shift count is reduced to its low five bits, so 32 acts like zero.

11. Which expression checks that all required bits are available?

A. (available & required) === required B. (available & required) !== 0 C. available === required D. available | required

Answer: A. Nonzero intersection tests any overlap. Equality with the required mask tests every required bit.

12. Why can `target[key()] += 1` differ from a naive expanded assignment?

A. Compound assignment never calls getters B. It always returns a boolean C. The compound form evaluates the destination once D. The expanded form cannot write properties

Answer: C. Repeating target/key expressions may repeat effects or select a different destination.

13. Which condition correctly checks an exclusive range?

A. `0 < n < 10` B. `0 < n && n < 10` C. `n > (0 && n) < 10` D. `0 < (n < 10)`

Answer: B. Each comparison must inspect n. A chained comparison instead compares an intermediate boolean.

14. What does `false || 'fallback'` return?

A. false B. true C. undefined D. 'fallback'

Answer: D. OR selects the right operand when the left is falsy; it does not normalize to a boolean.

15. Which is valid JavaScript?

A. (a ?? b) || c B. a ?? b || c C. a && b ?? c D. a || b ?? c

Answer: A. Mixing coalescing with AND/OR requires explicit grouping.

16. Why avoid `value | 0` as a general integer validator?

A. It rejects every numeric string B. It preserves all safe integers C. It can truncate and wrap values into signed 32-bit range D. It always throws for fractions

Answer: C. A conversion that discards information is not validation; validate the input domain first.

Revision Sheet and Chapter Summary

Rules to Recall

Topic	Rule
Precedence	Determines grouping; it does not grant permission to reorder observable operand calls.
Associativity	Determines grouping among operators at the same level; exponentiation and assignment group right.
AND	Return left when falsy; otherwise evaluate and return right.
OR	Return left when truthy; otherwise evaluate and return right.
Coalescing	Evaluate right only for null or undefined.
Optional chaining	Skip the guarded access/call when its base is nullish; other errors still propagate.
Assignment	Evaluate a destination and write a value; the expression returns the assigned value.
Logical assignment	Also skip the write when its condition does not select the right side.
Postfix update	Store the increment/decrement but return the old numeric value.
Conditional	Evaluate one selected branch.
Number bitwise	Operate in a 32-bit integer model, not the full safe-integer domain.
Remainder	A nonzero result follows the dividend's sign.

Prediction Check

```
console.log(0 || 4, 0 ?? 4);
console.log('ok' && '', false ?? 7);
console.log(2 ** 3 ** 2);
console.log((1 | 2) & 2);
console.log(5n / 2n === 2n);

// Expected output:
```

```
// 4 0
//  false
// 512
// 2
// true
```

The leading space on the second output line separates an empty string from false; logging JSON is often clearer when blank values matter.

Review Before Shipping

- Define which values count as missing and which are invalid.
- Validate types and bounds after choosing defaults.
- Trace side effects in calls, computed keys, getters, setters, and updates.
- Use two comparisons for a range.
- Preserve exact boolean requirements at access boundaries.
- Select bitwise operations only for an intentionally bounded representation.
- Test that skipped work really was skipped.

Chapter Summary

Operators combine values and effects. Parentheses clarify grouping; explicit contracts clarify meaning. A safe default preserves intended values, a safe feature gate defines malformed-data behavior, and a readable expression exposes which work can be skipped. Memory reasoning tracks shared objects separately from copied primitive values.

Continue with Type Conversion and Coercion to explain how an operator chooses string concatenation, numeric conversion, truthiness, or comparison.

References and Further Reading

- Operator precedence: grouping, associativity, and evaluation.
- Optional chaining: chain boundaries and invalid uses.
- Nullish coalescing: fallback selection.
- ECMAScript expressions: normative operator algorithms.

After the next chapter, revisit the traces and explain each conversion rather than only the final output.

Edge Cases and Debugging

Start With the Skipped Operation

When a fallback unexpectedly runs, inspect the left value and its type. Zero retries, a false flag, and an empty label are all falsy but may be valid settings. Defaulting is a domain decision, not a repair for every suspicious value.

```
for (const value of [0, false, '', null, undefined]) {
  console.log(JSON.stringify([typeof value, value || 'default', value ?? 'default']));
}

// Expected output:
// ["number","default",0]
// ["boolean","default",false]
// ["string","default",""]
// ["object","default","default"]
// ["undefined","default","default"]
```

JSON makes an empty string visible here. It does not preserve undefined values in every position, so include typeof when that distinction matters.

Optional Chains Do Not Catch Exceptions

```
const broken = { get profile() { throw new Error('profile unavailable'); } };
try { console.log(broken?.profile?.name); }
catch (error) { console.log(error.message); }
const partial = {};
try { console.log(partial?.profile.name); }
catch (error) { console.log(error.name); }
console.log(partial?.profile?.name);

// Expected output:
// profile unavailable
// TypeError
// undefined
```

In the first case the root exists, so the getter runs and throws. In the second only the root is guarded; the missing profile reaches an ordinary property access. The third explicitly permits both absences. Decide whether missing profile data is actually acceptable before adding another optional boundary.

Compound Assignment Is Not Text Substitution

```
const trace = [];
let stored = 5;
const record = {
  get count() { trace.push('read'); return stored; },
```

```
  set count(value) { trace.push('write'); stored = value; }
};
record.count ??= 9;
console.log(trace.join(','));
trace.length = 0;
record.count = record.count ?? 9;
console.log(trace.join(','));

// Expected output:
// read
// read,write
```

The second form writes even when no fallback was needed. A setter may persist data, invalidate a cache, or reject a change. Likewise, expanding `record[key()] += 1` by repeating `key()` may choose a different property or repeat work.

Range Tests and Numeric Shortcuts

```
const count = 25;
console.log(0 < count < 10);
console.log(count > 0 && count < 10);
console.log(2147483648 | 0);
console.log(Math.trunc(2147483648.75));
console.log(-5 % 3);

// Expected output:
// true
// false
// -2147483648
// 2147483648
// -2
```

The chained comparison becomes `true < 10`. The bitwise expression wraps to signed 32-bit range; Math.trunc discards a fraction without that wrapping. Neither validates an application's maximum count. Remainder may be negative; a nonnegative-modulo formula is a separate domain choice.

Syntax Errors Need a Parsing Diagnosis

An unparenthesized mixture such as `a ?? b || c` is invalid syntax. Choose `(a ?? b) || c` or `a ?? (b || c)` according to the intended policy; they can differ when a is falsy but present. Likewise, write `-(2 ** 2)` or `(-2) ** 2` rather than an unparenthesized unary minus to the left of exponentiation. A same-file try statement cannot execute when that file fails to parse.

A Reproducible Debugging Procedure

1. Preserve the original inputs; inspecting a later mutated object can mislead.
2. Parenthesize grouping without changing operand order.
3. Annotate each operand's value and type.

4. Record getter, function, and setter calls in a local trace array.
5. Stop at the first throw and mark skipped operands.
6. Assert the smallest failing case and its neighboring valid case.

For the feature gate, test an enabled member with a missing list and with one matching ID. Both must return booleans and neither may throw. The companion is `code/volume-1/chapter-03/example-04-edge-cases.js`.

The optional-chaining reference documents continuous-chain boundaries; the precedence reference separates grouping from evaluation.

Performance and Security

Count the Work Inside the Operands

A logical operator performs a small selection, but its operands may search collections, allocate strings, or invoke application code. `enabled && allowedIds.includes(id)` avoids the search when enabled is falsy. When enabled is true, searching n IDs is O(n) in the worst case; comparing each string also depends on ID length.

The production gate bounds both list length and ID length. Its complete validation pass precedes the staff shortcut because malformed configuration must deny everyone. Moving staff first changes that policy. Performance work must preserve decisions as well as happy-path results.

For repeated lookups against one stable configuration, validating once and building a Set may pay off. Construction consumes O(n) storage; rebuilding on every request loses the benefit. Define configuration ownership and invalidation before retaining such an index. ECMAScript requires average sublinear Set access, not one particular hash-table implementation or a universal constant-time guarantee. See Set performance.

Lazy Defaults Avoid Unneeded Work

```
let builds = 0;
function buildDefault() { builds += 1; return { retries: 3 }; }
const supplied = { retries: 0 };
const selected = supplied ?? buildDefault();
console.log(selected === supplied);
console.log(builds);
const fallback = null ?? buildDefault();
console.log(fallback.retries, builds);

// Expected output:
// true
// 0
// 3 1
```

This is useful when a fallback requires work. It does not imply every allocation should become a cache. Reusing a mutable fallback can create shared state; building a fresh record can be the correct ownership choice.

Avoid Unproven Numeric Micro-optimizations

`value | 0` and `~~value` convert to signed 32-bit values. They can wrap, truncate, or turn NaN into zero. Applying them to pagination, money, or identifiers changes accepted values. Choose a validation contract first, then measure the actual workload.

String concatenation may copy or defer copying depending on engine strategy. BigInt arithmetic depends on operand magnitude. Do not label every + operation constant-time simply because the source contains one operator. Bounded Number arithmetic is a useful constant-cost model here; arbitrary strings and integers need size-aware reasoning.

Defaults Can Change Access Decisions

Security-sensitive flags should be booleans from a trusted source. The string "false" is truthy, so a guard such as `if (input.enabled)` can enable an operation the sender appeared to disable. The chapter's gate requires `enabled === true` and `active === true`; malformed shapes deny access.

A missing allowlist needs a named policy. Falling back to true, or placing `|| true` after membership testing, can grant access when data is absent. An empty list denies ordinary members. A trusted staff bypass is a separate explicit policy, not a reason to trust a role submitted by a browser.

These guards operate on ordinary application data. Getters and proxies can run code during reads; shape checking is not a sandbox for arbitrary JavaScript objects. Bound serialized request size before parsing and obtain identity/roles from the authentication layer.

Keep Decisions Reviewable

Split an access expression when a reviewer cannot identify each condition. Name predicates when that clarifies policy, and preserve short-circuiting when a later operation depends on an earlier guard. Do not reorder effectful getters or callbacks based only on apparent cost.

Measure the complete request or batch with representative enabled/disabled ratios and list sizes. Repeated addition of constants says little about real validation, search, and I/O costs.

Professional Field Guide

Choose the Policy Before the Operator

Requirement	Useful expression	Review condition
Default only absent/null values	`value ?? fallback`	Zero, false, and empty strings remain meaningful.
Default every falsy value	`value		
Skip an optional read	`value?.field`	Absence at this exact boundary is valid.
Run an optional method	`value.method?.()`	A present nonfunction is an error.
Require boolean predicates	`a && b && c`	Each predicate produces a boolean, or the caller accepts operand results.
Cache only a nullish property	`record.key ??= build()`	Mutation, lifetime, and nullish results are understood.
Check all permission bits	`(flags & required) === required`	Masks were validated before 32-bit conversion.

A required dependency should fail clearly when missing; an optional dependency can legitimately short-circuit.

Review a Configuration Change

A worker used `config.retries || 3`. Operators report that zero retries still triggers three attempts. Use nullish defaulting and validate the selected count. Test zero, a positive count, null, absence, a numeric string, a negative number, and the upper bound. Verify that the helper does not mutate caller-owned configuration.

The root cause is a mismatch between falsy and missing. Renaming a variable cannot change that behavior. The production implementation makes the distinction executable.

Review an Access Expression

Ask where identity and role originate. Enumerate disabled feature, inactive account, missing list, invalid list, listed member, unlisted member, and trusted staff. Record allow/deny and whether malformed configuration should throw or deny before reviewing the expression.

An expression such as `user && feature.enabled && list.includes(user.id)` can return null or undefined. If callers require a boolean, validate roots first and return explicit booleans on rejected paths. Optional chaining improves syntax only after the domain decision is settled.

Explain an Interview Trace

For `a() + b() * c()`, draw addition above multiplication. Write call order beside the tree: a, b, c. Then combine b/c and add a. For `a() && b()`, draw a branch after a and show when b is absent from the trace. These distinguish grouping from conditional evaluation.

For `target[key()] += amount()`, identify the destination once, read its old value, evaluate the right operand, compute, and write. If computation throws, the final write does not happen, but earlier effects are not rolled back.

Practical Completion Check

Explain why zero survives ??, why optional invocation does not make 3 callable, why ??= can skip a setter, why a chained range comparison is wrong, and why a large count cannot be validated by bitwise conversion. Demonstrate the claims with the exercises.

Continue to Type Conversion and Coercion to explain what happens between obtaining an operand value and applying arithmetic or comparison.

CHAPTER 4

Type Conversion and Coercion

A query parameter arrives as the string "25", while a page-size calculation needs a Number. Conversion creates a value in the required type. Validation decides whether the original input and converted result meet the application's rules. Confusing those jobs allows values such as true, an empty string, or an unsafe integer to become apparently valid inputs.

JavaScript also converts values implicitly. Addition, comparisons, conditions, and property keys each request particular conversions. The useful question is which operation requests which type, not whether coercion is always good or always bad.

Learning Objectives

- Predict String, Number, Boolean, and BigInt conversion results.
- Distinguish full-value conversion from prefix parsing.
- Trace loose equality and relational comparisons from operand types.
- Explain object-to-primitive hints and conversion side effects.
- Preserve identity and original inputs while deriving normalized values.
- Implement bounded pagination and exact boolean parsing with regression assertions.

Prerequisites and Running Guide

Read Variables and Data Types and Operators and Expressions. Recognize primitive types, object identity, property access, calls, and short-circuit evaluation. Small functions group reusable steps; return supplies a result. Later foundation chapters develop functions, arrays, objects, and exceptions systematically.

Run each JavaScript block independently with Node.js 20 or newer. Expected-output comments specify deliberate console output; assertions use the built-in node:assert/strict module.

```
const raw = '25';
const parsed = Number(raw);
console.log(typeof raw, typeof parsed);
console.log(parsed + 5);
console.log(raw + 5);
console.log(Number(true));

// Expected output:
// string number
// 30
// 255
// 1
```

The original raw binding still holds a string. Number(true) succeeding is a language conversion rule; it does not make true a valid query parameter.

Reading and Practice Route

Start with the conversion rules, follow the execution diagrams, and inspect the production parsers. Attempt the six challenges before consulting their solutions, then test your reasoning with the sixteen MCQs.

Companion programs live in `code/volume-1/chapter-04/`:

```
node code/volume-1/chapter-04/example-00-coercion-model.js
node code/volume-1/chapter-04/example-01-query-parser.js
node code/volume-1/chapter-04/example-02-boolean-env.js
node code/volume-1/chapter-04/example-03-coercion-challenges.js
node code/volume-1/chapter-04/example-04-edge-cases.js
```

The chapter treats query/environment values as data. It does not evaluate input as JavaScript or assume an arbitrary object is safe to coerce.

Theory

Conversion Produces a Value; Validation Accepts a Domain

An explicit conversion names the target representation, such as Number(raw). An implicit conversion is requested by an operation, such as multiplication converting a numeric string. Neither inherently validates an application's schema. A valid Number may be negative, infinite, fractional, unsafe as an integer, or outside a business limit.

Primitive conversion does not rewrite the source binding. Object conversion may call methods and therefore cause effects; the returned primitive is still separate from the original object.

String, Number, and Boolean Conversion Matrix

The following table uses default built-ins and ordinary objects. Quoted entries in the String column are string results; NaN is the Number value representing an invalid numeric result.

Input	String(input)	Number(input)	Boolean(input)
undefined	"undefined"	NaN	false
null	"null"	0	false
false	"false"	0	false
true	"true"	1	true
""	""	0	false

Input	String(input)	Number(input)	Boolean(input)
" "	" "	0	true
"12"	"12"	12	true
"12px"	"12px"	NaN	true
[]	""	0	true
[7]	"7"	7	true
[7, 8]	"7,8"	NaN	true
{}	"[object Object]"	NaN	true

Object rows depend on their conversion methods; they are not universal results for every object. String conversion is also distinct from JSON serialization. `String({id: 1})` does not serialize the object's fields into JSON.

```
console.log(Number(''), Number('  '), Number(null));
console.log(Number(undefined), Number('12px'));
console.log(String([7, 8]));
console.log(Boolean('false'), Boolean([]), Boolean({}));
console.log(String(Symbol('job')));
try { console.log(Number(Symbol('job'))); }
catch (error) { console.log(error.name); }

// Expected output:
// 0 0 0
// NaN NaN
// 7,8
// true true true
// Symbol(job)
// TypeError
```

String explicitly accepts a Symbol primitive and returns a description; implicit string conversion in concatenation rejects a Symbol. Symbols are not numeric quantities. See String conversion and Number conversion.

Numeric Syntax Is Broader Than Most Input Schemas

Number consumes its complete string under the language's numeric-string rules. It accepts surrounding whitespace, hexadecimal notation, exponential notation, and Infinity. parseInt and parseFloat instead recognize a numeric prefix; parseInt's radix controls digit interpretation, not a requirement to consume the entire string.

```
console.log(Number('0x10'), Number('1e2'), Number(' 12 '));
console.log(Number('12px'));
console.log(parseInt('12px', 10), parseInt('1e2', 10));
```

```
console.log(parseFloat('1.5ms'));
console.log(Number.isFinite(Number('Infinity')));
console.log(Number.isInteger(9007199254740992));
console.log(Number.isSafeInteger(9007199254740992));

// Expected output:
// 16 100 12
// NaN
// 12 1
// 1.5
// false
// true
// false
```

Prefix parsing can be useful when the contract explicitly allows a suffix and separately validates it. It is wrong for a strict page number where trailing text must be rejected. Number.isSafeInteger checks that a Number is integral and within the exact safe-integer range; it cannot recover a decimal string already rounded during conversion. Preserve and validate the original text first. parseInt reference, safe integers.

Truthiness Does Not Parse Boolean Text

The falsy primitive values are undefined, null, false, positive/negative zero, NaN, zero BigInt, and the empty string. Other primitive values are truthy. Ordinary objects are truthy, even an empty array or a Boolean wrapper holding false. Boolean conversion does not call an object's valueOf or Symbol.toPrimitive method.

```
let calls = 0;
const value = { valueOf() { calls += 1; return 0; } };
console.log(Boolean(value), calls);
console.log(Boolean(new Boolean(false)));
console.log(Boolean('0'), Boolean('false'), Boolean(' '));
console.log(Boolean(0n));

// Expected output:
// true 0
// true
// true true true
// false
```

Use Boolean(value) to ask whether value is truthy. To parse an environment flag, compare accepted strings such as "true" and "false" explicitly. Avoid new Boolean for flags. Browsers retain a legacy exception for document.all; it is not a portable data-model tool and does not exist in these Node examples. Boolean reference.

BigInt Conversion Has Its Own Boundary

BigInt accepts integer strings and integral Numbers; nonintegral Numbers cause RangeError and malformed integer strings cause SyntaxError. Boolean values become 0n/1n. Null, undefined, and Symbol are rejected. Decimal fractions and exponent notation are not valid BigInt string syntax, though base-prefixed integer strings are accepted.

```
console.log(String(BigInt('9007199254740993')));
console.log(String(BigInt(Number('9007199254740993'))));
console.log(String(BigInt(true)));
try { BigInt(1.5); } catch (error) { console.log(error.name); }
try { BigInt('1e3'); } catch (error) { console.log(error.name); }

// Expected output:
// 9007199254740993
// 9007199254740992
// 1
// RangeError
// SyntaxError
```

Convert large integer text directly to BigInt when that is the required representation. Going through Number can lose precision permanently. Arithmetic normally requires both operands to have the same numeric type; comparisons can handle Number and BigInt without that arithmetic restriction. BigInt reference.

Addition and Numeric Operators Request Different Conversions

Binary + first obtains primitives. If either is a string, it concatenates their string forms; otherwise it performs numeric addition. Subtraction and multiplication request numeric values instead. Left associativity can therefore change the type midway through a chain.

```
console.log(2 + 3 + '4');
console.log('2' + 3 + 4);
console.log('6' - 2);
console.log('6' * '2');
console.log('6' + 2);

// Expected output:
// 54
// 234
// 4
// 12
// 62
```

Do not use subtraction by zero as an input schema. It silently admits the same broad numeric conversions that a strict boundary often needs to reject.

Strict and Loose Equality Ask Different Questions

Strict equality does not convert differing types. Objects compare by identity, NaN is unequal to itself, and positive/negative zero compare equal. Object.is also compares identity, but considers NaN equal to itself and distinguishes the two zeros.

Loose equality follows type-directed rules. Same-type values use strict-equality behavior. Null and undefined match each other. Boolean operands become Numbers; a Number/String pair converts the string numerically. Object/primitive comparisons can request primitive conversion. Two objects still

compare identity, and comparison with null/undefined does not generally coerce an ordinary object.

```
console.log('0' == false);
console.log('0' === false);
console.log(null == undefined, null == 0);
console.log([] == false);
console.log([] == []);
console.log(NaN === NaN, Object.is(NaN, NaN));
console.log(0 === -0, Object.is(0, -0));
console.log(1n == 1, 1n === 1);

// Expected output:
// true
// false
// true false
// true
// false
// false true
// true false
// true false
```

For [] == false: false becomes 0; the array becomes the empty string; that string becomes 0; 0 equals 0. None of these steps says the array is falsy. For most application comparisons, validate a shared representation and use ===. A deliberate `value == null` can test null/undefined together in ordinary data; spell out both checks when that is clearer for the team. Equality reference, Object.is.

Relational Comparison Is Not Equality With Ordering

When both resulting primitives are strings, relational comparison orders UTF-16 code units lexicographically. Otherwise it follows numeric comparison rules, including special handling for BigInt. NaN produces an unordered comparison: both < and >= can be false. Consequently >= is not always equivalent to negating <.

```
console.log('12' < '3');
console.log('12' < 3);
console.log(null < 1, null == 0, null >= 0);
console.log(undefined < 1, undefined >= 1);
console.log(2n < 2.5);

// Expected output:
// true
// false
// true false true
// false false
// true
```

Choose numeric conversion before comparing numeric input strings. Locale-aware human text ordering is a separate requirement from the language's relational string order. Relational comparison reference.

Object-to-Primitive Conversion

An operation may request the hint "string", "number", or "default". A callable Symbol.toPrimitive method receives that hint and must return a primitive; returning an object throws. Without that method, ordinary conversion tries toString before valueOf for a string hint, and valueOf before toString for a number hint. Ordinary objects treat a default hint like number; built-ins such as Date can behave differently.

```
const hints = [];
const price = {
  [Symbol.toPrimitive](hint) {
    hints.push(hint);
    return hint === 'string' ? 'USD 12' : 12;
  }
};
console.log(String(price));
console.log(Number(price));
console.log(price + 2);
console.log(hints.join(','));

// Expected output:
// USD 12
// 12
// 14
// string,number,default
```

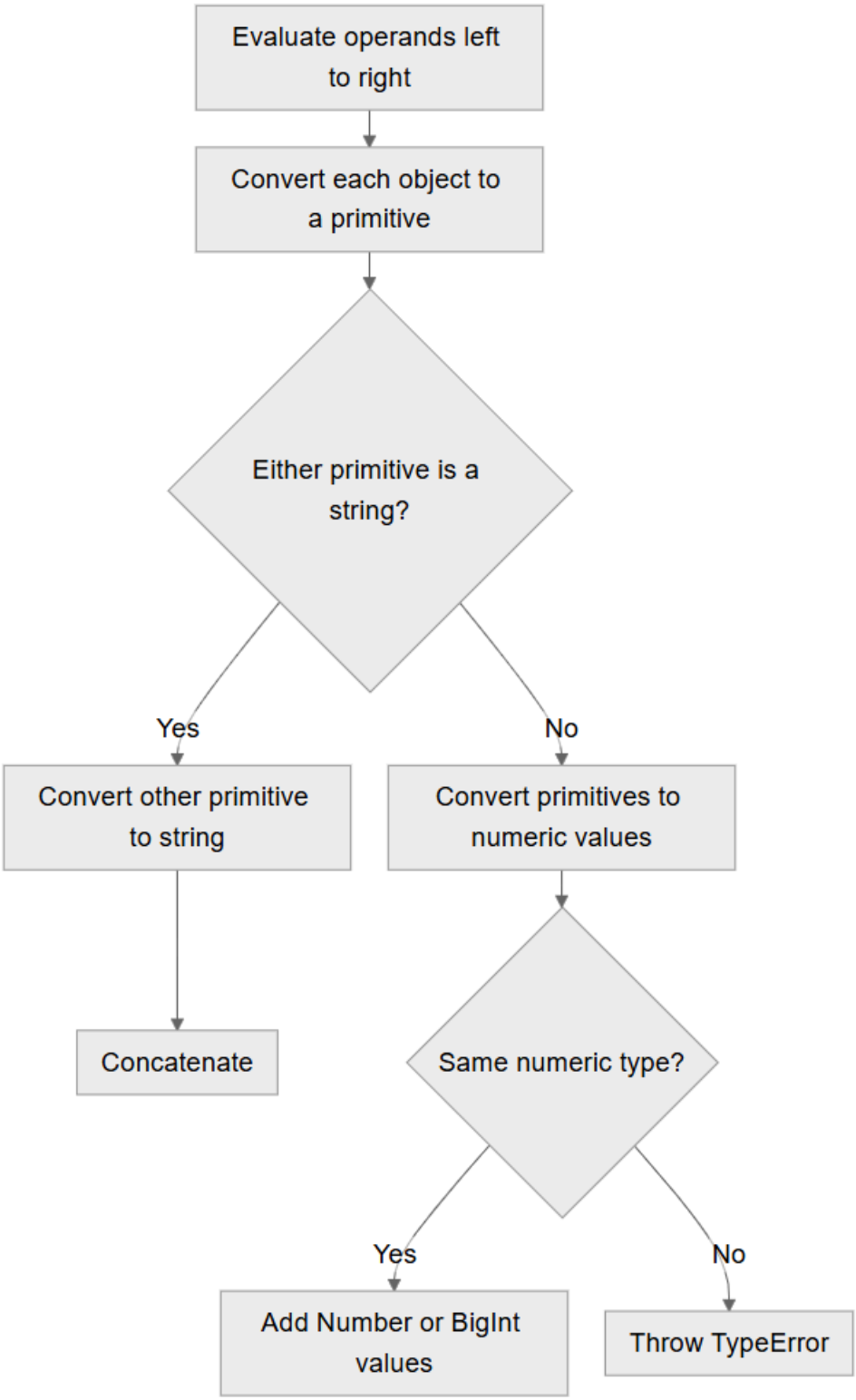
The method is useful for a carefully designed domain object, but implicit effects can make arithmetic and logging surprising. For ordinary DTOs, prefer named fields or explicit formatters. The Symbol.toPrimitive reference describes the customization point; the specification's abstract operations define fallback behavior.

Internal Working

Trace Values Before Selecting the Operation

For `invoice + ' USD'`, the engine first evaluates the operands and obtains their values. It then requests primitive values, using the default hint for an object. After conversion, a string operand selects concatenation. Conversion methods may throw or change state before the final operation happens.

Stored source: `diagrams/volume-1-chapter-04-addition-flow.mmd`.



This models binary +. It is not the algorithm for Boolean conversion, subtraction, or every comparison. A Symbol primitive reaches string conversion in the concatenation branch and throws; a Number/BigInt pair reaches the mismatched numeric-types branch and throws.

A Visible Conversion Trace

```
const events = [];
const left = {
  [Symbol.toPrimitive](hint) { events.push('left:' + hint); return 4; }
};
const right = {
  [Symbol.toPrimitive](hint) { events.push('right:' + hint); return '5'; }
};
console.log(left + right);
console.log(events.join(','));

// Expected output:
// 45
// left:default,right:default
```

Both primitives are obtained before + chooses concatenation. The number 4 then becomes "4", and the result is "45". If the left conversion throws, the right conversion does not run. Evaluating the right operand expression and invoking its conversion method are separate steps: both operand expressions are evaluated before + requests primitive conversions.

Fallback Conversion Order

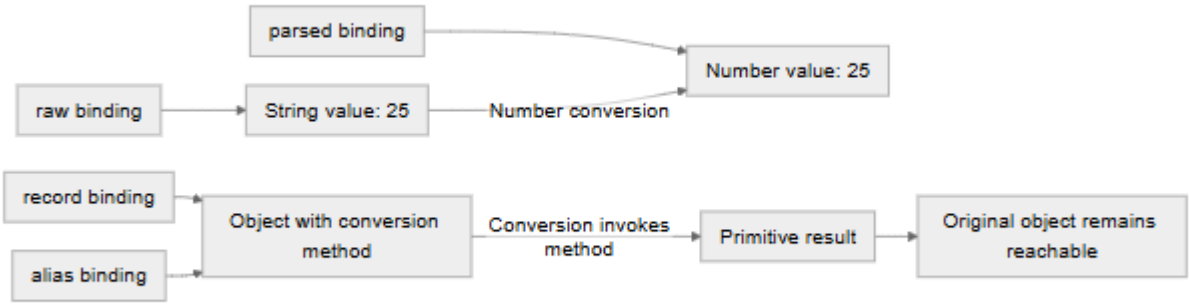
```
const events = [];
const amount = {
  valueOf() { events.push('valueOf'); return this; },
  toString() { events.push('toString'); return '25'; }
};
console.log(Number(amount));
console.log(events.join(','));
events.length = 0;
console.log(String(amount));
console.log(events.join(','));

// Expected output:
// 25
// valueOf,toString
// 25
// toString
```

Number conversion tries valueOf, receives an object, and tries toString. String conversion reverses the order and stops as soon as it receives a primitive. A primitive of the wrong eventual numeric meaning can still become NaN; the conversion method's success does not validate a business quantity. If neither method returns a primitive, conversion throws TypeError.

Bindings, Values, and Object Identity

Stored source: `diagrams/volume-1-chapter-04-conversion-memory.mmd`.



The diagram represents semantic values and references, not fixed physical stack/heap locations. Primitive conversion derives a result; it does not replace the source binding or mutate an object automatically. User-defined conversion methods can still mutate shared objects.

```
const raw = '25';
const parsed = Number(raw);
const record = { count: 0, valueOf() { this.count += 1; return 25; } };
const alias = record;
const normalized = Number(record);
console.log(typeof raw, typeof parsed);
console.log(normalized, alias.count);
console.log(record === alias);

// Expected output:
// string number
// 25 1
// true
```

Two aliases still refer to the same record. Its conversion hook increments shared state; the normalized Number is a separate primitive result. Repeated coercion of such an object is observable work, not a harmless repeated read.

Trace an Equality Failure

For `'false' == false`, the right boolean becomes 0; the left string then undergoes Number conversion and becomes NaN; equality with NaN is false. For `'0' == false`, the same steps reach 0 == 0 and return true. For Boolean('false'), neither equality nor numeric conversion runs: a nonempty string is simply truthy.

This explains why comparing a setting to false with loose equality is not a reliable parser for boolean words.

From Language Algorithms to Engines

Names such as ToPrimitive, ToNumber, and IsLooselyEqual describe specification operations. They are not public JavaScript functions or a promise that an engine emits a distinct function call for every step. A compiler can specialize operations for observed primitive types, but it must preserve conversion-hook

order, thrown errors, and visible effects.

Do not claim that one spelling is faster based on how many abstract operations appear in the specification. Compare measured workloads only after ensuring the same accepted inputs, outputs, and errors. The abstract-operation definitions settle semantic questions; they do not specify the engine's memory layout.

Production Examples

Parse Pagination at the Boundary

The gateway accepts page and pageSize as strings from a query adapter. Each must be a canonical positive decimal integer: ASCII digits, no leading zero, no sign, no whitespace, no fraction, and no suffix. The parser admits at most 16 digits, then requires a safe integer and a field-specific bound.

Page is limited to 1 through 1,000,000 and pageSize to 1 through 100. Missing fields default to "1" and "25"; explicit null is invalid. The helper returns a new frozen record, tolerates unrelated fields, and leaves its ordinary data-record input unchanged. Duplicate query parameters must be rejected or resolved by the HTTP adapter; an array of values is rejected here.

Input	Result and reason
"2"	Number 2, if within the field bound.
true or 2	TypeError: only source strings are accepted.
"01", "+2", "2.0", "2e0"	RangeError: outside the accepted grammar.
"2px"	RangeError: the entire string must match.
"9007199254740993"	RangeError: not a safe integer after conversion.
pageSize "101"	RangeError: exceeds the page-size policy.
Missing page	Default to 1.
page null	TypeError: malformed supplied value.

```
'use strict';
const assert = require('node:assert/strict');

function parsePositiveInteger(value, fieldName, maximum = Number.MAX_SAFE_INTEGER) {
  if (!Number.isSafeInteger(maximum) || maximum < 1) {
    throw new RangeError('maximum must be a positive safe integer');
  }
  if (typeof value !== 'string') throw new TypeError(fieldName + ' must be a string');
  if (value.length < 1 || value.length > 16 || value[0] === '0' || /[^0-9]/.test(value)) {
    throw new RangeError(fieldName + ' must contain canonical positive decimal digits');
```

```
  }
  const parsed = Number(value);
  if (!Number.isSafeInteger(parsed) || parsed > maximum) {
    throw new RangeError(fieldName + ' is outside its allowed range');
  }
  return parsed;
}

function parsePagination(query = {}) {
  if (query === null || typeof query !== 'object' || Array.isArray(query)) {
    throw new TypeError('query must be a data record');
  }
  const page = parsePositiveInteger(query.page === undefined ? '1' : query.page, 'page', 1000000);
  const pageSize = parsePositiveInteger(
    query.pageSize === undefined ? '25' : query.pageSize, 'pageSize', 100
  );
  return Object.freeze({ page, pageSize, offset: (page - 1) * pageSize });
}

const input = { page: '2', pageSize: '25' };
const parsed = parsePagination(input);
assert.deepEqual(parsed, { page: 2, pageSize: 25, offset: 25 });
assert.deepEqual(input, { page: '2', pageSize: '25' });
assert.equal(Object.isFrozen(parsed), true);
assert.throws(() => parsePositiveInteger('9007199254740993', 'page'), RangeError);
assert.throws(() => parsePositiveInteger(true, 'page'), TypeError);
assert.throws(() => parsePagination({ pageSize: '101' }), RangeError);
assert.throws(() => parsePagination({ page: null }), TypeError);
console.log(JSON.stringify(parsed));
console.log(JSON.stringify(parsePagination()));

// Expected output:
// {"page":2,"pageSize":25,"offset":25}
// {"page":1,"pageSize":25,"offset":0}

// O(k) parsing work for at most 16 digits per field; O(1) output storage.
```

The type check prevents true becoming 1 and prevents object conversion hooks from running. The length bound limits work before the regular expression. Grammar validation rejects representations that Number would otherwise accept. Safe-integer validation catches precision loss, and domain bounds keep request sizes and computed offsets controlled.

The largest offset is 99,999,900 under these bounds, so its integer multiplication remains safe. A different page-size or page limit requires revisiting that proof. Large database offsets may still be slow; cursor pagination is a separate design decision, not something numeric validation solves.

Companion: `code/volume-1/chapter-04/example-01-query-parser.js`. Additional assertions cover both maximums, unsafe integer neighbors, long strings, duplicate-value arrays, nonstring values, and canonical syntax.

Parse Environment Flags as an Explicit Vocabulary

The configuration layer accepts exactly "true" or "false". Case changes and whitespace are rejected, so a typo cannot silently switch behavior. An absent CACHE_ENABLED defaults to false. A present malformed value stops configuration loading with an error instead of quietly selecting a default.

The function receives a record rather than reading process.env directly. Tests can supply independent records without mutating the process environment, and browser/server adapters can use the same parsing policy.

```
'use strict';
const assert = require('node:assert/strict');

function parseBoolean(value, fieldName) {
  if (value === 'true') return true;
  if (value === 'false') return false;
  throw new TypeError(fieldName + ' must be "true" or "false"');
}

function parseCacheSettings(input = {}) {
  if (input === null || typeof input !== 'object' || Array.isArray(input)) {
    throw new TypeError('settings must be a data record');
  }
  const raw = input.CACHE_ENABLED === undefined ? 'false' : input.CACHE_ENABLED;
  return Object.freeze({ cacheEnabled: parseBoolean(raw, 'CACHE_ENABLED') });
}

assert.equal(parseBoolean('false', 'CACHE_ENABLED'), false);
assert.equal(parseBoolean('true', 'CACHE_ENABLED'), true);
assert.throws(() => parseBoolean('False', 'CACHE_ENABLED'), TypeError);
assert.throws(() => parseBoolean(false, 'CACHE_ENABLED'), TypeError);
console.log(JSON.stringify(parseCacheSettings({ CACHE_ENABLED: 'false' })));
console.log(JSON.stringify(parseCacheSettings({ CACHE_ENABLED: 'true' })));
console.log(JSON.stringify(parseCacheSettings()));

// Expected output:
// {"cacheEnabled":false}
// {"cacheEnabled":true}
// {"cacheEnabled":false}

// Fixed accepted literals and a one-field output: O(1) work and storage.
```

Boolean('false') would return true because the string is nonempty. JSON.parse could parse booleans, but it also accepts many values this schema does not allow. Explicit comparisons state the vocabulary directly and avoid coercing object values.

Companion: `code/volume-1/chapter-04/example-02-boolean-env.js`. Assertions cover capitalization, whitespace, numeric spellings, null, missing data, result freezing, and input ownership.

Error and Trust Policy

The helpers throw on malformed supplied values. An HTTP adapter can map query errors to an appropriate client response; a startup adapter can report an invalid setting and stop. Do not expose arbitrary raw input in diagnostic messages. Field names in these examples come from the program's fixed schema, not from an attacker.

The record wrappers expect ordinary application data, not hostile proxies/getters. Rejecting a nonstring field prevents value conversion hooks, but reading that field can itself execute a getter. Parsing data is not a sandbox for JavaScript objects. Enforce request-size limits before deserializing large inputs.

Why Separate Conversion From Business Logic?

Downstream code receives bounded Number and Boolean primitives and can compute without repeatedly interpreting strings. That gives every call site the same missing-value and invalid-value policy. A later request to accept whitespace or "yes" should change the boundary contract and its assertions, not introduce ad hoc conversions throughout the service.

See Number.isSafeInteger for the representable integer boundary and Boolean conversion for why truthiness is not boolean-text parsing.

Interview Perspective

Explain Conversion Before Quoting an Output

An answer is strongest when it identifies input types, names the operation, traces its requested conversions, and only then states the result. This gives you a method when an interviewer changes `[]` to `{}` or a Number to a BigInt.

Does Explicit Conversion Validate an Input?

No. Number(true) is 1 and Number('') is 0. Both are valid language conversions but may violate a query schema. A parser should specify source type, accepted syntax, representation, safe range, domain range, and missing-value policy. The pagination example checks all of those.

A follow-up may ask why Number.isInteger is insufficient. It accepts integral Numbers outside the safe-integer range; the original decimal digits may already have rounded. Number.isSafeInteger checks the resulting integer range, and grammar/type checks ensure the source itself was acceptable.

Why Is an Empty Array Truthy but Loosely Equal to False?

Boolean conversion sees an ordinary object and returns true without calling conversion methods. Loose equality applies a different algorithm: false becomes 0; the array becomes an empty string; the string becomes 0. The final numeric equality is true. Do not describe this as the array being "both true and false."

```
const array = [];
console.log(Boolean(array));
console.log(array == false);
console.log(array === false);
console.log(array == []);
console.log(array == array);

// Expected output:
// true
// true
// false
// false
// true
```

The final two cases compare object identity. Loose equality does not convert two objects just because they look alike.

What Does the Plus Operator Need to Know?

Binary + obtains primitives and then asks whether either is a string. If so, it concatenates; otherwise it adds same-type numeric values. Subtraction does not have the concatenation branch. For `2 + 3 + '4'`, left grouping first produces 5, then "54". For `'2' + 3 + 4`, both operations concatenate and produce "234".

The follow-up is order: operand expressions are evaluated left to right before the + operation requests primitive conversion. Conversion itself can call user code and throw.

Why Can Null Compare Greater Than or Equal to Zero but Not Equal It?

Relational comparison converts null numerically to zero. Loose equality's null/undefined rule does not equate null with Number zero. Different operators request different algorithms. It is inaccurate to explain all comparisons as "convert both sides to numbers."

```
console.log(null >= 0);
console.log(null == 0);
console.log(undefined < 0);
console.log(undefined >= 0);

// Expected output:
// true
// false
// false
// false
```

Undefined converts to NaN for these relational comparisons. That also explains why >= is not universally equivalent to negating <.

Does Strict Equality Mean Every Value Equals Itself?

No: NaN === NaN is false. Object.is considers NaN equal to itself but distinguishes positive and negative zero. Neither operation performs deep structural equality for objects. Choose the equality semantics required by the domain, not whichever name sounds strongest.

Why Can BigInt Fail to Recover a Large Integer?

If text passed through Number first, the precise digits may already have been lost. BigInt converts that rounded Number. For a large integer identifier, keep the string when textual identity matters, or parse directly to BigInt when arithmetic is required. BigInt is not a blanket fix for decimal fractions.

What Is the Object Conversion Protocol?

First inspect Symbol.toPrimitive. A callable method receives "string", "number", or "default" and must return a primitive. Without it, ordinary fallback order depends on the hint: toString first for string, valueOf first for number. Default behaves like number for ordinary objects, with built-in exceptions such as Date. Boolean conversion does not use this protocol.

In production, a named conversion method may communicate units and effects better than an implicit hook. Reject arbitrary objects at a string-input boundary before attempting Number conversion.

Worked Design Prompt

An endpoint receives pageSize as an unknown value. A candidate proposes `parseInt(value, 10) || 25`. Identify four failures: suffixes are accepted, zero and missing data collapse into the same fallback, nonstrings are coerced, and there is no safe/domain upper bound. A complete replacement accepts one documented string grammar, defaults only missing input, checks a safe Number and the page-size bound, and returns an error for supplied invalid data.

Test "25", "0", "25px", "1e2", true, null, undefined, the maximum allowed page size, and the next larger value. Each result should follow the stated contract without relying on accidental coercion.

Consult the equality rules when tracing a new mixed-type case; rehearse the exercise solutions as examples of actual boundary design.

Exercises and Coding Challenges

Write each contract and its boundary cases before reading the solution. Every program is independent. The companion `code/volume-1/chapter-04/example-03-coercion-challenges.js` runs all six solutions.

1. Parse a Network Port

Problem: Accept a string containing a canonical positive decimal port from 1 through 65535. Reject signs, whitespace, leading zeros, fractions, suffixes, booleans, and missing input. Return a Number.

Hint: Check the source type and digit vocabulary before numeric conversion.

Solution

```
const assert = require('node:assert/strict');
function parsePort(raw) {
  if (typeof raw !== 'string') throw new TypeError('port must be a string');
  if (raw.length < 1 || raw.length > 5 || raw[0] === '0' || /[^0-9]/.test(raw)) {
    throw new RangeError('invalid port syntax');
  }
  const port = Number(raw);
  if (port > 65535) throw new RangeError('port is too large');
  return port;
}
assert.equal(parsePort('1'), 1);
assert.equal(parsePort('65535'), 65535);
for (const raw of ['0', '01', '65536', '80px', '80\n', '+80', ' 80']) {
  assert.throws(() => parsePort(raw), RangeError);
}
assert.throws(() => parsePort(true), TypeError);
console.log(parsePort('8080'));

// Expected output:
// 8080

// O(k) work for at most five digits; O(1) additional storage.
```

Number('80px') rejects a suffix but accepts other forbidden forms such as whitespace and hexadecimal notation. The source grammar therefore remains necessary. The five-digit bound makes every syntactically accepted Number a safe integer.

2. Parse a Three-State Setting

Problem: Accept only the strings "enabled", "disabled", and "auto". Return true, false, and null respectively. Reject every other value, including boolean inputs. The null result means defer to a separate policy.

Hint: An explicit vocabulary can represent more than truthiness.

Solution

```
const assert = require('node:assert/strict');
function parseMode(raw) {
  if (raw === 'enabled') return true;
  if (raw === 'disabled') return false;
  if (raw === 'auto') return null;
```

```
  throw new TypeError('mode must be enabled, disabled, or auto');
}
assert.equal(parseMode('enabled'), true);
assert.equal(parseMode('disabled'), false);
assert.equal(parseMode('auto'), null);
for (const raw of [true, false, undefined, '', 'AUTO', 'false']) {
  assert.throws(() => parseMode(raw), TypeError);
}
console.log(JSON.stringify(['enabled', 'disabled', 'auto'].map(parseMode)));

// Expected output:
// [true,false,null]

// O(1) work and storage for each fixed vocabulary lookup.
```

Boolean('disabled') is true, so generic truthiness would erase the setting's meaning. Do not apply ?? until the caller is ready to resolve the intentional auto state.

3. Preserve an Opaque Decimal Identifier

Problem: Accept one to twenty ASCII digits as an identifier and return the original string unchanged, including leading zeros. Reject numeric inputs. Two long IDs that collide after Number conversion must remain distinct.

Hint: An identifier need not be an arithmetic quantity.

Solution

```
const assert = require('node:assert/strict');
function requireDecimalId(raw) {
  if (typeof raw !== 'string') throw new TypeError('id must be a string');
  if (raw.length < 1 || raw.length > 20 || /[^0-9]/.test(raw)) {
    throw new RangeError('id must contain 1 through 20 digits');
  }
  return raw;
}
const a = requireDecimalId('9007199254740992');
const b = requireDecimalId('9007199254740993');
assert.notEqual(a, b);
assert.equal(Number(a), Number(b));
assert.equal(requireDecimalId('0007'), '0007');
assert.throws(() => requireDecimalId(7), TypeError);
assert.throws(() => requireDecimalId('7\n'), RangeError);
console.log(a === b, Number(a) === Number(b));
console.log(requireDecimalId('0007'));

// Expected output:
// false true
// 0007

// O(k) validation for at most 20 digits; returns the original immutable string.
```

Converting to BigInt would preserve magnitude but discard leading zeros. The contract preserves textual identity, so neither Number nor BigInt is the right normalized representation.

4. Compare Decimal Sensor Readings Numerically

Problem: Accept signed decimal strings up to 32 characters, with optional fractional digits and no whitespace/exponent notation. Allow -0 and reject leading zeros except zero itself. Require magnitudes at most 1,000,000. Return -1, 0, or 1 using their Number values; this contract permits floating-point rounding.

Hint: Full-match validation and numeric comparison are separate steps.

Solution

```
const assert = require('node:assert/strict');
function compareReadings(left, right) {
  function parse(raw) {
    if (typeof raw !== 'string') throw new TypeError('reading must be a string');
    if (raw.length < 1 || raw.length > 32) throw new RangeError('invalid reading length');
    const match = /^-?(?:0|[1-9][0-9]*)(?:\.[0-9]+)?$/.exec(raw);
    if (match === null || match[0] !== raw) throw new RangeError('invalid decimal syntax');
    const number = Number(raw);
    if (!Number.isFinite(number) || Math.abs(number) > 1000000) {
      throw new RangeError('reading out of range');
    }
    return number;
  }
  const a = parse(left);
  const b = parse(right);
  return a < b ? -1 : a > b ? 1 : 0;
}
assert.equal(compareReadings('12', '3'), 1);
assert.equal(compareReadings('-0', '0'), 0);
assert.equal(compareReadings('1.25', '1.3'), -1);
assert.throws(() => compareReadings('1\n', '2'), RangeError);
assert.throws(() => compareReadings('1e2', '2'), RangeError);
assert.throws(() => compareReadings('1000001', '2'), RangeError);
console.log(compareReadings('12', '3'), compareReadings('1.25', '1.3'));

// Expected output:
// 1 -1

// O(k) validation with each input bounded to 32 characters; O(k) match storage.
```

The full-match equality rejects a final line terminator even though JavaScript's $ anchor can match immediately before it. This compares approximate sensor values, not exact money or arbitrary-precision decimals.

5. Give a Duration a Deliberate Primitive Representation

Problem: Create a duration from a nonnegative integer Number of milliseconds, at most one day. String conversion should produce a label such as '1500 ms'; numeric/default conversion should produce milliseconds. Reject other input types. Keep the stored value private to the created method's closure.

Hint: Inspect the hint passed to Symbol.toPrimitive.

Solution

```
const assert = require('node:assert/strict');
function makeDuration(milliseconds) {
  if (!Number.isInteger(milliseconds) || milliseconds < 0 || milliseconds > 86400000) {
    throw new RangeError('duration must be a bounded millisecond count');
  }
  return Object.freeze({
    [Symbol.toPrimitive](hint) {
      return hint === 'string' ? milliseconds + ' ms' : milliseconds;
    }
  });
}
const duration = makeDuration(1500);
assert.equal(String(duration), '1500 ms');
assert.equal(Number(duration), 1500);
assert.equal(duration + 500, 2000);
assert.equal(Boolean(duration), true);
assert.throws(() => makeDuration('1500'), RangeError);
assert.throws(() => makeDuration(-1), RangeError);
console.log(String(duration), duration + 500);

// Expected output:
// 1500 ms 2000

// O(1) creation and conversion for this bounded numeric domain.
```

The closure retains one bounded primitive; freezing prevents replacement of the conversion method. The method returns only primitives. In a larger API, named toMilliseconds/format methods may be easier to read than implicit arithmetic.

6. Sum Validated Counts Without Concatenation

Problem: Accept an array of at most 1000 canonical nonnegative decimal strings, each from 0 through 999999. Reject holes, nonstrings, leading zeros, and malformed values. Return the total as a Number without changing the array.

Hint: Validate each item before adding it; bound the largest possible total.

Solution

```
const assert = require('node:assert/strict');
function sumCounts(values) {
  if (!Array.isArray(values)) throw new TypeError('counts must be an array');
  if (values.length > 1000) throw new RangeError('too many counts');
  let total = 0;
  for (const raw of values) {
    if (typeof raw !== 'string') throw new TypeError('each count must be a string');
    if (raw.length < 1 || raw.length > 6 || /[^0-9]/.test(raw) ||
        (raw.length > 1 && raw[0] === '0')) throw new RangeError('invalid count');
    total += Number(raw);
  }
  return total;
}
const input = ['2', '3', '0'];
assert.equal(sumCounts(input), 5);
assert.deepEqual(input, ['2', '3', '0']);
assert.equal(sumCounts([]), 0);
assert.equal(sumCounts(Array(1000).fill('999999')), 999999000);
assert.throws(() => sumCounts(['01']), RangeError);
assert.throws(() => sumCounts(['2', 3]), TypeError);
assert.throws(() => sumCounts(new Array(1)), TypeError);
console.log(sumCounts(input));

// Expected output:
// 5

// O(n) time with at most six characters per item; O(1) additional space.
```

The maximum total is 999,999,000, well inside the safe-integer range. Starting with numeric zero is necessary but insufficient: adding unconverted strings would turn the accumulator into a string.

MCQs

Choose one answer before reading the explanation. Assume default built-ins and ordinary objects in Node.js unless stated otherwise.

1. What does Number(' ') return?

A. NaN B. 0 C. undefined D. TypeError

Answer: B. Whitespace-only strings convert to zero. A strict input schema can reject them before conversion.

2. What is Boolean('false')?

A. false B. TypeError C. The string "false" D. true

Answer: D. Every nonempty string is truthy; Boolean does not interpret a boolean word.

3. Which result follows from Number.isInteger(9007199254740992)?

A. true, despite being outside the safe-integer range B. false because every unsafe integer is fractional C. RangeError D. true and therefore every neighboring integer is exact

Answer: A. Being integral and being safe are separate properties. Number.isSafeInteger rejects this value.

4. What is parseInt('15px', 10)?

A. NaN B. TypeError C. 15 D. 150

Answer: C. parseInt consumes a valid digit prefix. It does not enforce a full-string numeric schema.

5. Why is [] == false true?

A. Empty arrays are falsy B. The array converts to an empty string, then both sides reach numeric zero C. Equality compares array length to the boolean D. Both values are converted to strings with identical text

Answer: B. Equality and truthiness request different conversion algorithms. Boolean([]) is true.

6. What does [] == [] produce?

A. false B. true C. TypeError D. undefined

Answer: A. The operands are distinct objects; loose equality does not convert two objects for structural comparison.

7. Which statement describes Object.is?

A. It converts numeric strings to Numbers B. It performs deep object comparison C. It treats NaN as equal to itself and distinguishes positive/negative zero D. It treats Number 1 and BigInt 1n as equal

Answer: C. Object.is uses same-value semantics while still comparing objects by identity.

8. What is `'12' < '3'`?

A. false because 12 exceeds 3 B. TypeError because both operands are strings C. NaN D. true because the comparison is lexicographic

Answer: D. Both primitives are strings, so code-unit ordering compares the first characters.

9. Which expression is true?

A. null == 0 B. null >= 0 C. undefined >= 0 D. `undefined < 0`

Answer: B. Relational conversion turns null into zero. The equality and undefined cases follow different rules.

10. Which conversion can preserve all digits of '9007199254740993' for integer arithmetic?

A. BigInt(Number(text)) B. parseInt(text, 10) C. Number(text) D. BigInt(text)

Answer: D. Direct BigInt parsing avoids an intermediate Number's precision loss.

11. What happens when BigInt('1.5') is evaluated?

A. SyntaxError B. It returns 1n C. It returns 2n D. It returns NaN

Answer: A. A fractional string is not valid integer syntax. This differs from a nonintegral Number argument, which raises RangeError.

12. Which hint does binary + pass to an object's Symbol.toPrimitive method?

A. "string" for every use B. "number" for every use C. "default" D. No hint argument

Answer: C. The primitive result then determines whether + selects concatenation or numeric addition.

13. What happens if Symbol.toPrimitive returns an object?

A. The engine always tries that object's toString B. TypeError C. The object becomes truthy D. The original object is returned from Number

Answer: B. The hook must directly return a primitive. Ordinary fallback is not used to rescue an invalid hook result.

14. Which operation does not call an ordinary object's conversion methods?

A. Boolean(object) B. Number(object) C. String(object) D. object + 1

Answer: A. Boolean conversion treats ordinary objects as truthy without requesting a primitive.

15. What does 2 + 3 + '4' produce?

A. Number 9 B. String "234" C. String "54" D. TypeError

Answer: C. Left grouping adds the first Numbers, then the string operand selects concatenation.

16. Which validation sequence fits a strict bounded page-size string?

A. Number conversion, then accept any truthy result B. parseInt, then default every falsy result C. Boolean conversion, then Number conversion D. Source type, full grammar, numeric conversion, safe/domain bounds

Answer: D. Conversion alone accepts representations and magnitudes that a boundary contract may forbid.

Revision and Summary

Conversion Rules at a Glance

Operation	Rule to recall
String(value)	Creates a string representation; this is not JSON serialization.
Number(value)	Converts a whole value using permissive numeric rules; empty text and null become zero.
Boolean(value)	Tests truthiness; nonempty "false" and ordinary objects are true.
BigInt(value)	Accepts integer representations; Number input may already have lost precision.
parseInt(text, radix)	Reads a numeric prefix, so trailing text may be ignored.
Binary +	Obtain primitives, concatenate if either is a string, otherwise add numeric values.
===	No mixed-type coercion; objects compare identity; NaN differs from itself.
==	Type-directed conversions, including null/undefined pairing and object/primitive conversion.

Operation	Rule to recall
Object.is	Same-value comparison: NaN matches itself, signed zeros differ.
Relational comparison	Two strings use code-unit order; otherwise numeric rules apply.
Symbol.toPrimitive	Receives a hint and must return a primitive.
Boolean(object)	Does not request an object's primitive conversion.

Prediction Check

```
console.log(Number(''), Boolean(''));
console.log(Number('false'), Boolean('false'));
console.log(null == undefined, null == 0, null >= 0);
console.log('2' + 1, '2' - 1);
console.log(Number.isSafeInteger(Number('9007199254740993')));

// Expected output:
// 0 false
// NaN true
// true false true
// 21 1
// false
```

For each output, name the operation before the rule. The same input can legitimately produce different results under numeric conversion, truthiness, equality, and ordering.

Boundary Parser Checklist

1. State the source type. A string parser must reject true before it becomes 1.
2. Define missing versus invalid. Default an absent field only when the contract permits it.
3. Bound source length and validate the complete grammar.
4. Convert once to the desired representation.
5. Check finite/safe status as appropriate and enforce domain limits.
6. Preserve caller ownership and test both sides of every bound.

Opaque identifiers may need no numeric conversion. Exact large integer arithmetic may need direct BigInt parsing. Approximate decimal measurements may legitimately use Number. Choose representation from the domain.

Common Mistakes to Retire

- Treating Boolean('false') as a text parser.

- Treating parseInt as full-string validation.
- Assuming Number.isInteger implies a safe integer.
- Explaining all mixed comparisons with one conversion rule.
- Converting a large integer through Number before BigInt.
- Assuming conversion methods cannot throw or mutate state.
- Comparing numeric strings lexicographically by accident.

References and Further Reading

- Number conversion: accepted primitive and string cases.
- Boolean conversion: falsy values, wrappers, and objects.
- Equality: type-directed comparison cases.
- Symbol.toPrimitive: conversion customization and hints.
- ECMAScript abstract operations: normative definitions when a trace is disputed.

Continue to Strings, Numbers, and Dates for representation details: text units, numeric precision, and explicit timestamp contracts.

Edge Cases and Debugging

Preserve the Original Representation

A log containing only the converted result can hide the defect. An empty string, whitespace, null, and zero can all reach Number zero. Record a safe field label, original type, and an appropriate redacted representation before conversion when investigating a parser. Do not place secrets or arbitrary request text in logs.

```
for (const raw of ['', ' ', null, 0]) {
  console.log(JSON.stringify(raw), typeof raw, Number(raw));
}

// Expected output:
// "" string 0
// " " string 0
// null object 0
// 0 number 0
```

The inputs are observably different even though the converted Number is equal. A parser can reject three of them and accept the fourth if that is its contract.

NaN Needs Its Own Check

```
const invalid = Number('missing');
console.log(invalid === NaN);
console.log(Number.isNaN(invalid));
console.log(Number.isNaN('missing'));
console.log(isNaN('missing'));
console.log(Number.isFinite('25'), isFinite('25'));

// Expected output:
// false
// true
// false
// true
// false true
```

The global isNaN/isFinite functions coerce; their Number counterparts inspect without coercing. Use Number.isNaN for a Number result and Number.isFinite for a required finite Number. Neither checks an application's range or syntax.

A Conversion Hook Can Fail Before Arithmetic

```
const bad = { [Symbol.toPrimitive]() { return {}; } };
try { console.log(Number(bad)); }
catch (error) { console.log(error.name); }
const throwing = { valueOf() { throw new Error('conversion failed'); } };
try { console.log(throwing * 2); }
catch (error) { console.log(error.message); }
console.log(Boolean(throwing));

// Expected output:
// TypeError
// conversion failed
// true
```

Boolean conversion does not run the hook. Numeric conversion does. Reject nonstrings at a string boundary before attempting to coerce their values; this avoids treating application objects as interchangeable input text.

Symbol and BigInt Are Not Universal Numeric Inputs

```
const id = Symbol('id');
console.log(String(id));
try { console.log('id=' + id); } catch (error) { console.log(error.name); }
try { console.log(+1n); } catch (error) { console.log(error.name); }
console.log(Number(1n));
console.log(1n == 1, 1n === 1);

// Expected output:
// Symbol(id)
```

```
// TypeError
// TypeError
// 1
// true false
```

String explicitly handles a Symbol primitive, while implicit concatenation does not. Unary + rejects BigInt even though Number can explicitly convert it. Explicit Number conversion of a large BigInt can lose precision.

Full Grammar Means Full Input

JavaScript's $ regular-expression anchor may match just before a final line terminator. If validating an entire input with an anchored match, verify the matched text equals the original text or use a grammar that rejects every non-digit character. The production integer parser takes the latter approach. Include newline and carriage-return cases in regression tests, not only spaces.

Also decide whether leading zeros, a plus sign, exponent notation, a hexadecimal prefix, and a decimal point are allowed. Their mathematical meaning may match a valid value while their source syntax violates a canonical representation policy.

Turn the Audit Failure Into a Regression

The old parser accepted true as 1 and rounded '9007199254740993'. The repaired companion asserts that both fail. Neighbor checks accept the documented maximum and reject the next value; they also verify that missing pagination defaults while explicit null fails.

When debugging a comparison, write the types beside both operands, identify the operator's algorithm, and show each intermediate primitive. If user-defined conversion is involved, instrument a local trace rather than assuming the object's printed label is its numeric value.

The Number.isNaN reference explains the noncoercing check, and Symbol.toPrimitive explains hook failures.

Performance and Security

Parse Once at a Named Boundary

Repeated Number conversion inside a loop repeats input interpretation and can repeat object conversion hooks. A boundary parser returns a stable primitive representation; downstream logic uses it without re-deciding what missing, empty, or malformed input means.

The pagination parser examines at most 16 characters per field. Its digit scan is O(k), with a small explicit bound on k. The two returned numeric fields and computed offset use constant output storage. Describing arbitrary numeric-string parsing as universally O(1) would hide dependence on input length.

BigInt parsing and arithmetic depend on digit/bit count. A huge decimal string can require substantial work and memory even though the source contains one BigInt call. Limit input size before parsing. Do not accept arbitrary-length integers simply because the type can represent them.

Choose an Optimization Only After Matching Semantics

Unary +, Number, parseInt, and bitwise truncation are not interchangeable parsers. They differ on suffixes, empty strings, BigInt, safe range, and fractions. A benchmark comparing them without matching contracts compares different computations.

Measure the whole boundary under representative input lengths and failure rates. Exception-heavy malformed traffic may have different costs from valid traffic. A service can use a structured validation result at its adapter if exceptions are undesirable there, but it must retain the same acceptance policy and clear failure reporting.

Coercion Can Execute Application Code

```
let calls = 0;
const supplied = { valueOf() { calls += 1; return 25; } };
function requireStringNumber(raw) {
  if (typeof raw !== 'string') throw new TypeError('string required');
  return Number(raw);
}
try { requireStringNumber(supplied); }
catch (error) { console.log(error.name); }
console.log(calls);
console.log(Number(supplied), calls);

// Expected output:
// TypeError
// 0
// 25 1
```

Checking the primitive source type prevents this value's numeric hook from running. It does not make arbitrary property access safe: an object containing the field may have a getter or proxy trap. These parsers assume ordinary data produced by the application's deserialization layer.

Ambiguous Conversion Can Cross a Policy Boundary

Boolean('false') can accidentally enable a feature. Loose equality can equate differently represented identifiers. Unsafe Number conversion can merge distinct integer IDs. Those are concrete reasons to specify source types and representations before comparing permissions, flags, or identifiers.

Keep identity opaque when arithmetic is unnecessary. Keep externally supplied flags separate from trusted authorization facts. A validly parsed "true" means the sender requested something; it does not establish that the sender may perform it.

Resource Limits Belong at Multiple Layers

The page-size cap bounds the result size requested by this helper; it does not bound the incoming HTTP body or make large database offsets cheap. Reject oversized requests before parsing, limit counts at the API boundary, and choose a pagination strategy suited to the storage layer.

Error messages should name the schema field and violated rule without echoing secrets or entire payloads. Defaults should apply only to permitted absence. Replacing every conversion error with a success-shaped default hides malformed traffic and configuration mistakes.

Use safe-integer checks and the BigInt reference to review representation limits before optimizing.

Professional Field Guide

Write a Parsing Contract in Six Lines

For each boundary field, record source type, accepted text grammar, missing-value policy, target representation, range, and error behavior. For pageSize this chapter chooses: string; positive decimal digits without leading zeros; default 25 only when undefined; Number; 1 through 100; reject malformed supplied input.

That contract explains why "25" succeeds while 25, true, "025", "25px", and null fail. If the product later accepts trimmed text, change the contract and its assertions deliberately.

Choose Representation From Meaning

Value	Useful representation	Reason
Bounded page count	Safe integer Number	Arithmetic and bounds are small and exact.
Approximate sensor reading	Finite Number	Floating-point approximation is an accepted part of the domain.
Large integral arithmetic quantity	BigInt parsed directly from text	Avoid an intermediate Number losing digits.
Identifier with leading zeros	String	Textual identity must survive.
Boolean setting	Boolean after explicit vocabulary parsing	Truthiness does not interpret words.
Three-state setting	Boolean or a documented third value	Absence/defer must remain distinguishable.

BigInt is not a decimal-money type, and converting an identifier to a number is not required merely because all its characters are digits.

Review a Parser Repair

The original query helper called Number(value) and checked positivity/integrality. That admitted true and unsafe integer strings. The repair adds a primitive type check, bounded complete digit grammar, safe-integer check, and page-specific maximums. It also separates missing from malformed values.

Review tests against the contract rather than counting assertions: smallest value, largest value, immediate outside neighbors, wrong primitive types, whitespace, suffixes, leading zeros, unsafe integers, and unchanged caller data. A happy-path printout cannot establish any of those boundaries.

Review Conversion in General Code

For +, ask whether either operand can become a string. For comparisons, ask whether both values share a validated representation. For object operands, ask whether conversion hooks can run more than once, mutate state, or throw. For boolean conditions, ask whether the program wants truthiness or exact boolean identity.

Use explicit named formatting/conversion methods when units or side effects would otherwise be hidden. Implicit hooks are appropriate only when their behavior is consistent and useful throughout the object's API.

Explain an Incident Clearly

A useful explanation is: "The boundary accepted true, Number converted it to 1, and the integer guard admitted it. We now require the declared string type before conversion and test that true fails." This identifies the entry point, language rule, incorrect policy, and regression.

For a precision incident, preserve the original text before it is rounded. Once two strings map to the same Number, inspecting that Number cannot reconstruct which ID arrived.

Complete the exercise set, rehearse the interview traces, and continue to Strings, Numbers, and Dates for text and numeric representation details.

CHAPTER 5

Strings, Numbers, and Dates

Introduction

A shipping label, a price, and an expiry time all look simple in a UI. Their storage contracts are different. A label needs a definition of "character"; a price needs units and a rounding policy; an expiry needs a time zone and boundary policy. This chapter connects those choices to the JavaScript values that implement them.

The running examples format bounded integer cents and check a UTC access window. They reject unsupported input instead of relying on convenient coercion. The examples are deliberately small enough to inspect line by line.

Learning Objectives

By the end, you can:

- Distinguish UTF-16 code units, Unicode code points, and grapheme clusters.
- Use immutable string transformations and bounded regular-expression validation.
- Explain finite values, safe integers, binary floating-point rounding, and BigInt.
- Separate an instant from its local calendar presentation.
- Validate canonical UTC timestamps and half-open time windows.
- Test negative values, overflow, malformed dates, and exact boundary instants.

Prerequisites

Read Conversion and Coercion. You should recognize primitive values, strict equality, logical expressions, and basic function calls. A function declaration groups a calculation under a name; `return` sends its result to the caller. The full functions chapter follows Control Flow.

Examples use arrays as small lists and `throw` to reject invalid arguments. These constructs receive dedicated treatment later in this volume; their local purpose is explained when used here.

Running Guide

Use Node.js 20 or later from the repository root:

```
node code/volume-1/chapter-05/example-00-value-model.js
node code/volume-1/chapter-05/example-01-money-formatting.js
node code/volume-1/chapter-05/example-02-date-window.js
node code/volume-1/chapter-05/example-03-text-number-challenges.js
```

JavaScript fences run independently. Their output is fixed: dates use UTC and numeric examples use an explicit locale. Production applications must select their own display locale; a formatted price is presentation text, not a storage format.

First Experiment

```
const label = 'A\u{1F680}';
const amountCents = 1299;
const instant = new Date('2026-07-06T10:00:00.000Z');

console.log(label.length, Array.from(label).length);
console.log(amountCents / 100);
console.log(instant.toISOString());
// Expected output:
// 3 2
// 12.99
// 2026-07-06T10:00:00.000Z
```

The rocket uses two code units but one code point. Dividing cents produces a Number suitable for the bounded formatter, while storage remains integer cents. The ISO string explicitly identifies UTC.

Continue with theory, then the internal model.

Strings, Numbers, and Dates: Theory

Strings Are Immutable Values

A string is a sequence of UTF-16 code units. Reading an index returns one code unit as a string; methods such as `slice`, `trim`, and `replace` return values instead of editing the original. A binding can be reassigned, but the previously stored string value does not change. Prefer primitive strings to `new String(...)` wrappers, whose object identity changes equality behavior. See the String reference.

```
const raw = '  invoice-042  ';
const clean = raw.trim();
console.log(JSON.stringify(raw));
console.log(clean.toUpperCase());
console.log(clean.slice(-3), clean.includes('invoice'));
// Expected output:
// "  invoice-042  "
// INVOICE-042
// 042 true
```

`slice(start, end)` excludes the end index. A negative index counts from the end. `includes` is case-sensitive. Template literals interpolate values into a string; they do not perform HTML escaping or SQL parameterization.

Three Meanings of Character

Unit	Example operation	Appropriate use
UTF-16 code unit	`text.length`, `text[i]`	JavaScript indexing and offsets
Unicode code point	`Array.from(text)`, `for...of`	Traversal without splitting valid surrogate pairs
Grapheme cluster	`Intl.Segmenter` with `granularity: 'grapheme'`	User-visible cursor or truncation units

A combining accent can be a separate code point while displaying as part of one grapheme. Emoji sequences can contain several code points. String iteration preserves lone surrogates in ill-formed strings; it is not Unicode validation. Neither `length` nor spread guarantees the count a user perceives. Intl.Segmenter supplies segmentation.

```
const text = 'e\u0301\u{1F680}';
const segmenter = new Intl.Segmenter('en', { granularity: 'grapheme' });
console.log(text.length);
console.log(Array.from(text).length);
console.log(Array.from(segmenter.segment(text)).length);
console.log(text.normalize('NFC') === '\u00e9\u{1F680}');
// Expected output:
// 4
// 3
// 2
// true
```

Normalization can make canonically equivalent spellings compare equally. It is a product decision: do not normalize an opaque token or cryptographic signature. Grapheme rules and locale data can evolve; persist original text when preservation matters.

Regular Expressions Match a Language

A regular expression describes accepted text patterns. Anchors constrain the whole string, character classes constrain characters, and quantifiers constrain counts. This identifier accepts 1 through 12 ASCII letters, digits, or hyphens; it intentionally rejects international names and spaces.

```
const idPattern = /^[A-Z0-9-]{1,12}$/;
console.log(idPattern.test('ORD-42'));
console.log(idPattern.test('ORD 42'));
console.log(idPattern.test('ord-42'));
console.log('order=42'.replace(/=(\d+)$/, ':$1'));
// Expected output:
// true
// false
// false
// order:42
```

Do not add `g` to a reusable validation expression: global and sticky tests advance `lastIndex` and may give different results on repeated calls. Capture groups support extraction or replacement; they do not establish business validity. A digit-shaped date still needs calendar validation. See RegExp.test.

Numbers: Representation and Boundaries

Number uses binary floating-point semantics. Many decimal fractions need rounding, so `0.1 + 0.2` is close to, but not exactly, `0.3`. Number also contains `NaN`, infinities, and signed zero.

```
console.log(0.1 + 0.2 === 0.3);
console.log(Number.isFinite('12'), Number.isFinite(12));
console.log(Number.isNaN(Number('unknown')));
console.log(Number.isInteger(2 ** 53), Number.isSafeInteger(2 ** 53));
console.log(Object.is(-0, 0), -0 === 0);
// Expected output:
// false
// false true
// true
// true false
// false true
```

`Number.isFinite` does not convert strings. A safe integer is within `-(2 ** 53 - 1)` through `2 ** 53 - 1`. Some larger integers are representable, but neighboring integer inputs are no longer distinguishable. Validate operands and results; safe inputs do not guarantee a safe sum. Safe-integer reference.

For measured quantities, choose an error tolerance tied to units and scale. `Number.EPSILON` describes spacing near `1`; it is not a universal threshold for all magnitudes. For monetary calculations, choose bounded integer minor units, a decimal representation, or a suitable decimal implementation with an explicit rounding rule. Number.EPSILON.

`Math.floor` rounds toward negative infinity; `Math.trunc` removes the fraction toward zero. This matters for negative adjustments. `toFixed` returns presentation text, not a repaired exact number.

```
console.log(Math.floor(-1.2), Math.trunc(-1.2));
console.log(typeof (12.5).toFixed(2));
console.log((12.5).toFixed(2));
// Expected output:
// -2 -1
// string
// 12.50
```

BigInt Preserves Integer Precision

BigInt represents integers with precision limited by available resources. Construct a large value from a literal ending in `n` or from a validated integer string. Converting an already-rounded Number to BigInt cannot recover lost input digits. Arithmetic generally cannot mix Number and BigInt operands; integer division truncates toward zero. BigInt reference.

```
const externalId = BigInt('9007199254740993');
console.log(String(externalId + 1n));
console.log(String(7n / 2n));
console.log(1n === 1, 1n < 2);
try {
  JSON.stringify({ id: externalId });
} catch (error) {
  console.log(error.name);
}
// Expected output:
// 9007199254740994
// 3
// false true
// TypeError
```

Serialize IDs as strings with an agreed schema. BigInt is not a fractional-decimal type, and ordinary `Math` functions do not accept it.

Dates Represent Instants, Not Stored Time Zones

A Date object contains a time value in milliseconds relative to the UTC epoch. It does not store the zone from its input. Local getters interpret the instant in the host's zone; UTC getters interpret it in UTC. An invalid Date contains an invalid time value. Date setters mutate the object, so aliases see changes. Date reference.

```
const instant = new Date('2026-07-06T15:30:00+05:30');
console.log(instant.toISOString());
console.log(instant.getUTCMonth());
console.log(Number.isNaN(new Date('invalid').getTime()));
// Expected output:
// 2026-07-06T10:00:00.000Z
// 6
// true
```

Months returned by `getUTCMonth` are zero-based. A duration of 86,400,000 milliseconds means 24 elapsed hours; "same local time tomorrow" is a calendar operation and can cross a daylight-saving offset change.

Parsing and Formatting Are Separate Contracts

`Date.parse` is a parser, not a validator for your API schema. Standard date-only strings imply UTC; standard date-time strings without an offset use local time. Other forms may depend on the implementation. Require an explicit accepted grammar at your boundary. Date.parse.

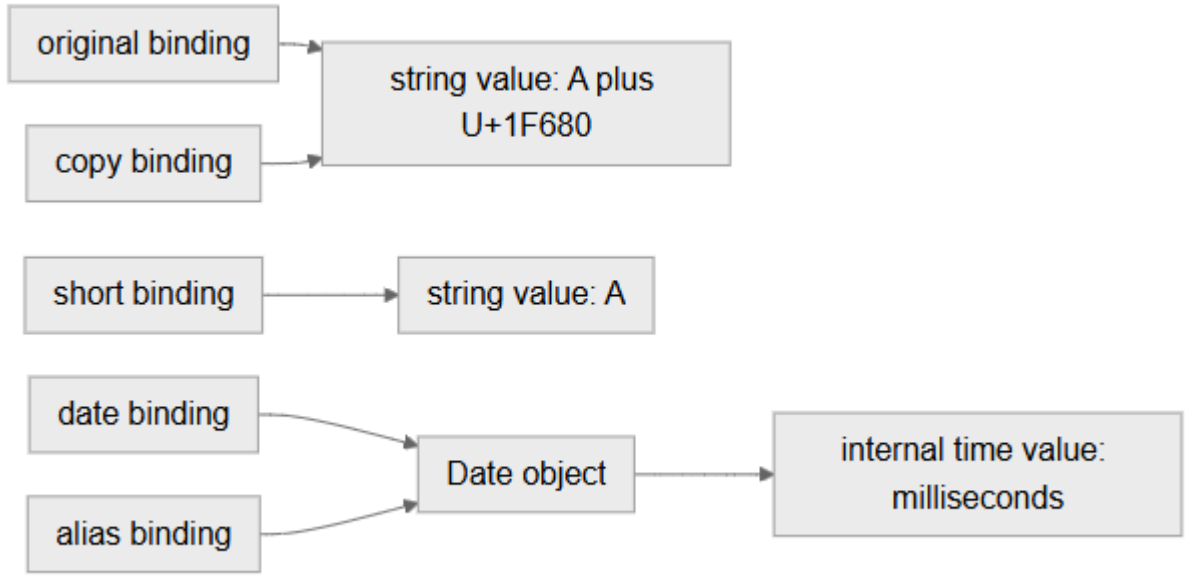
This chapter accepts exactly `YYYY-MM-DDTHH:mm:ss.sssZ`, then checks that parsing and converting back produces identical text. It rejects missing zones, alternate offsets, impossible calendar days, and normalized overflow. It is narrower than all valid ISO 8601 forms.

Use `Intl.DateTimeFormat` and `Intl.NumberFormat` for display. Preserve timestamps and integer units for calculations; do not parse a localized display string back into domain data.

Strings, Numbers, and Dates: Internal Working

Value and Reference Model

The diagram is conceptual: it describes observable relationships, not an engine's exact stack layout or string allocation strategy.



Source: `diagrams/volume-1-chapter-05-value-memory.mmd`.

Copying a string value cannot give another binding a way to mutate it. Copying a Date reference does share a mutable object. A fresh Date constructed from the original timestamp separates that state.

```
const original = 'A\u{1F680}';
const copy = original;
const short = original.slice(0, 1);
const date = new Date('2026-01-01T00:00:00.000Z');
const alias = date;
const snapshot = new Date(date.getTime());

alias.setUTCDate(2);
console.log(copy.length, short);
console.log(date.toISOString());
console.log(snapshot.toISOString());
// Expected output:
// 3 A
// 2026-01-02T00:00:00.000Z
// 2026-01-01T00:00:00.000Z
```

Execution steps:

1. Bind `original` and `copy` to the same string value.
2. Derive the one-code-unit prefix without changing that value.
3. Allocate one Date and bind both `date` and `alias` to it.
4. Allocate a second Date with the copied numeric time value.
5. Mutate the first object's time value; the second keeps its timestamp.

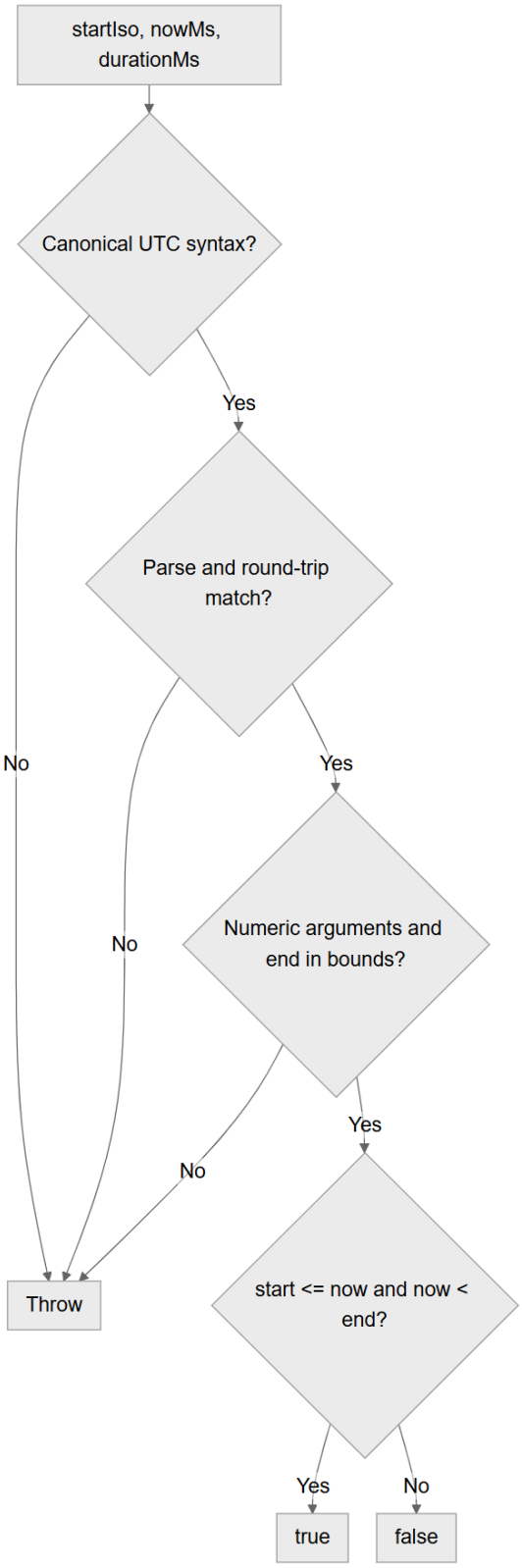
The language permits engines to share immutable string storage or optimize temporary values. Do not infer retention sizes from this picture; measure actual retained memory when that matters.

Binary Arithmetic Trace

In `0.1 + 0.2`, each literal is first represented as a binary floating-point value. Addition operates on those represented values and rounds the result. Decimal formatting later chooses text to represent that result. A formatting step cannot prove that earlier arithmetic matched the domain's intended decimal policy.

By contrast, the formatter's integer `1299` is exact. The accepted range is deliberately bounded. Division by 100 creates an approximation sufficiently precise for the two-decimal display at that bound; it does not change the integer storage value. Applying tax, splitting amounts, and rounding fractional cents require separate contracts.

UTC Window Validation Flow



Source: `diagrams/volume-1-chapter-05-utc-window.mmd`.

For start `2026-07-06T10:00:00.000Z`, now at 10:30 UTC, and duration 3,600,000:

Step	Result	Why it matters
Check fixed grammar	Accepted	Zone and precision are explicit
Parse and round-trip	Same text	Calendar normalization did not change the input
Check numeric arguments	Accepted	Addition cannot concatenate a duration string
Calculate end	11:00 UTC	Bounded integer arithmetic
Compare	`true`	Start is inclusive; end is exclusive

At exactly 11:00 the result is false. Adjacent windows can share that boundary without counting it twice. A zero-duration window contains no instants.

Engine Semantics Versus Optimization

A string method call on a primitive has wrapper-like property access semantics; it does not require a permanent wrapper object to remain allocated. Numeric optimizations must still preserve specified results, including `NaN` and negative zero. Date parsing and international formatting invoke built-in algorithms and runtime data; they are not simple string slicing operations.

For this chapter, correctness depends on the value rules, not on whether an engine uses a compact integer representation, a shared string buffer, or compiled fast paths. Benchmark a real workload before choosing an algorithm based on guessed engine behavior.

See the production examples for the complete boundary implementations.

Strings, Numbers, and Dates: Production Examples

Bounded Two-Decimal Money Display

Contract: accept an integer Number of cents from -1,000,000,000 through +1,000,000,000. Support USD, EUR, GBP, and INR only, all with two fractional digits in this example. Reject fractional, unsafe, nonnumeric, or out-of-range input. Normalize negative zero. Pass an explicit locale; the example uses `en-US`.

This is a display helper. It does not define tax arithmetic, exchange rates, allocation, or support for currencies with other minor-unit scales. The bound keeps conversion to display units comfortably below the precision at which cent differences disappear.

```
function formatMoneyFromCents(cents, locale = 'en-US', currency = 'USD') {
  const allowed = ['USD', 'EUR', 'GBP', 'INR'];
  if (!Number.isSafeInteger(cents) || Math.abs(cents) > 1000000000) {
    throw new RangeError('Expected integer cents within plus or minus 1,000,000,000.');
  }
  if (typeof locale !== 'string' || !allowed.includes(currency)) {
    throw new TypeError('Expected a locale string and a supported two-decimal currency.');
  }
  return new Intl.NumberFormat(locale, {
    style: 'currency',
    currency,
    minimumFractionDigits: 2,
    maximumFractionDigits: 2
  }).format((Object.is(cents, -0) ? 0 : cents) / 100);
}

console.log(formatMoneyFromCents(1299));
console.log(formatMoneyFromCents(-105));
console.log(formatMoneyFromCents(-0));
try {
  formatMoneyFromCents(1000000001);
} catch (error) {
  console.log(error.name);
}
// Expected output:
// $12.99
// -$1.05
// $0.00
// RangeError
```

The companion assertion program is `code/volume-1/chapter-05/example-01-money-formatting.js`. It covers both bounds, unsupported currencies, strings, fractions, infinities, and negative zero.

Input validation happens before formatting. `Intl.NumberFormat` may reject a malformed locale with `RangeError`; callers should choose supported product locales instead of passing arbitrary request values. Reuse a formatter for a fixed locale/currency in a repeated-render path, with a bounded cache if options vary.

Work and output size are bounded here. Constructing a formatter has a cost, but claiming a fixed number of machine operations would be misleading because locale processing belongs to the runtime.

Canonical UTC Access Window

Contract: `startIso` is a four-digit-year canonical UTC timestamp with exactly three fractional digits. `nowMs` is a safe integer Number within the Date range. `durationMs` is an integer Number from zero through seven elapsed days. The calculated end must remain representable and within the Date range. The result describes `[start, end)`.

Accepting the clock value as an argument makes tests deterministic and leaves clock acquisition to the caller. Validation errors throw; a valid time outside the window returns false.

```
function isWithinWindow(nowMs, startIso, durationMs) {
  const dateLimit = 8640000000000000;
  const maxDuration = 7 * 24 * 60 * 60 * 1000;
  const canonicalUtc = /^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}\.\d{3}Z$/;
  if (typeof startIso !== 'string' || !canonicalUtc.test(startIso)) {
    throw new TypeError('Expected YYYY-MM-DDTHH:mm:ss.sssZ.');
  }
  const startMs = Date.parse(startIso);
  if (!Number.isFinite(startMs) || new Date(startMs).toISOString() !== startIso) {
    throw new RangeError('Invalid UTC calendar timestamp.');
  }
  if (!Number.isSafeInteger(nowMs) || Math.abs(nowMs) > dateLimit) {
    throw new RangeError('Expected an integer timestamp within the Date range.');
  }
  if (!Number.isSafeInteger(durationMs) || durationMs < 0 || durationMs > maxDuration) {
    throw new RangeError('Expected an integer duration from zero to seven days.');
  }
  const endMs = startMs + durationMs;
  if (!Number.isSafeInteger(endMs) || Math.abs(endMs) > dateLimit) {
    throw new RangeError('Window end is out of range.');
  }
  return nowMs >= startMs && nowMs < endMs;
}

const start = '2026-07-06T10:00:00.000Z';
const startMs = Date.parse(start);
console.log(isWithinWindow(startMs, start, 3600000));
console.log(isWithinWindow(startMs + 3599999, start, 3600000));
console.log(isWithinWindow(startMs + 3600000, start, 3600000));
console.log(isWithinWindow(startMs, start, 0));
try {
  isWithinWindow(startMs, start, '3600000');
} catch (error) {
  console.log(error.name);
}
// Expected output:
// true
// true
// false
// false
// RangeError
```

The companion `code/volume-1/chapter-05/example-02-date-window.js` also rejects February 30, non-leap February 29, hour 24, missing milliseconds, non-UTC offsets, non-finite clocks, fractional durations, and values beyond the declared limits. Leap-year February 29 is accepted.

Parsing alone can normalize some calendar overflow. The round-trip check compares normalized UTC output with the exact input, so normalization cannot silently change a request. Since the grammar has fixed length, parsing and checking use bounded work and storage.

The four-digit start-year grammar is much narrower than the full Date range. The explicit end check documents a reusable arithmetic invariant; it also prevents a future widening of the grammar from introducing unchecked overflow.

Operational Boundaries

A scheduled local meeting needs a named zone and calendar policy, not this elapsed-duration helper. An authorization system also needs an authoritative clock and server-side enforcement. Formatting or client-side time checks do not enforce access control.

Return normalized domain values across service boundaries: integer minor units plus a currency identifier, or epoch milliseconds plus a documented meaning. Keep display strings at the presentation boundary.

Strings, Numbers, and Dates: Interview Perspective

Beginner: What Is a Character?

A useful answer separates storage from display: JavaScript indexes UTF-16 code units, its string iterator advances by code point, and user-facing text often needs grapheme clusters. Ask which unit the product limit means before writing a truncation helper.

```
const value = 'e\u0301';
console.log(value.length, Array.from(value).length);
console.log(Array.from(new Intl.Segmenter('en', { granularity: 'grapheme' }).segment(value)).length);
// Expected output:
// 2 2
// 1
```

Follow-up: "Would NFC normalization always make length a user-visible count?" No. It can combine some canonically equivalent sequences, but it does not turn all grapheme clusters into single code units.

Intermediate: Why Are Integer Cents Useful?

They make the unit explicit and allow exact integer addition inside a checked range. They do not automatically solve percentage rounding, unsafe totals, currencies with different minor-unit scales, or fractional-cent allocation.

A strong design answer names the accepted bound, validates the calculated total, and defines where rounding occurs. "Use toFixed everywhere" only changes presentation. "Use BigInt" still needs a scale and rounding policy.

Output Prediction: A Date Alias

```
const first = new Date('2026-01-01T00:00:00.000Z');
const second = first;
```

```
second.setUTCFullYear(2027);
console.log(first === second);
console.log(first.getUTCFullYear());
// Expected output:
// true
// 2027
```

Both variables refer to one mutable Date. A separate `new Date(first.getTime())` creates a snapshot of its numeric time value. It does not store a time zone.

Senior: Review an Expiry Check

Suppose code parses a request timestamp and tests `now < start + duration`. Identify four missing decisions:

1. Which timestamp grammar and zone are accepted?
2. Are now and duration validated Numbers with explicit units and bounds?
3. Is calendar overflow rejected after parsing?
4. Are the start and end inclusive or exclusive?

Then explain the operational boundary: the server clock is authoritative for access enforcement. A browser countdown is a display aid. A local recurring appointment needs named-zone calendar rules beyond an elapsed UTC interval.

Design Challenge: Imports With Large IDs

A service receives a 20-digit identifier. Keeping it as a string is sufficient if the service only compares or transmits it. Use BigInt when integer arithmetic is genuinely required, constructing it from validated text. Specify the JSON representation and input-length cap.

This answer avoids unnecessary conversion, protects precision, and bounds resource use. It also distinguishes an identifier from a measured quantity or price.

What a Complete Answer Contains

State the input contract, predict one normal result, demonstrate a failing boundary, and explain the representation that causes it. Use the production examples for concrete implementations and the exercises for practice.

Strings, Numbers, and Dates: Exercises and Coding Challenges

Attempt each contract before reading its solution. Each solution is independent. The companion `code/volume-1/chapter-05/example-03-text-number-challenges.js` asserts normal, empty, type, and boundary cases.

1. Normalize a Display Label

Accept a primitive string, normalize canonical Unicode spelling to NFC, trim its edges, and reject an empty result. Preserve internal spacing and case. Return the label; do not turn a non-string into text. Target O(n) work and output space for n code units.

Hint: Transform only after checking the primitive type; reject the empty transformed result.

Solution

```
function cleanLabel(value) {
  if (typeof value !== 'string') throw new TypeError('Expected text.');
  const clean = value.normalize('NFC').trim();
  if (clean === '') throw new RangeError('Label is empty.');
  return clean;
}
console.log(cleanLabel('  Cafe\u0301  ') === 'Caf\u00e9');
try {
  cleanLabel('   ');
} catch (error) {
  console.log(error.name);
}
// Expected output:
// true
// RangeError
```

Normalization and trimming create a new string. The contract does not collapse internal spaces or case-fold identity-sensitive text. Test an already-normalized label, a combining accent, whitespace only, and a number.

2. Truncate by Grapheme Cluster

Return at most limit user-visible clusters without adding an ellipsis. Require a nonnegative safe integer limit and a primitive string. Do not split a combining sequence or emoji sequence. A zero limit returns an empty string.

Hint: Segment before slicing; a code-point array still does not preserve every grapheme.

Solution

```
function takeGraphemes(text, limit) {
  if (typeof text !== 'string') throw new TypeError('Expected text.');
  if (!Number.isSafeInteger(limit) || limit < 0) throw new RangeError('Invalid limit.');
  const segmenter = new Intl.Segmenter('en', { granularity: 'grapheme' });
  const pieces = Array.from(segmenter.segment(text), entry => entry.segment);
  return pieces.slice(0, limit).join('');
}
console.log(takeGraphemes('e\u0301\u{1F680}Z', 2) === 'e\u0301\u{1F680}');
console.log(JSON.stringify(takeGraphemes('hello', 0)));
// Expected output:
// true
// ""
```

This straightforward solution segments the entire input before slicing: O(n) text work and O(n) temporary storage in the usual traversal model. A streaming loop can stop earlier; the following chapter teaches that control flow. A grapheme is the chosen UI unit, not an encoded-byte limit.

3. Parse a Restricted Price

Accept unsigned ASCII decimal text with no leading zeros except zero itself, exactly two decimal places, and at most six whole digits. Return integer cents. Reject whitespace, signs, exponent notation, booleans, and excess fractional digits. This parser intentionally handles one two-decimal amount syntax.

Hint: Validate the complete decimal grammar, then combine whole and fractional digits as integers.

Solution

```
function parsePrice(text) {
  if (typeof text !== 'string' || !/^(0|[1-9]\d{0,5})\.\d{2}$/.test(text)) {
    throw new TypeError('Expected bounded decimal price text.');
  }
  const [whole, fraction] = text.split('.');
  return Number(whole) * 100 + Number(fraction);
}
console.log(parsePrice('19.05'));
console.log(parsePrice('0.00'));
try {
  parsePrice('19.005');
} catch (error) {
  console.log(error.name);
}
// Expected output:
// 1905
// 0
// TypeError
```

The grammar bounds the result at 99,999,999 cents. Parsing whole and fractional digits separately avoids multiplying an approximate fractional Number by 100. Work and output are bounded by the fixed grammar. This is not a locale parser.

4. Add Safe Inventory Counts

Accept two nonnegative safe-integer Numbers. Return their sum only if it is also safe. Reject strings and reject an overflow even when both inputs separately satisfy the contract.

Hint: Validate both operands and the calculated total.

Solution

```
function addCounts(left, right) {
  if (!Number.isSafeInteger(left) || !Number.isSafeInteger(right) || left < 0 || right < 0) {
    throw new RangeError('Expected nonnegative safe integers.');
  }
  const total = left + right;
  if (!Number.isSafeInteger(total)) throw new RangeError('Count overflow.');
  return total;
}
console.log(addCounts(42, 8));
try {
  addCounts(Number.MAX_SAFE_INTEGER, 1);
} catch (error) {
  console.log(error.name);
}
// Expected output:
// 50
// RangeError
```

The result check is essential: validating only inputs leaves overflow possible. Arithmetic uses constant-sized Number values, so the operation has O(1) work and storage. This helper does not convert external strings.

5. Increment an External Integer ID

Accept 1 through 30 ASCII decimal digits in canonical unsigned form, with zero allowed and no leading zeros otherwise. Increment using BigInt and return a string for JSON transport. The output can have 31 digits when the input is thirty nines.

Hint: Build BigInt directly from accepted text and serialize the result as text.

Solution

```
function nextId(text) {
  if (typeof text !== 'string' || !/^(0|[1-9]\d{0,29})$/.test(text)) {
    throw new TypeError('Expected a canonical bounded integer ID.');
  }
  return String(BigInt(text) + 1n);
}
console.log(nextId('9007199254740993'));
console.log(JSON.stringify({ id: nextId('0') }));
// Expected output:
```

```
// 9007199254740994
// {"id":"1"}
```

Constructing BigInt directly from accepted text preserves digits. Returning a string avoids default JSON serialization failure for BigInt. The 30-digit cap bounds conversion and arithmetic work; without it, BigInt cost grows with operand size.

6. Measure Elapsed Whole Seconds

Accept two integer millisecond timestamps within the Date range, with end at or after start. Return the number of complete elapsed seconds. Reject an interval whose millisecond difference is outside safe-integer range. Treat this as elapsed time, not a local calendar duration.

Hint: Validate the direction and the subtraction result before dividing.

Solution

```
function elapsedSeconds(startMs, endMs) {
  const limit = 8640000000000000;
  if (!Number.isSafeInteger(startMs) || !Number.isSafeInteger(endMs) ||
      Math.abs(startMs) > limit || Math.abs(endMs) > limit || endMs < startMs) {
    throw new RangeError('Invalid interval endpoints.');
  }
  const duration = endMs - startMs;
  if (!Number.isSafeInteger(duration)) throw new RangeError('Interval too large.');
  return Math.floor(duration / 1000);
}
console.log(elapsedSeconds(1000, 3999));
console.log(elapsedSeconds(1000, 1000));
try {
  elapsedSeconds(3000, 1000);
} catch (error) {
  console.log(error.name);
}
// Expected output:
// 2
// 0
// RangeError
```

The direction check makes the duration nonnegative, so floor counts completed seconds. Valid individual Date values can span an unsafe difference, which is why the result also needs checking. This uses O(1) arithmetic and storage.

Strings, Numbers, and Dates: MCQs

Choose one answer per question, then explain the rule before checking the answer.

1. What does `"A\u{1F680}".length` return?

A. 2, because there are two code points B. 3, because the rocket uses a surrogate pair C. 1, because both symbols form a string D. 4, because each visible symbol requires two units

Answer: B. String length counts UTF-16 code units: A uses one and the rocket uses two.

2. Which operation can preserve a base letter plus combining accent when truncating visible text?

A. `slice(0, 1)` B. `split("")` C. `charAt(0)` D. Grapheme segmentation before selecting clusters

Answer: D. Code-unit and code-point operations can separate a combining sequence. Grapheme segmentation matches the stated UI unit.

3. After `const a = " x "; const b = a.trim();`, what is `a`?

A. `" x "` B. `"x"` C. A String wrapper D. `undefined`

Answer: A. trim returns a value. It does not mutate the original string.

4. Why might repeated `/A/g.test("A")` alternate results?

A. The input changes case B. The regex consumes the source string C. Global testing advances lastIndex D. The parser recompiles the expression differently

Answer: C. The regex object holds search position. The original string stays unchanged.

5. Which value satisfies `Number.isSafeInteger`?

A. `"42"` B. `Infinity` C. `42` D. `2 ** 53`

Answer: C. The method requires a Number that is an integer within the safe range; it does not coerce.

6. Why is `0.1 + 0.2 === 0.3` false?

A. Binary representations and arithmetic rounding differ B. The equality operator rounds decimals C. All decimal literals are invalid D. Addition converts to text

Answer: A. The decimal literals map to binary floating-point values; the sum is not the same represented value as 0.3.

7. What does `(2.5).toFixed(2)` return?

A. A BigInt B. The Number `2.50` with stored scale C. The integer `250` D. The string `"2.50"`

Answer: D. toFixed formats text; Number does not retain a decimal display scale.

8. Which constructs the exact integer 9007199254740993?

A. `BigInt(9007199254740993)` B. `BigInt("9007199254740993")` C. `Number("9007199254740993")` D. `parseInt("9007199254740993", 10)`

Answer: B. The string reaches BigInt without first being rounded as a Number.

9. What is `7n / 2n`?

A. `3n` B. `3.5` C. `4n` D. A TypeError

Answer: A. BigInt division truncates the fractional part toward zero.

10. What happens to `JSON.stringify({ id: 1n })` without customization?

A. It emits `{"id":1}` B. It emits `{"id":"1"}` C. It throws TypeError D. It silently removes id

Answer: C. BigInt needs an explicit transport representation such as a decimal string.

11. Which statement describes a Date created with an explicit offset?

A. It stores the original string permanently B. It stores an instant; the original offset is not retained C. Every local getter returns the input zone fields D. It is immutable

Answer: B. The Date time value identifies an instant; getters apply UTC or the host local zone.

12. Why use a parse-and-ISO round-trip after a strict timestamp regex?

A. To change local time to the user locale B. To accept more human date forms C. To restore a missing time zone D. To reject impossible or normalized calendar values

Answer: D. Syntax checks the shape. Round-trip identity checks whether the accepted text denotes that exact canonical calendar instant.

13. At the exact end of `[start, end)`, is now inside?

A. Yes, both boundaries are inclusive B. Only in UTC C. Only when the duration is an integer D. No, the end boundary is exclusive

Answer: D. The interval explicitly excludes its end; a zero-duration interval is empty.

14. What is the best default policy for a duration argument in this chapter?

A. Coerce any truthy input B. Require a bounded nonnegative integer Number C. Use parseInt and ignore trailing text D. Accept strings because Date.parse accepts strings

Answer: B. A numeric domain contract prevents concatenation and rejects fractional or unbounded durations.

15. What is `Math.floor(-1.2)`?

A. `1` B. `-1` C. `-2` D. `-1.2`

Answer: C. Floor moves toward negative infinity; truncation would produce `-1`.

16. Which claim about Number.EPSILON is accurate?

A. It describes spacing near `1`, so tolerance still needs a scale policy B. It is the largest rounding error possible C. It makes every decimal sum exact D. It is a money rounding rule

Answer: A. Measurement comparison needs a domain-specific tolerance; EPSILON alone does not define one.

Strings, Numbers, and Dates: Revision and Summary

Revision Sheet

Question	Rule
What does string `length` count?	UTF-16 code units
What does string iteration yield?	Code points, not necessarily graphemes
Do string methods mutate?	They return values; strings are immutable
How do I count display clusters?	Segment with `Intl.Segmenter` and a defined policy
Why can regex validation alternate?	`g`/`y` tests maintain `lastIndex`
Does "integer" mean "safe integer"?	No; use `Number.isSafeInteger` and domain bounds
Does `toFixed` fix arithmetic?	No; it produces a string

Question	Rule
Is `Number.EPSILON` a universal tolerance?	No; tolerance depends on scale and units
What does BigInt solve?	Exact integer precision beyond the safe Number range
Does BigInt represent fractions?	No; division truncates toward zero
What does a Date store?	A numeric time value, not its input zone
What makes a timestamp acceptable?	A declared grammar plus calendar/range validation
What is `[start, end)`?	Inclusive start and exclusive end
Is one local calendar day always 24 hours?	No; zone-offset changes can alter elapsed time

Explain It in One Minute

"JavaScript text is indexed by UTF-16 code units, so I define whether a limit counts storage units, code points, or user-visible clusters. Numbers use binary floating-point semantics; I validate finiteness, integer safety, and business bounds separately. I keep bounded money in integer minor units and define rounding before calculations. A Date represents an instant; I use explicit UTC input for elapsed-time checks and an explicit zone for presentation."

Review the Boundary Matrix

Before shipping a helper, write down behavior for empty text, combining marks, maximum length, negative zero, NaN, infinity, unsafe integers, malformed timestamps, impossible dates, start-minus-one, exact start, end-minus-one, and exact end. Include only relevant cases, but give each a deliberate policy.

This chapter's formatter is intentionally bounded and restricted to four two-decimal currencies. Its UTC window rejects duration strings and normalized invalid dates. Those constraints are part of the public contract.

Summary

Representation determines which operations are valid. Text processing needs a unit, numeric work needs precision and bounds, and time work needs an instant/calendar distinction. A parser, formatter, and validator each solve a different part of that problem.

Practice with the exercises and MCQs. Continue to Control Flow.

Strings, Numbers, and Dates: Edge Cases and Debugging

A Cut Through a Surrogate Pair

```
const rocket = '\u{1F680}';
const firstUnit = rocket.slice(0, 1);
console.log(firstUnit.length);
console.log(firstUnit.charCodeAt(0).toString(16));
console.log(Array.from(rocket)[0] === rocket);
// Expected output:
// 1
// d83d
// true
```

The first result is a lone surrogate code unit, not the whole rocket. When a bug concerns truncation, log code points or escaped text as well as visible text. A console glyph can hide the distinction. For user-visible limits, inspect grapheme segmentation.

Stateful Regex Validation

```
const stateful = /^OK$/g;
console.log(stateful.test('OK'));
console.log(stateful.test('OK'));
const stable = /^OK$/;
console.log(stable.test('OK'), stable.test('OK'));
// Expected output:
// true
// false
// true true
```

The global regex starts its second test at its saved position. Removing `g` is appropriate for this whole-string validator. Resetting `lastIndex` can also control a deliberately stateful scan, but do not hide that state inside an otherwise stateless-looking validator.

A Safe-Looking Integer Was Already Rounded

```
const parsed = Number('9007199254740993');
console.log(String(parsed));
console.log(Number.isInteger(parsed), Number.isSafeInteger(parsed));
console.log(String(BigInt('9007199254740993')));
// Expected output:
// 9007199254740992
// true false
// 9007199254740993
```

Debug the original input and each conversion, not only the final value. If a caller already supplied a rounded Number, no downstream conversion can infer which original decimal string was intended.

Invalid Date Behaviors Differ by Operation

```
const invalid = new Date('not-a-date');
console.log(Number.isNaN(invalid.getTime()));
console.log(JSON.stringify({ when: invalid }));
try {
  invalid.toISOString();
} catch (error) {
  console.log(error.name);
}
// Expected output:
// true
// {"when":null}
// RangeError
```

A successful JSON serialization does not prove the input was a valid date. Validate before serialization; otherwise an invalid Date can become null and lose the reason for failure.

Debugging the Window Contract

For an unexpected result, inspect `typeof durationMs` first. A string can make addition concatenate. Next compare `startIso` to `new Date(startMs).toISOString()` after checking that parsing succeeded. Finally print start, now, and end as numeric timestamps and test the two inequalities separately.

Use a table of start-minus-one, start, end-minus-one, and end. Add a zero-duration case. Never make a test depend on the wall clock advancing during the test. The production helper accepts now explicitly for that reason.

Locale Output Is Not a Wire Format

Different locale data versions may affect spacing or punctuation. Keep exact display tests scoped to a selected runtime and locale; use semantic values for service contracts. Avoid asserting that every currency has two decimal places: this chapter's formatter uses an explicit allowlist.

For the underlying invalid-time behavior, see Date.toISOString and Date.toJSON.

Strings, Numbers, and Dates: Performance and Security

Bound Input Before Expensive Work

Normalize, segment, or match only text that fits a documented input-size limit. Grapheme segmentation may produce many segments; collecting them with `Array.from` also allocates an array. If only a prefix is required, a loop can stop after the necessary cluster count.

Avoid assuming that string concatenation, slicing, or normalization has one universal allocation strategy. Engines can share or flatten storage. Measure the complete workload and retained objects, especially when keeping a tiny substring of a very large input.

Reuse Formatters Deliberately

Repeatedly creating an `Intl.NumberFormat` or `Intl.DateTimeFormat` for unchanged options adds work. Construct one formatter per fixed configuration. If configuration comes from requests, bound the supported locale/currency combinations before caching; an unbounded cache is a memory-growth policy.

Keep formatting outside calculation loops when the result is only displayed once. Do not optimize away validation at an external boundary because a microbenchmark measures it.

Regex Resource Limits

A pattern with nested overlapping repetition can require excessive backtracking for a near-match. Prefer a simple bounded grammar when the domain permits one, cap input length, and test long failing inputs. A safe result for a short example does not establish a worst-case runtime guarantee.

Do not construct an executable pattern directly from arbitrary user text unless the product intentionally accepts regex syntax and supplies resource controls. Plain-text search often needs a string method instead.

Numeric Integrity

A malformed numeric value can be a correctness or authorization bug: infinity may bypass a range assumption, an unsafe integer may merge distinct IDs, and string concatenation may extend an expiry window. Validate type, syntax where applicable, finite/safe range, units, and computed results.

BigInt permits large values but computation and memory grow with digit count. Length limits belong before conversion. Native BigInt operations should not be assumed constant-time for secret-dependent cryptographic work. See BigInt cryptography notes.

Text Safety Is Contextual

Normalization is not sanitization. Template interpolation is not HTML escaping. A valid identifier grammar does not make text safe for every output context. Use the destination's safe API: text content for ordinary DOM text and parameters for database values.

A normalized label may still contain visually confusing characters. Decide whether labels are display names, trusted IDs, or credentials before applying a policy. Never silently transform a password or signature through a convenience text-cleaning helper.

Clock and Calendar Costs

Reading a wall clock is not a reliable way to benchmark short operations: clock adjustments can change it. Use the runtime's monotonic performance clock for elapsed measurements. For business expiry, use the authoritative system clock with an explicit service policy.

UTC arithmetic avoids host-zone dependence for elapsed windows. It does not implement local recurring schedules, business days, or holiday calendars. Choose the representation from the requirement before optimizing it.

Strings, Numbers, and Dates: Professional Field Guide

Choose the Representation First

Requirement	Representation and policy
Opaque external ID	Validated string; preserve digits and case
Large integer arithmetic	Bounded BigInt with string serialization
Small inventory count	Nonnegative safe Number with checked results
Two-decimal price in this chapter	Bounded integer cents plus supported currency
User-visible label limit	Grapheme policy plus a separate storage-size limit
Exact event instant	Validated UTC timestamp or integer epoch milliseconds
Recurring local appointment	Calendar fields, named zone, and ambiguity policy
Localized presentation	Explicit Intl formatter; keep source domain data

Review a Text Helper

Ask what its unit is and what transformations it promises. Trimming whitespace is often appropriate for a label but not for an opaque token. NFC normalization merges canonical spellings; case conversion can depend on language and does not define a universal identifier policy.

Then examine allocation and limits. Does the helper build all grapheme segments to return a short prefix? That may be fine for a bounded display field, but the limit should be stated.

Review a Numeric Helper

Write the unit beside every argument: cents, milliseconds, seconds, or item count. Check both accepted inputs and intermediate results. If an operation produces fractions, identify who owns the rounding decision and whether totals must reconcile after allocation.

A display function should not silently become an accounting function. The money example supports a narrow, testable contract rather than arbitrary currency precision.

Review a Time Helper

Specify the accepted text grammar, the date-validity check, the supported range, the source of now, and the interval boundaries. Distinguish an invalid request from a valid instant outside the interval. Keep that distinction in logs and error handling.

Test around midnight and year boundaries without relying on the machine's local zone. A named-zone scheduling feature also needs tests at offset transitions; this chapter's UTC elapsed-window helper does not claim that feature.

References

- String: indexing, immutability, and methods.
- Intl.Segmenter: grapheme boundaries.
- Number.isSafeInteger: representable integer limits.
- BigInt: arithmetic and serialization boundaries.
- Date.parse: standard and implementation-dependent parsing behavior.

Further Reading

Read String.normalize before designing a Unicode identity policy. Read Intl.NumberFormat and Intl.DateTimeFormat when building localized presentation.

Next, Control Flow shows how to validate, skip, stop, and accumulate work without losing the domain rules.

CHAPTER 6

Control Flow

Introduction

Control flow determines which operations execute, in what order, and when execution stops. An import can skip an invalid row, stop on a fatal record, or process a normal record. Those decisions look like small `if` statements, but their order encodes a business policy.

This chapter uses a synchronous job batch and an order-state dispatcher. The job batch validates each record, gives a valid fatal marker precedence over cancellation, and returns fresh result records. The dispatcher permits payment before shipping and rejects unsupported events. Neither example performs external work or claims retry, persistence, or asynchronous shutdown behavior.

Learning Objectives

You will learn to:

- Choose branches, switch dispatch, and value-selecting conditional expressions.
- Trace `for`, `while`, and `do...while`, including their update and test positions.
- Distinguish iterable values from enumerable property keys.
- Apply `break`, `continue`, `return`, and labels to the correct target.
- State a loop invariant and a termination argument.
- Validate records before reading fields and preserve deliberate policy precedence.
- Test empty batches, malformed rows, exact limits, and early exits.

Prerequisites

Read Operators and Expressions, Conversion and Coercion, and Strings, Numbers, and Dates. You should recognize booleans, strict equality, short-circuiting, block-scoped variables, and safe integer checks.

Small examples use functions as named groups of statements and arrays as ordered lists. `return` exits a function with a value; `push` appends to an array. The next chapter systematically teaches functions and callbacks, followed by objects and arrays.

Running Guide

Run each companion with Node.js 20 or later from the repository root:

```
node code/volume-1/chapter-06/example-00-control-flow-model.js
node code/volume-1/chapter-06/example-01-job-runner.js
```

```
node code/volume-1/chapter-06/example-02-switch-dispatch.js
node code/volume-1/chapter-06/example-03-control-flow-challenges.js
```

Each JavaScript fence is independent and has exact expected output. Companions include assertions for policy and failure cases.

First Trace

```
const priorities = ['normal', 'cancelled', 'normal', 'fatal', 'normal'];
let processed = 0;
for (const priority of priorities) {
  if (priority === 'fatal') break;
  if (priority === 'cancelled') continue;
  processed++;
}
console.log(processed);
// Expected output:
// 2
```

The cancellation skips one iteration. The fatal entry exits the loop, so the final normal entry is never visited. Read theory to see how each control statement chooses its next step.

Control Flow: Theory

Branches Select Executed Statements

An `if` evaluates its condition and converts the result to Boolean. Only the selected branch executes. An `else if` is another conditional evaluated only after earlier conditions fail. Use explicit comparisons when values such as zero or an empty string are valid data; truthiness alone does not express every domain rule.

```
const remaining = 0;
let label;
if (remaining < 0) {
  label = 'invalid';
} else if (remaining === 0) {
  label = 'sold out';
} else {
  label = 'available';
}
console.log(label);
console.log(remaining === 0 ? 'disable purchase' : 'allow purchase');
// Expected output:
// sold out
// disable purchase
```

The conditional operator chooses a value. `if` chooses a block of work. Prefer a short conditional expression for a simple assignment, and blocks for validation, multiple effects, or an important policy ordering. Braces prevent accidental ambiguity as branches grow.

Guard Clauses Reduce Nesting

A guard rejects or returns for an exceptional path before the main work. Guard order still matters: validate shape before reading nested fields, then validate the domain before causing effects.

```
function shippingLabel(order) {
  if (order === null || typeof order !== 'object') return 'invalid';
  if (order.paid !== true) return 'payment required';
  if (order.shipped === true) return 'already shipped';
  return 'ready';
}
console.log(shippingLabel(null));
console.log(shippingLabel({ paid: true, shipped: false }));
// Expected output:
// invalid
// ready
```

This helper selects a label; it is not a general order schema validator. Production input validation below is stricter. A guard returning false and a guard throwing an error communicate different contracts.

Switch Uses Strict Matching and Allows Fallthrough

`switch` evaluates its discriminant once and looks for a case using strict-equality matching. After a match, execution continues through statements until an exit; later labels do not automatically stop it. Grouped empty cases can deliberately share a body. A `default` handles unmatched values. Switch reference.

```
function category(status) {
  switch (status) {
    case 'queued':
    case 'running':
      return 'active';
    case 'complete':
      return 'finished';
    default:
      return 'unknown';
  }
}
console.log(category('queued'));
console.log(category('complete'));
console.log(category(1));
// Expected output:
// active
// finished
// unknown
```

Use braces around a case body if it declares a block-scoped variable that would conflict with another case. Labels alone do not create separate scopes. A `return` exits the enclosing function, so it also prevents fallthrough; a `break` exits only the switch.

Three Loop Schedules

Loop	Schedule	Good fit
`for (init; test; update)`	Initialize once; test; body; update; repeat	Counted or indexed traversal
`while (test)`	Test; body; repeat	Work controlled by changing state
`do { body } while (test)`	Body; test; repeat	A first attempt is required

A loop's test is checked at its specified point, not continuously while the body executes. A test can be false initially; then `for` and `while` skip their bodies, while `do...while` still runs once. While, do...while.

```
let pretested = 0;
while (false) {
  pretested++;
}
let posttested = 0;
do {
  posttested++;
} while (false);
const visited = [];
for (let index = 0; index < 3; index++) {
  visited.push(index);
}
console.log(pretested, posttested);
console.log(visited.join(','));
// Expected output:
// 0 1
// 0,1,2
```

Write the progress step where every non-exiting path reaches it. In a classic `for` loop, `continue` proceeds to the update expression, then the next test. In a `while` loop, it proceeds to the test; it does not run any skipped update statement in the body. Continue reference.

```
const kept = [];
for (let index = 0; index < 4; index++) {
  if (index === 1) continue;
  kept.push(index);
}
console.log(kept.join(','));
// Expected output:
// 0,2,3
```

Values Versus Property Keys

`for...of` consumes an iterable, such as an array or string. For an ordinary array it visits values; for a string it visits code points. A plain object is not automatically iterable. `for...in` enumerates enumerable string property keys, including inherited ones; it omits symbol keys. For...of, for...in.

```
const stock = Object.create({ inherited: 7 });
stock.own = 3;
const allKeys = [];
for (const key in stock) allKeys.push(key);
const ownValues = [];
for (const key of Object.keys(stock)) ownValues.push(stock[key]);
console.log(allKeys.join(','));
console.log(ownValues.join(','));

const items = ['a', 'b'];
items.note = 'metadata';
const values = [];
for (const item of items) values.push(item);
console.log(values.join(','));
// Expected output:
// own,inherited
// 3
// a,b
```

`Object.keys` supplies own enumerable string keys when that is the intended object traversal. Do not use `for...in` as an array-value loop. A sparse array's ordinary value iterator yields `undefined` for a hole; property enumeration only finds present keys.

`const item` in a `for...of` loop creates a binding for each iteration. It does not freeze an object held by that binding. Copy or construct output records deliberately when shared mutation would be wrong.

Exits Have Different Targets

Statement	Target
`break`	Nearest enclosing loop or switch
`continue`	Next iteration of the nearest enclosing loop
`return value`	Entire current function call
`throw error`	Nearest applicable catch, or outward propagation
`break label`	The enclosing statement with that label
`continue label`	The enclosing loop with that label

A labeled break can exit nested loops without an extra flag. Use a descriptive target and keep its scope small. A labeled continue must target a loop; labels are not general-purpose jumps. Labeled statement reference.

```
const shelves = [['empty', 'book'], ['book']];
let found = 'none';
search:
for (let row = 0; row < shelves.length; row++) {
  for (let column = 0; column < shelves[row].length; column++) {
    if (shelves[row][column] === 'book') {
      found = row + ':' + column;
      break search;
    }
  }
}
console.log(found);
// Expected output:
// 0:1
```

A small search function can often `return` the result instead, making a label unnecessary. Use the construct whose target expresses the requirement most clearly.

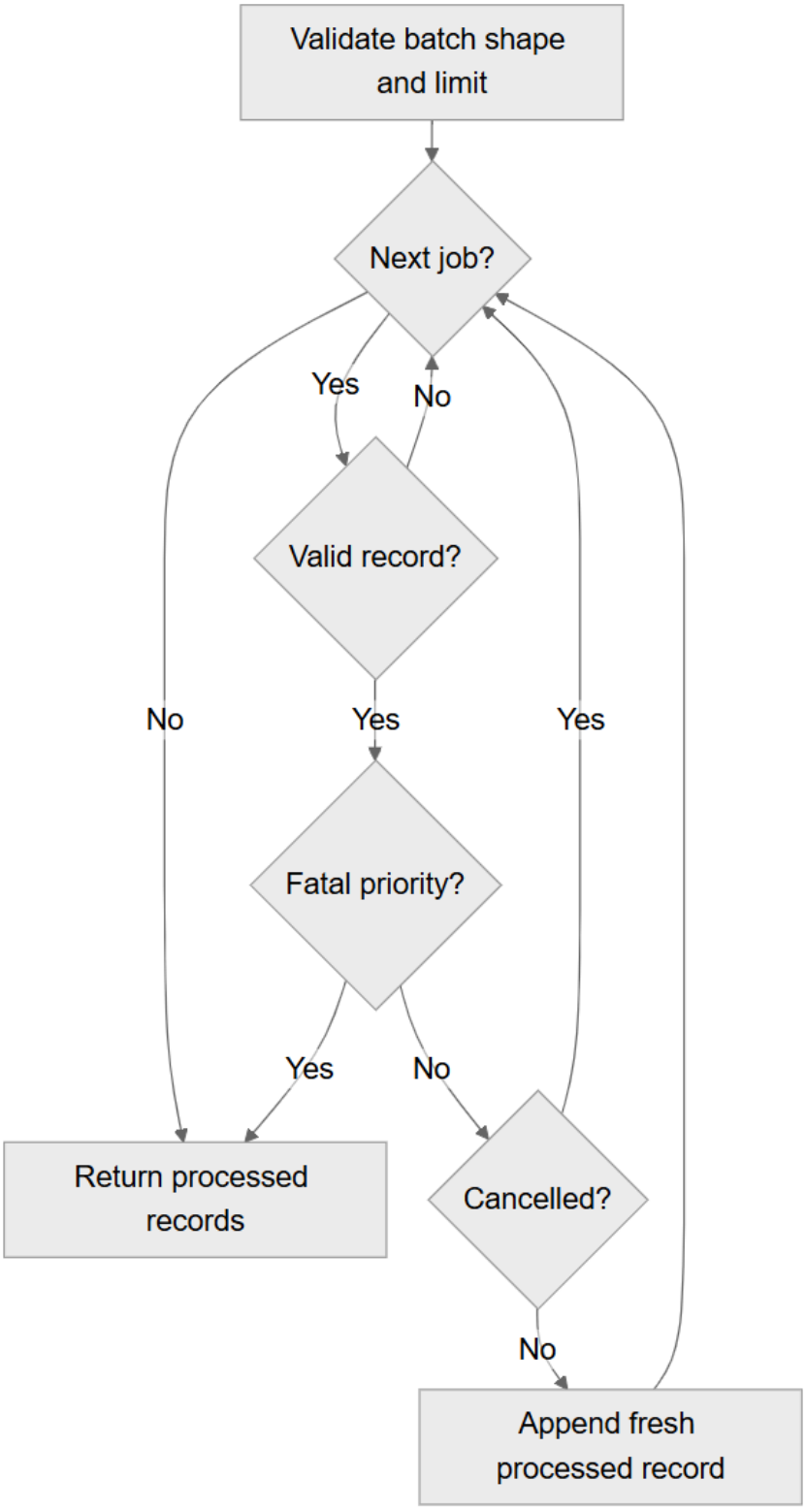
Termination and Invariants

A termination argument states what moves toward an exit. For an indexed scan over an unchanged finite array, the index increases until it reaches length. A loop invariant states what remains true at a boundary: before each job iteration, the output contains only valid, non-cancelled normal jobs from the already-visited prefix.

The invariant explains correctness; the progress argument explains termination. Changing the input while traversing it can invalidate both arguments. The production batch treats its input as an unchanged finite list during the synchronous call.

Control Flow: Internal Working

From a Policy to a Flowchart



Source: `diagrams/volume-1-chapter-06-job-flow.mmd`.

Validation precedes business decisions. A malformed record that happens to contain `priority: 'fatal'` is skipped under this contract. A valid fatal record stops the batch even if its cancellation flag is true. Reversing those two business checks would implement a different policy.

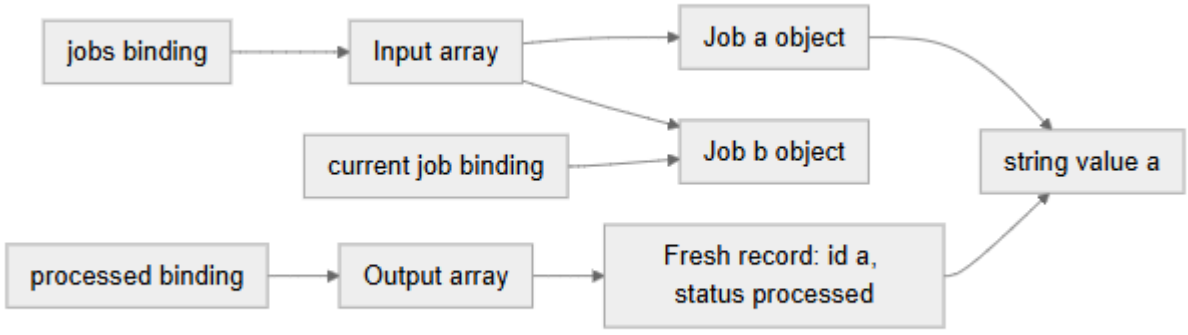
Execution Trace

For a batch containing normal a, null, cancelled b, cancelled fatal c, and normal d:

Visit	Validation	Decision	Output IDs
a	Valid	Append a fresh result	a
null	Invalid	Continue	a
b	Valid	Continue because cancelled	a
c	Valid	Break because fatal	a
d	Not visited	None	a

No output is produced for a fatal marker. The array returned after the loop is the accumulated prefix result. An empty batch returns an empty output.

Reference and Accumulator Model



Source: `diagrams/volume-1-chapter-06-job-memory.mmd`.

The current loop variable refers to the current input object; the loop does not copy it automatically. The implementation creates a fresh result object and copies the immutable ID string into it. Mutating an output record therefore does not mutate an input job record.

The diagram describes observable ownership. It does not claim a particular engine memory layout or allocation count after optimization.

```
const input = [{ id: 'a', cancelled: false }];
const output = [];
for (const job of input) {
  output.push({ id: job.id, status: 'processed' });
}
output[0].id = 'changed';
console.log(input[0].id);
console.log(input[0] === output[0]);
// Expected output:
// a
// false
```

A Classic For Loop's Next Instruction

```
const trace = [];
for (let index = 0; index < 3; index++) {
  trace.push('visit ' + index);
  if (index === 1) continue;
  trace.push('keep ' + index);
}
console.log(trace.join(' | '));
// Expected output:
// visit 0 | keep 0 | visit 1 | visit 2 | keep 2
```

At index 1, `continue` skips the rest of the body, but the update expression still increments index. At index 3, the test fails and the loop completes. Moving the update into a skipped path of a `while` loop can remove this progress guarantee.

Iterator and Completion Semantics

Conceptually, `for...of` obtains an iterator, repeatedly requests a step, checks whether it is done, and binds the yielded value for the body. This explains why it works with several collection types without interpreting their property names.

The specification models control transfers using completion records, including normal, break, continue, return, and throw. A construct consumes the completions it handles and lets others propagate. This is a language model, not a requirement that an engine allocate a JavaScript object per statement. ECMAScript statements and declarations.

On an early exit from `for...of`, iterator closing can invoke an iterator's `return` method. Generators and custom iterators are covered later; the practical rule here is that loop exit can have cleanup semantics, so manually replacing iteration with indexed access is not always equivalent.

Engine Internals and Cost

An engine can compile branches and loops into different machine instructions while preserving observable evaluation order. A switch does not promise a jump table, and source-level nesting does not prove poor performance.

For n input jobs, the batch performs at most n visits and creates at most n result records: O(n) work and O(n) output space, with constant-sized checks under the bounded ID contract. A fatal record may stop earlier. The output array is the main retained growth; control statements themselves do not produce one output per branch.

Continue with the production contracts.

Control Flow: Production Examples

Validated Synchronous Job Batch

Input contract: an array of at most 10,000 ordinary data records. A valid job has its own nonempty string `id` of at most 128 code units with no surrounding whitespace, its own boolean `cancelled`, and its own `priority` equal to `'normal'` or `'fatal'`. Missing or malformed records are skipped. Duplicate IDs represent separate occurrences.

Policy: skip invalid entries first. A valid fatal marker stops the batch, including when it is also cancelled. Otherwise skip a cancelled job and append a fresh processed record for a normal job. Do not mutate the input. The helper accepts ordinary data objects from a trusted decoding layer, not hostile getters or proxies.

The example records which jobs would be processed. It does not execute callbacks, retry work, persist results, or monitor an asynchronous shutdown signal.

```
function processJobs(jobs) {
  if (!Array.isArray(jobs)) throw new TypeError('Expected an array.');
  if (jobs.length > 10000) throw new RangeError('Batch limit is 10,000.');
  const processed = [];
  for (const job of jobs) {
    const valid = job !== null && typeof job === 'object' && !Array.isArray(job) &&
      Object.hasOwn(job, 'id') && typeof job.id === 'string' &&
      job.id.length > 0 && job.id.length <= 128 && job.id.trim() === job.id &&
      Object.hasOwn(job, 'cancelled') && typeof job.cancelled === 'boolean' &&
      Object.hasOwn(job, 'priority') && (job.priority === 'normal' || job.priority === 'fatal');
    if (!valid) continue;
    if (job.priority === 'fatal') break;
    if (job.cancelled) continue;
    processed.push({ id: job.id, status: 'processed' });
  }
  return processed;
}

const jobs = [
```

```
  { id: 'a', cancelled: false, priority: 'normal' },
  null,
  {},
  { id: 'b', cancelled: true, priority: 'normal' },
  { id: 'stop', cancelled: true, priority: 'fatal' },
  { id: 'after', cancelled: false, priority: 'normal' }
];
console.log(JSON.stringify(processJobs(jobs)));
console.log(JSON.stringify(processJobs([null, {}])));
console.log(jobs[0].id);
// Expected output:
// [{"id":"a","status":"processed"}]
// []
// a
```

The companion `code/volume-1/chapter-06/example-01-job-runner.js` asserts malformed entries, inherited fields, invalid fatal-like records, fatal/cancel precedence, empty input, duplicate IDs, batch limits, and output ownership.

Why This Order Is Deliberate

Accessing a field before shape validation caused the original null-record failure. Treating an absent ID as valid produced undefined results. Here a single validity expression short-circuits before unsafe field reads and a guard skips records that fail.

Fatal precedes cancellation because the batch policy says a valid stop marker must be observed. The inverse order would allow a cancelled fatal marker to be ignored. Write a test containing both flags; separate fatal and cancelled tests cannot detect that precedence bug.

The traversal is O(n) in batch length and O(k) output space for k accepted jobs before the stop marker. No callback mutates the list during traversal under this contract.

Order State Dispatcher

Contract: state has own boolean `paid` and `shipped` fields, with shipped implying paid. An event has its own string `type`. `PAID` marks payment; `SHIPPED` requires payment. Repeating a valid event is allowed and returns an equivalent fresh state. Unknown events and invalid transitions throw.

Return only the two modeled state fields. Extra input metadata is outside this state projection. The function selects a next state; persistence and concurrent update control belong to its caller.

```
function reduceOrderState(state, event) {
  if (state === null || typeof state !== 'object' || Array.isArray(state) ||
      !Object.hasOwn(state, 'paid') || typeof state.paid !== 'boolean' ||
      !Object.hasOwn(state, 'shipped') || typeof state.shipped !== 'boolean') {
    throw new TypeError('Expected paid and shipped booleans.');
  }
  if (state.shipped && !state.paid) throw new RangeError('Shipped requires paid.');
  if (event === null || typeof event !== 'object' || Array.isArray(event) ||
      !Object.hasOwn(event, 'type') || typeof event.type !== 'string') {
    throw new TypeError('Expected an event type.');
  }
```

```
  switch (event.type) {
    case 'PAID':
      return { paid: true, shipped: state.shipped };
    case 'SHIPPED':
      if (!state.paid) throw new RangeError('Pay before shipping.');
      return { paid: true, shipped: true };
    default:
      throw new RangeError('Unsupported event type.');
  }
}

const initial = { paid: false, shipped: false };
const paid = reduceOrderState(initial, { type: 'PAID' });
const shipped = reduceOrderState(paid, { type: 'SHIPPED' });
console.log(JSON.stringify(initial));
console.log(JSON.stringify(shipped));
try {
  reduceOrderState(initial, { type: 'SHIPPED' });
} catch (error) {
  console.log(error.name);
}
// Expected output:
// {"paid":false,"shipped":false}
// {"paid":true,"shipped":true}
// RangeError
```

Returning from each case prevents fallthrough. A default branch makes unsupported events explicit instead of silently leaving state unchanged. The companion `code/volume-1/chapter-06/example-02-switch-dispatch.js` verifies that both input state and event remain unchanged.

This dispatcher has a fixed number of branches and returns a fixed-size object: O(1) work and storage. A larger state model may need a transition table and separate validation, but that is a design decision rather than an automatic switch optimization.

What These Examples Establish

Both examples make branch order reviewable, validate before field-dependent decisions, and define whether invalid input is skipped or rejected. A real worker adds operational behavior only with separate contracts and tests for side effects, retry limits, idempotency, and shutdown. Those features cannot be inferred from a loop alone.

Control Flow: Interview Perspective

Beginner: Trace Continue and Break

```
const seen = [];
for (let index = 0; index < 6; index++) {
  if (index === 2) continue;
  if (index === 4) break;
```

```
  seen.push(index);
}
console.log(seen.join(','));
// Expected output:
// 0,1,3
```

At 2, continue skips the append but still reaches the for update. At 4, break exits the loop before appending. Index 5 is never visited. A good explanation names both the skipped statements and the next executed step.

Intermediate: A Switch Inside a Loop

```
const seen = [];
for (const value of ['skip', 'keep']) {
  switch (value) {
    case 'skip':
      break;
    default:
      seen.push('case');
  }
  seen.push('after');
}
console.log(seen.join(','));
// Expected output:
// after,case,after
```

The break exits the switch, so code after the switch still runs. If the intended policy is to skip the rest of the loop iteration, use continue in that context. If the policy is to leave the whole function, return expresses that target.

Intermediate: Values or Keys?

For an array of jobs, use `for...of` for values. For a dictionary, use `Object.keys` or `Object.entries` for own enumerable properties. `for...in` also considers inherited enumerable string keys, which can be surprising for data processing.

Follow-up: "Does const inside for...of make a job immutable?" No. Each iteration has a binding whose object can still be mutated. Show whether the output should retain the original object or create a separate result.

Senior: Review a Batch Policy

State the precedence explicitly: malformed records are skipped; valid fatal markers stop; other cancelled jobs are skipped; remaining normal jobs become outputs. Then prove the invariant: every result corresponds to a valid, normal, non-cancelled record before the first valid fatal marker.

A useful test contains a cancelled fatal record followed by a normal record. If the normal record is processed, the implementation ignored the intended stop policy.

Design Challenge: Termination Under Mutation

A loop appends to the same array it is traversing. Ask whether newly appended items are part of the work set. Without a fixed snapshot or explicit count budget, an iterator can observe continuing growth and fail to finish.

For a fixed batch, forbid mutation during traversal or process a snapshot according to a documented shallow-copy policy. For a queue, design an explicit stopping condition, maximum work budget, and scheduling boundary. Calling a growing queue "just an array loop" hides those requirements.

How to Explain Complexity

Count visited records and output records. An early break improves some inputs but does not remove the worst-case linear scan. Nested loops over a rectangular matrix with r rows and c columns visit at most r times c cells; ragged matrices are better described by total cell count plus row overhead.

Do not claim that switch is always faster than if. Runtime strategy and workload determine performance; branch correctness comes first.

Control Flow: Exercises and Coding Challenges

Write each contract and trace a boundary input before reading the solution. The companion `code/volume-1/chapter-06/example-03-control-flow-challenges.js` includes assertions for all six exercises.

1. Classify Inventory With Guards

Accept a nonnegative safe-integer count and a positive safe-integer low-stock threshold. Return sold-out for zero, low below the threshold, and available at or above it. Reject invalid numeric arguments.

Hint: Validate before comparing; zero must have its own branch.

Solution

```
function stockState(count, threshold) {
  if (!Number.isSafeInteger(count) || count < 0 ||
      !Number.isSafeInteger(threshold) || threshold <= 0) {
    throw new RangeError('Invalid inventory arguments.');
  }
  if (count === 0) return 'sold-out';
  if (count < threshold) return 'low';
  return 'available';
}
console.log(stockState(0, 5));
console.log(stockState(4, 5));
console.log(stockState(5, 5));
// Expected output:
```

```
// sold-out
// low
// available
```

The ordering makes the boundary explicit. Zero is not simply another low value. Time and additional space are O(1). Test zero, threshold minus one, exact threshold, negative input, and numeric strings.

2. Sum Valid Counts Until a Stop Marker

Accept an array. Stop at the first null. Before it, skip entries that are not nonnegative safe integers. Sum accepted counts and reject an unsafe total. Do not examine later entries after null.

Hint: Place the stop check before the validity guard; validate the running result too.

Solution

```
function sumUntilStop(values) {
  if (!Array.isArray(values)) throw new TypeError('Expected an array.');
  let total = 0;
  for (const value of values) {
    if (value === null) break;
    if (!Number.isSafeInteger(value) || value < 0) continue;
    const next = total + value;
    if (!Number.isSafeInteger(next)) throw new RangeError('Total overflow.');
    total = next;
  }
  return total;
}
console.log(sumUntilStop([2, '3', -1, 4, null, 99]));
console.log(sumUntilStop([]));
// Expected output:
// 6
// 0
```

The invariant is that total equals the accepted prefix sum. Each iteration either exits or advances the finite iterator. Time is O(n) in visited entries; additional space is O(1). A null-first input returns zero.

3. Count a Closed Set of States

Accept an array containing only queued, running, or done strings. Return an object containing the count of each state. Reject an unsupported value, including numbers. Do not mutate the input.

Hint: Use one switch case per state and an explicit default error.

Solution

```
function countStates(states) {
  if (!Array.isArray(states)) throw new TypeError('Expected an array.');
  const counts = { queued: 0, running: 0, done: 0 };
```

```
  for (const state of states) {
    switch (state) {
      case 'queued': counts.queued++; break;
      case 'running': counts.running++; break;
      case 'done': counts.done++; break;
      default: throw new RangeError('Unsupported state.');
    }
  }
  return counts;
}
console.log(JSON.stringify(countStates(['queued', 'done', 'queued'])));
try {
  countStates(['unknown']);
} catch (error) {
  console.log(error.name);
}
// Expected output:
// {"queued":2,"running":0,"done":1}
// RangeError
```

Each break leaves the switch and then the loop continues. O(n) time and O(1) result size follow from the fixed three-state vocabulary. Without a break, one entry could increment several counters.

4. Find the First Available Seat

Accept a matrix whose rows are arrays and whose cells are booleans: true means available. Search rows and columns in ascending index order and return the first coordinate, or null. Validate all rows and cells before searching so invalid data is rejected even after an early match. Empty or ragged rows are allowed.

Hint: Keep schema validation separate from the early-exit search.

Solution

```
function firstSeat(rows) {
  if (!Array.isArray(rows)) throw new TypeError('Expected rows.');
  for (const row of rows) {
    if (!Array.isArray(row)) throw new TypeError('Expected a row array.');
    for (const cell of row) {
      if (typeof cell !== 'boolean') throw new TypeError('Expected boolean seats.');
    }
  }
  for (let row = 0; row < rows.length; row++) {
    for (let column = 0; column < rows[row].length; column++) {
      if (rows[row][column]) return { row, column };
    }
  }
  return null;
}
console.log(JSON.stringify(firstSeat([[false], [], [false, true]])));
console.log(firstSeat([[false]]));
// Expected output:
// {"row":2,"column":1}
// null
```

A return exits both loops because it exits the function. Validation makes the acceptance contract independent of search order. For r rows and c total cells, time is O(r + c), including empty rows; extra space is O(1).

5. Build Inclusive Page Ranges

Accept a total from 0 through 1000 and a page size from 1 through 1000, both safe-integer Numbers. Return inclusive one-based start/end ranges with no overlap or missing item. A total of zero yields an empty array.

Hint: Advance the start by page size and clamp each end to total.

Solution

```
function pageRanges(total, size) {
  if (!Number.isSafeInteger(total) || total < 0 || total > 1000 ||
      !Number.isSafeInteger(size) || size < 1 || size > 1000) {
    throw new RangeError('Invalid page arguments.');
  }
  const ranges = [];
  for (let start = 1; start <= total; start += size) {
    ranges.push({ start, end: Math.min(start + size - 1, total) });
  }
  return ranges;
}
console.log(JSON.stringify(pageRanges(7, 3)));
console.log(JSON.stringify(pageRanges(0, 3)));
// Expected output:
// [{"start":1,"end":3},{"start":4,"end":6},{"start":7,"end":7}]
// []
```

Positive size proves progress, and the bounds prevent unsafe arithmetic. If p pages are returned, time and output space are O(p). Check exact multiples, a final partial page, zero total, and a rejected zero size.

6. Apply a Bounded Attempt Budget

Given an array of boolean outcomes and a safe-integer budget from 0 through 100, inspect at most that many outcomes. Return attempts and succeeded, stopping at the first true. Validate the entire outcomes array first. This models decisions from recorded outcomes; it performs no external operation or retry.

Hint: Increment attempts before the success check so every examined outcome is counted.

Solution

```
function inspectAttempts(outcomes, budget) {
  if (!Array.isArray(outcomes)) throw new TypeError('Expected outcomes.');
  if (!Number.isSafeInteger(budget) || budget < 0 || budget > 100) {
    throw new RangeError('Invalid budget.');
  }
```

```
  for (const outcome of outcomes) {
    if (typeof outcome !== 'boolean') throw new TypeError('Expected booleans.');
  }
  let attempts = 0;
  while (attempts < budget && attempts < outcomes.length) {
    const succeeded = outcomes[attempts];
    attempts++;
    if (succeeded) return { attempts, succeeded: true };
  }
  return { attempts, succeeded: false };
}
console.log(JSON.stringify(inspectAttempts([false, true, true], 3)));
console.log(JSON.stringify(inspectAttempts([true], 0)));
// Expected output:
// {"attempts":2,"succeeded":true}
// {"attempts":0,"succeeded":false}
```

Both budget and finite input length bound the loop. Validation takes O(n) time for n outcomes; the decision scan examines at most min(n, budget), and extra space is O(1). Zero budget performs no attempts.

Control Flow: MCQs

Choose one answer and identify the execution rule that supports it.

1. When an if condition is false, which branch executes?

A. Both branches, with the result discarded B. Neither branch even if else exists C. Only the selected else branch, if present D. The branch with fewer statements

Answer: C. Only the chosen branch executes; an absent else means no branch body runs.

2. Which construct naturally selects a short expression value?

A. The conditional operator B. A for...in loop C. A switch case label D. A labeled block

Answer: A. The conditional operator yields one of two expression results. A branch statement is better for a multi-step action.

3. How does switch compare its discriminant with a case value?

A. Object.is matching B. Loose equality with coercion C. String conversion for every case D. Strict-equality matching

Answer: D. A Number 1 does not match string "1"; NaN also does not match NaN as a case value.

4. What happens after a matched case body without break, return, or throw?

A. The switch restarts B. Execution continues through following statements C. Only matching later cases execute D. The engine inserts an implicit break

Answer: B. Fallthrough follows statements regardless of later labels.

5. How often can a do...while body run if its condition is initially false?

A. Zero times B. It throws before running C. Until the condition becomes true D. Once

Answer: D. The first test follows the first body execution.

6. In a classic for loop, where does continue transfer control?

A. Immediately after the loop B. Back to initialization C. To the update expression and then the test D. To the start of the body without a test

Answer: C. The update still runs; initialization happens only once.

7. What does continue do in a while loop?

A. Proceed to the next condition test B. Run every skipped statement first C. Increment every numeric variable D. Exit the function

Answer: A. An update skipped in the body is not automatic, which can prevent progress.

8. What does for...of over an ordinary array provide?

A. Enumerable property names B. Element values through its iterator C. Only own non-index properties D. Every inherited value

Answer: B. Array value iteration follows the iterator, not property enumeration.

9. Which keys does for...in include?

A. Only own symbol keys B. Enumerable string keys, including inherited ones C. Only numeric indices as Numbers D. All properties, including non-enumerable ones

Answer: B. Use Object.keys when own enumerable string keys are intended.

10. What is the ordinary array value iterator result for a hole?

A. It stops the loop B. It throws ReferenceError C. It skips the index without yielding D. It yields undefined

Answer: D. A hole differs from an absent iteration step for ordinary array value iteration.

11. A break inside a switch inside a loop exits what?

A. The function B. The outer loop C. The switch D. Both switch and loop

Answer: C. Unlabeled break targets the nearest enclosing switch or loop.

12. What can continue label target?

A. An enclosing labeled loop B. Any enclosing function C. Any earlier source line D. Any labeled block

Answer: A. Continue must select an iteration target; it is not a general goto.

13. Does const job in for...of freeze each job object?

A. Yes, including nested data B. Only when the input array is const C. No; it prevents binding reassignment within that iteration D. Only if the loop has no continue

Answer: C. Binding immutability and object mutability are separate.

14. Why test a record that is both valid-fatal and cancelled?

A. To prove it is an array B. To verify which policy has precedence C. To ensure all branches execute twice D. To test numeric overflow

Answer: B. Separate cancellation and fatal tests do not expose an ordering bug when both conditions hold.

15. Which is a loop invariant for the validated job batch?

A. Output contains only accepted records from the visited prefix B. The loop will always visit every input C. The current job can never be invalid D. The output shares every input object

Answer: A. The invariant remains true after append, skip, and exit paths; fatal records can shorten traversal.

16. What establishes termination for an unchanged finite indexed scan?

A. Using const for the array binding B. Writing the loop in a single line C. Adding a console log inside the loop D. A bounded index advances toward length on each continuing path

Answer: D. Progress and a bound form the argument. Syntax style alone does not ensure termination.

Control Flow: Revision and Summary

Revision Sheet

Construct	Rule to remember
`if / else`	Evaluate a condition and execute only its selected branch
Conditional operator	Select one expression value
`switch`	Strictly match cases; statements fall through without an exit
Case with declarations	Use braces when an independent lexical scope is needed
`for`	Initialize, test, body, update, test again
`while`	Test before each body execution
`do...while`	Run the body once before the first test
`for...of`	Consume iterable values
`for...in`	Enumerate enumerable string keys, including inherited keys
`Object.keys`	Produce own enumerable string keys
`break`	Exit the nearest loop or switch
`continue`	Move to the next iteration; classic for still runs its update
`return`	Exit the current function
Labeled break	Exit the named enclosing statement
Invariant	A statement that remains true at a chosen loop boundary
Termination argument	A bounded measure moves toward completion

One-Minute Explanation

"I choose a branch structure that makes the policy order visible. I validate a record before reading fields, distinguish skip from stop, and state which condition has precedence. For loops, I identify the visited unit, the progress step, the exit, and the ownership of output records. I use values iteration for arrays and deliberate own-key enumeration for records."

Trace Checklist

For any loop, answer:

1. Can the body run zero times?
2. What happens after continue?
3. What does break exit here?
4. Which state changes on every non-exiting path?
5. Can the traversed collection change?
6. What has been accumulated before the current iteration?
7. What happens at the first invalid, fatal, or boundary record?

Summary

Control flow translates a policy into an execution path. Different loops have different test and update schedules; exits have different targets. Correct code needs both a valid result and an argument that processing ends.

The batch example skips invalid records, prioritizes valid fatal markers over cancellation, and creates separate output records. The order dispatcher enforces payment before shipping. Neither claims external processing or retries.

Review the exercises and MCQs. Continue to Functions and Callbacks.

Control Flow: Edge Cases and Debugging

Missing a While Update

A continue placed before the only increment can make a while loop repeat the same input forever. Keep this demonstration safe by placing the update before the guard:

```
const values = [0, 2, 0, 4];
let index = 0;
let sum = 0;
while (index < values.length) {
  const value = values[index];
```

```
  index++;
  if (value === 0) continue;
  sum += value;
}
console.log(index, sum);
// Expected output:
// 4 6
```

To debug a hanging loop, inspect the condition variables and every continue path. Set a breakpoint at the loop test and confirm that at least one bounded progress measure changes. Do not reproduce an unbounded loop in a shared runtime merely to demonstrate the failure.

Accidental Fallthrough

```
const actions = [];
switch ('paid') {
  case 'paid':
    actions.push('payment');
    // Deliberate fallthrough for this demonstration.
  case 'shipped':
    actions.push('shipment');
    break;
}
console.log(actions.join(','));
// Expected output:
// payment,shipment
```

The shipment statement runs even though the discriminant is paid. A real transition handler should return or break unless shared execution is part of the policy. A comment documents intended fallthrough; it does not make unintended behavior correct.

Sparse Arrays and Extra Properties

```
const list = [];
list[1] = 'job';
list.note = 'metadata';
const keys = [];
for (const key in list) keys.push(key);
const values = [];
for (const value of list) values.push(String(value));
console.log(keys.join(','));
console.log(values.join(','));
// Expected output:
// 1,note
// undefined,job
```

Different iteration forms traverse different abstractions. Pick one based on the contract; do not assume changing loop syntax preserves which entries are visited.

Mutating a Collection During Traversal

```
const queue = ['a'];
const visited = [];
for (const item of queue) {
  visited.push(item);
  if (item === 'a') queue.push('b');
}
console.log(visited.join(','));
// Expected output:
// a,b
```

The ordinary array iterator observes the appended element here. An append on every iteration could prevent completion. A fixed-batch function should establish its no-mutation assumption or use a deliberate snapshot; an evolving queue needs an explicit budget.

Labels Make the Target Explicit

```
const accepted = [];
rows:
for (const row of [[1, 2], [3, -1], [4]]) {
  for (const value of row) {
    if (value < 0) continue rows;
  }
  accepted.push(row.join(':'));
}
console.log(accepted.join(','));
// Expected output:
// 1:2,4
```

The labeled continue skips the rest of the outer row iteration. It does not merely continue the inner value loop. A helper returning whether the row is acceptable could express the same policy when it improves readability.

Debugging Checklist

Build a tiny input containing one normal, one malformed, one cancelled, and one fatal record. Include a record satisfying competing conditions. Record the branch taken and the output length after each visit.

Distinguish an invalid batch from an invalid entry: the production batch rejects a non-array but skips a malformed record. Check mutation separately by comparing input state before and after and by changing a returned record in a test.

Control Flow: Performance and Security

Count Work, Then Measure

A single scan is O(n) in visited entries. Creating one result object per accepted entry adds O(k) output space. A nested scan over every pair may be O(n squared); an early break reduces some executions but leaves the worst case unchanged.

Prefer an algorithmic reduction before a syntax micro-optimization. Searching repeatedly through the same large list may justify an index or Set, taught in the collections chapter. Replacing for with while alone does not change the amount of work.

Bound Synchronous Processing

A finite array can still be too large to process responsively in one synchronous call. Put explicit limits at input boundaries. The job example accepts at most 10,000 records, making its operational expectation visible.

A while loop that waits for an external signal without yielding cannot let queued JavaScript callbacks run in the same thread. Real cancellation and scheduling require cooperation with the host and asynchronous design. The synchronous example therefore makes no claim of live shutdown handling.

Preserve Validation Order

Validate a record before using its fields for a decision. An inherited property or unexpected type should not silently satisfy an authorization or transition rule. The examples require own modeled fields and exact booleans or strings.

These checks are written for ordinary data records. Arbitrary objects can contain getters or proxies whose property access executes code. Decode and validate untrusted serialized input into your accepted data model; a series of field reads is not a sandbox.

Make Resource Growth Visible

A loop can grow its own input, append unbounded output, or create repeated temporary copies. Identify the retained structures in a review. Do not assume that an early-exit branch bounds output unless every path enforces that limit.

For a queue that intentionally grows, define both a work budget and what happens to unprocessed items. A cap that silently discards remaining work changes the business contract.

Avoid Timing-Based Correctness

The result of a branch should depend on validated state, not a guessed machine speed. Retry and timeout policies need explicit budgets, clocks, and failure semantics; a large busy loop is not a delay primitive.

Benchmark representative accepted and rejected inputs. Keep logging out of a hot loop when it dominates runtime, but preserve useful aggregate counts or diagnostics at the surrounding boundary.

Security Is in the Policy

A branch can correctly execute the wrong policy. Test combinations: paid and shipped states, fatal and cancelled flags, a missing required ID, and an inherited field. A single happy path does not establish that an access or workflow check is sound.

Unknown transitions should have a deliberate default. The dispatcher rejects them so they cannot silently advance or overwrite state.

Control Flow: Professional Field Guide

Choose the Construct From the Decision

Requirement	Useful starting point
Several ordered validation checks	Guard clauses
One simple selected value	Conditional expression
Dispatch on a closed event vocabulary	Switch with explicit default
Indexed work with clear bounds	Classic for
Consume iterable values	For...of
Own object fields	Object.keys or Object.entries
Repeat while state permits	While with a visible progress step
Perform at least one attempt	Do...while
Return the first search result	Small function with early return
Exit a tightly scoped nested traversal	Descriptive labeled break

Review Loop Correctness

Name the input unit, invariant, progress measure, exit target, and output ownership. For the batch example, the unit is a job; the invariant is the accepted prefix; progress comes from the finite iterator; fatal exits the loop; outputs are fresh records.

Then ask how each assumption could fail. A malformed object can break a field read. A continue can skip a while update. Appending to the traversed input can change termination. Sharing original records can make later callers mutate your source data.

Review Policy Precedence

Write competing conditions as a small decision table. Valid fatal plus cancelled means stop in this chapter. Invalid plus fatal-looking means skip. Those are application choices, not universal JavaScript rules.

Keep the table beside the tests when the policy matters. Rearranging guards for style can alter precedence even when every individual condition remains present.

Make Failure Behavior Consistent

For the job batch, invalid entries are an expected skip path; an invalid top-level batch is a caller error. For the state dispatcher, unsupported events are rejected. Do not silently change one contract into the other.

A production service may need counts or structured rejection reasons. Add them as explicit return fields and test them; do not claim diagnostics the function does not return.

References

- Loops and iteration: language constructs and their schedules.
- Switch: matching, scope, and fallthrough.
- For...of: iterable values and early exit.
- For...in: enumerable string-key traversal.
- Continue: update behavior and targets.
- ECMAScript statements: specification-level execution semantics.

Further Reading

Read labeled statements when reviewing nested exits. Then study Functions and Callbacks to place early returns inside clear, reusable operations. Later error-handling material develops throw, catch, and finally; asynchronous volumes explain scheduling and cooperative cancellation.

CHAPTER 7

Functions and Callbacks

Introduction

A function packages an operation behind a name or another function value. Its caller supplies arguments; a particular invocation creates local parameter bindings, executes a body, and returns a value or throws. This lets a program reuse a rule while keeping the work of each call understandable.

```
'use strict';
function pageCount(total, pageSize) {
  return Math.ceil(total / pageSize);
}
const count = pageCount;
console.log(count(52, 25));
console.log(count === pageCount);
// Expected output:
// 3
// true
```

`count` contains the same function value. Assigning a function does not call it. The parentheses in `count(52, 25)` perform a call. This introductory example assumes valid numbers; the production section turns those assumptions into explicit validation.

Learning Objectives

- Distinguish a function value, its declaration or expression, and an invocation.
- Trace arguments, parameters, local variables, return values, and thrown errors.
- Explain why parameter reassignment differs from mutation of an argument object.
- Use defaults and rest parameters without silently changing the meaning of `null`, zero, or missing input.
- Select ordinary functions, arrows, and methods according to their call contract.
- Pass a callback with the correct signature and explain when the receiving function invokes it.
- Separate deterministic calculation from logging, mutation, and other effects.
- Trace recursion and choose an iterative implementation when stack depth is unbounded.

Prerequisites

Read Variables and Data Types, Operators and Expressions, and Control Flow. Here an object literal is a small record of named values, and an array is an ordered collection; the following chapters teach these structures in detail.

Running the Examples

Each `js` block is an independent Node.js 20+ program with exact expected output. The scripts under `code/volume-1/chapter-07` add assertions for normal and failing calls:

```
node code/volume-1/chapter-07/example-00-function-model.js
node code/volume-1/chapter-07/example-01-batch-planning.js
node code/volume-1/chapter-07/example-02-label-callbacks.js
node code/volume-1/chapter-07/example-03-function-challenges.js
```

Read Theory, trace Internal Working, and attempt the exercises. Deep closure behavior and receiver selection are developed in Volume 2; this chapter supplies the function vocabulary those lessons require.

Theory

Definition, Value, and Invocation

A function declaration introduces a binding and a function. A function expression produces a function value where an expression is expected. An arrow is another function-expression form. Defining a body does not execute it; a call evaluates arguments and invokes the selected function. MDN: functions.

```
'use strict';
console.log(double(3));
function double(value) { return value * 2; }
const triple = function (value) { return value * 3; };
const quadruple = value => value * 4;
console.log(triple(3), quadruple(3));
// Expected output:
// 6
// 9 12
```

The declaration is available before its statement in this scope. The expressions assigned to `const` are available only after those declarations initialize. Do not extend the simple declaration rule to every sloppy-mode block declaration; use modules or strict code with declarations in clear scopes.

A function can be stored in a variable, an array, or an object property; passed as an argument; or returned. These operations copy the function value, not its body. Calling a non-callable value such as `undefined` throws a `TypeError`.

Arguments Become Local Parameter Values

The names in a definition are parameters. Values supplied by a call are arguments. Missing arguments ordinarily initialize their parameters to `undefined`; extra arguments do not automatically cause an error. Validate the domain contract explicitly.

```
'use strict';
function inspect(first, second) {
  console.log(first, second);
}
inspect('ready');
inspect('ready', 'queued', 'ignored by this body');
// Expected output:
// ready undefined
// ready queued
```

JavaScript passes values. When the value identifies an object, caller and callee can reach the same object. Reassigning the local parameter does not reassign the caller's variable, but mutating that object can be observed by the caller.

```
'use strict';
function revise(record) {
  record.status = 'reviewed';
  record = { status: 'local replacement' };
  return record.status;
}
const original = { status: 'draft' };
console.log(revise(original));
console.log(original.status);
// Expected output:
// local replacement
// reviewed
```

Do not describe this as pass-by-reference: the caller's binding itself was never handed to the callee. Ownership determines whether mutation is appropriate. A calculation can instead return a new record and leave the original untouched.

Return Values and Completion

`return expression` evaluates an expression, completes this invocation, and gives the resulting value to the caller. Reaching the end of a normal function, or using `return;`, produces `undefined`. Logging is an effect; it is not a return value.

```
'use strict';
function label(id) {
  if (id === '') return 'missing';
  return `item:${id}`;
}
function announce(message) { console.log(message); }
console.log(label('7'));
console.log(announce('saved'));
// Expected output:
// item:7
// saved
// undefined
```

A newline immediately after `return` can end that statement through automatic semicolon insertion. Keep a returned expression on the same line or open parentheses on that line. A thrown error is another completion path: statements after the throw do not run unless control is later resumed outside the throwing invocation by a handler.

Defaults Express a Missing-Value Policy

Default parameter expressions run when the corresponding argument is missing or explicitly `undefined`. They do not run for `null`, `false`, zero, or an empty string. Defaults are evaluated at call time, from left to right, and can refer to earlier parameters. MDN: default parameters.

```
'use strict';
function settings(limit = 10, offset = limit * 2) {
  return `${limit}:${offset}`;
}
console.log(settings());
console.log(settings(0));
console.log(settings(null));
console.log(settings(undefined, 4));
// Expected output:
// 10:20
// 0:0
// null:0
// 10:4
```

The `null` example demonstrates the rule, not a recommended numeric contract. A domain API should reject `null` when a number is required. Likewise, `function f(options = {})` does not make `f(null)` a valid call. Default object/array expressions create fresh values per call; defaults referencing an outer object retain that shared reference.

Parameter initialization has its own ordering and scope rules. A default cannot safely read a later uninitialized parameter, and cannot use a variable declared only inside the function body. Keep complicated preparation in the body so its dependencies are visible.

Rest Collects Arguments; Spread Supplies Them

A final rest parameter gathers the remaining argument values into a real array. Spread at a call site expands an iterable into separate arguments. They use the same punctuation for different operations. MDN: rest parameters.

```
'use strict';
function joinTags(prefix, ...tags) {
  return `${prefix}:${tags.join('|')}`;
}
const tags = ['draft', 'review'];
console.log(joinTags('doc', ...tags));
console.log(joinTags('empty'));
// Expected output:
```

```
// doc:draft|review
// empty:
```

An ordinary function also has an `arguments` object, but rest communicates which arguments the implementation intends to collect and provides array methods directly. Arrows do not create their own `arguments`. Avoid spreading an unbounded collection into a call: engines impose argument-count limits. Iterate when the input may be large.

Choose an Arrow for Its Semantics

An expression-bodied arrow implicitly returns its expression. A block-bodied arrow requires an explicit return. Parenthesize an object literal used as the returned expression so braces are not parsed as a body.

```
'use strict';
const increment = value => value + 1;
const forgotten = value => { value + 1; };
const record = id => ({ id });
console.log(increment(4), forgotten(4));
console.log(record('A').id);
// Expected output:
// 5 undefined
// A
```

An arrow captures surrounding `this`; it does not choose a receiver from a later property call. It is not a constructor and has no own `arguments` binding. Ordinary methods are appropriate when behavior should use the object on which it is called. MDN: arrow functions.

```
'use strict';
const formatter = {
  prefix: 'report',
  format(id) { return `${this.prefix}:${id}`; }
};
console.log(formatter.format('7'));
const detached = formatter.format;
try { detached('7'); } catch (error) { console.log(error.name); }
// Expected output:
// report:7
// TypeError
```

For a callback, `id => formatter.format(id)` deliberately preserves a property call. Volume 2 explains receivers in detail; here the key design question is whether the function expects a receiver or only explicit arguments.

A Callback Is a Function Supplied to Another Operation

The receiving API decides when the callback runs, how often it runs, which arguments it receives, what its return value means, and how errors propagate. “Callback” does not mean “asynchronous.”

```
'use strict';
function transformPair(first, second, transform) {
  return [transform(first, 0), transform(second, 1)];
}
const seen = [];
const result = transformPair('a', 'b', (value, index) => {
  seen.push(index);
  return value.toUpperCase();
});
console.log(result.join(','));
console.log(seen.join(','));
// Expected output:
// A,B
// 0,1
```

This API invokes the callback synchronously, twice, in order, with a value and index. Passing a built-in with a different second parameter can change behavior: a callback API's index might be interpreted as a radix or options argument. Use a wrapper to select and adapt the intended arguments. Do not pass `transform()` when the API expects the function value `transform`.

Local State, Pure Calculation, and Effects

Each ordinary invocation gets new parameter and local bindings. It can still read or mutate reachable outer objects. A pure calculation depends on its explicit inputs and returns a result without externally visible mutation. Reading the current time, generating randomness, logging, and writing shared state make additional dependencies observable.

Inject such dependencies when testing needs control. For example, accept a trusted timestamp argument rather than calling `Date.now()` deep inside a calculation. Do not expose a callback as arbitrary user-supplied executable code; callback injection is a design technique for cooperating application code.

Recursion Is Repeated Invocation

A recursive function calls itself directly or indirectly. It needs a terminating base case and progress toward that case. Each active call normally consumes stack capacity; a mathematically terminating algorithm can still exceed the runtime's stack limit on large input.

```
'use strict';
function sumDown(n) {
  if (n === 0) return 0;
  return n + sumDown(n - 1);
}
console.log(sumDown(4));
// Expected output:
// 10

// For this bounded nonnegative-integer domain: O(n) calls and stack depth.
```

This demonstration assumes a small nonnegative integer. Production validation and an iterative alternative belong at the boundary. Do not rely on tail-call optimization for portable Node.js examples. Choose recursion when the data structure and a known depth bound justify it; choose iteration when stack depth is unnecessary.

Internal Working

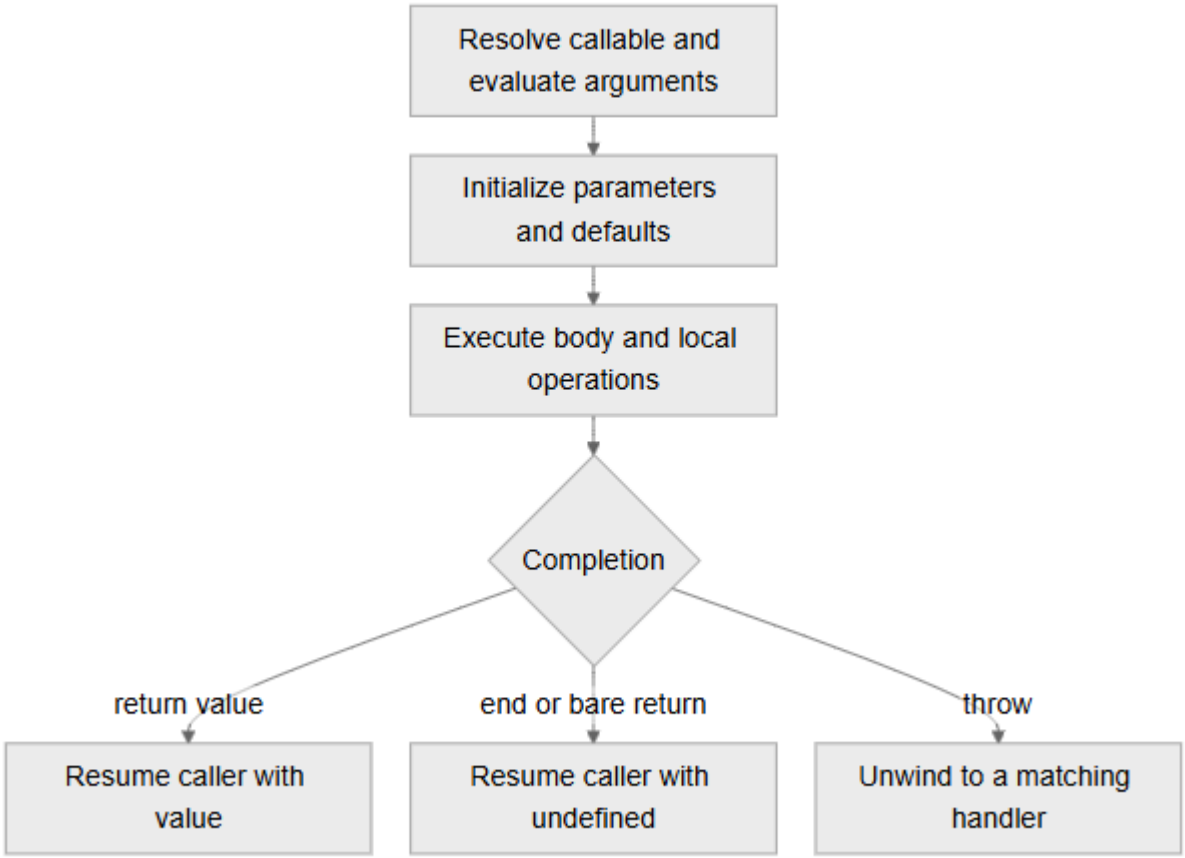
Trace a Call From Arguments to Result

```
'use strict';
const events = [];
function input(value) { events.push(`argument ${value}`); return value; }
function combine(left, right) {
  events.push('body');
  const total = left + right;
  return total;
}
const result = combine(input(2), input(3));
console.log(events.join(' -> '));
console.log(result);
// Expected output:
// argument 2 -> argument 3 -> body
// 5
```

1. Resolve the function value being called.
2. Evaluate the argument expressions in order; their effects occur before this function body starts.
3. Establish this invocation's execution context and initialize its parameters.
4. Execute the body with local bindings `left`, `right`, and `total`.
5. Evaluate the return expression and complete the invocation.
6. Resume the caller with the returned value, here assigning `5` to `result`.

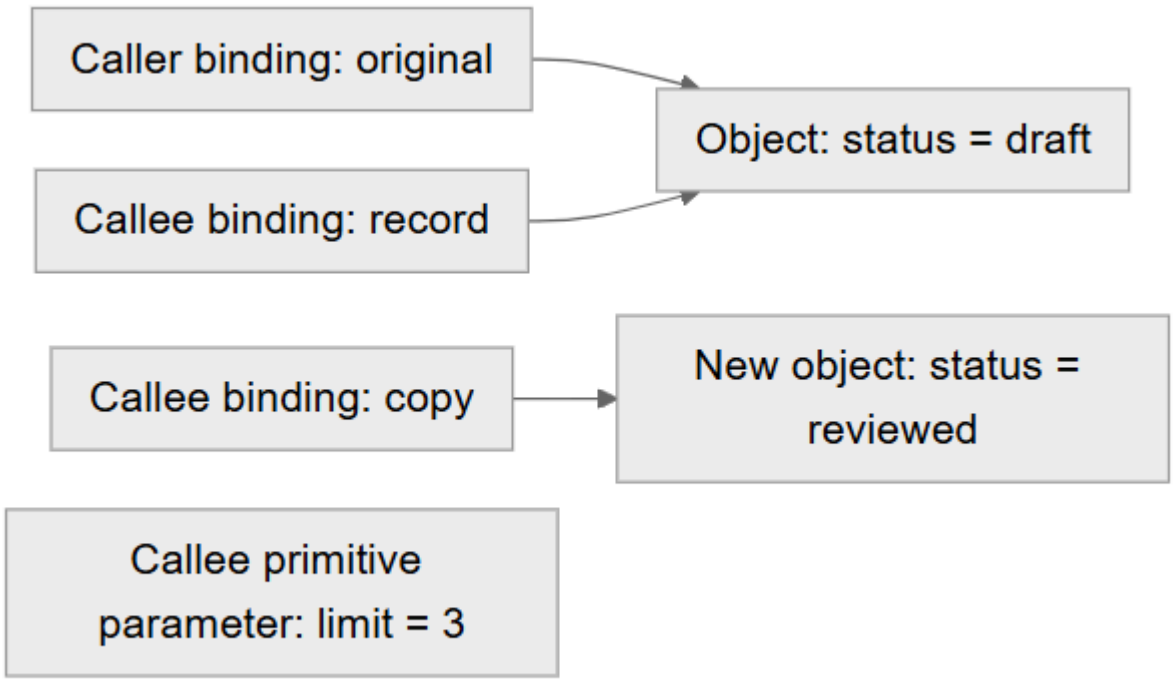
If the first argument evaluation throws, later arguments and the called body do not run. If a default parameter throws, the body also does not run. A callback call follows these same rules; it is not a separate execution mechanism.

Call Flow



Source: `diagrams/volume-1-chapter-07-call-flow.mmd`. Argument evaluation and parameter initialization can also throw before reaching the body. Exception handling is developed in Chapter 10.

Bindings and Shared Object Identity



Source: `diagrams/volume-1-chapter-07-argument-memory.mmd`.

The caller and parameter initially refer to one object. Reassigning `record` moves the parameter's reference; it does not move the caller's reference. Mutating the shared object's property changes that object for both observers. Creating `copy` allocates a separate outer object; any nested values copied by reference need their own ownership decision.

These boxes represent language-level bindings and object identity. They do not require primitive values to occupy a physical stack location or every conceptual activation to survive engine optimization.

Nested Calls and Stack Order

```
'use strict';
const trace = [];
function inner(value) { trace.push('inner'); return value * 2; }
function outer(value) {
  trace.push('outer starts');
  const result = inner(value);
  trace.push('outer resumes');
  return result + 1;
}
console.log(outer(3));
console.log(trace.join(' | '));
// Expected output:
// 7
// outer starts | inner | outer resumes
```

The inner invocation completes before `outer` continues. Returning from a function removes its active execution from the call stack; it does not guarantee that every value created by that function becomes unreachable. Returned objects and functions can retain data. Volume 2's closure chapter explains preserved lexical bindings.

Engine Internals and Observable Semantics

ECMAScript describes function execution through abstract operations and execution contexts, including parameter initialization and completion records. An engine can inline calls or optimize away temporary storage while preserving observable results, receiver behavior, errors, and effects. ECMAScript: function declaration instantiation.

Do not claim that arrows always run faster, or count diagram boxes as allocated bytes. Measure the actual workload. Function creation inside a loop can produce distinct function identities; calling a previously created function many times is a different operation. Recursion adds active invocation depth unless the runtime optimizes it, which portable code cannot assume.

Execution Cost Depends on the Body

A function call is an interface boundary, not a complexity class. A function returning a supplied number does fixed bookkeeping. A function inspecting `n` records performs work determined by that traversal and its callbacks. A callback can allocate, throw, perform I/O, or call back into its caller; include those behaviors in the contract and analysis.

Continue with Production Examples to turn this call model into bounded calculations and an explicit synchronous callback interface.

Production Examples

A Bounded Batch-Planning Function

A document service needs the number of batches required for a known number of records. The calculation is synchronous and deterministic. Inputs are Numbers: `total` is an integer from 0 to 1,000,000, and `size` is an integer from 1 to 1,000. Missing or `undefined` size selects 25; other invalid inputs throw. The function returns a fresh record and performs no scheduling or I/O.

```
'use strict';
const assert = require('node:assert/strict');
function planBatches(total, size = 25) {
  if (!Number.isSafeInteger(total) || total < 0 || total > 1000000) {
    throw new RangeError('total must be an integer from 0 to 1000000');
  }
  if (!Number.isSafeInteger(size) || size < 1 || size > 1000) {
    throw new RangeError('size must be an integer from 1 to 1000');
  }
```

```
  return { total, size, batches: Math.ceil(total / size) };
}
assert.deepEqual(planBatches(51), { total: 51, size: 25, batches: 3 });
assert.equal(planBatches(0).batches, 0);
assert.equal(planBatches(50).batches, 2);
assert.throws(() => planBatches('51'), RangeError);
assert.throws(() => planBatches(51, null), RangeError);
assert.throws(() => planBatches(51, 0), RangeError);
assert.throws(() => planBatches(1000001), RangeError);
console.log(JSON.stringify(planBatches(51)));
console.log('batch planning assertions passed');
// Expected output:
// {"total":51,"size":25,"batches":3}
// batch planning assertions passed

// O(1) arithmetic and fixed-size result storage in the bounded Number domain.
```

The upper bounds are part of this example's application contract, not JavaScript limits. They keep operational inputs explicit and make numeric precision straightforward. A caller must parse external text before calling this function. Returning a record makes the chosen default visible to the caller.

The function does not allocate all batches or claim that the service can process one million records at once. Scheduling, backpressure, and persistence belong to another layer. This separation lets tests cover the calculation without a network or database.

The matching assertion program is `code/volume-1/chapter-07/example-01-batch-planning.js`.

A Synchronous Callback With a Defined Signature

A preview screen builds labels from IDs. The application injects a formatter so the traversal does not need to know the presentation policy. The input is a trusted in-process array with at most 1,000 nonempty string IDs; a hole or invalid entry is rejected. All IDs are validated before any formatter call. The formatter is a receiver-independent synchronous function `(id, index) => string`.

```
'use strict';
const assert = require('node:assert/strict');
function buildLabels(ids, format) {
  if (!Array.isArray(ids) || ids.length > 1000) throw new TypeError('Invalid ID list');
  if (typeof format !== 'function') throw new TypeError('Formatter must be callable');
  const input = [];
  for (let index = 0; index < ids.length; index += 1) {
    const id = ids[index];
    if (!Object.hasOwn(ids, index) || typeof id !== 'string' || id.length === 0) {
      throw new TypeError('Each ID must be a nonempty string');
    }
    input.push(id);
  }
  const labels = [];
  for (let index = 0; index < input.length; index += 1) {
    const label = format(input[index], index);
    if (typeof label !== 'string') throw new TypeError('Formatter must return a string');
    labels.push(label);
  }
  return labels;
```

```
}
const source = ['A', 'B'];
const labels = buildLabels(source, (id, index) => `${index + 1}. ${id}`);
assert.deepEqual(source, ['A', 'B']);
assert.deepEqual(labels, ['1. A', '2. B']);
let calls = 0;
assert.throws(() => buildLabels(['A', null], () => { calls += 1; return ''; }), TypeError);
assert.equal(calls, 0);
assert.throws(() => buildLabels(new Array(1), id => id), TypeError);
assert.throws(() => buildLabels(['A'], () => 7), TypeError);
const failure = new Error('formatter failed');
assert.throws(() => buildLabels(['A'], () => { throw failure; }), error => error === failure);
console.log(labels.join(' | '));
console.log('callback contract assertions passed');
// Expected output:
// 1. A | 2. B
// callback contract assertions passed

// O(n) wrapper work and copied references/results, plus callback work/output text.
```

The two passes establish different guarantees. The first rejects malformed IDs before formatter effects begin and copies the primitive IDs so a formatter mutating the caller's array cannot change the pending traversal. The second calls the formatter once per copied ID in order. It returns a fresh output array.

A formatter may still perform an effect and then throw. The wrapper does not roll back those effects. It also does not await promises; an async formatter returns a non-string and violates the contract. Plain object access can invoke getters or proxy traps, so this is an in-process API, not a sandbox for executable hostile objects.

If a collaborator needs its own receiver, inject an adapter such as `id => printer.format(id)`. Passing `printer.format` directly loses that property call. Do not change the wrapper to guess a receiver.

The matching assertion program is `code/volume-1/chapter-07/example-02-label-callbacks.js`. The exercises ask you to design additional call contracts rather than copy these implementations.

Interview Perspective

What Happens When a Function Is Assigned?

Question: Does `const run = processItem` execute `processItem`?

Answer: No. The initializer evaluates to a function value and stores it in a new binding. `run(item)` later invokes that function. `const run = processItem(item)` instead executes immediately and stores the returned value, which may not be callable.

Follow-up: Does the alias change the function's lexical scope? No. Definition location determines lexical access; the new binding does not rewrite it. Volume 2 develops that point with closures.

Is JavaScript Pass-by-Reference?

```
'use strict';
function update(value, record) {
  value = 99;
  record.count += 1;
  record = { count: 99 };
}
let value = 1;
const record = { count: 1 };
update(value, record);
console.log(value, record.count);
// Expected output:
// 1 2
```

Answer: Argument values initialize local parameters. Assignments to those parameters do not assign to the caller's bindings. The parameter initially identifies the same record, so its property mutation is visible. Draw two bindings pointing to one object before drawing the local reassignment.

Does a Callback Run Later?

Answer: The receiving API decides. An array transformation may call synchronously; a timer callback is scheduled by the host. Read the contract instead of inferring timing from the word callback. Also establish argument order, number of calls, return handling, receiver, and error behavior.

Follow-up: Why can passing a built-in directly be wrong? The receiving API may supply more arguments than expected. A wrapper selects the intended arguments explicitly.

Does a Default Handle Every Missing-Looking Value?

```
'use strict';
function choose(value = 'default') { return String(value); }
console.log(choose(), choose(undefined), choose(null), choose(0));
// Expected output:
// default default null 0
```

Answer: Only absent arguments and `undefined` trigger this default. Zero and null remain supplied values. Decide whether null is allowed, rejected, or explicitly mapped by a separate rule.

Why Did an Arrow Return Undefined?

Answer: A block body does not implicitly return its final expression. `x => { x + 1; }` returns undefined. Use an expression body or `return`. An object literal expression needs parentheses: `id => ({ id })`.

Follow-up: Are arrows interchangeable with methods? No. An arrow has lexical `this` and cannot be used as a constructor. Choose based on receiver and call semantics.

How Would You Make a Function Testable?

Answer: State input and output types, ranges, ownership, effects, and failure behavior. Separate deterministic calculations from I/O. Inject a clock or formatter only when a real dependency needs control. Assert boundaries and invariants, including unchanged caller-owned data after invalid input.

Whiteboard drill: Implement a bounded batch calculator, trace a call with an omitted size, then test size zero and a numeric string. Explain why explicit parsing belongs at the external-data boundary rather than inside every calculation.

What Does Recursion Cost?

Answer: Count calls, work per call, retained output, and maximum active depth separately. A linear recursive sum has O(n) calls and normally O(n) stack depth. An iterative loop can do the same arithmetic with O(1) extra state. Do not infer an optimization guarantee from tail-call syntax.

Exercises and Coding Challenges

Try each problem before reading its solution. All solution blocks run independently in Node.js 20+. The companion assertions are in `code/volume-1/chapter-07/example-03-function-challenges.js`.

Exercise 1: Return a Result Instead of Only Logging

Requirements: Write `formatRun(id, status = 'queued')`. Accept a nonempty string ID and one of `queued`, `running`, or `done`. Return `id:status` without logging inside the function. Reject invalid status and ID with `TypeError`.

Hint: A default handles undefined; it does not validate a supplied value.

```
'use strict';
function formatRun(id, status = 'queued') {
  if (typeof id !== 'string' || id.length === 0 ||
      !['queued', 'running', 'done'].includes(status)) throw new TypeError('Invalid run');
  return `${id}:${status}`;
}
console.log(formatRun('R1'));
console.log(formatRun('R1', 'done'));
try { formatRun('R1', null); } catch (error) { console.log(error.name); }
// Expected output:
// R1:queued
// R1:done
// TypeError
```

Explanation: All successful branches return a string; output is the caller's decision. Formatting takes O(L) result work for label length L. A fixed status list has bounded lookup cost. An object of options may be preferable if the signature gains many independent settings.

Exercise 2: Keep Mutation Out of a Status Change

Requirements: Given a trusted flat record `{ id, status }` whose fields are strings, implement `finish(record)` returning a new `{ id, status: 'done' }` record. Do not change the argument or retain additional fields. Demonstrate both identity and value behavior.

Hint: A new outer object is sufficient because the selected values are strings.

```
'use strict';
function finish(record) { return { id: record.id, status: 'done' }; }
const before = { id: 'R1', status: 'running' };
const after = finish(before);
console.log(before.status, after.status);
console.log(before === after);
// Expected output:
// running done
// false
```

Explanation: The new record preserves the selected identifier and replaces state without writing through the shared input reference. This fixed shape needs O(1) bookkeeping. For nested records, define which nested values should be copied rather than calling every outer copy a deep clone.

Exercise 3: Collect and Validate Rest Arguments

Requirements: Implement `sumScores(...scores)`. Accept zero or more integer scores from 0 to 100, at most 100 scores. Return their sum, using zero for no arguments. Reject an invalid count or score with `RangeError`.

Hint: The rest parameter is an array; loop over it rather than recursively calling the function.

```
'use strict';
function sumScores(...scores) {
  if (scores.length > 100) throw new RangeError('Too many scores');
  let total = 0;
  for (const score of scores) {
    if (!Number.isInteger(score) || score < 0 || score > 100) throw new RangeError('Invalid score');
    total += score;
  }
  return total;
}
console.log(sumScores(), sumScores(10, 20, 30));
try { sumScores(10, '20'); } catch (error) { console.log(error.name); }
// Expected output:
// 0 60
// RangeError
```

Explanation: The limits keep the total below 10,001, well within exact Number integer arithmetic. Work is O(n); rest collection retains O(n) values, while loop bookkeeping is O(1). If callers already hold a large array, an array-taking API avoids spreading it into arguments.

Exercise 4: Adapt an Extra Callback Argument

Requirements: Implement `parseTwo(first, second)` for trusted decimal strings. A supplied helper `applyTwo` calls its callback with `(value, index)`. Use `parseInt` with radix 10 deliberately, and show why passing it directly gives an unwanted result.

Hint: The index is not the intended radix.

```
'use strict';
function applyTwo(a, b, callback) { return [callback(a, 0), callback(b, 1)]; }
function parseTwo(a, b) { return applyTwo(a, b, value => Number.parseInt(value, 10)); }
console.log(applyTwo('10', '10', Number.parseInt).join(','));
console.log(parseTwo('10', '10').join(','));
// Expected output:
// 10,NaN
// 10,10
```

Explanation: The first direct call uses radix zero's inferred behavior; the second uses invalid radix one and returns NaN. The wrapper ignores the index and supplies ten. Parsing costs O(L) in the input characters examined. This exercise assumes trusted strings; strict external numeric parsing also needs syntax and range validation.

Exercise 5: Inject a Deterministic Clock

Requirements: `stamp(id, now)` accepts a string ID and a synchronous zero-argument function returning a nonnegative safe-integer timestamp. Call the clock once and return `{ id, at }`; reject bad input or return values with `TypeError`. Do not call `Date.now` internally.

Hint: Validate the callable before invoking it; validate its result before constructing the output.

```
'use strict';
function stamp(id, now) {
  if (typeof id !== 'string' || typeof now !== 'function') throw new TypeError('Invalid dependency');
  const at = now();
  if (!Number.isSafeInteger(at) || at < 0) throw new TypeError('Invalid timestamp');
  return { id, at };
}
let calls = 0;
console.log(JSON.stringify(stamp('R1', () => { calls += 1; return 1000; })));
console.log(calls);
try { stamp('R1', () => NaN); } catch (error) { console.log(error.name); }
// Expected output:
// {"id":"R1","at":1000}
// 1
// TypeError
```

Explanation: The caller can inject a real clock or a fixed test value without changing the function. It has O(1) wrapper bookkeeping plus the clock's work. A clock exception propagates unchanged; the wrapper cannot undo clock effects. An optional default clock could be convenient, but explicit injection keeps this exercise deterministic.

Exercise 6: Replace Linear Recursive Depth

Requirements: Implement `sumTo(n)` for safe integers from 0 to 10,000. Use iteration, reject invalid input with `RangeError`, and explain why a recursively defined equivalent needs a depth bound.

Hint: Maintain the running total and the next term. No nested call is required.

```
'use strict';
function sumTo(n) {
  if (!Number.isSafeInteger(n) || n < 0 || n > 10000) throw new RangeError('Invalid bound');
  let total = 0;
  for (let term = 1; term <= n; term += 1) total += term;
  return total;
}
console.log(sumTo(0), sumTo(4), sumTo(10000));
try { sumTo(-1); } catch (error) { console.log(error.name); }
// Expected output:
// 0 10 50005000
// RangeError
```

Explanation: This has O(n) additions and O(1) extra state. A linear recursive version generally uses O(n) active calls and can exceed stack capacity. The arithmetic-series formula gives O(1) arithmetic for this bounded domain, but the loop makes progress and termination visible for the exercise.

MCQs

Predict the behavior before reading the expected output and answer. Each program is independent.

1. Assigning a Function

After `const run = processItem`, where `processItem` is a function, which statement is correct?

- A. `processItem` has already executed with no arguments.
- B. `run` contains a copy of the most recent return value.
- C. `run` contains the same function value and can invoke it later.
- D. The function now resolves free names from the scope that calls `run`.

Answer: C. Assignment stores the function value. Invocation requires a call. An alias does not change lexical scope.

2. Missing Arguments

```
'use strict';
function report(first, second) { return `${first}:${second}`; }
console.log(report('ready'));
```

```
// Expected output:
// ready:undefined
```

- A. Missing parameters cause an immediate TypeError.
- B. The omitted parameter receives undefined.
- C. The last supplied value is repeated.
- D. The function is not invoked until both arguments are supplied.

Answer: B. JavaScript does not enforce this argument count automatically. Validate required inputs inside the API boundary.

3. Logging and Returning

A normal function prints a message and reaches its closing brace. What does its caller receive?

- A. The printed string.
- B. The last expression evaluated in the body.
- C. Null.
- D. Undefined.

Answer: D. Console output is an effect. A normal function needs an explicit return expression to return another value.

4. A Supplied Null

```
'use strict';
function choose(value = 5) { return value; }
console.log(choose(undefined), choose(null), choose(0));
// Expected output:
// 5 null 0
```

- A. Only undefined selects the default among these arguments.
- B. Both null and undefined select the default.
- C. All falsy arguments select the default.
- D. Explicit undefined differs from omission and bypasses the default.

Answer: A. Parameter defaults apply to missing or undefined arguments, not every falsy value.

5. Rest Parameters

What is `values` inside `function collect(...values) {}`?

- A. An alias of the caller's array, regardless of how it calls the function.
- B. The legacy arguments object.

- C. An array containing the collected argument values.
- D. A lazy iterator that executes the caller again when read.

Answer: C. Rest collects values into an array. Object values inside it may still identify shared objects.

6. Arrow Body Syntax

```
'use strict';
const first = n => n + 1;
const second = n => { n + 1; };
console.log(first(2), second(2));
// Expected output:
// 3 undefined
```

- A. Both functions return 3.
- B. The block body throws because it lacks return.
- C. Both functions return undefined.
- D. Only the expression body implicitly returns its result.

Answer: D. A block body is a statement list. Its final expression is not automatically the returned value.

7. Reassignment Versus Mutation

```
'use strict';
function revise(item) {
  item.value = 2;
  item = { value: 3 };
}
const item = { value: 1 };
revise(item);
console.log(item.value);
// Expected output:
// 2
```

- A. It prints 1 because object arguments are deeply copied.
- B. It prints 2 because mutation reaches the shared object, while reassignment is local.
- C. It prints 3 because the parameter aliases the caller's binding.
- D. Const makes the property write throw.

Answer: B. The local parameter and caller initially hold the same object value. They remain different bindings.

8. Callback Timing

Which statement is valid for all callback-based APIs?

- A. Timing and invocation rules must be learned from the receiving API's contract.
- B. A callback always runs after the current function returns.
- C. A callback always runs exactly once.
- D. A callback's returned value automatically becomes the outer function's return value.

Answer: A. A callback can run synchronously, later, repeatedly, or not at all. Its name alone establishes none of those guarantees.

9. Adapting a Signature

An API calls a callback as `(text, index)`. You want decimal parsing. Which callback is appropriate?

- A. `Number.parseInt()`
- B. `Number.parseInt`
- C. `text => Number.parseInt(text, 10)`
- D. `(text, index) => Number.parseInt(text, index)`

Answer: C. The wrapper fixes the radix and ignores the unrelated index. Directly passing parseInt lets the index become the radix; calling it immediately passes its result instead of a function.

10. Argument Evaluation

```
'use strict';
const trace = [];
function value(n) { trace.push(n); return n; }
function add(a, b) { trace.push('body'); return a + b; }
add(value(1), value(2));
console.log(trace.join(','));
// Expected output:
// 1,2,body
```

- A. The body runs before either argument expression.
- B. Argument expressions run in order before the body.
- C. The runtime can freely reverse these observable argument effects.
- D. Arguments are evaluated only when their parameters are first read.

Answer: B. The call evaluates its arguments before invoking the function body. A throw during argument evaluation can prevent that body from running.

11. Arrows and Receivers

Which requirement favors an ordinary object method over an arrow stored in a property?

- A. The function must return an object.

- B. The function needs parameters.
- C. The function must be passed as a value.
- D. A property call should supply the object's receiver as `this`.

Answer: D. Arrows capture surrounding this rather than acquiring it from the later call. Both forms can take parameters, return objects, and be stored as values.

12. Recursion Depth

A linear recursive traversal visits n records and keeps one active invocation per record. What portable concern remains even when every input terminates?

- A. The runtime can run out of call-stack capacity.
- B. JavaScript forbids recursive declarations.
- C. Every recursive function is automatically asynchronous.
- D. Tail-call-shaped source guarantees constant stack in Node.js.

Answer: A. Termination and available stack capacity are separate concerns. Iteration can remove unnecessary active depth.

13. Default Allocation

```
'use strict';
function items(value = []) { value.push('x'); return value; }
console.log(items().length, items().length);
// Expected output:
// 1 1
```

- A. The default array is shared, producing 1 then 2.
- B. Defaults execute only once when the function is defined.
- C. The array expression executes separately for each call that needs the default.
- D. Arrays cannot be used as defaults.

Answer: C. The expression creates a fresh array for each of these calls. A default referring to an outer array would instead reuse that array.

14. A Callback Throws After Writing

A callback writes to a log array, then throws. Its caller catches the error. What follows automatically?

- A. The log array is restored to its previous state.
- B. The callback is retried once.
- C. Earlier callbacks are undone in reverse order.

- D. No rollback occurs unless the application implements one.

Answer: D. Error propagation transfers control. It is not a transaction mechanism.

15. A Returned Record

```
'use strict';
function rename(item, name) { return { id: item.id, name }; }
const before = { id: 'A', name: 'old' };
const after = rename(before, 'new');
console.log(before.name, after.name, before === after);
// Expected output:
// old new false
```

- A. The original is mutated because it was an argument.
- B. A separate outer record is returned and the original stays unchanged.
- C. Both variables refer to the same object with two names.
- D. Returning an object freezes the original automatically.

Answer: B. Object construction creates the new outer identity. No automatic copying or freezing occurs merely because a function returns.

16. Preserving a Collaborator's Receiver

`printer.format(id)` reads `this.prefix`. A function-taking API will call its argument without a receiver. Which adapter preserves the intended property call?

- A. `id => printer.format(id)`
- B. `printer.format`
- C. `printer.format()`
- D. `() => printer.format`

Answer: A. The wrapper invokes the method through printer. B loses that call form, C invokes immediately, and D returns the method without invoking it.

Revision Sheet and Chapter Summary

Revision Sheet

Concept	Rule
Function value	Can be assigned, passed, or returned without being called
Invocation	Evaluates arguments, initializes parameters, executes the body, and completes
Declaration	Available before its statement in the ordinary scope examples used here
Const-bound expression	Readable only after the binding is initialized
Parameter	A local binding initialized from an argument value
Object argument	Its value may identify the same object visible to the caller
Reassignment	Changes the local parameter, not the caller's binding
Property mutation	Can change shared state visible outside the function
No return expression	A normally completed ordinary call produces undefined
Default parameter	Selected by omission or undefined, not by null or all falsy values
Rest parameter	Collects remaining argument values into a new array
Call spread	Expands an iterable into separate arguments; avoid unbounded argument counts
Arrow expression body	Implicitly returns the expression
Arrow block body	Requires return for a non-undefined normal result
Arrow receiver	Lexical this; ordinary methods choose their receiver through the call
Callback	Receiving API defines timing, arguments, return handling, and failures
Recursion	Needs progress, a base case, and a practical depth bound

Predicting a Call

Resolve the function, evaluate argument effects in order, initialize defaults, and follow the body until it returns, throws, or finishes. Draw bindings separately from objects. At a nested call, suspend the current body until the nested call completes. At a callback boundary, read the receiving API's signature rather than guessing.

Chapter Summary

Functions turn behavior into values that can be named, reused, and composed. A good function boundary specifies accepted values, returned values, ownership, effects, and errors. Those decisions matter more than choosing the shortest syntax.

Use the batch planner and label builder to practice explicit contracts. Use the exercises to distinguish returning from logging, copying from mutation, defaults from validation, and callback adaptation from immediate invocation.

References

- MDN: functions guide.
- MDN: default parameters.
- MDN: rest parameters.
- MDN: arrow functions.
- ECMAScript: function declaration instantiation.

Further Reading

Continue with Objects and Data Ownership, then Arrays and Collections. Revisit Errors and Debugging for error propagation and assertions. Volume 2 develops lexical scope and closures and receiver selection in depth.

Edge Cases, Debugging, and Failure Modes

A Newline Changes Return Behavior

```
'use strict';
function broken() {
  return
  { status: 'ready' };
}
function fixed() {
  return { status: 'ready' };
}
```

```
console.log(broken());
console.log(fixed().status);
// Expected output:
// undefined
// ready
```

The newline after return ends the statement. The following braces are not its result expression. Keep the expression on the same line, or put an opening parenthesis there before wrapping a long expression. A linter can catch this pattern before runtime.

Defaults Cannot Read Later Parameters

```
'use strict';
function later(first = second, second = 2) { return first + second; }
try { later(); } catch (error) { console.log(error.name); }
console.log(later(1));
// Expected output:
// ReferenceError
// 3
```

At the first default, second is not initialized. Supplying first bypasses that default. Do not silently depend on callers avoiding a fragile initialization path; place the dependency first or compute it in the body.

A Callback Can Change Shared Inputs

```
'use strict';
function visit(items, callback) {
  for (const item of items) callback(item);
}
const items = [{ count: 1 }];
visit(items, item => { item.count += 1; });
console.log(items[0].count);
// Expected output:
// 2
```

Passing the item does not clone it. If callbacks should receive snapshots, select and copy the required fields. If they may mutate, document that permission. Copying an array of objects alone does not isolate the objects.

Extra Arguments Are Observable

Some callback APIs supply a value, index, and collection. A function written for `(text, radix)` can misinterpret that signature. The callback adaptation exercise demonstrates a concrete failure. Wrap the target to select the intended arguments rather than relying on matching names.

Definitions Can Succeed While Calls Fail

```
'use strict';
const parse = value => value.trim();
console.log(typeof parse);
try { parse(null); } catch (error) { console.log(error.name); }
// Expected output:
// function
// TypeError
```

A file that only defines parse can exit successfully without testing its body. Meaningful tests must invoke the function with accepted inputs, boundaries, and required failure cases. This distinction is why the companion programs contain assertions.

Debugging Procedure

1. Confirm that the value being called is actually a function.
2. Inspect the supplied values and the receiving signature, including extra callback arguments.
3. Check defaults and whether null was incorrectly treated as omission.
4. Trace the return path; do not confuse console output with a returned value.
5. Draw shared references before blaming parameter reassignment for a mutation.
6. Check whether a method lost its receiver or an arrow captured an unintended one.
7. Separate callback errors from effects already performed; catching does not roll back.
8. For recursion, verify both progress and realistic maximum depth.

Performance and Security Notes

Count Work Inside the Boundary

A function call does not make an algorithm constant time. The batch planner performs bounded arithmetic and returns a fixed-size record. The label builder validates n IDs, stores a snapshot, calls the formatter n times, and retains the resulting strings. Its cost includes the formatter's work and output lengths.

Avoid repeated string parsing or invariant computation inside a callback when the caller could do it once. Preserve correctness: moving a call changes behavior if that call reads changing state or has effects. Measure an equivalent workload before treating the change as an optimization.

Function Identity and Retention

Calling one function repeatedly differs from creating many functions. A function expression evaluated for each record can create distinct identities; a stored callback can retain objects it needs. An event registry holding a callback can therefore prolong the lifetime of captured data.

Prefer clear lifetimes and stable callback identity when registration/removal requires it. Do not infer physical storage from the memory diagram or assume every conceptual call creates a fixed number of heap objects. Engine optimizations must preserve behavior but can change representation.

Stack and Argument Limits

Bound recursive depth when processing external structures. An iterative traversal can remove unnecessary call depth, but it still needs a limit on total work. Similarly, avoid spreading an arbitrarily large array into a function call. Iteration handles large collections without relying on a portable maximum argument count.

Validate at the Public Boundary

Specify types, ranges, maximum input sizes, ownership, and errors. `typeof callback === 'function'` establishes callability for ordinary application values; it does not prove the callback is pure, safe, synchronous, or inexpensive. Validate returned values when the caller relies on a narrower contract.

A callback can read application state, mutate objects, or throw after an effect. Inject only trusted application behavior. Do not compile user text with eval or Function to make it callable. For user-selected behavior, map an allowlisted name to an existing operation and validate that operation's data inputs.

Preserve the Return and Error Contract

An adapter that logs an error and returns undefined changes the caller's observable behavior. A wrapper around a synchronous function must not secretly introduce promises. State whether errors propagate, are translated, or become explicit result values; use the error-handling chapter to implement that decision.

Avoid logging full arguments when they can contain credentials or personal data. Prefer a field name, error category, and safe identifier. Test invalid data without storing sensitive values in permanent fixtures.

Measurement Checklist

Measure creation separately from repeated calls, keep console output outside timed regions, and record runtime and input sizes. Compare results and error behavior first. Inspect retained data if memory is the concern. A microbenchmark of arithmetic-only arrows cannot answer whether a real formatting callback allocates too much output.

Professional Field Guide

Write the Function's Promise

Describe a function in terms a caller can rely on: accepted arguments, successful result, changes to supplied objects, dependencies, and failure behavior. For the batch planner, the promise is a bounded numeric calculation with a fresh result record. For the label builder, it includes callback order and validating all IDs before formatting begins.

A long parameter list often mixes independent concerns. A named options record can clarify those concerns, but destructuring it is not validation. Use a focused constructor or parser for the record when the boundary needs stronger guarantees.

Case Study: A Logger Becomes a Calculator

A utility prints a total and returns nothing. A later caller assigns its result and sends undefined downstream. The fix is to return the calculated value and let the boundary decide whether to log it. Tests should assert the returned value directly; checking that a console message appeared would miss the interface defect.

Case Study: A Callback Is Invoked Too Soon

A caller writes `register(makeHandler())` where the API expects the handler function itself. This may be intentional if makeHandler is a factory, or a bug if it performs the operation immediately and returns undefined. Trace the initializer and inspect the returned value before changing syntax. Function-producing factories and action functions have different contracts.

Choose names that expose the distinction: createHandler returns a function; handleRecord performs the action. Avoid names that force every reader to inspect the implementation to know which form to pass.

Case Study: Mutation Hides Behind a Parameter

A helper receives a draft record and updates a nested array. Reassigning the helper's parameter later does not undo that mutation. Reproduce with two references to the same object, then decide whether mutation is authorized. If it is not, copy the specific structure the result owns. A shallow outer spread is insufficient when mutable nested values remain shared.

Case Study: A Convenient Callback Signature Drifts

An API initially calls a formatter with one argument, then adds an index. A directly supplied built-in now interprets the index as another option. Keep the receiving API signature documented and use adapters at the composition boundary. Regression checks should cover argument order and exact call count when those are part of the promise.

Review Questions

- Can each branch return the documented type or throw the documented failure?
- Are defaults applied only to values that the domain considers missing?
- Does the function mutate an input or share a nested reference in its output?
- Does the callback contract specify timing, arguments, result, receiver, and failure behavior?
- Are external effects separated from calculations that could be deterministic?
- Can an invalid call perform partial work before it is rejected?
- Do recursion and input size have bounds?
- Are assertions invoking the implementation rather than merely loading its definition?

Mastery Check

Explain a call without executing it: identify the function value, argument evaluation order, parameter defaults, shared objects, return path, and any callback effects. Then change one argument to null and repeat the reasoning. Continue with Objects and Data Ownership to make the shared-reference part of that explanation precise.

CHAPTER 8

Objects and Data Ownership

An object groups properties under an identity. Reading a property, checking whether it belongs to the object, and copying that object's values are distinct operations. Many production bugs come from treating those operations as interchangeable: an inherited setting is accepted as submitted data, an optional value hides a missing field, or a spread copy still shares nested state.

This chapter develops ordinary records and explicit ownership contracts. It introduces just enough prototype and descriptor behavior to make property access predictable. Volume 2 develops inheritance, descriptors, classes, and proxies in depth.

Learning Objectives

- Use literal, computed, string, and symbol property keys appropriately.
- Distinguish a missing property, an own property, and an inherited property.
- Choose between property lookup, `in`, `Object.hasOwn`, `Object.keys`, and `Reflect.ownKeys`.
- Explain data-property flags and accessor behavior without treating every property as a plain slot.
- Apply destructuring and defaults while preserving null/undefined distinctions.
- Trace shallow copies and copy the exact nested records an update owns.
- Define safe ordinary-JSON boundaries without trusting arbitrary keys or inherited configuration.
- Test mutation isolation, unknown keys, and descriptor-related failures.

Prerequisites and Running Guide

Read Variables and Data Types, Operators and Expressions, and Functions and Callbacks. Be comfortable with conditionals, loops, parameters, and return values. Arrays in this chapter hold short key lists; the next chapter develops collections systematically.

Use Node.js 20 or later. Each JavaScript block is independent and includes exact output. Run these companion assertion programs from the repository root:

```
node code/volume-1/chapter-08/example-00-property-model.js
node code/volume-1/chapter-08/example-01-owned-profile.js
node code/volume-1/chapter-08/example-02-safe-preferences.js
node code/volume-1/chapter-08/example-03-ownership-challenges.js
```

First Observation

```
'use strict';

const before = { id: 'U-1', preferences: { theme: 'light' } };
```

```
const after = { ...before };
after.preferences.theme = 'dark';
console.log(before === after);
console.log(before.preferences === after.preferences);
console.log(before.preferences.theme);

// Expected output:
// false
// true
// dark
```

The outer objects differ, but the nested preference object is shared. A complete ownership explanation names which records are new and which remain shared. Start with property theory, draw the reference graph, and then test the production boundaries.

Theory

Properties Have Keys and Behavior

An ordinary object literal creates a new object. Most literal entries create writable, enumerable, configurable own data properties. Keys are strings or symbols; a numeric key expression is converted to a string key. Dot notation uses a literal identifier name, while bracket notation evaluates an expression for the key.

```
'use strict';

const field = 'delivery-zone';
const internalId = Symbol('internalId');
const shipment = { id: 'S-1', [field]: 'north', [internalId]: 17, 1: 'first' };
console.log(shipment.id, shipment[field]);
console.log(shipment[1], shipment['1']);
console.log(shipment[internalId]);

// Expected output:
// S-1 north
// first first
// 17
```

Use brackets for computed keys or names that are not valid dot-notation identifiers. Distinct directly created symbols remain distinct keys even if their descriptions match. A symbol key is discoverable through reflection; it is not an access-control mechanism.

Own Properties and Inherited Lookup

Reading `record.key` first asks the object's property machinery for a value. For ordinary objects, lookup can follow the prototype relationship when an own property is absent. `Object.hasOwn(record, key)` asks a narrower question: does this object itself have that property? The `in` operator asks whether the property

exists anywhere in ordinary lookup, including prototypes.

```
'use strict';

const defaults = { region: 'APAC' };
const request = Object.create(defaults);
request.id = 'R-1';
request.note = undefined;
console.log(request.region, 'region' in request, Object.hasOwn(request, 'region'));
console.log(request.note, request.missing);
console.log(Object.hasOwn(request, 'note'), Object.hasOwn(request, 'missing'));

// Expected output:
// APAC true false
// undefined undefined
// true false
```

An undefined result alone does not establish that a property is absent. Use ownership checks when a request must explicitly supply its own field. Prefer the static method to `record.hasOwnProperty(...)`: data can shadow that method, and a null-prototype dictionary does not inherit it. See Object.hasOwn.

Enumerating Keys Deliberately

Operation	Own properties	Inherited properties	Non-enumerable properties	Symbol keys
`Object.keys`	Yes	No	No	No
`Object.entries`	Yes, with values	No	No	No
`for...in`	Enumerable string keys	Enumerable string keys	No	No
`Reflect.ownKeys`	Yes	No	Yes	Yes
Object spread	Copies enumerable values	No	No	Yes

```
'use strict';

const token = Symbol('token');
const record = { visible: 1, [token]: 2 };
Object.defineProperty(record, 'hidden', { value: 3, enumerable: false });
console.log(Object.keys(record).join('|'));
console.log(Reflect.ownKeys(record).map(String).join('|'));
const copy = { ...record };
console.log(copy[token], Object.hasOwn(copy, 'hidden'));

// Expected output:
// visible
```

```
// visible|hidden|Symbol(token)
// 2 false
```

For ordinary own keys, array-index string keys (canonical nonnegative integers below 2^32 - 1) come first in numeric order, followed by other string keys in insertion order, then symbols in insertion order. A key such as `01` or `4294967295` is in the ordinary string group. Do not use an object's enumeration order as a business priority list. Keep an explicit ordered array when order is part of the contract. Details and operation differences are documented in MDN enumerability and ownership.

A Property Descriptor Is More Than a Value

A data property has a value and a writable flag. An accessor property has getter/setter functions instead. Both kinds have enumerable and configurable flags. Enumerable controls inclusion in selected traversal operations; configurable controls deletion and many descriptor changes. It is not a synonym for "the value is writable."

When a new property is created by `Object.defineProperty`, omitted boolean flags default to false. Object literal data properties normally start with those flags true. Do not assume the two creation mechanisms have identical defaults.

```
'use strict';

const order = {};
Object.defineProperty(order, 'id', { value: 'O-1', enumerable: true });
console.log(Object.keys(order).join('|'));
try { order.id = 'O-2'; } catch (error) { console.log(error.name); }
console.log(Object.getOwnPropertyDescriptor(order, 'id').writable);
const pricing = {
  cents: 450,
  get label() { return `${this.cents} cents`; }
};
console.log(pricing.label);

// Expected output:
// id
// TypeError
// false
// 450 cents
```

Reading `pricing.label` calls code; it is not simply loading a stored label slot. Getters can have effects or throw. Destructuring and copying that read properties can therefore execute code on arbitrary JavaScript objects. This chapter's external-data parsers accept JSON text, whose parsed values do not contain functions or accessors. The descriptor reference gives the formal flag behavior.

Destructuring Reads Values Into Bindings

```
'use strict';

const input = { id: 'U-1', label: undefined, note: null };
```

```
const { id: userId, label = 'Guest', note = 'none', ...remaining } = input;
console.log(userId, label, note);
console.log(Object.keys(remaining).length);
input.id = 'U-2';
console.log(userId);

// Expected output:
// U-1 Guest null
// 0
// U-1
```

The syntax `id: userId` reads property `id` and initializes the binding `userId`. A default runs only when the retrieved value is undefined, whether caused by absence or an explicit undefined value. Null does not trigger that default. Destructuring a nested object value still shares its identity; it does not recursively clone it.

Ordinary property lookup during destructuring can read inherited fields. Object rest gathers remaining own enumerable properties. Therefore destructuring is concise extraction, not by itself a schema or ownership validator. See MDN destructuring.

Spread Creates an Outer Copy

Object spread reads own enumerable source properties and creates data properties on the new object. Later entries overwrite earlier values at matching keys. It does not preserve a source's prototype or descriptor flags, and it does not recursively clone property values.

```
'use strict';

const previous = { id: 'U-1', options: { theme: 'light', email: true } };
const next = { ...previous, options: { ...previous.options, theme: 'dark' } };
console.log(previous === next, previous.options === next.options);
console.log(previous.options.theme, next.options.theme, next.options.email);

// Expected output:
// false false
// light dark true
```

The update copies every record on the modified path: the root and `options`. Fields outside that path remain shared if they contain objects. This is useful structural sharing when callers respect the ownership or immutability policy; it is not complete graph independence. MDN spread syntax documents the shallow-copy contract.

`Object.assign(target, source)` also reads source values, but writes by assignment to an existing target and can invoke its setters. Object spread into a new literal defines own properties instead. Neither operation is a general sanitizer for unknown objects.

Optional Fields and Deletion

Choose a policy for absent, undefined, and null values. A patch API might interpret absence as "leave unchanged" and null as "clear the field." A defaulting API might accept only absence as a request for its default. Read presence with `Object.hasOwn` when these distinctions matter.

```
'use strict';

const record = { note: 'call ahead' };
record.note = undefined;
console.log(Object.hasOwn(record, 'note'));
delete record.note;
console.log(Object.hasOwn(record, 'note'));

// Expected output:
// true
// false
```

Setting undefined preserves the property; deleting a configurable own property removes it. A later ordinary lookup can then expose an inherited property of the same name. Use explicit output schemas instead of treating undefined, deletion, and inheritance as interchangeable.

Freezing Is Shallow

`Object.freeze` prevents extensions and makes own properties non-configurable; own data properties also become non-writable. It does not recursively freeze nested objects or make accessor behavior pure.

```
'use strict';

const settings = Object.freeze({ options: { theme: 'light' } });
settings.options.theme = 'dark';
try { settings.options = {}; } catch (error) { console.log(error.name); }
console.log(settings.options.theme);

// Expected output:
// TypeError
// dark
```

A frozen outer object can still point to mutable nested state. Freeze selected owned records when the contract requires it, and do not claim that freezing is a security boundary. Object.freeze describes these limits.

Internal Working

Trace an Update Through the Graph

```
'use strict';

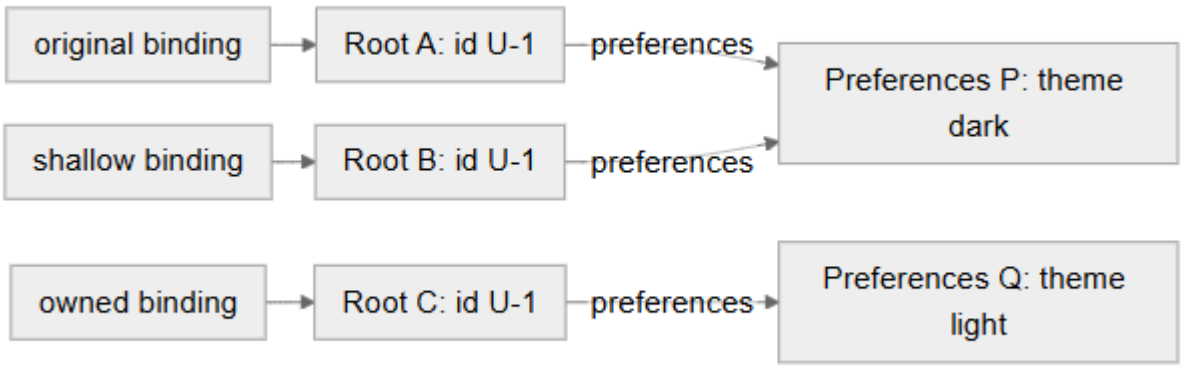
const original = { id: 'U-1', preferences: { theme: 'light' } };
const shallow = { ...original };
const owned = { ...original, preferences: { ...original.preferences } };
shallow.preferences.theme = 'dark';
console.log(original.preferences.theme, owned.preferences.theme);
console.log(original === shallow, original.preferences === shallow.preferences);
console.log(original.preferences === owned.preferences);

// Expected output:
// dark light
// false true
// false
```

1. The first literal creates root object A and nested preference object P.
2. The first spread creates root B. Reading A's preference property supplies P's identity, so B points to P too.
3. The second outer spread creates root C. Its explicit `preferences` entry overrides the initially copied value with new object Q.
4. The nested spread copies P's current theme string into Q.
5. Mutating through B's preference reference changes P. A observes dark; Q still holds light.

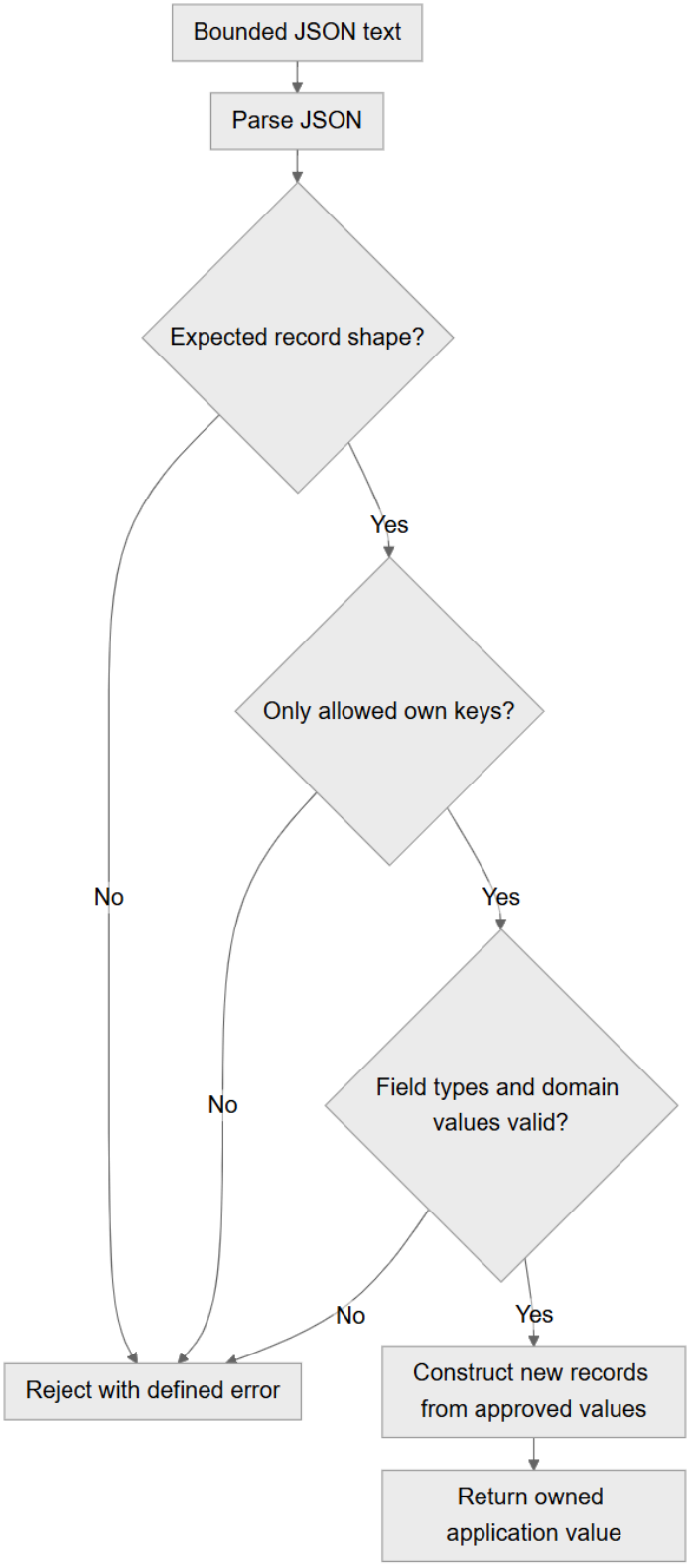
Memory Diagram

The snapshot after the final mutation is:



Source: `diagrams/volume-1-chapter-08-ownership.mmd`. Each arrow denotes a property or binding designating an object identity. The diagram makes no claim about concrete memory addresses or allocation size.

Safe Boundary Flow



Source: `diagrams/volume-1-chapter-08-record-boundary.mmd`. JSON parsing establishes a data representation, not an application schema. The subsequent checks decide whether that data belongs in the application.

How a Property Read Differs From an Own Check

For an ordinary data property, an own-property descriptor identifies its stored value. If the property is absent, ordinary lookup can continue through a prototype. An accessor descriptor instead invokes a getter to obtain a value. `Object.hasOwn` checks whether an own descriptor exists without using an inherited method on the record.

This explanation is intentionally about ordinary objects. Proxies can intercept operations, including own-property inspection. A schema that accepts parsed JSON starts from data objects instead of promising effect-free reflection over every possible JavaScript object. Volume 2 covers the underlying object protocols and proxies.

What Spread Actually Copies

For an object source, spread obtains its own keys, considers enumerable properties, reads their values, and creates data properties on the new target. Reading an accessor gets its current result; it does not copy the getter function as an accessor descriptor. A non-enumerable property is omitted. An object-valued result remains a shared identity.

These steps explain three common observations: a getter can run during spread; a read-only source property can become a writable property in the copy; and nested state can remain shared. Copying descriptors explicitly would be a different operation with a different contract.

Engine Representation and Cost

Engines may optimize predictable property access with internal shape information and caches, but those implementation choices do not change ownership semantics. V8 describes such choices in its fast properties article. Do not translate that article into a universal rule that adding one field always makes an application slow.

Copying `k` own enumerable properties requires considering those properties and constructing a result. Getter work can add arbitrary cost. A schema-specific copy visits only the records and fields required by its contract. Profile a real workload when allocation or traversal matters; first make the intended sharing visible and correct.

Production Examples

A Shipment Snapshot With Owned Nested Records

A shipping view accepts an ordinary data record containing string `id`, `address.city`, and `address.postalCode`. It keeps only these fields, trims them, and returns new root and address records. All retained leaves are primitive strings. The contract promises isolation for this schema, not a general clone of arbitrary input graphs.

```
'use strict';

function shipmentView(input) {
  function record(value, name) {
    if (value === null || typeof value !== 'object' || Array.isArray(value)) {
      throw new TypeError(`${name} must be a record`);
    }
  }
  function requiredText(value, name) {
    if (typeof value !== 'string') throw new TypeError(`${name} must be text`);
    const text = value.trim();
    if (text === '') throw new RangeError(`${name} must be nonempty`);
    return text;
  }
  record(input, 'shipment');
  if (!Object.hasOwn(input, 'id') || !Object.hasOwn(input, 'address')) {
    throw new TypeError('own id and address required');
  }
  record(input.address, 'address');
  if (!Object.hasOwn(input.address, 'city') || !Object.hasOwn(input.address, 'postalCode')) {
    throw new TypeError('own city and postalCode required');
  }
  return {
    id: requiredText(input.id, 'id'),
    address: {
      city: requiredText(input.address.city, 'city'),
      postalCode: requiredText(input.address.postalCode, 'postalCode')
    }
  };
}
const source = { id: ' S-1 ', address: { city: ' Pune ', postalCode: '411001' }, internalNote: 'private'
};
const view = shipmentView(source);
source.address.city = 'Mumbai';
view.address.postalCode = 'changed locally';
console.log(view.id, view.address.city, source.address.postalCode);
console.log(view === source, view.address === source.address, Object.hasOwn(view, 'internalNote'));

// Expected output:
// S-1 Pune 411001
// false false false
```

Copying the root alone would leave the address shared. Explicit construction also excludes the unrelated internal note. Validating nonempty postal-code text does not verify an actual postal format or deliverable address; a geographic service owns those rules. The companion `code/volume-1/chapter-08/example-01-owned-profile.js` checks bidirectional mutation isolation, missing own fields, wrong types, and blank text.

For total retained text length `L`, normalization takes O(L) work and output text storage. The object graph has a fixed number of records. If the schema gains an object-valued field, revisit the ownership contract before adding it to the result.

Parse Preferences With an Allowlist

The next boundary accepts JSON text of at most 4096 UTF-16 code units. Root keys may be `theme` and `notifications`; nested notification keys may be `email` and `sms`. Missing fields receive defaults. Explicit null or unsupported values are rejected. Unknown keys are rejected at both levels, including prototype-related key names.

```
'use strict';

function parsePreferences(text) {
  if (typeof text !== 'string') throw new TypeError('JSON text required');
  if (text.length > 4096) throw new RangeError('preferences text too large');
  const input = JSON.parse(text);
  function checkRecord(value, allowed, name) {
    if (value === null || typeof value !== 'object' || Array.isArray(value)) {
      throw new TypeError(`${name} must be a record`);
    }
    for (const key of Object.keys(value)) {
      if (!allowed.includes(key)) throw new TypeError(`unknown ${name} field: ${key}`);
    }
  }
  checkRecord(input, ['theme', 'notifications'], 'preferences');
  const theme = Object.hasOwn(input, 'theme') ? input.theme : 'light';
  if (theme !== 'light' && theme !== 'dark') throw new RangeError('unsupported theme');
  const notifications = Object.hasOwn(input, 'notifications') ? input.notifications : {};
  checkRecord(notifications, ['email', 'sms'], 'notifications');
  const email = Object.hasOwn(notifications, 'email') ? notifications.email : false;
  const sms = Object.hasOwn(notifications, 'sms') ? notifications.sms : false;
  if (typeof email !== 'boolean' || typeof sms !== 'boolean') {
    throw new TypeError('notification values must be booleans');
  }
  return { theme, notifications: { email, sms } };
}
console.log(JSON.stringify(parsePreferences('{}')));
console.log(JSON.stringify(parsePreferences('{"theme":"dark","notifications":{"email":true}}')));
for (const text of ['null', '{"theme":null}', '{"notifications":{"sms":"false"}}', '{"__proto__":
{"admin":true}}']) {
  try { parsePreferences(text); } catch (error) { console.log(error.name); }
}

// Expected output:
// {"theme":"light","notifications":{"email":false,"sms":false}}
// {"theme":"dark","notifications":{"email":true,"sms":false}}
// TypeError
// RangeError
```

```
// TypeError
// TypeError
```

The parser never assigns arbitrary input keys into a target or walks user-selected property paths. It constructs a fixed output schema with owned nested state. The input limit bounds text processed here; an HTTP server should also enforce its byte-level request limit before reading an oversized body. Malformed JSON propagates a SyntaxError so the caller can report a parse failure separately from a schema failure.

JSON input contains data rather than getters or proxies, which makes this boundary's assumptions concrete. This is not a validator for arbitrary live JavaScript objects. The fixed key allowlists mean validation work is linear in parsed keys and text size; the output has a fixed record shape. The companion `code/volume-1/chapter-08/example-02-safe-preferences.js` includes malformed input, nested unknown keys, independent defaults, null, array, and size-limit tests.

A Dictionary for Arbitrary String Labels

An application sometimes needs data keys rather than a fixed schema. A null-prototype dictionary avoids inherited Object.prototype property names. It does not decide whether any particular key should be authorized or copied into another target later.

```
'use strict';

const totals = Object.create(null);
for (const label of ['north', 'constructor', 'north', '__proto__']) {
  totals[label] = Object.hasOwn(totals, label) ? totals[label] + 1 : 1;
}
console.log(totals.north, totals.constructor, totals.__proto__);
console.log(Object.getPrototypeOf(totals) === null);

// Expected output:
// 2 1 1
// true
```

These names are ordinary own data keys on this dictionary. Use `Object.hasOwn` instead of expecting inherited methods. If keys need arbitrary object identity, a `Map` is a better fit; Arrays and Collections develops that choice.

Review the Returned Contract

For each API, state accepted representation, allowed keys, defaults, rejected values, mutation behavior, and returned ownership. A new outer object is one piece of evidence, not the entire contract. Assertions should change nested output and input records to verify the promised isolation.

Interview Perspective

Is a Spread Copy Independent?

```
'use strict';

const source = { label: 'before', nested: { enabled: true } };
const copy = { ...source };
copy.label = 'after';
copy.nested.enabled = false;
console.log(source.label, source.nested.enabled);
console.log(copy === source, copy.nested === source.nested);

// Expected output:
// before false
// false true
```

The copy owns new outer property slots. Its nested value is the same object identity. Explain independence per graph edge rather than saying "spread clones the object." A correct repair copies the records whose mutable state must be isolated, or documents intentional sharing.

Which Presence Test Fits a Request Field?

Question: A request must explicitly contain its own `enabled` field. Should a guard use `request.enabled !== undefined`, `in`, or `Object.hasOwn`?

Answer: Use `Object.hasOwn` to test the ownership requirement, then validate the value. A present own field can hold undefined, and `in` can accept inherited fields. Whether undefined is valid is a separate schema decision.

Explain a Destructuring Default

```
'use strict';

const first = { value: undefined };
const second = { value: null };
const { value: a = 'default' } = first;
const { value: b = 'default' } = second;
console.log(a, b);

// Expected output:
// default null
```

Defaults apply to undefined, not all falsy values. If null means explicit clearing, preserving it is often exactly what a patch API needs. If null is invalid, reject it rather than hoping a default will replace it.

Does Spread Preserve Descriptors?

No. It reads enumerable source values and creates own data properties on the new object. It can invoke a source getter but does not preserve that getter as an accessor. Non-enumerable properties are omitted, and a non-writable source property can produce a writable property in the copy. Descriptor-preserving copying requires a different operation, covered in Volume 2.

Can Property Access Execute Code?

Yes. A getter or proxy can execute during an apparently simple property read. `Object.entries`, destructuring, and spread can read values too. For a parsed-JSON contract, values are data objects and do not contain executable accessors. For arbitrary objects supplied by other code, specify whether effects and exceptional reads are permitted.

Does Object.freeze Make a Graph Immutable?

It changes only the object passed to it. A nested object can remain mutable, and accessor behavior does not become pure. Freezing the outer record blocks replacing a data-property value, not mutating the object already stored there. Ask whether each mutable record is owned and whether any mutable identity escapes.

Design Question: Safe User Preferences

Accept bounded JSON text, parse it, validate the root and nested record shapes, allowlist keys, validate values, apply defaults only according to a defined presence policy, and construct a fresh output schema. Do not recursively merge arbitrary request keys into application state. Test malformed JSON, null, arrays, unknown keys at both levels, wrong booleans, and independent defaults across calls.

Explain the Complexity

Copying `k` own enumerable properties requires inspecting those keys and values. Getter work adds its own cost. A fixed-schema snapshot with `L` characters of normalized text costs O(L) work plus a fixed number of record allocations. Repeatedly copying a growing dictionary can create quadratic cumulative work; correctness of ownership comes first, then profiling and data-structure choices.

Exercises and Coding Challenges

Run each solution independently in Node.js 20 or later. Assertions test ownership and boundary behavior rather than only logging a happy path. All input-object contracts here refer to ordinary data objects unless a getter is deliberately included to demonstrate property semantics.

Exercise 1: Require an Own Field

Requirements: `readOwn(record, key)` returns the value of an own property, even when that value is undefined. Reject null/nonobjects and nonstring keys with `TypeError`. Throw `RangeError` when the key is absent or only inherited. Do not call a method obtained from `record`.

Hint: Presence and value are separate checks; use the static own-property operation.

```
'use strict';
const assert = require('node:assert/strict');

function readOwn(record, key) {
  if (record === null || typeof record !== 'object' || typeof key !== 'string') {
    throw new TypeError('object and string key required');
  }
  if (!Object.hasOwn(record, key)) throw new RangeError('own field required');
  return record[key];
}
const record = Object.create({ inherited: 7 });
record.present = undefined;
record.hasOwnProperty = false;
assert.equal(readOwn(record, 'present'), undefined);
assert.equal(readOwn(record, 'hasOwnProperty'), false);
assert.throws(() => readOwn(record, 'inherited'), RangeError);
assert.throws(() => readOwn(record, 'missing'), RangeError);
assert.throws(() => readOwn(null, 'id'), TypeError);
assert.throws(() => readOwn({}, 4), TypeError);
console.log(Object.hasOwn(record, 'present'), readOwn(record, 'present'));

// Expected output:
// true undefined
```

The algorithm performs a fixed number of operations; property-operation cost depends on the engine and object representation. Reading an accessor could execute code, so the ordinary-data assumption matters.

Exercise 2: Apply a Patch With Three Presence States

Requirements: A trusted current delivery record contains primitive `id` and optional `note`. A patch may have only a `note` field. Missing means keep the existing note; a string, including an empty string, replaces it; null removes the property. Explicit undefined or another type must throw. Return a new outer record without mutating current or patch.

Hint: A destructuring default cannot distinguish all three states; check own presence first.

```
'use strict';
const assert = require('node:assert/strict');

function applyNotePatch(current, patch) {
  if (patch === null || typeof patch !== 'object' || Array.isArray(patch)) {
    throw new TypeError('patch record required');
  }
  for (const key of Reflect.ownKeys(patch)) {
    if (key !== 'note') throw new TypeError('unknown patch field');
```

```
  }
  const next = { ...current };
  if (!Object.hasOwn(patch, 'note')) return next;
  if (patch.note === null) delete next.note;
  else if (typeof patch.note === 'string') next.note = patch.note;
  else throw new TypeError('note must be text or null');
  return next;
}
const current = Object.freeze({ id: 'D-1', note: 'ring bell' });
assert.equal(applyNotePatch(current, {}).note, 'ring bell');
assert.equal(applyNotePatch(current, { note: '' }).note, '');
const cleared = applyNotePatch(current, { note: null });
assert.equal(Object.hasOwn(cleared, 'note'), false);
assert.equal(current.note, 'ring bell');
assert.notEqual(cleared, current);
assert.throws(() => applyNotePatch(current, { note: undefined }), TypeError);
assert.throws(() => applyNotePatch(current, { admin: true }), TypeError);
assert.throws(() => applyNotePatch(current, { [Symbol('extra')]: true }), TypeError);
assert.throws(() => applyNotePatch(current, null), TypeError);
console.log(current.note, Object.hasOwn(cleared, 'note'));

// Expected output:
// ring bell false
```

For `k` current fields and `p` patch fields, work is O(k + p), with O(k) output property storage. The current record is already trusted and has primitive leaves; a general nested patch API needs an explicit deeper policy.

Exercise 3: Copy the Changed Path

Requirements: Change a trusted profile's `preferences.theme` to light or dark. Return a new root and new preferences record. Preserve the existing email flag. Intentionally share the untouched `metadata` object. Reject an unsupported theme. Verify each identity claim.

Hint: Copy the root and preferences separately; do not claim full graph independence.

```
'use strict';
const assert = require('node:assert/strict');

function withTheme(profile, theme) {
  if (theme !== 'light' && theme !== 'dark') throw new RangeError('unsupported theme');
  return { ...profile, preferences: { ...profile.preferences, theme } };
}
const profile = {
  id: 'U-1',
  preferences: Object.freeze({ theme: 'light', email: true }),
  metadata: Object.freeze({ source: 'import' })
};
const next = withTheme(profile, 'dark');
assert.notEqual(next, profile);
assert.notEqual(next.preferences, profile.preferences);
assert.equal(next.metadata, profile.metadata);
assert.equal(next.preferences.email, true);
assert.equal(profile.preferences.theme, 'light');
assert.throws(() => withTheme(profile, 'sepia'), RangeError);
```

```
console.log(profile.preferences.theme, next.preferences.theme, next.metadata === profile.metadata);

// Expected output:
// light dark true
```

If the root has `r` enumerable fields and preferences has `p`, copying costs O(r + p) property operations and output slots. The shared metadata is intentionally frozen here. A mutable shared subtree would need a documented mutation policy.

Exercise 4: Explain the Descriptor Loss in a Copy

Requirements: Create an object with an enumerable read-only `id`, a non-enumerable `secret`, and an enumerable getter `status`. Spread it into a new object. Assert that secret is omitted, the getter runs once, and the copied id is writable. Explain why this is a value copy rather than a descriptor copy.

Hint: Inspect descriptors on the result, not just the printed values.

```
'use strict';
const assert = require('node:assert/strict');

let reads = 0;
const source = {
  get status() { reads += 1; return 'ready'; }
};
Object.defineProperty(source, 'id', { value: 'R-1', enumerable: true });
Object.defineProperty(source, 'secret', { value: 'hidden' });
const copy = { ...source };
assert.equal(reads, 1);
assert.equal(Object.hasOwn(copy, 'secret'), false);
assert.equal(Object.getOwnPropertyDescriptor(copy, 'id').writable, true);
assert.equal(Object.getOwnPropertyDescriptor(copy, 'status').get, undefined);
copy.id = 'R-2';
assert.equal(source.id, 'R-1');
console.log(copy.id, copy.status, reads);

// Expected output:
// R-2 ready 1
```

Spread considers the source's own keys and reads its enumerable values. General work depends on key count plus getter work. This fixed example has constant-sized state, and only the local `reads` counter is affected.

Exercise 5: Count Arbitrary String Labels Safely

Requirements: Accept an array of string labels, including empty strings and names such as constructor or **proto**. Return a null-prototype dictionary of counts. Reject non-array input and nonstring elements. Do not modify any prototype or the input array.

Hint: Start with a dictionary that has no inherited property names and test own presence.

```
'use strict';
const assert = require('node:assert/strict');

function countLabels(labels) {
  if (!Array.isArray(labels)) throw new TypeError('labels array required');
  const counts = Object.create(null);
  for (const label of labels) {
    if (typeof label !== 'string') throw new TypeError('string labels required');
    counts[label] = Object.hasOwn(counts, label) ? counts[label] + 1 : 1;
  }
  return counts;
}
const labels = Object.freeze(['__proto__', 'constructor', '__proto__', '']);
const counts = countLabels(labels);
assert.equal(Object.getPrototypeOf(counts), null);
assert.equal(counts.__proto__, 2);
assert.equal(counts.constructor, 1);
assert.equal(counts[''], 1);
assert.equal(Object.keys(countLabels([])).length, 0);
assert.throws(() => countLabels(null), TypeError);
assert.throws(() => countLabels(['north', 1]), TypeError);
console.log(counts.__proto__, counts.constructor, counts['']);

// Expected output:
// 2 1 1
```

The algorithm makes O(n) dictionary operations for `n` labels and stores O(u) entries for `u` distinct labels, with string-key processing costs additional. Do not feed the dictionary into a generic unsafe merge later. Map is another option covered in the next chapter.

Exercise 6: Publish a Frozen Two-Level Status View

Requirements: A trusted status record has primitive string `id` and `details.state` fields. Publish a new root and a new details record containing only those values. Freeze both new records so strict-mode callers cannot change the published fields. The source must remain mutable and independent; extra source fields must be excluded.

Hint: Copy before freezing, and freeze each owned record explicitly. Do not write a universal recursive-freeze utility.

```
'use strict';
const assert = require('node:assert/strict');

function publishStatus(source) {
  if (source === null || typeof source !== 'object' || Array.isArray(source) ||
      !Object.hasOwn(source, 'id') || !Object.hasOwn(source, 'details') ||
      typeof source.id !== 'string') throw new TypeError('status record required');
  const details = source.details;
  if (details === null || typeof details !== 'object' || Array.isArray(details) ||
      !Object.hasOwn(details, 'state') || typeof details.state !== 'string') {
    throw new TypeError('details state text required');
  }
  return Object.freeze({
    id: source.id,
```

```
    details: Object.freeze({ state: details.state })
  });
}
const source = { id: 'S-1', details: { state: 'ready', internal: 'private' } };
const view = publishStatus(source);
source.details.state = 'sent';
assert.equal(view.details.state, 'ready');
assert.equal(Object.isFrozen(source), false);
assert.equal(Object.isFrozen(source.details), false);
assert.equal(Object.hasOwn(view.details, 'internal'), false);
assert.throws(() => { view.id = 'changed'; }, TypeError);
assert.throws(() => { view.details.state = 'changed'; }, TypeError);
assert.throws(() => publishStatus({ id: 'S-1', details: null }), TypeError);
console.log(view.details.state, source.details.state, Object.isFrozen(view.details));

// Expected output:
// ready sent true
```

This fixed record schema requires O(1) property operations and two new objects. The retained leaves are immutable strings, so freezing those two records establishes the stated graph policy. New object-valued fields would require a fresh ownership and freezing decision.

Companion Program

`code/volume-1/chapter-08/example-03-ownership-challenges.js` runs all six solutions and their assertions. Keep the identity assertions: checking only equal values would miss several of the original sharing defects.

Multiple-Choice Questions

Choose one answer per question. Each explanation distinguishes the operation or ownership contract being tested.

1. Which test requires a property to belong directly to a record?

- A. Boolean(record[key])
- B. Object.hasOwn(record, key)
- C. key in record
- D. record[key] !== undefined

Answer: B. Own presence is distinct from inherited lookup, value, and truthiness.

2. What does object spread copy from an ordinary object?

- A. All inherited properties

- B. All property descriptors
- C. Only string keys, whether enumerable or not
- D. Own enumerable string and symbol property values

Answer: D. Spread copies enumerable own values. It does not preserve descriptors or recursively clone nested objects.

3. Which destructuring value triggers a default?

- A. undefined
- B. null
- C. 0
- D. false

Answer: A. A default applies when the extracted value is undefined; other falsy values are preserved.

4. A spread copy contains a nested object. What is true by default?

- A. The nested object is recursively copied
- B. The nested object is frozen
- C. The nested identity is shared
- D. The nested property is omitted

Answer: C. A shallow copy stores the same object-valued reference in its new outer property.

5. What does Object.keys include?

- A. Own enumerable string keys
- B. Every own symbol and string key
- C. Inherited enumerable keys only
- D. Own non-enumerable keys only

Answer: A. Use Reflect.ownKeys when all own string and symbol keys are needed.

6. What are omitted flags for a newly defined Object.defineProperty data property?

- A. Inherited from the prototype
- B. Writable true, others false
- C. False

- D. All true

Answer: C. For a new property, omitted writable, enumerable, and configurable flags default to false.

7. Why is record.hasOwnProperty(key) unsafe as a general record operation?

- A. It always includes inherited keys
- B. The record can shadow the method or have no inherited method
- C. It reads every field recursively
- D. It can only accept numeric keys

Answer: B. Object.hasOwn avoids relying on a method obtained from the record itself.

8. What can happen while spreading an accessor property?

- A. The getter is always copied without running
- B. The property becomes inherited
- C. The target becomes a proxy
- D. The source getter runs to produce the copied value

Answer: D. Spread reads the source value and creates an own data property containing that result.

9. How does Object.assign differ from spread into a new literal?

- A. It never reads getters
- B. It ignores symbol keys in every case
- C. It always deep-clones nested values
- D. Its writes can invoke setters on the existing target

Answer: D. Assignment writes and data-property creation have different observable behavior.

10. What does Object.freeze guarantee for an ordinary object with a nested data object?

- A. Its nested values are cloned
- B. No getter can have effects
- C. Every reachable object becomes immutable
- D. Its own data properties cannot be reassigned, while nested objects may remain mutable

Answer: D. Freeze applies to the object passed, not automatically to its reachable graph.

11. A property has value undefined. What follows about its presence?

- A. Presence still requires a separate check
- B. It must be absent
- C. It cannot be enumerable
- D. It must be inherited

Answer: A. Undefined can be stored in an own property or obtained from a missing property.

12. What can deletion of an own property reveal?

- A. A new lexical binding
- B. An automatic clone
- C. A frozen descriptor for that key
- D. An inherited value at the same key

Answer: D. Ordinary lookup may continue to the prototype after the own property is removed.

13. Why use explicit public-view fields instead of removing one secret field with rest?

- A. Rest always copies inherited fields
- B. Rest never copies strings
- C. An allowlist authorizes every caller
- D. An allowlist avoids automatically exposing future internal fields

Answer: D. Publishing only approved fields keeps future schema additions private unless deliberately exposed.

14. What is true of a null-prototype dictionary?

- A. It authorizes all copied keys for every future consumer
- B. It cannot store **proto** as data
- C. It has no Object.prototype inheritance and can hold arbitrary string data keys
- D. It inherits hasOwnProperty

Answer: C. The dictionary avoids inherited name collisions, but subsequent consumers still need safe key handling.

15. Which boundary best prevents arbitrary preference keys from controlling a merge?

- A. Validate allowed keys at every supported level and construct a fixed result
- B. Only call JSON.parse
- C. Only freeze the output after a generic recursive merge
- D. Only reject unknown root keys, ignoring nested keys

Answer: A. Parsing and freezing do not substitute for validation before keys drive operations.

16. An immutable update copies root and preferences but shares metadata. What is the correct claim?

- A. All old records become unreachable
- B. No additional objects were allocated
- C. The entire graph is independent
- D. Only the copied path is independent; metadata remains shared

Answer: D. Ownership must be described per retained identity, including intentionally shared subgraphs.

Revision Sheet and Summary

Property Operations

Need	Operation or rule
Read a fixed identifier-named property	`record.name`
Read a computed or non-identifier key	`record[key]`
Check own presence	`Object.hasOwn(record, key)`
Check presence including inheritance	`key in record`
Enumerate own enumerable string keys	`Object.keys(record)`
Enumerate every own string/symbol key	`Reflect.ownKeys(record)`
Read a default only for undefined	Destructuring default
Remove a configurable own property	`delete record.key`

Need	Operation or rule
Copy own enumerable values into a new outer record	Object spread

Ownership Rules

A second binding or property can hold the same object identity. Spreading an object makes a new outer record while retaining object-valued references. Copy each modified path when the update must preserve an earlier view. A schema-specific snapshot should name exactly which nested records it owns and which leaves are primitive.

Freezing is shallow. Symbol keys are discoverable. Destructuring can read inherited properties. A missing field, undefined, null, false, zero, and an empty string are different states; defaults and presence checks should reflect the API's policy.

Descriptor Rules

A data property has a value and writable flag; an accessor property has getter/setter behavior. Both have enumerable and configurable flags. New properties created by `Object.defineProperty` default omitted flags to false. Spread reads values and does not preserve source descriptors or prototypes.

Boundary Rules

Parse data using a documented representation. Validate shape, own keys, field types, and domain values. Build the allowed output schema explicitly. A null-prototype dictionary is useful for string data keys, but copying those keys later into another target still requires care. Do not use a generic recursive merge as a substitute for a schema.

Summary

Property access explains where a value comes from; ownership explains who can change the object behind that value. Correct record APIs make both explicit. Prove isolation by mutating input and output nested records in tests, and prove schema discipline with missing, inherited, unknown, and malformed fields.

Continue with Arrays and Collections. Volume 2's Prototypes and Inheritance develops inherited lookup beyond the ordinary-record model used here.

References and Further Reading

- MDN enumerability and ownership: compare traversal operations.
- MDN Object.hasOwn: distinguish own presence from lookup.
- MDN destructuring: extraction, renaming, defaults, and rest.
- MDN Object.defineProperty: descriptor defaults and restrictions.

- MDN prototype pollution: data-key hazards and defenses.
- V8 fast properties: implementation context, not a portable allocation guarantee.

Edge Cases and Debugging

A Data Field Can Shadow a Method

```
'use strict';

const record = { hasOwnProperty: false, id: 'R-1' };
console.log(Object.hasOwn(record, 'id'));
try { record.hasOwnProperty('id'); } catch (error) { console.log(error.name); }
const dictionary = Object.create(null);
dictionary.id = 'R-2';
console.log(Object.hasOwn(dictionary, 'id'), typeof dictionary.hasOwnProperty);

// Expected output:
// true
// TypeError
// true undefined
```

The record's data key hides the inherited method. A null-prototype dictionary does not inherit the method at all. The static own-property operation avoids both assumptions.

Copying Can Evaluate a Getter

```
'use strict';

let reads = 0;
const source = {
  get status() { reads += 1; return 'ready'; }
};
const copy = { ...source };
console.log(reads, copy.status, reads);
console.log(typeof Object.getOwnPropertyDescriptor(copy, 'status').get);
copy.status = 'sent';
console.log(copy.status);

// Expected output:
// 1 ready 1
// undefined
// sent
```

The getter runs while spread reads its value. The result is an ordinary data property, so later reads of the copy do not invoke the source getter. If copying triggers an unexpected exception or expensive computation, inspect descriptors before assuming the copy operation was passive.

Assignment and Spread Differ at a Target Setter

```
'use strict';

const writes = [];
const target = { set status(value) { writes.push(value); } };
Object.assign(target, { status: 'assigned' });
const copy = { ...target, status: 'spread' };
console.log(writes.join('|'), copy.status);
console.log(Object.getOwnPropertyDescriptor(copy, 'status').writable);

// Expected output:
// assigned spread
// true
```

`Object.assign` writes to the supplied target and invokes its setter. Spread builds data properties on a new object. This matters for observable behavior and for dangerous property names; the two forms are not interchangeable merges.

Deleting Can Expose an Inherited Value

```
'use strict';

const defaults = { region: 'APAC' };
const local = Object.create(defaults);
local.region = 'EU';
console.log(local.region, Object.hasOwn(local, 'region'));
delete local.region;
console.log(local.region, Object.hasOwn(local, 'region'));

// Expected output:
// EU true
// APAC false
```

Deletion removed the local property, not the prototype's property. If absence means "disabled" in your API, ordinary lookup through defaults may implement the wrong policy. Inspect own presence separately.

A Special Literal Key Is Not the Same as Parsed Data

```
'use strict';

const parsed = JSON.parse('{"__proto__":{"admin":true}}');
const literal = { __proto__: { admin: true } };
console.log(Object.hasOwn(parsed, '__proto__'), Object.hasOwn(parsed, 'admin'));
console.log(Object.hasOwn(literal, '__proto__'), literal.admin);
console.log(Object.getPrototypeOf(parsed) === Object.prototype);

// Expected output:
// true false
```

```
// false true
// true
```

The parsed JSON contains an own data key. The special colon-form object-literal entry sets the new object's prototype instead. Neither example mutates Object.prototype. Trouble arises when later code treats attacker-controlled keys as assignment targets or recursive traversal paths. Reject unknown keys before building trusted state; do not mistake successful parsing for safe merging.

Debugging Sequence

Locate the exact property and owner. Check own presence, value, and descriptor. Draw the identities reachable through nested fields. Identify whether an operation reads inherited data, invokes an accessor, or writes through a setter. Then reduce the failure to an assertion about the promised schema or ownership boundary.

Performance and Security

Copy the Needed Graph

A shallow copy visits the source's own enumerable properties; a schema-specific nested copy visits the records and leaves that the contract retains. A blanket recursive clone can process far more state and needs policies for cycles, dates, collections, functions, and special objects. Do not introduce a generic clone merely to copy a three-field shipping view.

If `n` updates each spread a growing `n`-key object, cumulative property-copy work can become O(n squared). A single owned mutable accumulator may be appropriate during construction, followed by one published result. For immutable application state, structural sharing can preserve old views while avoiding copies of unchanged subgraphs, provided shared subgraphs obey the intended mutation policy.

Measure Shapes Rather Than Guessing

Engines can specialize property access based on observed object layouts. That does not make every dynamic key or deletion a production bottleneck. First determine record counts, copied fields, string processing, and retention. Profile the workload before trading clear schema code for opaque shape-management tricks. V8's fast properties article gives one implementation's context.

Keep Untrusted Keys Out of Control Paths

Prototype pollution can occur when untrusted property names become assignment targets or recursive paths through objects. The preference parser avoids that class of merge by allowing a fixed list of keys at each level and constructing the result explicitly. Checking only the root is insufficient if nested keys later drive traversal. MDN's prototype-pollution guidance explains these paths and defenses.

A null-prototype dictionary treats names such as `__proto__` as ordinary own data keys and avoids inherited lookup. It does not make those keys safe to feed into an unrelated generic merge later. A Map also separates entries from object properties and is often useful for arbitrary keys; choose it based on the API and serialization contract.

Own Presence Is Not Authorization

An own `admin: true` property is still client-controlled. A valid boolean is not evidence of permission. Obtain privileged decisions from the authoritative application layer. Similarly, symbol properties and non-enumerable fields are still discoverable and accessible to code that has the object; they do not protect secrets.

Publish an Allowlist, Not a Growing Denylist

Removing only a field called `password` with object rest can accidentally expose a new sensitive field added later. A public view should explicitly name the fields it publishes and copy or omit nested data deliberately. Tests should assert exact output keys as well as expected values.

JSON text boundaries remove executable getters and proxies from the accepted representation, but parsing still needs size limits and schema validation. Do not use serialization as a universal deep-clone or sanitization guarantee; it has its own supported types and loss behavior.

Retention and Error Evidence

Copies can retain shared nested graphs, while full snapshots can multiply storage. Measure what remains reachable after a workflow completes. For errors, record the rejected field and failure category without dumping complete request records or internal notes into logs.

Professional Field Guide

Case Study: Canceling an Edit Does Not Restore Preferences

A form stores `draft = { ...saved }`, then changes `draft.notifications.email`. Canceling the form discards the outer draft, but the saved record has already changed because both roots pointed to the same notification object.

The repair creates a new notification record for the draft and any other mutable records the form edits. A regression test changes the draft's nested property, discards the draft, and asserts the saved value. A second test changes saved state and verifies whether the draft should remain a snapshot or intentionally reflect live updates. The product's expectation determines the ownership contract.

Case Study: A New Field Leaks Through a Public View

A service removes `password` using object rest and returns everything else. Later a developer adds an internal review note to the user record. The rest operation publishes that new field automatically.

Construct an explicit public view with approved fields. If it includes an address object, copy the approved address fields too. Assert exact keys at both levels. This change turns a denylist that must anticipate future secrets into an output schema that must deliberately opt into new data.

Case Study: Preferences Accept an Unexpected Key

A configuration endpoint recursively merges all parsed keys into defaults. A request supplies a prototype-related key at a nested level. The bug is not JSON parsing; it is allowing data keys to control object traversal and assignment.

Replace the generic merge with a fixed preference schema. Validate own keys at every supported record level, distinguish absent settings from explicit invalid values, and return new records containing only allowed primitive values. The production example demonstrates this design with no global prototype mutation.

Choose the Contract Before the Syntax

Requirement	Appropriate starting point	What to verify
Read a possibly inherited default	Ordinary property lookup	Is inheritance intended?
Require submitted field presence	`Object.hasOwn` plus value validation	Does undefined or null have a special meaning?
Create a historical view	Explicit schema copy	Are all retained mutable records owned?
Update immutable state	Copy each changed path	Are unchanged references intentionally shared?
Count arbitrary string labels	Null-prototype dictionary or Map	Will later consumers treat keys safely?
Publish external data	Explicit allowed output keys	Can future internal fields leak?

A Code Review Walkthrough

Ask which object owns each field and who can still reach the nested values. Inspect presence checks and default policies. Identify any operation that reads getters or invokes setters. For external data, identify the accepted representation and where unknown keys are rejected. Then inspect tests that mutate both sides of a claimed snapshot and tests for nested unknown keys.

A good explanation uses a small reference graph and an exact schema. It does not rely on the phrases "spread makes it immutable" or "the object is const." Use the revision sheet and exercises to rehearse these decisions before continuing to Arrays and Collections.

CHAPTER 9

Arrays and Collections

Arrays model ordered sequences. Sets model uniqueness and membership. Maps associate keys with values without forcing every key into a string. Choosing among them makes operations and ownership clearer: an invoice has ordered lines, a deduplication step has seen IDs, and a catalog has a product lookup.

This chapter connects collection syntax to behavior that affects real programs: holes differ from stored undefined, copying an array does not copy its records, callbacks receive more arguments than just a value, and sorting needs an explicit numeric comparator.

Learning Objectives

- Read and update indexed arrays while understanding length and holes.
- Choose map, filter, reduce, find, some, every, and ordinary loops by the required result.
- Distinguish mutation from copying and explain shallow ownership.
- Sort validated numeric values and preserve equal-key ordering.
- Create arrays with Array.from, destructuring, and spread.
- Use Set/Map membership, identity, insertion order, and SameValueZero equality.
- Build a bounded, validated collection pipeline with explicit output ownership.

Prerequisites and Running Guide

Read Control Flow, Functions and Callbacks, and Objects. Be comfortable with loops, callbacks, property access, reference identity, and shallow object copying.

Every JavaScript block runs independently with Node.js 20 or newer. This baseline includes copying methods such as toSorted. Assertions use node:assert/strict; expected-output comments specify deliberate console output.

```
const ids = ['p2', 'p1', 'p2'];
const unique = [...new Set(ids)];
const stock = new Map([['p1', 8], ['p2', 3]]);
console.log(ids.join(','));
console.log(unique.join(','));
console.log(stock.get('p2'));

// Expected output:
// p2,p1,p2
// p2,p1
// 3
```

The array preserves repeated positions. The Set preserves first-seen unique IDs. The Map answers a keyed lookup. None requires treating an ID as a numeric quantity.

Reading and Practice Route

Read theory, trace iteration and references, then review the validated production pipelines. Solve the six challenges before checking their solutions and the MCQs.

Companion programs live in `code/volume-1/chapter-09/`:

```
node code/volume-1/chapter-09/example-00-collection-model.js
node code/volume-1/chapter-09/example-01-order-summary.js
node code/volume-1/chapter-09/example-02-member-index.js
node code/volume-1/chapter-09/example-03-collection-challenges.js
node code/volume-1/chapter-09/example-04-edge-cases.js
```

Theory

Arrays Are Objects With Indexed Sequence Behavior

Array indices start at zero. The length is a nonnegative integer greater than every present own array index; it is not a count of present elements. Increasing length creates absent positions. Deleting the final element leaves length unchanged, so an array can also have trailing holes without an explicit length increase. Reducing length removes ordinary configurable elements beyond the new boundary. Reading an absent index ordinarily returns undefined, which does not prove that an element exists there.

Array.isArray(value) distinguishes arrays from other objects. Array elements can hold different types, but a collection with a clear item schema is easier to validate and process.

```
const items = ['a', 'b'];
items[4] = 'e';
items.label = 'batch';
console.log(items.length, Object.keys(items).join(','));
console.log(items[2], Object.hasOwn(items, 2));
console.log(items.at(-1), items[-1]);
items.length = 2;
console.log(items.join(','), items.length);
delete items[1];
console.log(items.length, Object.hasOwn(items, 1));

// Expected output:
// 5 0,1,4,label
// undefined false
// e undefined
// a,b 2
// 2 false
```

A named property such as label is not a sequence element and does not affect length. at(-1) requests the last element; bracket access with -1 requests the property named "-1". Avoid using arrays as arbitrary-key dictionaries. Array reference.

Dense Arrays, Holes, and Undefined

A dense array has a present element at each index below length. A hole is an absent property at one of those positions. An explicit undefined element is present. Creating Array(3), assigning a far index, increasing length, or deleting an element can create holes.

```
const sparse = [10, , undefined];
const visited = [];
const mapped = sparse.map((value, index) => { visited.push(index); return value; });
console.log(visited.join(','));
console.log(Object.hasOwn(mapped, 1), Object.hasOwn(mapped, 2));
console.log([...sparse].length, Object.hasOwn([...sparse], 1));
console.log(sparse.includes(undefined));

// Expected output:
// 0,2
// false true
// 3 true
// true
```

Under ordinary unmodified prototypes, map skips holes and preserves them in its result. filter skips holes and packs selected values densely. for...of, array spread, and find read holes as undefined. includes can therefore match undefined in a hole. If indexed properties exist on a prototype, some inherited reads/presence checks differ; production dense-array validation can require own indices explicitly.

Deleting an array element does not shift later indices or shrink length. Use splice when removal should shift positions, or filter when building a new selected sequence.

Choose the Method From the Result

Need	Method	Result and stopping behavior
Transform each present item	map	New array, preserving length/holes.
Select items	filter	New dense array of accepted items.
Aggregate	reduce	One accumulator; supply an initial value.
First matching value	find	Value or undefined; stops at a match.
First matching index	findIndex	Index or -1; distinguishes a found undefined.

Need	Method	Result and stopping behavior
Any item matches	some	Boolean; stops at first truthy predicate.
Every present item matches	every	Boolean; stops at first falsy predicate.
Perform effects per present item	forEach	undefined; no normal break.
Explicit control, early exit, or multiple state updates	for...of / for loop	Statements make control flow visible.

Callbacks receive value, index, and array for many of these methods. Passing a function that interprets those extra arguments differently can cause a bug. Wrap a callback when its intended interface is narrower.

```
const prices = [12, 3, 8];
console.log(prices.map(value => value * 2).join(','));
console.log(prices.filter(value => value >= 8).join(','));
console.log(prices.reduce((total, value) => total + value, 0));
console.log(prices.find(value => value > 5));
console.log(prices.some(value => value < 5), prices.every(value => value > 0));
console.log(['10', '10', '10'].map(parseInt).join(','));
console.log(['10', '10', '10'].map(value => parseInt(value, 10)).join(','));

// Expected output:
// 24,6,16
// 12,8
// 23
// 12
// true true
// 10,NaN,2
// 10,10,10
```

map passes indices 0, 1, 2 as parseInt's radix argument. Radix 0 infers decimal for "10", radix 1 is invalid, and radix 2 interprets it as binary. The wrapper deliberately supplies radix 10. map reference.

Empty Collections Have Defined Answers

A supplied reduce accumulator makes an empty sum zero. Without it, reduce needs a first present element and throws when there is none. Empty some is false; empty every is true because there is no counterexample. Neither says a collection is nonempty.

```
console.log([].reduce((sum, value) => sum + value, 0));
console.log([].some(() => true), [].every(() => false));
try { [].reduce((sum, value) => sum + value); }
catch (error) { console.log(error.name); }
const values = [undefined];
console.log(values.find(value => value === undefined));
console.log(values.findIndex(value => value === undefined));
```

```
// Expected output:
// 0
// false true
// TypeError
// undefined
// 0
```

A find result of undefined may mean no match or a matching undefined value. Choose findIndex or a tagged result when that distinction matters. reduce reference.

Mutation and Copying Are Separate API Choices

Mutates the receiver	Common copying alternative
sort	toSorted
reverse	toReversed
splice	toSpliced
Indexed replacement	with
push/pop, shift/unshift	Construct a new array when ownership requires it.
fill, copyWithin	Copy first if modifying caller-owned storage would be wrong.

slice, concat, map, and filter create arrays but do not recursively copy their items. A new outer array can still share every contained object with the source.

```
const original = [{ count: 2 }, { count: 5 }];
const copied = original.slice();
copied[0].count = 9;
copied.push({ count: 1 });
console.log(original.length, copied.length);
console.log(original[0].count);
console.log(original[0] === copied[0]);
const detached = original.map(item => ({ count: item.count }));
detached[0].count = 4;
console.log(original[0].count, detached[0].count);

// Expected output:
// 2 3
// 9
// true
// 9 4
```

The explicit projection owns the one-level record schema used here. If a record contains nested objects, copying its fields is still shallow. Choose a contract-specific snapshot instead of claiming that spread or map automatically performs a deep clone.

Numeric Sorting Requires a Comparator

Default sorting compares string representations. For finite Numbers, a - b supplies ascending numeric order; b - a supplies descending order. Comparator results below zero, zero, and above zero mean before, tied, and after. A boolean comparator cannot consistently express all three relationships.

```
const counts = [10, 2, 30];
console.log(counts.toSorted().join(','));
console.log(counts.toSorted((a, b) => a - b).join(','));
console.log(counts.join(','));
const rows = [{ id: 'a', rank: 2 }, { id: 'b', rank: 1 }, { id: 'c', rank: 2 }];
console.log(rows.toSorted((a, b) => a.rank - b.rank).map(row => row.id).join(','));
console.log(counts.sort((a, b) => a - b) === counts);

// Expected output:
// 10,2,30
// 2,10,30
// 10,2,30
// b,a,c
// true
```

Sorting is stable: tied items retain their original relative order. Comparators should be pure, consistent, reflexive, antisymmetric, and transitive. Validate finite numeric keys before subtracting them; NaN or unintended strings can undermine the intended ordering. The language does not guarantee one sort algorithm or universal time/space bound. sort reference.

Construction, Destructuring, and Spread

Array.from consumes an iterable or array-like object and can map each position during construction. It fills positions rather than preserving holes from an array-like source. Spread consumes an iterable. Array destructuring consumes positions; defaults apply to undefined, and rest collects remaining values into a new array.

```
const sequence = Array.from({ length: 3 }, (_, index) => index + 1);
const [first, second = 20, ...remaining] = [1, undefined, 3, 4];
console.log(sequence.join(','));
console.log(first, second, remaining.join(','));
console.log(Array(3).map(() => 1).join('|'));
console.log(Array.from({ length: 3 }, () => 1).join('|'));

// Expected output:
// 1,2,3
// 1 20 3,4
// ||
// 1|1|1
```

Array(3).map never invokes its callback for the absent positions. Array.from with a mapping function creates real elements. Bound length before allocating an array from external data. Array.from.

Set: Uniqueness and Membership

Set stores each distinct value once and iterates in insertion order. add, has, delete, and size express membership operations directly. Equality is SameValueZero: NaN matches NaN, positive/negative zero match, and objects match only by identity. A new object with equal fields is a different member.

```
const key = { id: 'u1' };
const seen = new Set([NaN, NaN, 0, -0, key, { id: 'u1' }]);
console.log(seen.size);
console.log(seen.has(NaN), seen.has({ id: 'u1' }), seen.has(key));
console.log([...new Set(['b', 'a', 'b'])].join(','));

// Expected output:
// 4
// true false true
// b,a
```

Deduplicate records by an explicit ID projection if equal IDs should represent one entity. A Set of the records themselves tests reference uniqueness. The specification requires average sublinear access, without promising a particular implementation. Set reference.

Map: Values Indexed by Keys

Map permits primitive or object keys and uses the same key equality as Set. It preserves key insertion order. Updating an existing key changes its value without moving its position; deleting then reinserting places it at the end. get returns undefined for both an absent key and a stored undefined, so use has when presence matters.

```
const catalog = new Map([['b', 2], ['a', undefined]]);
catalog.set('b', 3);
console.log([...catalog.keys()].join(','));
console.log(catalog.get('a'), catalog.has('a'), catalog.has('missing'));
catalog.delete('b');
catalog.set('b', 4);
console.log([...catalog.keys()].join(','));
catalog.set('__proto__', 'data');
console.log(catalog.get('__proto__'));

// Expected output:
// b,a
// undefined true false
// a,b
// data
```

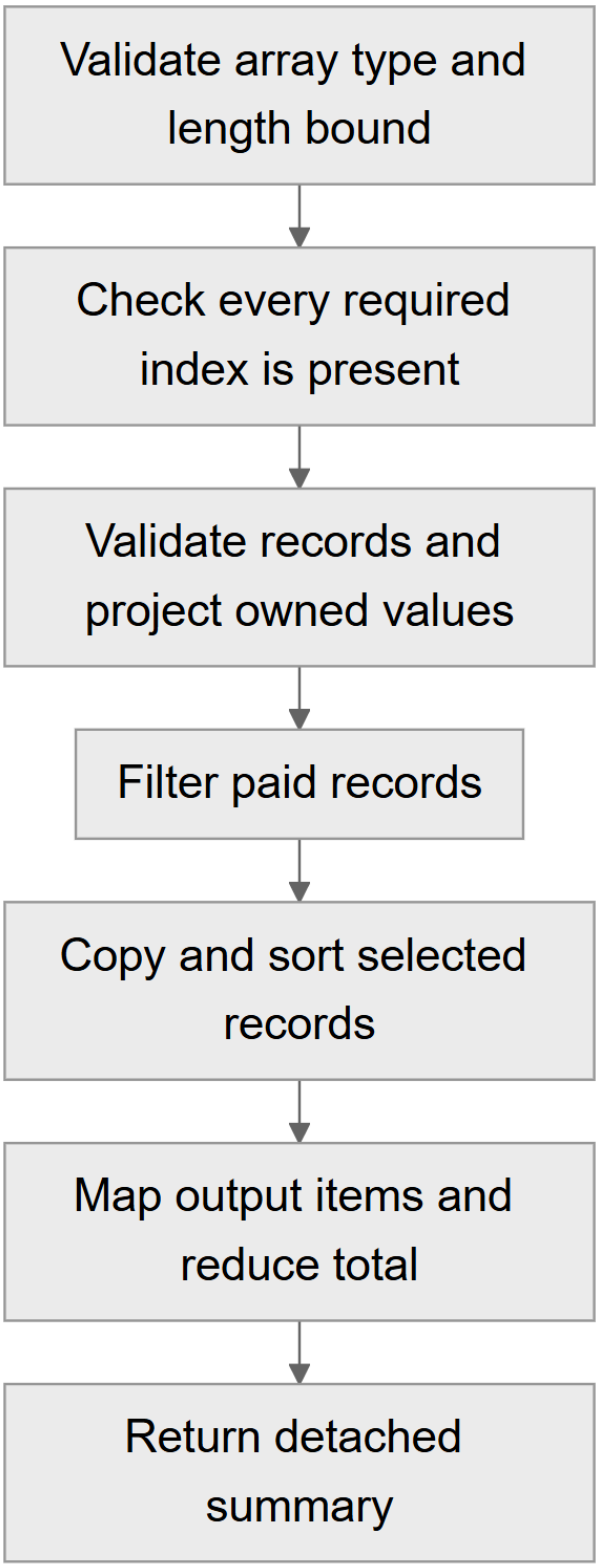
Use set/get/has for entries. Bracket assignment such as map[key] writes an ordinary object property and does not create a Map entry. Keys such as "**proto**" are ordinary Map entry keys, avoiding the prototype-linked behavior of some object-property operations. A Map still needs size limits and a clear lifetime. Map reference.

Internal Working

A Collection Pipeline Has Separate Stages

The production order summary validates shape and ownership before transformation. It rejects holes, creates normalized records containing only the supported primitive fields, selects paid orders, sorts a copy, and constructs the result.

Stored source: `diagrams/volume-1-chapter-09-pipeline-flow.mmd`.



Validation before filtering matters. If an invalid cancelled order were filtered out first, malformed input could pass unnoticed. The contract rejects an invalid item anywhere in the batch.

How Map Visits an Array

Conceptually, map obtains the initial length, creates a result array, then considers indices in increasing order. At each index it checks presence, obtains the current value when present, calls the callback, and places the returned value at the corresponding output index. Absent indices remain absent under ordinary prototypes.

The initial length is captured, but future values are not snapshotted. Appending beyond that initial range does not add callbacks; changing a not-yet-visited element can affect what the callback sees.

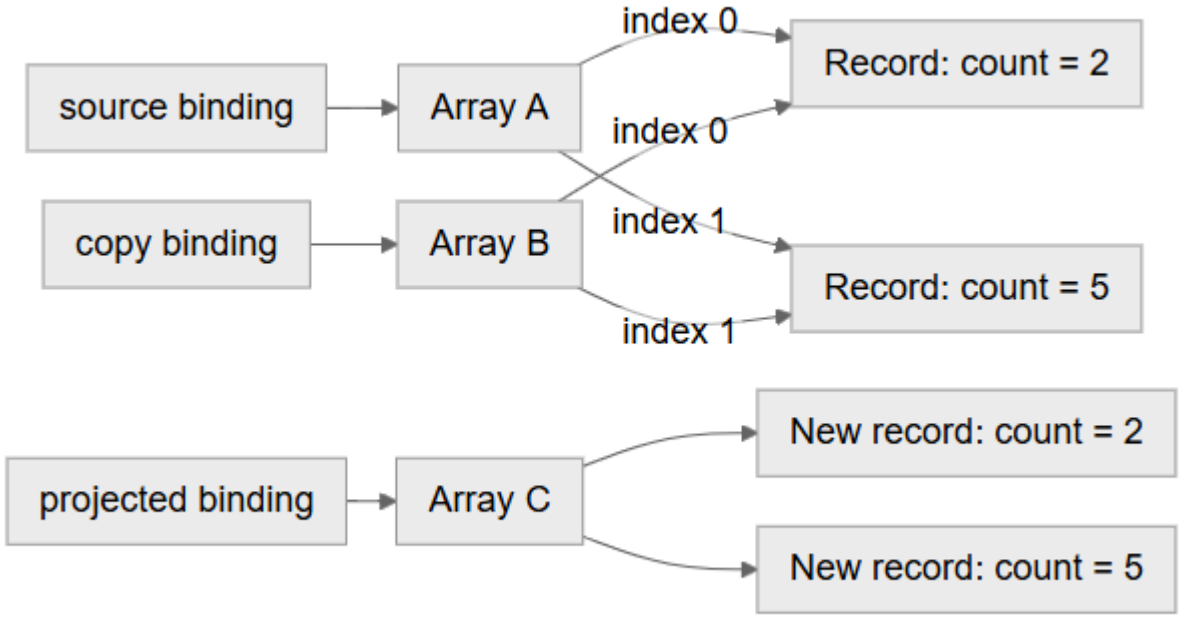
```
const input = [1, 2, 3];
const visited = [];
const output = input.map((value, index) => {
  visited.push(value);
  if (index === 0) {
    input[1] = 20;
    input.push(4);
  }
  return value * 2;
});
console.log(visited.join(','));
console.log(output.join(','));
console.log(input.join(','));

// Expected output:
// 1,20,3
// 2,40,6
// 1,20,3,4
```

This demonstrates the rule, not a recommended transformation style. A callback that mutates its input makes reasoning harder; normally keep mapping callbacks pure and place intentional mutation in explicit statements. Callback execution here is synchronous and ordered. Returning promises from map creates an array of promises; it does not automatically wait for them.

Shallow Copying in Memory

Stored source: `diagrams/volume-1-chapter-09-shallow-memory.mmd`.



Array B has independent index slots, but those slots point to the same records as Array A. Replacing B[0] leaves A[0] alone; changing B[0].count changes the shared record. Array C comes from projecting each primitive count into a new record, so that specific schema is detached.

This is an identity model, not a guarantee of physical engine addresses. Engines may use different storage layouts while preserving aliasing and observable operations.

A Reduce Trace Has an Accumulator

```
const trace = [];
const total = [4, 6, 3].reduce((accumulator, value, index) => {
  trace.push(index + ':' + accumulator + '+' + value);
  return accumulator + value;
}, 0);
console.log(trace.join(','));
console.log(total);

// Expected output:
// 0:0+4,1:4+6,2:10+3
// 13
```

The callback's return becomes the next accumulator. Forgetting return supplies undefined to the next call. Without an initial value, the first present element becomes the accumulator and callback iteration starts at the next present index. A fixed initial value makes empty behavior explicit.

An accumulator can be an object or array when that is useful. Mutating a newly created, locally owned accumulator can avoid repeatedly copying its entire contents. Do not mistake every local mutation for mutation of caller-owned input.

Collection Identity and Equality

A Map stores a key/value association independently from how the key's object fields later change. Changing key.id does not reindex the Map by that property; the key remains the same object identity. If the domain needs lookup by ID text, store the ID text as the key.

```
const user = { id: 'u1' };
const byObject = new Map([[user, 'active']]);
const byId = new Map([[user.id, 'active']]);
user.id = 'u2';
console.log(byObject.get(user));
console.log(byObject.has({ id: 'u2' }));
console.log(byId.has('u1'), byId.has('u2'));

// Expected output:
// active
// false
// true false
```

The string key is the original primitive value; it does not follow later property changes. The object key retains the object, which also has implications for a long-lived Map's memory usage.

Engines and Complexity

Engines may specialize arrays based on element kinds and density; those are implementation strategies, not additional language types. Avoid holes and mixed representations when the domain naturally supplies dense homogeneous data, but do not invent correctness rules from a specific engine's optimization.

A scan over n bounded records is O(n). A copying operation usually requires O(n) result storage. Sorting has an implementation-dependent cost; denote it S(n) when making a portable claim. Map/Set access is specified as average sublinear, so an expected constant-time model is a common engineering assumption rather than a universal language guarantee.

The map algorithm describes observable iteration, while V8's elements kinds article provides one engine's storage perspective.

Production Examples

Build a Detached Paid-Order Summary

A report accepts at most 1000 ordinary order records. Every index must be present. Each record requires a unique nonempty ID of at most 64 characters, status paid/pending/cancelled, and an integer-cent amount from zero through 100,000,000. Unknown fields are ignored.

The function validates every record, including records later excluded from the report. It sorts paid orders by descending amount, then by ID code-unit order for a deterministic tie break. It returns fresh item records containing only primitive fields and does not mutate or retain caller-owned records.

```
'use strict';
const assert = require('node:assert/strict');

function summarizeOrders(input) {
  if (!Array.isArray(input)) throw new TypeError('orders must be an array');
  if (input.length > 1000) throw new RangeError('at most 1000 orders');
  const seen = new Set();
  const normalized = [];
  for (let index = 0; index < input.length; index += 1) {
    if (!Object.hasOwn(input, index)) throw new TypeError('orders must be dense');
    const row = input[index];
    if (row === null || typeof row !== 'object' || Array.isArray(row)) {
      throw new TypeError('order must be a data record');
    }
    const { id, status, amountCents } = row;
    if (typeof id !== 'string' || id.length < 1 || id.length > 64) {
      throw new TypeError('order id must be a bounded string');
    }
    if (!['paid', 'pending', 'cancelled'].includes(status)) throw new RangeError('unknown status');
    if (!Number.isSafeInteger(amountCents) || amountCents < 0 || amountCents > 100000000) {
      throw new RangeError('amountCents out of range');
    }
    if (seen.has(id)) throw new RangeError('duplicate order id');
    seen.add(id);
    normalized.push({ id, status, amountCents });
  }
  const paid = normalized.filter(row => row.status === 'paid');
  const ordered = paid.toSorted((a, b) =>
    b.amountCents - a.amountCents || (a.id < b.id ? -1 : a.id > b.id ? 1 : 0)
  );
  return {
    paidCount: ordered.length,
    totalCents: ordered.reduce((sum, row) => sum + row.amountCents, 0),
    items: ordered.map(({ id, amountCents }) => ({ id, amountCents }))
  };
}

const input = [
  { id: 'b', status: 'paid', amountCents: 500 },
  { id: 'a', status: 'paid', amountCents: 500 },
  { id: 'c', status: 'pending', amountCents: 900 }
];
const result = summarizeOrders(input);
assert.deepEqual(result, {
  paidCount: 2, totalCents: 1000,
  items: [{ id: 'a', amountCents: 500 }, { id: 'b', amountCents: 500 }]
});
assert.deepEqual(input.map(row => row.id), ['b', 'a', 'c']);
assert.notEqual(result.items[0], input[1]);
assert.throws(() => summarizeOrders(new Array(1)), TypeError);
assert.throws(() => summarizeOrders([input[0], input[0]]), RangeError);
console.log(JSON.stringify(result));
console.log(JSON.stringify(summarizeOrders([])));
```

```
// Expected output:
// {"paidCount":2,"totalCents":1000,"items":[{"id":"a","amountCents":500},{"id":"b","amountCents":500}]}
// {"paidCount":0,"totalCents":0,"items":[]}

// O(n) scans plus sorting cost S(p), under usual expected Set lookup costs.
// O(n) owned records/index storage plus the sort implementation's workspace.
```

The first loop establishes the complete input contract and creates owned normalized records. filter selects by status; toSorted preserves the paid array's order while creating a sorted result; map projects output fields; reduce produces the total from an explicit zero accumulator.

With 1000 orders and at most 100,000,000 cents per order, the maximum total is 100,000,000,000 cents. It remains an exact safe integer. Increasing either bound requires reviewing that product. These are integer-cent inputs; accepting decimal currency strings would need a separate parser and rounding policy.

The output is detached for the stated primitive-field schema, not recursively cloned arbitrary data. A caller may edit result.items without changing the source order records. Unknown nested metadata is deliberately omitted.

Companion: `code/volume-1/chapter-09/example-01-order-summary.js`. It also asserts both ownership directions, all invalid root shapes, unsafe/nonintegral amounts, invalid excluded records, duplicate IDs, and both batch-size boundaries.

Build a Team-to-Member Index

A membership report receives at most 1000 dense events. Each event contains a team string of length 1 through 40 and a member string of length 1 through 64. Values are case-sensitive opaque labels; the function does not trim or normalize identities.

The result is a new Map from team to a new Set of member IDs. Teams and members retain first-seen order. Duplicate membership events are intentionally idempotent. No input event object is retained.

```
'use strict';
const assert = require('node:assert/strict');

function buildMemberIndex(events) {
  if (!Array.isArray(events)) throw new TypeError('events must be an array');
  if (events.length > 1000) throw new RangeError('at most 1000 events');
  const groups = new Map();
  for (let index = 0; index < events.length; index += 1) {
    if (!Object.hasOwn(events, index)) throw new TypeError('events must be dense');
    const event = events[index];
    if (event === null || typeof event !== 'object' || Array.isArray(event)) {
      throw new TypeError('event must be a data record');
    }
    const { team, member } = event;
    if (typeof team !== 'string' || team.length < 1 || team.length > 40 ||
        typeof member !== 'string' || member.length < 1 || member.length > 64) {
      throw new TypeError('team and member must be bounded strings');
    }
    if (!groups.has(team)) groups.set(team, new Set());
    groups.get(team).add(member);
```

```
  }
  return groups;
}

const input = [
  { team: 'blue', member: 'u1' }, { team: 'blue', member: 'u1' },
  { team: 'red', member: 'u2' }, { team: 'blue', member: 'u3' }
];
const groups = buildMemberIndex(input);
assert.deepEqual([...groups.keys()], ['blue', 'red']);
assert.deepEqual([...groups.get('blue')], ['u1', 'u3']);
assert.equal(groups.get('red').size, 1);
assert.throws(() => buildMemberIndex([null]), TypeError);
assert.throws(() => buildMemberIndex(new Array(1)), TypeError);
const report = Array.from(groups, ([team, members]) => ({ team, members: [...members] }));
console.log(JSON.stringify(report));

// Expected output:
// [{"team":"blue","members":["u1","u3"]},{"team":"red","members":["u2"]}]

// Expected O(n) time with conventional Map/Set implementations; O(n) entry storage.
// ECMAScript guarantees average sublinear access, not a universal O(1) bound.
```

A Map models the grouping key directly; each Set models unique membership. Updating an existing team does not move its insertion position. Using a Set of event objects would not deduplicate two distinct objects with equal team/member text.

The returned Map and Sets are mutable and owned by the caller. That is an explicit API choice: callers can extend their local index without altering input records. The report projection creates arrays suitable for JSON output; JSON.stringify on a Map itself would not serialize entries as this report expects.

Companion: `code/volume-1/chapter-09/example-02-member-index.js`. Assertions cover duplicate events, "**proto**" as ordinary entry data, input/output independence, empty input, holes, invalid labels, and the 1000-event bound.

Failure and Trust Contracts

Both functions reject malformed batches rather than silently dropping bad records. Local temporary collections become unreachable when a validation error escapes, and inputs remain unchanged. The caller can decide how to report a rejected batch; the helpers do not log raw records or perform external writes.

Inputs are ordinary application data, typically produced by parsing a supported format. Reading a getter or iterating a proxy can run arbitrary JavaScript; these functions are not object sandboxes. The explicit own-index checks ensure a dense data contract, while request-size limits must also be enforced before deserialization.

Trade-offs to Review

The order example favors visible stages and ownership over minimizing temporary arrays. Its input is bounded to 1000 items. For a measured memory bottleneck, a single loop may combine projection, selection, and aggregation while preserving validation of every item. Sorting still needs the selected items.

The membership index retains at most the accepted batch's keys and members. A service-wide persistent index needs independent lifetime, eviction, and update policies. Map and Set make collection operations explicit; they do not provide those policies automatically.

Read Map and Set for entry semantics, and toSorted for the copying sort behavior.

Interview Perspective

Explain the Collection Contract First

Before choosing a method, identify whether order matters, duplicates are meaningful, lookup is by position or key, and callers share the input objects. Then state size and item-type assumptions. Those choices explain why an array, Set, Map, or a combination is appropriate.

Is an Array Copy Independent?

Its element slots are independent, but object values may remain shared. A slice or spread copy can be reordered without reordering the source. Mutating a shared item's property still affects both arrays. A detached report should project the required primitive fields into fresh records.

```
const original = [{ id: 'a', count: 1 }];
const copy = [...original];
copy[0].count = 2;
copy.push({ id: 'b', count: 3 });
console.log(original.length, original[0].count);
console.log(copy === original, copy[0] === original[0]);

// Expected output:
// 1 2
// false true
```

A follow-up may introduce a nested address object. Spreading each record once still shares its address; ownership must be described for the actual graph, not inferred from a single spread expression.

Why Does Array(3).map Produce No Mapped Values?

`Array(3)` creates a length-three array with absent positions. map skips those positions. `Array.from({length: 3}, mapper)` instead visits each generated position and creates a dense array. Storing undefined explicitly also creates a present element that map will visit.

Which Method Expresses the Desired Question?

Use some for existence, every for universal validation, find for a first matching value, and filter for every matching value. filter followed by a length check traverses the full collection and allocates a result even when only one match was needed.

Empty every is true. If a rule requires at least one valid item, include a nonempty check and handle holes deliberately. A sparse array can pass every without validating absent positions.

What Does Map Promise About Mutation During Iteration?

The array map method captures its initial length. Its callback reads later values when their indices are reached, so mutations to those values can be observed. Appending beyond the initial range does not add mapping callbacks. Explain this as a visitation rule, not an immutable snapshot.

Also distinguish array.map from the Map collection. Their similar names describe different APIs.

Why Is a Boolean Sort Comparator Wrong?

A comparator must express negative, zero, and positive ordering. Returning a > b provides only true/false, numerically 1/0, and treats one direction as a tie. Use a - b for validated finite Numbers and a deliberate multi-field comparator for records. Do not claim an exact output for an inconsistent comparator.

Stable sorting preserves input order for tied keys. toSorted returns a new array; sort mutates and returns the receiver. Neither deep-clones records.

Does Set Deduplicate Equal-Looking Objects?

Only if they are the same object reference. To deduplicate business entities by ID, compare their ID values and define first-wins, last-wins, merge, or reject behavior. A Map keyed by ID can express the policy directly.

```
const first = { id: 'u1' };
const second = { id: 'u1' };
console.log(new Set([first, second, first]).size);
console.log(new Set([first.id, second.id]).size);
const lookup = new Map([['u1', undefined]]);
console.log(lookup.get('u1'), lookup.has('u1'));
console.log(lookup.get('missing'), lookup.has('missing'));

// Expected output:
// 2
// 1
// undefined true
// undefined false
```

The last two lines demonstrate why get alone does not always answer existence. In a schema that forbids undefined values, the ambiguity may be excluded by contract; state that assumption.

Worked Design Prompt: Order Summary

A report must list paid orders, show the total, reject duplicate IDs, and leave input unchanged. A strong solution validates every bounded record, uses a Set for duplicate detection, selects paid items, sorts a copy, and projects owned output records. Test an invalid excluded record as well as a valid paid one. Otherwise filtering before validation could hide malformed input.

Discuss the maximum total before writing arithmetic. The production contract limits count and cents per item so the sum remains a safe integer. Discuss scan costs, retained arrays, and sorting separately; the language does not fix one sorting algorithm.

The production examples implement both this design and a Map-of-Sets membership index. The exercises test different ownership and duplication policies.

Exercises and Coding Challenges

Solve each contract before reading its solution. All programs are independent and assert ownership as well as values. Map/Set complexity uses conventional expected lookup costs; the language itself requires average sublinear access.

The companion `code/volume-1/chapter-09/example-03-collection-challenges.js` runs the six solutions.

1. Chunk a Dense Sequence

Problem: Accept a dense array of at most 10,000 values and an integer chunk size from 1 through 1000. Return new outer and chunk arrays in source order. Keep element references unchanged; do not mutate the input. Reject holes.

Hint: Validate density, then copy bounded slices.

Solution

```
const assert = require('node:assert/strict');
function chunk(values, size) {
  if (!Array.isArray(values)) throw new TypeError('values must be an array');
  if (values.length > 10000 || !Number.isInteger(size) || size < 1 || size > 1000) {
    throw new RangeError('invalid length or chunk size');
  }
  for (let index = 0; index < values.length; index += 1) {
    if (!Object.hasOwn(values, index)) throw new TypeError('values must be dense');
  }
  const chunks = [];
  for (let start = 0; start < values.length; start += size) {
    chunks.push(values.slice(start, start + size));
  }
  return chunks;
}
const object = { id: 'a' };
```

```
const input = [object, 2, 3, 4, 5];
const result = chunk(input, 2);
assert.deepEqual(result, [[object, 2], [3, 4], [5]]);
assert.notEqual(result[0], input);
assert.equal(result[0][0], object);
assert.equal(input.length, 5);
assert.deepEqual(chunk([], 2), []);
assert.throws(() => chunk(new Array(1), 2), TypeError);
assert.throws(() => chunk(input, 0), RangeError);
console.log(result.map(part => part.length).join(','));

// Expected output:
// 2,2,1

// O(n) time and O(n) copied element slots, plus chunk-array overhead.
```

The copies own their sequence slots, not the objects stored in those slots. Changing result[0][0].id would affect the same object held by the input; replacing result[0][0] would not.

2. Deduplicate IDs in First-Seen Order

Problem: Accept at most 1000 dense string IDs, each with length 1 through 64. Return a new array containing the first occurrence of each ID. Matching is exact and case-sensitive. Reject invalid items instead of coercing them.

Hint: Set iteration already preserves insertion order.

Solution

```
const assert = require('node:assert/strict');
function uniqueIds(values) {
  if (!Array.isArray(values)) throw new TypeError('ids must be an array');
  if (values.length > 1000) throw new RangeError('too many ids');
  const seen = new Set();
  for (let index = 0; index < values.length; index += 1) {
    if (!Object.hasOwn(values, index)) throw new TypeError('ids must be dense');
    const id = values[index];
    if (typeof id !== 'string' || id.length < 1 || id.length > 64) {
      throw new TypeError('invalid id');
    }
    seen.add(id);
  }
  return [...seen];
}
const input = ['b', 'a', 'b', 'A'];
assert.deepEqual(uniqueIds(input), ['b', 'a', 'A']);
assert.deepEqual(input, ['b', 'a', 'b', 'A']);
assert.deepEqual(uniqueIds([]), []);
assert.throws(() => uniqueIds(['a', 1]), TypeError);
assert.throws(() => uniqueIds(new Array(1)), TypeError);
console.log(uniqueIds(input).join(','));

// Expected output:
// b,a,A
```

```
// Expected O(n) time under conventional Set lookup costs; O(u) unique-ID storage.
```

Sorting before deduplication would change first-seen order. Lowercasing would merge A and a and therefore change the identity contract. The Set holds primitive strings rather than input record references.

3. Count Labels With a Map

Problem: Count a dense array of at most 1000 nonempty string labels, each at most 40 characters. Return a Map in first-seen key order. Accept labels such as '**proto**' as ordinary data.

Hint: Use get/set for Map entries and zero as the absent count.

Solution

```
const assert = require('node:assert/strict');
function countLabels(labels) {
  if (!Array.isArray(labels)) throw new TypeError('labels must be an array');
  if (labels.length > 1000) throw new RangeError('too many labels');
  const counts = new Map();
  for (let index = 0; index < labels.length; index += 1) {
    if (!Object.hasOwn(labels, index)) throw new TypeError('labels must be dense');
    const label = labels[index];
    if (typeof label !== 'string' || label.length < 1 || label.length > 40) {
      throw new TypeError('invalid label');
    }
    counts.set(label, (counts.get(label) ?? 0) + 1);
  }
  return counts;
}
const counts = countLabels(['open', '__proto__', 'open']);
assert.equal(counts.get('open'), 2);
assert.equal(counts.get('__proto__'), 1);
assert.deepEqual([...counts.keys()], ['open', '__proto__']);
assert.equal(countLabels([]).size, 0);
assert.throws(() => countLabels(['']), TypeError);
console.log(JSON.stringify([...counts]));

// Expected output:
// [["open",2],["__proto__",1]]

// Expected O(n) time with conventional Map lookup costs; O(u) distinct-key storage.
```

Every stored count is a positive integer, so undefined from get unambiguously denotes absence in this schema. For a general Map that permits undefined values, use has when existence itself matters.

4. Sort Tasks Without Changing Their Owner's Order

Problem: Accept at most 1000 dense task records with an integer priority from 0 through 5. Return a new array ordered from highest priority to lowest. Preserve arrival order for equal priorities and retain the same task objects. Do not mutate input records or order.

Hint: A pure numeric comparator plus stable copying sort satisfies the ownership contract.

Solution

```
const assert = require('node:assert/strict');
function orderTasks(tasks) {
  if (!Array.isArray(tasks)) throw new TypeError('tasks must be an array');
  if (tasks.length > 1000) throw new RangeError('too many tasks');
  for (let index = 0; index < tasks.length; index += 1) {
    if (!Object.hasOwn(tasks, index)) throw new TypeError('tasks must be dense');
    const task = tasks[index];
    if (task === null || typeof task !== 'object' || Array.isArray(task) ||
        !Number.isInteger(task.priority) || task.priority < 0 || task.priority > 5) {
      throw new TypeError('invalid task priority');
    }
  }
  return tasks.toSorted((a, b) => b.priority - a.priority);
}
const input = [
  { id: 'a', priority: 2 }, { id: 'b', priority: 5 }, { id: 'c', priority: 2 }
];
const sorted = orderTasks(input);
assert.deepEqual(sorted.map(task => task.id), ['b', 'a', 'c']);
assert.deepEqual(input.map(task => task.id), ['a', 'b', 'c']);
assert.equal(sorted[0], input[1]);
assert.notEqual(sorted, input);
assert.deepEqual(orderTasks([]), []);
assert.throws(() => orderTasks([{ priority: NaN }]), TypeError);
console.log(sorted.map(task => task.id).join(','));

// Expected output:
// b,a,c

// O(n) validation plus sort cost S(n); O(n) output slots plus sort workspace.
```

Stable sorting preserves the two priority-2 tasks in a,c order. The result shares records intentionally; copying order is not a detached snapshot. The amount and sequence of comparator calls are implementation details.

5. Partition Readings and Preserve Rejected Positions

Problem: Accept a dense array of at most 1000 arbitrary values. Accept only finite Number readings with absolute value at most 1,000,000. Return accepted values and rejected indices in input order; invalid readings are reported, while a malformed root or hole throws. Do not coerce strings.

Hint: A locally owned accumulator can collect both outputs in one reduce.

Solution

```
const assert = require('node:assert/strict');
function partitionReadings(values) {
  if (!Array.isArray(values)) throw new TypeError('values must be an array');
```

```
  if (values.length > 1000) throw new RangeError('too many readings');
  for (let index = 0; index < values.length; index += 1) {
    if (!Object.hasOwn(values, index)) throw new TypeError('values must be dense');
  }
  return values.reduce((result, value, index) => {
    if (Number.isFinite(value) && Math.abs(value) <= 1000000) result.accepted.push(value);
    else result.rejectedIndices.push(index);
    return result;
  }, { accepted: [], rejectedIndices: [] });
}
const input = [3, '4', NaN, -2, 1000001, null];
assert.deepEqual(partitionReadings(input), {
  accepted: [3, -2], rejectedIndices: [1, 2, 4, 5]
});
assert.deepEqual(partitionReadings([]), { accepted: [], rejectedIndices: [] });
assert.throws(() => partitionReadings(new Array(1)), TypeError);
assert.equal(input[1], '4');
console.log(JSON.stringify(partitionReadings(input)));

// Expected output:
// {"accepted":[3,-2],"rejectedIndices":[1,2,4,5]}

// O(n) time and O(n) output storage.
```

The accumulator belongs exclusively to this call, so push avoids repeated copying without mutating the source. Returning the accumulator on every path is essential. The explicit precheck prevents reduce from silently skipping a hole.

6. Join Requests to a Validated Catalog

Problem: A catalog contains at most 1000 unique product records with a nonempty SKU (maximum 40 characters) and integer priceCents from 0 through 1,000,000. Requests contain at most 1000 SKU strings. Both arrays must be dense. Return new product records in request order, retaining repeated requests. Reject duplicate catalog keys and unknown requested SKUs.

Hint: Build a Map once, then use has/get to resolve each requested key.

Solution

```
const assert = require('node:assert/strict');
function resolveProducts(catalog, requested) {
  function requireDense(values) {
    if (!Array.isArray(values)) throw new TypeError('array required');
    if (values.length > 1000) throw new RangeError('too many items');
    for (let index = 0; index < values.length; index += 1) {
      if (!Object.hasOwn(values, index)) throw new TypeError('dense array required');
    }
  }
  function validSku(sku) {
    return typeof sku === 'string' && sku.length >= 1 && sku.length <= 40;
  }
  requireDense(catalog);
  requireDense(requested);
  const prices = new Map();
```

```
  for (const row of catalog) {
    if (row === null || typeof row !== 'object' || Array.isArray(row) ||
        !validSku(row.sku) || !Number.isInteger(row.priceCents) ||
        row.priceCents < 0 || row.priceCents > 1000000) throw new TypeError('invalid product');
    if (prices.has(row.sku)) throw new RangeError('duplicate catalog sku');
    prices.set(row.sku, row.priceCents);
  }
  return requested.map(sku => {
    if (!validSku(sku)) throw new TypeError('invalid requested sku');
    if (!prices.has(sku)) throw new RangeError('unknown sku');
    return { sku, priceCents: prices.get(sku) };
  });
}
const catalog = [{ sku: 'a', priceCents: 0 }, { sku: 'b', priceCents: 250 }];
const output = resolveProducts(catalog, ['b', 'a', 'b']);
assert.deepEqual(output.map(row => row.priceCents), [250, 0, 250]);
assert.notEqual(output[0], output[2]);
output[0].priceCents = 1;
assert.equal(catalog[1].priceCents, 250);
assert.equal(output[2].priceCents, 250);
assert.throws(() => resolveProducts(catalog, ['missing']), RangeError);
assert.throws(() => resolveProducts([catalog[0], catalog[0]], []), RangeError);
assert.throws(() => resolveProducts(catalog, new Array(1)), TypeError);
console.log(JSON.stringify(resolveProducts(catalog, ['b', 'a'])));

// Expected output:
// [{"sku":"b","priceCents":250},{"sku":"a","priceCents":0}]

// Expected O(c + r) time with conventional Map lookup costs.
// O(c + r) storage for the index and fresh result records.
```

Repeated find calls could scan the catalog for every request. The Map gives one index-building pass followed by keyed lookups. Presence checking must not use truthiness: a valid free product has price zero.

MCQs

Assume ordinary unmodified prototypes and Node.js 20 or newer. Choose one answer before reading the explanation.

1. What happens to length after deleting index 1 from [1, 2, 3]?

A. It becomes 2 B. It becomes 1 C. It remains 3, with a hole at index 1 D. Deletion throws for every array

Answer: C. delete removes a property without shifting later elements or changing length.

2. How many callbacks does Array(3).map(() => 1) invoke?

A. 0 B. 1 C. 3 D. It throws before invoking any callback

Answer: A. The three positions are absent. map skips holes.

3. Which statement about [undefined].map(callback) is correct?

A. It behaves exactly like Array(1).map(callback) B. It invokes callback only if the callback accepts an index C. It always throws D. It invokes callback once for the present undefined element

Answer: D. An explicit undefined value is a present element, unlike a hole.

4. What does [].every(() => false) return?

A. false B. true C. undefined D. TypeError

Answer: B. There is no counterexample in an empty array. A nonempty requirement needs a separate check.

5. What happens to [].reduce((a, b) => a + b) without an initial value?

A. TypeError B. It returns 0 C. It returns undefined D. It returns an empty array

Answer: A. There is no first present element to use as the accumulator.

6. Which method tests whether any value matches without constructing every match?

A. filter B. map C. some D. reduce without an initial value

Answer: C. some stops at the first truthy predicate result and returns a boolean.

7. Why can find returning undefined be ambiguous?

A. It converts all falsy values to undefined B. It always skips explicit undefined elements C. It returns an index for successful matches D. A matching item itself may be undefined

Answer: D. Use findIndex when the distinction between a found undefined and no match matters.

8. Which call returns a numerically sorted copy of finite Numbers?

A. values.sort() B. values.toSorted((a, b) => a - b) C. values.sort((a, b) => a > b) D. values.toSorted()

Answer: B. toSorted copies the array, and the comparator supplies numeric order.

9. If two records compare equal in a stable sort, what happens?

A. Their original relative order is preserved B. They are always reversed C. One record is removed D. The comparator must be called exactly once for that pair

Answer: A. Stability concerns output order, not a fixed comparison schedule.

10. What does an array spread copy do to object elements?

A. Deep-clones each object B. Freezes each object C. Copies their references into new array slots D. Converts each object to a string

Answer: C. Reordering the outer array is independent; mutating a shared record is still visible through both arrays.

11. What is new Set([NaN, NaN, 0, -0]).size?

A. 4 B. 3 C. 1 D. 2

Answer: D. SameValueZero equates NaN with NaN and positive zero with negative zero.

12. What does `new Set([{id: 1}, {id: 1}]).size` return?

A. 1 B. 2 C. 0 D. TypeError

Answer: B. The two objects have distinct identities even though their fields match.

13. Which operation creates a Map entry?

A. map.set(key, value) B. map[key] = value C. map.key = value D. `Object.assign(map, {[key]: value})`

Answer: A. Map entries use set/get/has; ordinary property assignment does not populate the entry collection.

14. Updating an existing Map key with set does what to iteration order?

A. Moves it to the front B. Moves it to the end C. Retains its position D. Sorts all keys lexicographically

Answer: C. Deleting and then reinserting would move it to the end.

15. Why does ['10', '10', '10'].map(parseInt) produce surprising values?

A. map invokes callbacks in reverse order B. parseInt mutates the source array C. map omits every repeated string D. map passes its index as parseInt's radix argument

Answer: D. The radices become 0, 1, and 2. A wrapper can pass the intended radix explicitly.

16. What is guaranteed about Map/Set access complexity?

A. Every operation is worst-case O(1) B. Average access must be sublinear in collection size C. Every implementation must use a hash table D. Every lookup scans the entire collection

Answer: B. The standard permits different implementations; a typical expected constant-time model is not a universal guarantee.

Revision and Summary

Collection Decision Sheet

Requirement	Starting point
Ordered sequence, duplicates meaningful	Array
Unique values or membership checks	Set
Keyed association with arbitrary key types	Map
Transform each item	map
Select all matching items	filter
Aggregate to a result	reduce with an explicit initial value, or a loop
Find the first item/index	find / findIndex
Test any/all items	some / every
Sort without reordering the source	toSorted with an appropriate comparator
Detach output records	Explicitly project the supported schema into fresh objects

Rules to Recall Precisely

- length counts possible positions, not present elements.

- A hole differs from a stored undefined.
- map skips holes; Array.from and spread can create present undefined values from them.
- map/filter/reduce callbacks operate synchronously; map does not await promises.
- Most callback methods receive value, index, and array.
- Empty some is false; empty every is true.
- reduce without an initial value fails when no present element exists.
- Array copying is shallow unless your own projection copies the relevant records.
- Default sorting compares strings; finite numeric ordering needs a comparator.
- Stable sorting retains tie order; comparator call count is not fixed.
- Map/Set use SameValueZero and compare object keys by identity.
- Map.get(undefined-valued key) and Map.get(missing key) both return undefined; has distinguishes them.

A Compact Prediction Check

```
const sparse = [1, , 3];
console.log(sparse.length, Object.keys(sparse).length);
console.log(sparse.map(value => value * 2).join(','));
console.log([...sparse].includes(undefined));
console.log([10, 2].toSorted((a, b) => a - b).join(','));
console.log(new Set([NaN, NaN, 0, -0]).size);
console.log(new Map([['x', undefined]]).has('x'));

// Expected output:
// 3 2
// 2,,6
// true
// 2,10
// 2
// true
```

Production Review Questions

State the maximum batch size, dense/sparse policy, item schema, duplicate policy, sort order, and ownership of every returned record. Prove aggregate integer bounds when totals matter. Decide whether invalid items reject the batch, are skipped, or appear in a rejection report.

Count traversal and storage separately. A clear bounded pipeline can be preferable to a fused loop; a measured allocation problem can justify combining stages. Repeated catalog scans may benefit from an index, but an index needs a lifetime.

References and Further Reading

- Array: indices, length, and holes.

- map: callback arguments and visitation.
- reduce: initial values and accumulator behavior.
- sort: stable ordering and comparator contracts.
- Map and Set: key equality, iteration, and operation semantics.
- V8 elements kinds: one engine's array storage optimizations; keep them separate from portable semantics.

Continue to Errors and Debugging to make collection failures, assertion failures, and propagation policies explicit.

Edge Cases and Debugging

Length Does Not Prove Density

```
const input = Array(2);
console.log(input.length);
console.log(input.every(value => Number.isFinite(value)));
console.log(Object.hasOwn(input, 0), Object.hasOwn(input, 1));
const dense = Array.from(input);
console.log(Object.hasOwn(dense, 0), dense[0]);

// Expected output:
// 2
// true
// false false
// true undefined
```

every skips absent positions, so this check does not establish two finite numbers. Require own indices or reject undefined after a deliberate densification step, according to the contract. Densifying changes holes into values and is therefore a semantic change.

Mutation Can Skip Work or Change Later Values

```
const input = [1, 2, 3];
const seen = [];
input.forEach((value, index) => {
  seen.push(value);
  if (index === 0) input.splice(1, 1);
});
console.log(seen.join(','));
console.log(input.join(','));

// Expected output:
// 1,3
// 1,3
```

After removing the second element, 3 moves into index 1 and is visited there. The original 2 never reaches the callback. If processing must reflect the original batch, use a stable owned snapshot and define whether nested record mutations are allowed.

Repeated Fill Values Share Identity

```
const shared = Array(3).fill({ count: 0 });
shared[0].count = 1;
console.log(shared.map(item => item.count).join(','));
const independent = Array.from({ length: 3 }, () => ({ count: 0 }));
independent[0].count = 1;
console.log(independent.map(item => item.count).join(','));

// Expected output:
// 1,1,1
// 1,0,0
```

fill repeats one reference. The Array.from callback constructs a new record per position. The same concern applies to pushing the same mutable accumulator into a result repeatedly.

Map Entries Are Not Object Properties

```
const lookup = new Map();
lookup['u1'] = 'property';
lookup.set('u2', 'entry');
console.log(lookup.size, lookup.has('u1'));
console.log(lookup['u1'], lookup.get('u2'));
console.log(JSON.stringify(lookup));
console.log(JSON.stringify([...lookup]));

// Expected output:
// 1 false
// property entry
// {"u1":"property"}
// [["u2","entry"]]
```

JSON serializes enumerable object properties here, not the Map's entries. Define a serialization format explicitly, such as an array of entries or a schema-specific report. Objects used as Map keys may also need a separate identity encoding for transport.

Includes, IndexOf, and NaN

```
const values = [NaN, undefined];
console.log(values.includes(NaN), values.indexOf(NaN));
console.log([,].includes(undefined), [,].indexOf(undefined));
console.log(new Set([NaN]).has(NaN));

// Expected output:
// true -1
```

```
// true -1
// true
```

includes uses SameValueZero and treats holes as undefined for this ordinary-array case. indexOf uses strict equality and skips holes. Replacing one search method with another can change behavior even when both sound like membership checks.

Diagnose the First Divergent Stage

For a pipeline failure, inspect validated input, selected IDs, sorted IDs, projected output, and final aggregate separately. Keep logs bounded and redact sensitive fields. Ask whether the first discrepancy is missing visitation, coercion, duplicate policy, mutation, comparator behavior, or shared ownership.

The edge companion `code/volume-1/chapter-09/example-04-edge-cases.js` asserts these distinctions. The Array reference summarizes hole-sensitive methods, and the Map reference distinguishes entries from properties.

Performance and Security

Count Scans, Comparisons, and Stored Results

map, filter, reduce, find, some, and every perform at most a linear scan over their captured index range for ordinary arrays, excluding callback work. Early-exit methods may stop sooner. A callback that scans another collection changes the whole operation: mapping n orders and finding each in a catalog of m items can require O(nm) comparisons.

An index can replace repeated scans with a build pass and keyed lookups. Under conventional expected Map costs, that becomes O(n + m), at the price of O(m) retained index storage. The language specifies average sublinear access rather than a particular hash-table bound.

Array copying consumes storage for new element slots. It may still retain the same records, so shallow copying can increase reachability without isolating ownership. Name the lifetime of snapshots and indexes in a long-lived service.

Avoid Quadratic Accumulator Copying

```
let copiedSlots = 0;
const copied = [1, 2, 3, 4].reduce((result, value) => {
  copiedSlots += result.length;
  return [...result, value * 2];
}, []);
const local = [1, 2, 3, 4].reduce((result, value) => {
  result.push(value * 2);
  return result;
}, []);
```

```
console.log(copied.join(','));
console.log(local.join(','));
console.log(copiedSlots);

// Expected output:
// 2,4,6,8
// 2,4,6,8
// 6
```

The first pattern copies 0 + 1 + … + (n - 1) earlier slots, producing quadratic copying work. The second mutates a new local accumulator and builds the result in linear work under ordinary array-growth assumptions. map expresses this particular transform even more directly.

This does not justify mutating caller-owned input. Local construction and externally shared mutation have different ownership consequences.

Sorting and Queue Costs

Sorting needs a valid comparator and has an implementation-dependent cost. Keep expensive key extraction out of repeated comparator calls by computing validated keys beforehand when measurement justifies it. Do not add logging or mutations inside a comparator: call order/count is not an application contract.

Repeated shift/unshift can move many index positions. For a large queue, an advancing read index can avoid repeatedly removing the front; periodically release consumed references or compact according to measured memory needs. A simple bounded array queue may still be sufficient. Choose based on workload, not slogans about one syntax being fastest.

Bound Input Before Allocation

Validate count limits before Array.from, spread, sorting, or building large indexes. Also bound relevant string lengths and numeric fields. A single collection-size check does not limit the size of every record or the serialized request. Enforce transport/deserialization limits at the surrounding boundary.

Do not pass an unbounded array to a function through argument spread merely to compute an aggregate; engines have argument-count limits. Use an explicit loop or reduce with a documented empty-input result.

Keys and Retained Data

Map treats "**proto**" as ordinary entry data, which avoids some hazards of dynamic object-property dictionaries. It does not sanitize values, authorize access, cap memory, or make an arbitrary object safe to read. Identity and role fields still need trusted origins.

A Map strongly retains its object keys and values while the Map is reachable. Long-lived caches need eviction or explicit deletion. A Set of seen IDs also grows without bound unless its scope or retention period is limited. The production examples keep these collections local to bounded batches.

Practical Measurement

Measure realistic sizes, duplicate rates, match positions, sort-key distributions, and callback cost. Include allocation and retained memory, not just elapsed loop time. Engine specialization may favor dense predictable arrays, but correctness must not depend on a particular storage representation. V8 elements kinds explains one implementation's trade-offs; Map defines portable entry semantics.

Professional Field Guide

Start With Five Decisions

1. Is the collection ordered, keyed, unique, or a combination?
2. Are duplicates preserved, rejected, merged, or resolved by first/last occurrence?
3. Are holes allowed, and what is the empty-input result?
4. Does the output own only array slots, or fresh records too?
5. What bounds the input size, item size, aggregate range, and retained lifetime?

These decisions make method selection straightforward. A Set helps with uniqueness but cannot decide a duplicate policy for conflicting records. A copying sort protects order but cannot decide whether records must be detached.

Review a Pipeline

Check validation before selection. If a report promises to reject malformed records anywhere, filtering them out first violates the contract. Check the numeric bound before summing. Check the comparator's ties and whether input order is preserved. Finally, modify a returned record in a test to demonstrate the claimed ownership.

The order summary validates all rows and projects fresh primitive-field items. The task-sorting challenge intentionally shares records while returning a new order. Both are legitimate because their contracts differ.

Choose Simplicity at the Right Scale

A short map/filter/reduce pipeline is easy to inspect when each stage has one purpose and the batch is bounded. A single explicit loop can be clearer when the task needs several accumulators, multiple rejection reasons, or early exit. Avoid chaining methods merely to eliminate statements.

Use some rather than filter when only existence matters, and findIndex when a returned undefined could be a real item. Supply reduce's initial value. Wrap callbacks such as parseInt when the array method supplies extra arguments with incompatible meaning.

Make Identity Deliberate

A Set of object references does not deduplicate entities by equal fields. A Map keyed by a mutable object does not follow changes to an id property. Choose a stable primitive ID key when that represents domain identity; preserve textual IDs rather than coercing them numerically.

For each Map, state whether get returning undefined means absent or can mean a stored value. Use has when the distinction remains possible. Use set/get/has for entries and an explicit transport projection for JSON.

A Useful Incident Explanation

"The input contained a hole. every skipped that position, so validation succeeded without inspecting an item there. We now require own indices before validating each record, and the regression includes a sparse array."

Another example: "We copied the array but shared its records. Editing a report item changed live state. The report now projects its supported primitive fields into new records, and tests mutate both input and output to check isolation."

These explanations name the input, language rule, incorrect assumption, and measurable repair. They are more useful than saying that an array method behaved unexpectedly.

Before Moving On

Demonstrate density checks, shallow copying, stable numeric ordering, Map/Set identity, and bounded aggregation with the six exercises. Use the revision sheet to explain each rule aloud, then continue to Errors and Debugging for failure handling across calls.

CHAPTER 10

Errors and Debugging

Introduction

A program can reject invalid input, fail because a dependency throws, or produce the wrong answer without throwing anything. Those situations need different responses. Error handling controls failure propagation; debugging gathers evidence about the cause; assertions state behavior the program must satisfy.

This chapter develops those skills through a bounded JSON import parser and a synchronous resource helper. The parser distinguishes malformed JSON from invalid domain data. The resource helper calls cleanup exactly once after successful acquisition and preserves both failures when operation and cleanup fail together.

Learning Objectives

By the end, you can:

- Distinguish source syntax, runtime exceptions, and logical defects.
- Throw an Error, choose an appropriate built-in type, and preserve a cause.
- Trace try, catch, finally, rethrow, and pending return behavior.
- Handle expected failures without swallowing unrelated defects.
- Explain why a synchronous catch does not surround a later callback invocation.
- Choose explicit result values or exceptions for an API contract.
- Reduce a failure to a deterministic reproduction and inspect a call stack.
- Write assertions for results, failure types, side effects, and cleanup.

Prerequisites

Read Functions and Callbacks, Objects, and Arrays and Collections. You should understand function calls, object identity, arrays, and loop exits.

Examples use Node's built-in assertion module. `require('node:assert/strict')` loads it in the CommonJS companion scripts. The next chapter explains module and execution modes. No package installation is needed.

Running Guide

Use Node.js 20 or later:

```
node code/volume-1/chapter-10/example-00-error-model.js
node code/volume-1/chapter-10/example-01-import-parser.js
node code/volume-1/chapter-10/example-02-safe-cleanup.js
node code/volume-1/chapter-10/example-03-error-challenges.js
```

Each JavaScript example runs independently. It catches intentional errors and prints stable names or application-defined messages, not engine-specific stack strings.

First Experiment

```
function requireCount(value) {
  if (!Number.isSafeInteger(value) || value < 0) {
    throw new RangeError('Count must be a nonnegative safe integer.');
  }
  return value;
}
try {
  requireCount(-1);
  console.log('unreachable');
} catch (error) {
  console.log(error.name);
}
console.log('continued after handling');
// Expected output:
// RangeError
// continued after handling
```

Throwing skips the remainder of the protected operation. The catch handles the failure; execution then continues after the construct. It does not resume at the statement that threw.

Read theory, then follow the execution model.

Errors and Debugging: Theory

Three Kinds of Failure

Kind	Example	Detection and response
Source syntax failure	An unmatched brace in the current script	Parsing fails before that script executes; fix source
Runtime exception	Calling a non-callable value	Execution throws; handle or propagate deliberately
Logical defect	A discount applied twice	May not throw; assertions and domain checks reveal it

A try block in a script cannot catch that same script failing to parse, because the block never starts executing. By contrast, `JSON.parse` runs while the caller is executing and can throw a catchable SyntaxError for malformed input. Do not use evaluation of untrusted source as a parsing shortcut.

```
try {
  JSON.parse('{"id":');
} catch (error) {
  console.log(error.name);
}
const incorrectTotal = 100 - 10 - 10;
console.log(incorrectTotal === 90);
// Expected output:
// SyntaxError
// false
```

The second line exposes a logic mismatch without any exception. A clean process exit does not establish correctness.

Error Values Carry Context

`throw` can carry any JavaScript value, including null or undefined. Prefer Error objects for application failures: they provide a message, a category, and runtime diagnostic support. Creating an Error does not itself throw it. Error reference.

Type	Typical meaning
`TypeError`	An operation received a value of an inappropriate kind
`RangeError`	A value is of a relevant kind but outside an accepted range
`SyntaxError`	Parsed syntax is invalid
`ReferenceError`	A required binding cannot be resolved or accessed
`Error`	A general application failure
`AggregateError`	Several failures need to be preserved together

These built-in meanings guide the chapter's contracts; applications may also define stable domain codes. Do not decide behavior by matching an engine's message wording.

```
const issue = new Error('Import failed.');
console.log(issue.name, issue.message);
try {
  throw null;
} catch (error) {
  console.log(error === null);
}
// Expected output:
```

```
// Error Import failed.
// true
```

Because caught values can be arbitrary, accessing `error.message` without checking can itself fail. A robust boundary handles unknown thrown values. `instanceof Error` is useful for values from the same realm, but is not a universal cross-realm test.

Propagation Follows the Active Call Chain

If a function throws and has no applicable handler, its call does not return normally. The failure propagates outward until an active catch handles it, or to the host if no handler does. A catch handles exceptions from its protected execution, including synchronous calls made there.

```
function parseRecord() {
  throw new TypeError('Missing ID.');
}
function importRecord() {
  return parseRecord();
}
try {
  importRecord();
} catch (error) {
  console.log(error.message);
}
// Expected output:
// Missing ID.
```

A catch does not undo mutations already performed. Validation before mutation and an explicit rollback or transaction policy are separate design concerns.

Catch Narrowly and Rethrow Unexpected Failures

Put only the operation whose failure you intend to classify inside the protected region. Check the expected category, convert it to a documented outcome if appropriate, and rethrow everything else.

```
function parseOrDefault(text) {
  if (typeof text !== 'string') throw new TypeError('Expected text.');
  try {
    return JSON.parse(text);
  } catch (error) {
    if (error instanceof SyntaxError) return { kind: 'invalid-json' };
    throw error;
  }
}
console.log(JSON.stringify(parseOrDefault('{')));
console.log(JSON.stringify(parseOrDefault('{"ok":true}')));
// Expected output:
// {"kind":"invalid-json"}
// {"ok":true}
```

The function uses a fallback only for syntax failure. In a larger API, an explicit result wrapper avoids collisions between fallback objects and successful parsed objects. The production parser instead throws because its caller requires a valid domain record.

Add Context Without Destroying the Cause

Use `new Error(message, { cause })` when a boundary adds useful context. The cause property holds the original thrown value; for an object, it preserves that object's identity. It can hold any value, not only an Error. Rethrow the original value when no extra context is needed. Error.cause.

```
const original = new TypeError('Invalid quantity.');
const contextual = new Error('Could not import inventory.', { cause: original });
console.log(contextual.message);
console.log(contextual.cause === original);
console.log(contextual.cause.name);
// Expected output:
// Could not import inventory.
// true
// TypeError
```

Do not copy a message into a fresh Error and discard the cause. Do not wrap at every layer with identical text. Add the context a caller could not otherwise infer, while keeping secrets out of messages.

Finally Runs Before the Construct Exits

A finally block runs when normal JavaScript control leaves the associated try/catch, including through return or throw. If finally completes normally, the earlier result or failure continues. If finally returns or throws, that new control transfer replaces the pending one. Try/catch/finally.

```
const events = [];
function finish() {
  try {
    events.push('work');
    return 7;
  } finally {
    events.push('cleanup');
  }
}
console.log(finish());
console.log(events.join(','));
// Expected output:
// 7
// work,cleanup
```

Do not return from finally. It can silently replace a successful result or suppress an exception. Cleanup can also throw, so define which error should be reported if both work and cleanup fail. Finally is not a guarantee against process termination, crashes, or a loop that never exits.

Synchronous Catch Boundaries

A callback created inside a try is not permanently protected by that catch. The callback must execute while the relevant protected call chain is active. This example stores work and invokes it later without using an asynchronous API:

```
let pending;
try {
  pending = () => { throw new Error('Later failure.'); };
} catch {
  console.log('registration catch');
}
try {
  pending();
} catch (error) {
  console.log(error.message);
}
// Expected output:
// Later failure.
```

A timer callback similarly runs after the registration call has returned. Promise rejection and `await` need their own handling model, taught in the asynchronous volume. A synchronous resource helper must not pretend that returning a Promise means the operation has finished.

Results Versus Exceptions

An expected "not found" result may be `null` or a tagged result such as `{ ok: false, code: 'NOT_FOUND' }`. An exception can signal that a function cannot fulfill its promised result. Either style can work; specify it consistently and make callers handle both paths.

Use exceptions for failed invariants or invalid inputs when that is the contract. Use explicit results for frequent alternative outcomes when that makes control flow clearer. Avoid a mixture where some failures return undefined, others return strings, and others throw without documentation.

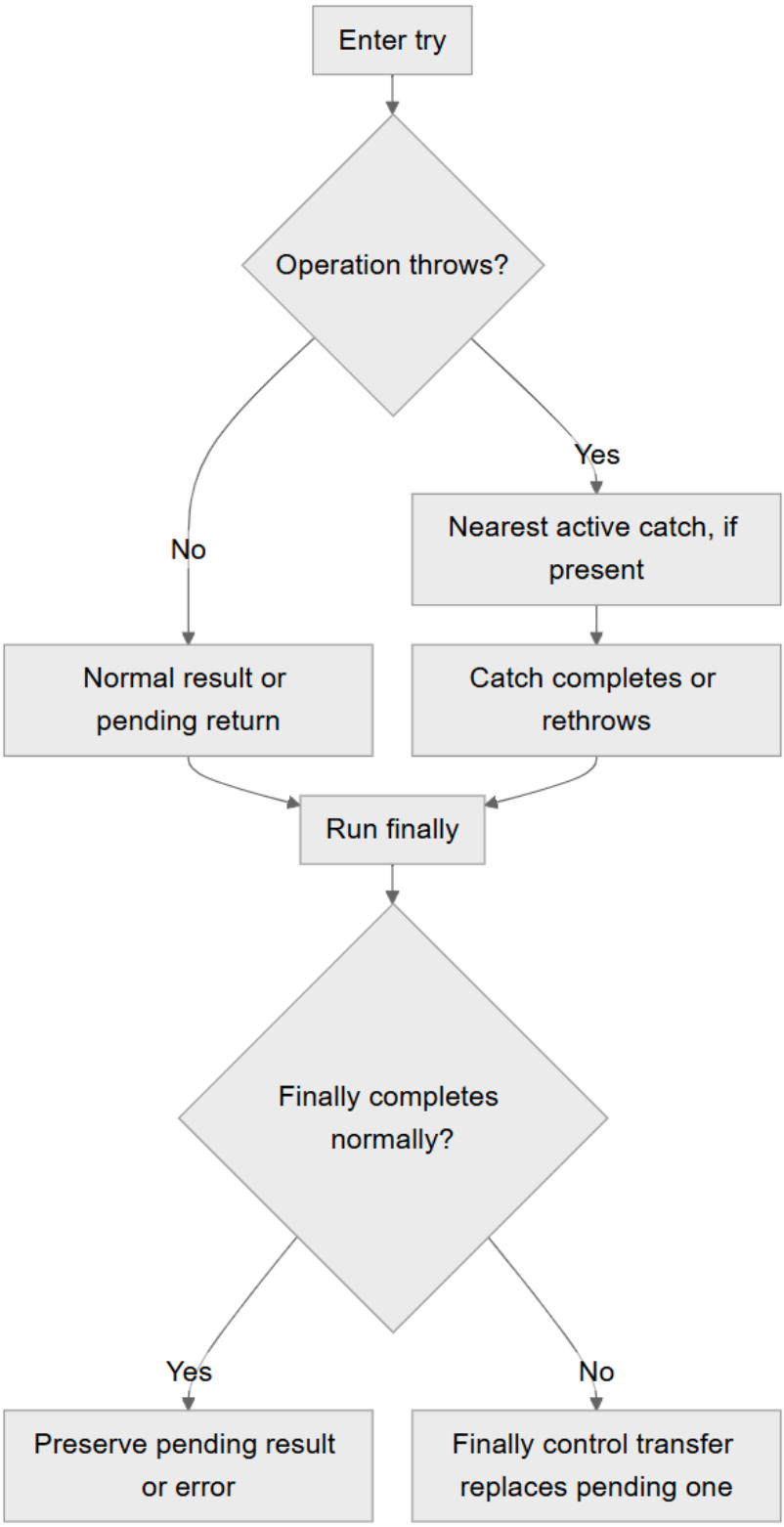
Assertions and Debugging

An assertion encodes an expectation and fails when the expectation is violated. Use strict equality for primitive results, structural equality for record/array shape, and exception assertions for failure contracts. Assert side effects as well as return values. Node assert.

Debugging begins with observed behavior, expected behavior, a minimal reproducer, and a hypothesis. Breakpoints and call stacks help test the hypothesis; they do not replace a specification of the intended result.

Errors and Debugging: Internal Working

The Pending Completion Model



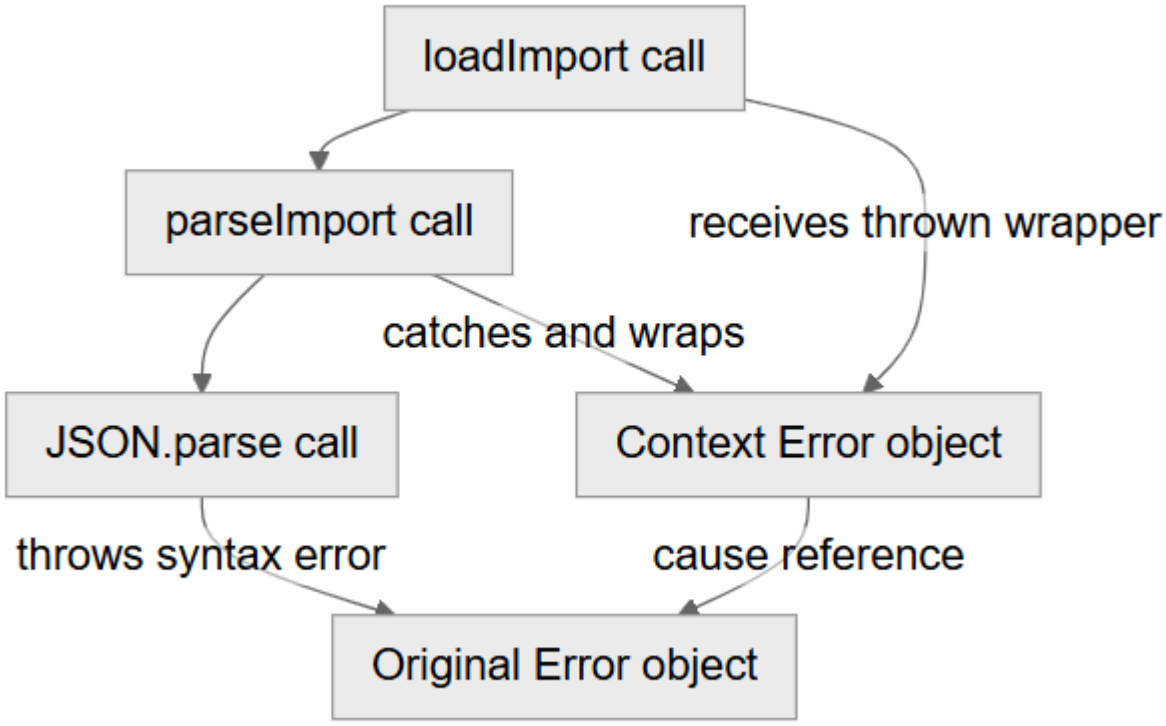
Source: `diagrams/volume-1-chapter-10-error-flow.mmd`.

The diagram describes control flow, not a physical queue of Error objects. Think of a pending completion as "continue normally," "return this value," or "throw this value." Finally gets a chance to execute before that completion leaves the construct.

For a failing operation with a catch and finally:

1. The throw abandons the remaining statements in the protected try.
2. The catch receives the thrown value.
3. The catch either completes normally or creates a new pending completion.
4. Finally runs.
5. Normal cleanup preserves the pending completion; a new transfer replaces it.

Call Stack and Cause References



Source: `diagrams/volume-1-chapter-10-error-cause.mmd`.

The active calls show where the failure propagates. The cause link shows retained object identity after wrapping. These are different relationships: the original function calls can finish unwinding while the two Error objects remain reachable from a logged or retained failure.

Stack text is a diagnostic view supplied by runtimes; its exact formatting and capture details vary. The language's observable throw/catch behavior does not depend on a portable stack-string format.

Return Values Are Selected Before Finally

```
function inspectReturn() {
  let count = 1;
  try {
    return count;
  } finally {
    count = 2;
  }
}
console.log(inspectReturn());
// Expected output:
// 1
```

The return expression produced the primitive value 1 before finally reassigned the local binding. If the pending returned value is an object, finally can still mutate that object through a shared reference:

```
function inspectObjectReturn() {
  const result = { closed: false };
  try {
    return result;
  } finally {
    result.closed = true;
  }
}
console.log(JSON.stringify(inspectObjectReturn()));
// Expected output:
// {"closed":true}
```

This distinction follows value and reference semantics. A rule such as "finally cannot affect a return value" would be too broad.

Two Failures Require an Explicit Policy

The resource helper acquires first. Once acquisition returns, use and release belong to one scope. An operation failure is retained even if its thrown value is undefined, so a separate boolean records whether failure occurred.

Operation	Release	Outcome
Succeeds	Succeeds	Return operation value
Throws	Succeeds	Rethrow the same value
Succeeds	Throws	Throw release failure
Throws	Throws	Throw AggregateError preserving both
Acquisition throws	Not called	Propagate acquisition failure

Keeping only an error variable and testing its truthiness would lose failures such as `throw undefined`. The companion specifically tests that case.

Engine Internals and Memory

An implementation may optimize normal execution and create diagnostic information lazily. Do not claim that merely placing code inside try always disables optimization. Measure actual hot paths rather than treating an exception mechanism as a fixed-cost primitive.

Retaining an Error with cause can retain everything reachable from that cause. Avoid attaching a complete request, large buffer, or credential object just to add context. Use a small identifier and carefully selected metadata when it satisfies diagnostic needs.

For the formal control-transfer model, see ECMAScript completion records. The production section makes the failure matrix executable.

Errors and Debugging: Production Examples

Bounded JSON Import Parser

Contract: accept a primitive JSON string of at most 2,048 UTF-16 code units. It must decode to an object with exactly own `id` and `quantity` fields. ID is 1 through 32 uppercase ASCII letters, digits, or hyphens. Quantity is an integer Number from zero through 10,000.

A malformed document throws a SyntaxError with the parsing error as cause. Invalid shape or field type throws TypeError. Oversize input or an out-of-range quantity throws RangeError. The result is a fresh projection containing only accepted fields.

```
function parseImport(text) {
  if (typeof text !== 'string') throw new TypeError('Expected JSON text.');
  if (text.length > 2048) throw new RangeError('Import text is too long.');
  let value;
  try {
    value = JSON.parse(text);
  } catch (error) {
    if (!(error instanceof SyntaxError)) throw error;
    throw new SyntaxError('Invalid import JSON.', { cause: error });
  }
  if (value === null || typeof value !== 'object' || Array.isArray(value)) {
    throw new TypeError('Expected an import object.');
  }
  const keys = Object.keys(value);
  if (keys.length !== 2 || !Object.hasOwn(value, 'id') || !Object.hasOwn(value, 'quantity')) {
    throw new TypeError('Expected exactly id and quantity.');
  }
  if (typeof value.id !== 'string' || !/^[A-Z0-9-]{1,32}$/.test(value.id) ||
      typeof value.quantity !== 'number') {
    throw new TypeError('Invalid import field types or ID syntax.');
```

```
  }
  if (!Number.isSafeInteger(value.quantity) || value.quantity < 0 || value.quantity > 10000) {
    throw new RangeError('Quantity must be an integer from zero to 10,000.');
  }
  return { id: value.id, quantity: value.quantity };
}

console.log(JSON.stringify(parseImport('{"id":"ORD-42","quantity":3}')));
try {
  parseImport('{"id":');
} catch (error) {
  console.log(error.name, error.cause.name);
}
try {
  parseImport('{"id":"ORD-42","quantity":-1}');
} catch (error) {
  console.log(error.name);
}
// Expected output:
// {"id":"ORD-42","quantity":3}
// SyntaxError SyntaxError
// RangeError
```

The parsing catch encloses only JSON.parse. A schema defect cannot be accidentally reported as malformed JSON. No reviver executes caller code. The fixed text cap bounds the accepted work; before applying the cap, the caller must still control how a large request body is read into memory.

JSON parsing does not reject repeated member names; ordinary JSON.parse retains the last value. This contract validates the resulting object and does not claim duplicate-key detection. If an application needs that stronger policy, use a parser that can report duplicates. JSON field-name strings such as **proto** do not become accepted fields here because the schema rejects extras and returns an explicit projection.

The companion `code/volume-1/chapter-10/example-01-import-parser.js` tests all error categories, null/array roots, extra fields, syntax cause, exact bounds, and unsupported types. Parsing and validation use O(n) work and space in input size under the small fixed cap.

Synchronous Resource Scope With Preserved Failures

Contract: acquire, use, and release are synchronous callbacks. Validate all three before acquisition. If acquire throws, release is not called. Otherwise call release exactly once, whether use returns or throws. Preserve the original operation error when cleanup succeeds; preserve both errors in ordered `AggregateError.errors` when both fail.

This helper does not await Promises or implement rollback. Its callbacks must finish their work before returning. Passing async callbacks would release too early and violates this contract.

```
function withResource(acquire, use, release) {
  if (typeof acquire !== 'function' || typeof use !== 'function' || typeof release !== 'function') {
    throw new TypeError('Expected synchronous callbacks.');
  }
  const resource = acquire();
  let operationFailed = false;
```

```
  let operationError;
  try {
    return use(resource);
  } catch (error) {
    operationFailed = true;
    operationError = error;
    throw error;
  } finally {
    try {
      release(resource);
    } catch (cleanupError) {
      if (operationFailed) {
        throw new AggregateError(
          [operationError, cleanupError],
          'Operation and cleanup failed.',
          { cause: operationError }
        );
      }
      throw cleanupError;
    }
  }
}

const events = [];
const result = withResource(
  () => ({ id: 'buffer-1' }),
  resource => { events.push('use ' + resource.id); return 42; },
  resource => { events.push('release ' + resource.id); }
);
console.log(result);
console.log(events.join(','));
try {
  withResource(
    () => ({}),
    () => { throw new Error('work failed'); },
    () => { throw new Error('release failed'); }
  );
} catch (error) {
  console.log(error.name);
  console.log(error.errors.map(item => item.message).join(','));
}
// Expected output:
// 42
// use buffer-1,release buffer-1
// AggregateError
// work failed,release failed
```

A throw from finally normally replaces a pending failure. Here that replacement is deliberate: AggregateError retains both original values and identifies the operation failure as cause. There is no return from finally.

The companion `code/volume-1/chapter-10/example-02-safe-cleanup.js` checks every row of the failure matrix, callback order, identical resource identity, and arbitrary thrown values. Wrapper overhead is O(1), excluding callback work and any error diagnostics.

Boundary Reporting

At a UI or service boundary, map expected validation failures to a safe, stable message or code. Keep full diagnostics in an appropriate internal channel, avoiding credentials and complete input payloads. Do not expose parser internals or stack traces merely because an Error contains them.

Unexpected failures should remain failures. Returning an empty import object or a fake success after a cleanup error would hide the fact that the function did not meet its contract.

Errors and Debugging: Interview Perspective

Beginner: Constructing Versus Throwing

An Error is a value; throw transfers control. Creating an Error without throwing does not enter catch. Prefer Error objects, but remember that catch can receive any value.

Distinguish source parsing from runtime data parsing. An unmatched brace in the current script prevents its try block from executing. A running JSON.parse call can throw into an active catch.

Intermediate: Predict Finally Behavior

```
function choose() {
  try {
    return 'work';
  } finally {
    return 'cleanup';
  }
}
console.log(choose());
// Expected output:
// cleanup
```

This intentionally demonstrates a bad production pattern. Return in finally replaces the pending result. If try had thrown, the final return could suppress that failure. Cleanup should normally finish without selecting a new successful result.

Intermediate: Catch and Rethrow

Ask what a catch can recover from. If a parser converts syntax failures into results, protect only the parser call. A TypeError from unrelated code in a large try should not become a misleading invalid-input response.

Wrapping can add context while preserving the original Error as cause. Wrapping with identical messages at every layer adds noise and retains unnecessary references.

Senior: Work and Cleanup Both Fail

Name the policy before coding. The resource helper returns work's result when both succeed, propagates either single failure, and uses AggregateError for two failures. Acquisition failure does not call release.

Why keep a separate failure flag? Because an operation may throw undefined or null. Testing the truthiness of the saved error would miss those failures.

Senior: Catch Around Registration

A callback registered inside try can execute after that try has finished. Source-code nesting is not the same as an active error-handling boundary. The invocation and asynchronous mechanism determine where a failure is handled.

A synchronous wrapper must not claim to await a returned Promise. Callback-local handling and Promise rejection handling are distinct topics.

Design Challenge: Batch Imports

Separate request-size limits, JSON syntax, schema validation, row validation, and persistence. Decide whether an invalid row rejects the batch or produces an indexed failure. Specify whether writes can happen before validation finishes and whether rollback exists.

Test malformed JSON, a valid non-object value, wrong fields, exact quantity bounds, and an unexpected dependency failure. A successful process exit does not prove any of those contracts.

Explain a Debugging Result

Describe one reproducible input, the first incorrect state, the evidence supporting its cause, and the assertion that prevents recurrence. A clear explanation is stronger than a list of debugging tools.

Errors and Debugging: Exercises and Coding Challenges

Implement each contract before reading the solution. The companion `code/volume-1/chapter-10/example-03-error-challenges.js` tests normal and failure paths.

1. Validate a Positive Count

Accept a Number that is a positive safe integer. Throw TypeError for other types and RangeError for invalid numeric values. Return accepted input.

Hint: Check type before range.

Solution

```
function requirePositiveCount(value) {
  if (typeof value !== 'number') throw new TypeError('Expected a Number.');
  if (!Number.isSafeInteger(value) || value <= 0) throw new RangeError('Invalid count.');
  return value;
}
console.log(requirePositiveCount(3));
for (const value of ['3', 0]) {
  try { requirePositiveCount(value); } catch (error) { console.log(error.name); }
}
// Expected output:
// 3
// TypeError
// RangeError
```

O(1) work and storage. Test NaN, infinity, fractions, zero, the maximum safe integer, and overflow.

2. Return a JSON Parse Result

Accept primitive text of at most 1,024 code units. Return `{ ok: true, value }` for valid JSON or `{ ok: false, code: "INVALID_JSON" }` for syntax failure. Wrong types and excessive length still throw.

Hint: Keep the catch around parsing and rethrow unexpected errors.

Solution

```
function tryJson(text) {
  if (typeof text !== 'string') throw new TypeError('Expected text.');
  if (text.length > 1024) throw new RangeError('Text too long.');
  try {
    return { ok: true, value: JSON.parse(text) };
  } catch (error) {
    if (!(error instanceof SyntaxError)) throw error;
    return { ok: false, code: 'INVALID_JSON' };
  }
}
console.log(JSON.stringify(tryJson('null')));
console.log(JSON.stringify(tryJson('{')));
// Expected output:
// {"ok":true,"value":null}
// {"ok":false,"code":"INVALID_JSON"}
```

The ok tag distinguishes valid null from failure. Parsing uses O(n) work and storage under the cap. Successful syntax does not establish a domain schema.

3. Recover From One Expected Category

Accept a synchronous operation callback. Return its result. Return zero only when it throws a same-realm RangeError; propagate every other value unchanged.

Hint: A catch should not convert unrelated defects to fallback data.

Solution

```
function countOrZero(operation) {
  if (typeof operation !== 'function') throw new TypeError('Expected callback.');
  try {
    return operation();
  } catch (error) {
    if (error instanceof RangeError) return 0;
    throw error;
  }
}
console.log(countOrZero(() => { throw new RangeError('Unavailable.'); }));
const failure = new TypeError('Broken dependency.');
try {
  countOrZero(() => { throw failure; });
} catch (error) {
  console.log(error === failure);
}
// Expected output:
// 0
// true
```

Wrapper cost is O(1), excluding callback work. A larger domain may use a dedicated error category or stable code rather than recovering from every RangeError.

4. Preserve the Cause at an Import Boundary

Accept a synchronous loader. Return its result or throw a new Error with message "Import unavailable." and the exact original thrown value as cause. Validate callability before the catch.

Hint: Use the Error options object; do not copy only the original message.

Solution

```
function loadWithContext(loader) {
  if (typeof loader !== 'function') throw new TypeError('Expected loader.');
  try {
    return loader();
  } catch (cause) {
    throw new Error('Import unavailable.', { cause });
  }
}
const failure = new Error('Storage failed.');
try {
  loadWithContext(() => { throw failure; });
} catch (error) {
  console.log(error.message);
  console.log(error.cause === failure);
}
// Expected output:
```

```
// Import unavailable.
// true
```

The wrapper adds one useful boundary description and retains identity. Its overhead is O(1), excluding the loader and runtime diagnostics. A cause can also be null or undefined.

5. Collect Expected Row Failures

Accept an array and synchronous validator. Collect returned values in accepted and indices of TypeError/RangeError failures in rejected. Propagate other failures. Do not mutate rows or claim rollback of validator effects.

Hint: Put a separate expected-error boundary inside each iteration.

Solution

```
function validateRows(rows, validate) {
  if (!Array.isArray(rows) || typeof validate !== 'function') throw new TypeError('Invalid arguments.');
  const accepted = [];
  const rejected = [];
  for (let index = 0; index < rows.length; index++) {
    try {
      accepted.push(validate(rows[index]));
    } catch (error) {
      if (!(error instanceof TypeError) && !(error instanceof RangeError)) throw error;
      rejected.push(index);
    }
  }
  return { accepted, rejected };
}
const result = validateRows([1, '2', 3], value => {
  if (typeof value !== 'number') throw new TypeError('Expected number.');
  return value * 2;
});
console.log(JSON.stringify(result));
// Expected output:
// {"accepted":[2,6],"rejected":[1]}
```

There are O(n) visits and O(n) result storage, plus validator costs. Accepted objects retain the identity supplied by the validator; the wrapper does not deep-copy them.

6. Restore a Busy Flag

Accept an ordinary mutable object with an own boolean busy field and a synchronous callback. Reject already-busy state. Set busy during the call and restore false on success or throw, preserving the callback outcome. Exclude accessors, proxies, and frozen state.

Hint: Restore in finally without returning from finally.

Solution

```
function withBusyFlag(state, operation) {
  if (state === null || typeof state !== 'object' || !Object.hasOwn(state, 'busy') ||
      typeof state.busy !== 'boolean' || typeof operation !== 'function') {
    throw new TypeError('Invalid state or callback.');
  }
  if (state.busy) throw new RangeError('Already busy.');
  state.busy = true;
  try {
    return operation();
  } finally {
    state.busy = false;
  }
}
const state = { busy: false };
try {
  withBusyFlag(state, () => {
    console.log(state.busy);
    throw new Error('Failed.');
  });
} catch (error) {
  console.log(error.message);
}
console.log(state.busy);
// Expected output:
// true
// Failed.
// false
```

Wrapper time and space are O(1). Under the ordinary mutable-object contract, restoring the flag succeeds. This is a local state invariant, not a rollback of other callback effects.

Errors and Debugging: MCQs

Choose one answer and explain its control-flow rule.

1. What does new Error("failed") do by itself?

A. It terminates the process B. It constructs a value without throwing C. It invokes catch D. It returns a rejected Promise

Answer: B. Throwing is a separate control operation.

2. Which values can JavaScript throw?

A. Only Error instances B. Only objects with message C. Only strings and errors D. Any JavaScript value

Answer: D. Catch can receive null, undefined, primitives, or objects.

3. Can try catch a syntax failure of its own script before that script executes?

A. No, its try block never starts B. Yes, if finally exists C. Yes, if variables use let D. Only in strict mode

Answer: A. Parsing must succeed before a handler in that script executes.

4. Why can JSON.parse failure be caught?

A. JSON parsing never throws B. The source parser rewrites try C. It is an operation in an active call D. JSON syntax failures are booleans

Answer: C. The invalid text is data passed to a running parser.

5. What does catch automatically roll back?

A. All assignments B. All external effects C. Nothing D. Only primitive assignments

Answer: C. Control flow changes; prior mutations remain unless explicitly reversed.

6. What should happen to an unexpected caught failure?

A. Rethrow it B. Return an empty object C. Log and report success D. Convert it to false

Answer: A. Recovery must be limited to the documented policy.

7. What does cause preserve?

A. Only formatted stack text B. A cloned request C. Only the original message D. The exact supplied original value

Answer: D. The cause can be any value, including the original Error object.

8. If finally completes normally after a return, what happens?

A. The return is cancelled B. The pending returned value continues C. The result is undefined D. The return expression runs again

Answer: B. The return expression was evaluated before finally.

9. Why avoid return in finally?

A. It can replace a result or suppress an error B. It is always invalid syntax C. It prevents finally execution D. It creates AggregateError automatically

Answer: A. Finally can create a new completion that replaces the pending one.

10. If cleanup throws during error propagation, what is the default effect?

A. Cleanup failure is ignored B. Both failures are automatically collected C. The cleanup failure replaces the pending failure D. The original is always placed in cause

Answer: C. Preserving both failures requires an explicit policy.

11. Why keep a separate operationFailed flag?

A. Errors cannot be stored B. Thrown values can be falsy or undefined C. Finally cannot read locals D. It detects asynchronous rejection

Answer: B. Occurrence of failure is separate from the truthiness of its value.

12. Does a catch around timer registration catch a later callback throw?

A. Always B. Only with zero delay C. Only for arrow callbacks D. No

Answer: D. The callback executes after that protected registration call has returned.

13. What does a tagged parse result distinguish?

A. Runtime from operating system B. All stack formats C. Only objects from arrays D. Valid parsed null from invalid syntax

Answer: D. The ok tag avoids using null ambiguously for valid data and failure.

14. Which test verifies cleanup on operation failure?

A. Print the resource B. Assert the throw and release-call count C. Check the helper exists D. Run only the success path

Answer: B. Observe both failure propagation and cleanup effects.

15. What helps reproduce a clock-dependent boundary bug?

A. Rewrite everything B. Add random delays C. Inject a fixed clock value D. Ignore every exception

Answer: C. Deterministic input makes the hypothesis and regression test repeatable.

16. What should be assumed about error.stack?

A. It is diagnostic and runtime-dependent B. It is a stable wire format C. It contains only public data D. It replaces the error message

Answer: A. Use stack text for diagnosis, not as an unfiltered public response or portable protocol.

Errors and Debugging: Revision and Summary

Revision Sheet

Question	Rule
Does new Error throw?	No; throw transfers control
Must a thrown value be an Error?	No; catch may receive any value
Can a try catch a syntax failure in its own source?	No; that source never starts executing
Can it catch JSON.parse syntax failure?	Yes; parsing input occurs during the active call
What happens after throw?	Remaining protected statements are skipped and propagation begins
Does catch roll back mutations?	No
What should an unexpected caught failure do?	Propagate unless the boundary has a deliberate policy
What does cause preserve?	The original value, including identity for an object
Does finally run before a return exits?	Yes, during normal language control flow
Why avoid return in finally?	It replaces the pending result or failure
Can cleanup hide the original error?	Yes, unless the failure policy preserves it
Does registration-time catch protect a later callback?	No
What detects a wrong answer without a throw?	Domain checks and meaningful assertions
Is a stack string a stable API?	No; exact formatting is runtime-dependent

Debugging Sequence

1. State expected and observed behavior.
2. Freeze variable inputs: clock, randomness, environment, and dependency results.

3. Reduce to the smallest reproduction that still fails.
4. Pause at the throw or first incorrect state.
5. Inspect local values and the active call chain.
6. Test one hypothesis with a targeted change.
7. Add an assertion for the behavior that was missing.
8. Re-run the relevant normal and failure paths.

Summary

Exceptions move failure through active calls; they do not undo work or prove that all failures were anticipated. Error objects and causes preserve useful context. Finally manages scope exit, but its own control transfer can replace the original outcome.

The parser defines syntax, shape, and range boundaries separately. The resource helper defines acquisition, operation, and cleanup behavior for synchronous callbacks and preserves competing failures.

Use the exercises and MCQs to practice. Continue with Modules and Execution Modes.

Errors and Debugging: Edge Cases and Debugging

Caught Values May Not Have Messages

```
function describeFailure(value) {
  return value instanceof Error ? value.message : 'Unknown thrown value.';
}
for (const failure of [new Error('Known failure.'), null, undefined]) {
  try {
    throw failure;
  } catch (error) {
    console.log(describeFailure(error));
  }
}
// Expected output:
// Known failure.
// Unknown thrown value.
// Unknown thrown value.
```

This same-realm display policy avoids reading a property of null. It also avoids stringifying arbitrary objects, which might execute custom conversion code or expose sensitive data.

Catch Does Not Roll Back State

```
const state = { count: 0 };
try {
  state.count++;
  throw new Error('Later failure.');
} catch {
  console.log('handled');
}
console.log(state.count);
// Expected output:
// handled
// 1
```

If count must stay unchanged on failure, validate or calculate before assigning, or define a rollback policy. Catch only changes control flow.

Meaningful Assertions

```
const assert = require('node:assert/strict');
function readCount(value) {
  if (!Number.isSafeInteger(value) || value < 0) throw new RangeError('Invalid count.');
  return value;
}
assert.equal(readCount(0), 0);
assert.throws(() => readCount(-1), RangeError);
assert.throws(() => readCount('1'), RangeError);
assert.deepEqual({ accepted: [1, 2] }, { accepted: [1, 2] });
console.log('Result, boundary, and failure assertions passed.');
// Expected output:
// Result, boundary, and failure assertions passed.
```

Pass a function to assert.throws so it controls the invocation. Calling the operation first would throw before the assertion can inspect it. Prefer the expected category or stable application code to native message text.

A test runner must let failed assertions fail the test and produce a failing process exit. Catching every failure and then printing success invalidates the check.

Reduce a Failure to Evidence

Keep one failing input and one nearby passing input. Remove unrelated UI, network work, and formatting while retaining the failure. Inject the clock or dependency result if it changes between runs.

Write the first violated expectation as an assertion. Trace types, values, ownership, and branch order from input to that point. Test one hypothesis at a time; a broad rewrite may hide the cause.

Breakpoints and Call Stacks

Pause before the first incorrect state or enable pause on exceptions. Inspect parameters and locals in the current frame, then caller frames to see how values arrived. Step over a trusted call to see its result, step into a suspicious call to inspect decisions, and step out when its contract is understood.

A debugger statement requests a pause if a debugger is attached; otherwise it has no observable effect. Debugger statement.

```
const input = { quantity: '2' };
debugger;
const total = 10 + input.quantity;
console.log(typeof input.quantity);
console.log(total);
// Expected output:
// string
// 102
```

The type at the breakpoint explains concatenation before any formatting code runs. The fix belongs at the input boundary: parse only accepted numeric syntax and validate the domain.

Stack Traces Have Limits

Stack text varies by runtime and build. Bundled code may require source maps. A later callback runs with a different active call chain, so its stack need not contain every earlier registration frame.

Cleanup errors can obscure the useful original failure. Inspect causes and AggregateError.errors according to the application's policy instead of assuming the outermost message is the only failure.

Errors and Debugging: Performance and Security

Match the Mechanism to the Contract

Do not throw thousands of exceptions merely to implement ordinary filtering when a boolean or tagged result expresses the expected alternative. Error construction and diagnostics add work, and frequent throwing can behave differently from normal completion.

Avoid the opposite folklore that a try block always makes code slow. Measure the complete workload after correctness is established. Parsing, allocation, validation, and logging often dominate.

Protect a Small Region

A broad catch around unrelated work can turn a programming defect into a misleading user-input error. Catch only failures the boundary understands. Preserve identity when rethrowing, or add useful context with cause.

Do not report success after an authorization failure or broken invariant. Callers need to distinguish fulfilled contracts from rejected operations.

Bound Parsing and Retention

The parser checks text length before JSON.parse and validates the decoded schema before producing an explicit projection. A string-length cap does not control the earlier allocation of a request body; its reader needs a separate byte limit.

Retaining an Error retains its cause and objects reachable from that cause. Attach small identifiers and selected metadata rather than entire requests, buffers, or credentials.

Keep Diagnostics From Leaking Data

Errors can expose paths, internal identifiers, raw payloads, or dependency details. Map expected public failures to safe messages and stable codes. Keep detailed diagnostics at an appropriate internal boundary.

Do not serialize arbitrary thrown values into responses. Some are not serializable, and custom serialization can execute code. Use a bounded response schema.

Cleanup Is Not a Transaction

Releasing a resource does not undo a payment, file write, or database update. Define side-effect ordering and transaction guarantees separately.

If release can fail, preserve the operation failure according to policy. The synchronous resource helper deliberately aggregates both failures; it does not promise asynchronous lifetime management.

Debugging and Logging Costs

Breakpoints and detailed object inspection belong in a controlled development workflow. Remove accidental debugger statements from production paths. Avoid logging a secret merely to reproduce a type mismatch.

Repeatedly logging and rethrowing at every layer multiplies noise. Report once at the boundary that has the context to decide what to do, preserving the cause chain for diagnosis.

Errors and Debugging: Professional Field Guide

Choose a Failure Contract

Situation	Useful response
Expected absence	Null or a tagged result
Invalid input type	TypeError or a domain category
Out-of-range value	RangeError or a domain category
Malformed syntax	SyntaxError with safe context
Unexpected dependency failure	Propagate or wrap once with cause
Competing scope failures	Preserve both under an explicit aggregate policy
Wrong answer without a throw	Assertion, reproduction, and state inspection

Consistency matters more than a favorite error style. Callers must know which outcomes are normal, which reject the request, and which indicate a defect.

Review a Catch

Identify the exact operation it protects and the failures it can recover from. Verify that unexpected values propagate. Check whether it could swallow AssertionError or an unrelated TypeError.

Ask whether wrapping adds new context. If yes, retain the cause; otherwise rethrow the original value.

Review a Finally

List all exit paths, including return and throw. Ensure cleanup does not accidentally select a successful result. If cleanup can throw, verify what happens to a pending operation failure.

Acquire before entering the release scope when failed acquisition produces no usable resource. After successful acquisition, release must receive the same resource exactly once.

Build a Useful Regression

Test the original failure with a minimal realistic input. Add a nearby accepted boundary so a fix cannot merely reject everything. Assert output shape, failure category, mutation policy, call order, and cleanup count where relevant.

A demonstration prints a result. A regression assertion must distinguish the defective implementation from the corrected one.

References

- Try/catch/finally: execution and completion.
- Error: failure values.
- Error.cause: retained context.
- JSON.parse: runtime data parsing.
- Node assert: result and failure assertions.
- Debugger: requesting a pause.

Further Reading

Read AggregateError for competing failures and Node debugging guidance for inspector workflows. The asynchronous volume covers Promise rejection, await, and cancellation boundaries.

Continue to Modules and Execution Modes.

CHAPTER 11

Modules and Execution Modes

Introduction

Splitting a program into files creates a new question: how does one file obtain a value from another? Modules answer that question with explicit interfaces and a loader. A module can hide a helper, expose a calculation, and declare its dependencies. Understanding its execution mode also explains why a snippet works in a browser console but fails in a Node.js file.

```
// Runtime: Node.js ES module
import assert from 'node:assert/strict';
const result = [2, 4, 6].map(value => value / 2);
assert.deepEqual(result, [1, 2, 3]);
console.log(JSON.stringify(result));
console.log(this === undefined);
// Expected output:
// [1,2,3]
// true
```

The import selects a Node built-in dependency. The array remains local to this module. At module top level, `this` is undefined; moving the same expression to another execution mode can change its meaning.

Learning Objectives

- Identify browser classic scripts, browser modules, Node CommonJS, and Node ES modules.
- Select a file extension or package setting deliberately.
- Define named and default exports and import them with the corresponding syntax.
- Explain live bindings, shared module state, and shallow object ownership across an interface.
- Trace resolution, linking, and evaluation without treating imports as textual substitution.
- Separate reusable library definitions from application startup effects.
- Diagnose missing files, mismatched exports, cycles, and strict-mode errors.
- Recognize dynamic import and top-level await while leaving promise scheduling to Volume 3.

Prerequisites

Read Functions and Callbacks, Objects and Data Ownership, Arrays and Collections, and Errors and Debugging. You should be able to return a value, validate an argument, and distinguish mutation from assignment.

Running the Examples

The baseline is Node.js 20+. Blocks marked `Runtime: Node.js ES module` must run as `.mjs` files or through `node --input-type=module`; other `js` blocks use CommonJS. The examples do not depend on automatic format detection, which has changed across Node releases.

This chapter is a self-contained module project in `code/volume-1/chapter-11`. Relative imports in its snippets refer to files in that folder. Keep those files together, or run the assertion entries from the repository root:

```
node code/volume-1/chapter-11/example-00-module-model.mjs
node code/volume-1/chapter-11/example-01-quote.mjs
node code/volume-1/chapter-11/example-02-report.mjs
node code/volume-1/chapter-11/example-03-module-challenges.mjs
node code/volume-1/chapter-11/example-04-commonjs.cjs
```

Library files intentionally produce no output when run alone. Their entries import and exercise them. Browser examples are described separately; a Node built-in import cannot run in a browser unchanged. Start with Theory.

Theory

Choose the Execution Mode First

Context	How it is selected	Scope and top-level `this`	Dependency mechanism
Browser classic script	A script element without `type="module"`	Global script environment; top-level `this` is the global object	No static import declarations
Browser ES module	A script element with `type="module"`	Module scope; top-level `this` is undefined	Standard import/export and host URL resolution
Node CommonJS	An explicit `.cjs` file	Per-module wrapper; top-level `this` initially refers to `module.exports`	`require` and `module.exports`
Node ES module	An explicit `.mjs` file	Module scope; top-level `this` is undefined	Standard import/export with Node resolution

In a browser classic script, a top-level `let` binding is not a property of the global object. Do not equate all global bindings with properties. In Node CommonJS, top-level declarations belong to the module wrapper, even though names such as `process` are supplied by the host.

For `.js` files in Node, declare the intended package format with the nearest relevant `package.json` `type` field: `module` or `commonjs`. Explicit `.mjs` and `.cjs` remain useful when both formats coexist. Do not rename every file in a repository or change its root package type without reviewing scripts and tooling. See Node package format rules.

Exports Define an Interface

An export makes a binding available through a module interface. Other declarations remain local. A named export has an interface name; a default export uses the interface name `default`, although the importer can choose its local identifier.

This complete library can run alone without output:

```
// Runtime: Node.js ES module
function validName(value) {
  return typeof value === 'string' && value.length > 0;
}
export function greeting(name) {
  if (!validName(name)) throw new TypeError('name must be non-empty');
  return `Hello, ${name}`;
}
export const formatVersion = 1;
export default greeting;
// Expected output:
// (none)
```

For a file saved as `greeting.mjs`, named import syntax is `import { greeting } from './greeting.mjs';` default syntax is `import greet from './greeting.mjs'`. An alias such as `import { greeting as greet } ...` changes only the local name. Braces here select named exports; they do not perform ordinary object destructuring. Export only what callers need to depend on. Export reference.

Imports Are Live Connections

The repository's `counter.mjs` exports a mutable `count` binding and an `increment` function. A named import observes the binding's current value. An ordinary local assignment takes the value at the time that assignment runs.

```
// Runtime: Node.js ES module
import { count, increment } from './counter.mjs';
const initial = count;
increment();
console.log(initial, count);
// Expected output:
// 0 1
```

An importer cannot reassign the imported binding. If an exported value is an object, that restriction does not make the object immutable. A mutation of a shared object's properties can still affect every caller. Module scope organizes access; it does not supply a deep copy. Import reference.

Static Imports and Dependency Order

A static import declaration belongs at module top level. The loader uses it to connect the dependency graph before executing the importing module's body. Placing an import after a console call in the source does not defer loading until that line.

For an acyclic graph without top-level await, dependencies evaluate before the module that needs them. Shared dependencies are reused for the same resolved module identity in that loader context. Two importers therefore do not automatically receive two independent counters. If callers need independent state, export a factory that allocates state per call.

Cycles connect modules back to themselves indirectly. Linking a cycle can succeed while evaluating an early read of an uninitialized `let`, `const`, or class binding fails. Avoid initializing A from B's value when B simultaneously initializes from A. Move shared constants to a third module, or move dependent work into an explicitly called function. Detailed asynchronous evaluation belongs in Volume 3.

Resolution Belongs to the Host

Node resolves `./pricing.mjs` relative to the importing module, not relative to the shell's current directory. Native Node ESM requires explicit extensions for relative file imports. `node:assert/strict` identifies a built-in; a bare name such as `some-package` uses package resolution. Browsers use URLs and can use import maps for bare names. A bundler may accept syntax that native Node or a browser loader does not. Node ESM resolution.

To derive a resource URL beside a module, use `new URL('./resource.json', import.meta.url)`. To turn a file URL into a Node filesystem path, use `fileURLToPath` from `node:url`; string slicing mishandles escaping and Windows paths. `import.meta.url` describes the current module's URL. Import metadata.

CommonJS Is a Different Interface

CommonJS calls `require` during execution and returns the dependency's exported value. The initial `exports` variable refers to `module.exports`; replacing `exports` alone does not replace the module's exported value. The following assertion program models the aliasing with ordinary objects:

```
'use strict';
const moduleRecord = { exports: {} };
let exportsAlias = moduleRecord.exports;
exportsAlias.ready = true;
exportsAlias = { ready: false };
console.log(moduleRecord.exports.ready);
console.log(exportsAlias === moduleRecord.exports);
// Expected output:
// true
// false
```

The real API uses `module.exports = value` to replace the export. CommonJS caching normally reuses exports for the same resolved file. Node interoperability rules depend on the version and dependency graph; this book does not assume every ESM module can be synchronously required. Keep interfaces in a single format where possible. CommonJS documentation.

Strict Mode Makes Failures Visible

ES modules use strict mode automatically. CommonJS does not become strict merely because it is a module in Node terminology; use a directive when appropriate. Strict mode rejects accidental assignments to undeclared names and throws for many invalid writes that would otherwise fail silently.

```
// Runtime: Node.js ES module
try {
  moduleLessonUndeclared = 1;
} catch (error) {
  console.log(error.name);
}
const policy = Object.freeze({ enabled: true });
try {
  policy.enabled = false;
} catch (error) {
  console.log(error.name);
}
// Expected output:
// ReferenceError
// TypeError
```

Strict mode is an execution rule, not a security sandbox or an input validator. It neither validates a URL nor limits what an imported package can do. Strict mode reference.

Dynamic Import Is an Asynchronous Boundary

`import(specifier)` is an expression that returns a promise for a module namespace. Use it when a dependency is genuinely conditional or deferred. It is not a synchronous replacement for `require`.

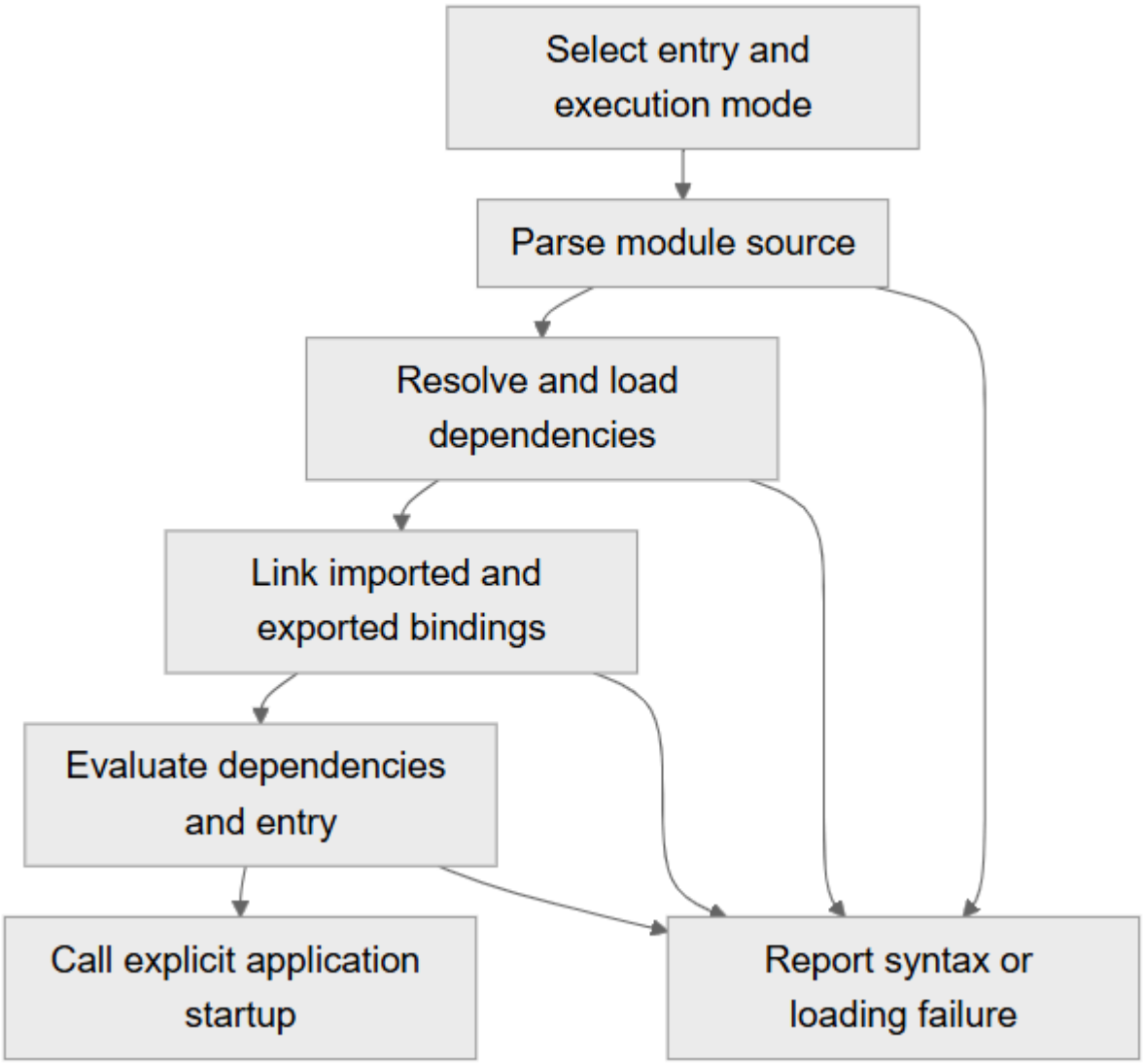
```
// Runtime: Node.js ES module
const namespace = await import('node:path');
console.log(namespace.posix.basename('/reports/daily.csv'));
// Expected output:
// daily.csv
```

This uses top-level await to wait for the result in an ES module. If loading fails, the promise rejects. A surrounding synchronous try block that merely starts an import does not handle a later rejection; await the result inside that try or attach a rejection handler. Volume 3 explains promises, task scheduling, and concurrent loading. Dynamic import reference.

Internal Working

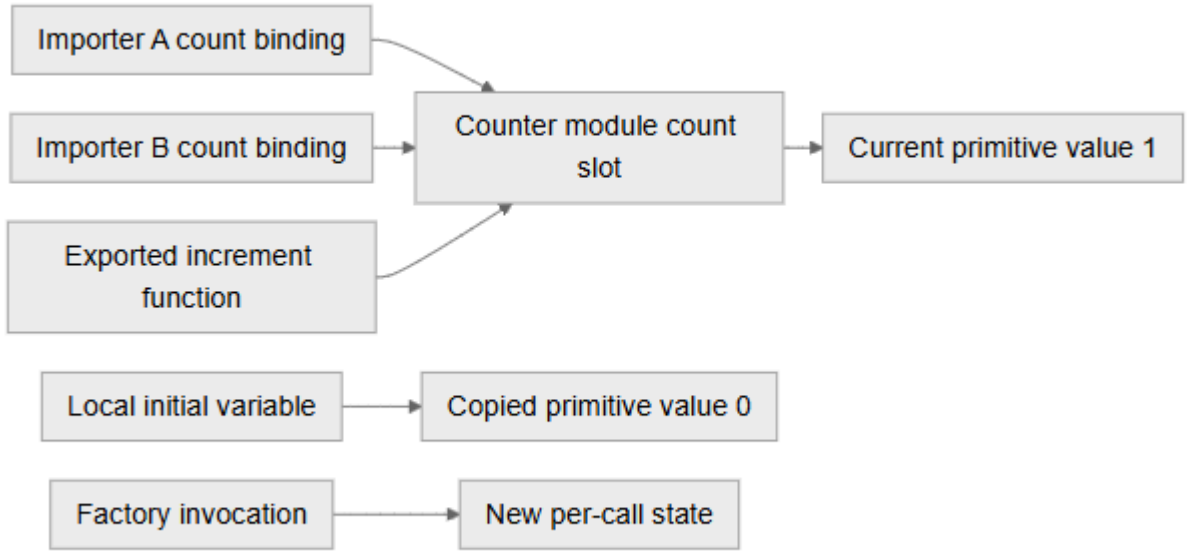
A Loader Builds a Graph

The entry file requests a dependency; that dependency can request more dependencies. Before a function call in the entry can use an imported name, the loader must resolve the requested module and connect the requested export. A missing export is different from a function receiving an invalid argument: the application may fail before its entry body begins.



Source: `diagrams/volume-1-chapter-11-module-flow.mmd`. The arrows show conceptual responsibilities, not an engine's exact internal scheduling. Real loaders can interleave discovery and loading. Top-level await adds asynchronous evaluation dependencies.

Binding Connections and Ordinary Values



Source: `diagrams/volume-1-chapter-11-binding-memory.mmd`. A module environment and its bindings are language-level concepts. This drawing does not require a particular engine to allocate a physical box per arrow.

The counter library has a deliberately shared state contract:

```
// Runtime: Node.js ES module
export let count = 0;
export function increment() {
  count += 1;
  return count;
}
// Expected output:
// (none)
```

Save it as `counter.mjs` to reproduce the theory's live-binding program. The repository already contains that file. No increment happens merely because the function is declared. This teaching counter has no persistence or concurrency contract; its assertions run in a fresh process.

Execution Trace

1. Node selects ESM for `example-00-module-model.mjs`.
2. The loader resolves `./counter.mjs` beside the entry and connects the named exports. During module instantiation, the function declaration initializes `increment` with its function value; `count` remains uninitialized.
3. Evaluation runs `let count = 0`, initializing that binding to zero before the entry body runs.
4. The entry saves the current primitive in a local variable.

5. Calling increment assigns a new number to the exported binding.
6. Reading count through the import sees one; reading the saved local still sees zero.
7. Importing the same resolved counter again returns a namespace connected to the same module instance in this process.

A fresh Node process creates a fresh loader context and count starts at zero again. Module caching is neither disk persistence nor shared memory across processes.

Scope, Lifetime, and Engine Work

An unexported helper can still be retained because an exported function uses it. Removing a local variable in an importer does not imply that a module or its resources are unloaded. Module caches, active functions, and other references affect reachability. Do not use dynamic import as an automatic memory release strategy.

The engine parses and executes JavaScript; the host supplies module resolution, filesystem access, and other facilities. Optimization may inline calls, but it must preserve observable behavior. Moving code into another file does not by itself make it faster, asynchronous, or isolated.

Cycles and Initialization

Consider a dependency diagram A -> B -> A. The names can be linked without all their values being initialized. If B's top-level initializer immediately reads an uninitialized exported constant from A, evaluation throws. If the read occurs later inside a function after both modules have finished initializing, that particular early-read failure may disappear. This is a timing distinction, not a reason to add cycles indiscriminately.

Further reading: ECMAScript module records and cyclic module records. Use the specification to distinguish binding creation, initialization, and evaluation; host-specific path lookup comes from the runtime documentation.

Production Examples

A Calculation Library With an Explicit Interface

An order system needs a line quote without reading environment variables, printing logs, or starting network requests when imported. The `pricing.mjs` library accepts integer cents from zero through 1,000,000 and integer quantities from zero through 1,000. Their product stays within safe integer range. It returns a fresh flat record and rejects coerced strings, negative values, and fractional values.

```
// Runtime: Node.js ES module
export function quoteLine(unitCents, quantity) {
  if (!Number.isInteger(unitCents) || unitCents < 0 || unitCents > 1_000_000) {
```

```
    throw new RangeError('unitCents must be an integer from 0 through 1000000');
  }
  if (!Number.isInteger(quantity) || quantity < 0 || quantity > 1_000) {
    throw new RangeError('quantity must be an integer from 0 through 1000');
  }
  return { unitCents, quantity, totalCents: unitCents * quantity };
}
// Expected output:
// (none)

// Time O(1); additional space O(1).
```

The entry file is the place for execution and assertions:

```
// Runtime: Node.js ES module
import assert from 'node:assert/strict';
import { quoteLine } from './pricing.mjs';
const first = quoteLine(250, 3);
assert.deepEqual(first, { unitCents: 250, quantity: 3, totalCents: 750 });
assert.notStrictEqual(first, quoteLine(250, 3));
assert.equal(quoteLine(1_000_000, 1_000).totalCents, 1_000_000_000);
assert.equal(quoteLine(0, 0).totalCents, 0);
for (const bad of ['250', null, NaN, Infinity, -1, 1.5, 1_000_001]) {
  assert.throws(() => quoteLine(bad, 1), RangeError);
}
for (const bad of ['3', undefined, -1, 0.5, 1_001]) {
  assert.throws(() => quoteLine(250, bad), RangeError);
}
console.log(JSON.stringify(first));
console.log('quote assertions passed');
// Expected output:
// {"unitCents":250,"quantity":3,"totalCents":750}
// quote assertions passed
```

Stored entry: `code/volume-1/chapter-11/example-01-quote.mjs`. This is a bounded quote calculation, not a tax, currency-conversion, or payment-settlement implementation. Naming the unit in the interface prevents an accidental cents-versus-units conversion.

Explicit Startup With an Injected Output

A report module should be importable by a test without printing a report. `reporting.mjs` exposes a factory that validates a trusted synchronous writer and returns a command. The command validates all arguments before writing exactly once. A writer failure propagates unchanged; there is no retry that could duplicate an external effect.

```
// Runtime: Node.js ES module
import { quoteLine } from './pricing.mjs';
export function createReport(write) {
  if (typeof write !== 'function') throw new TypeError('write must be a function');
  return function report(unitCents, quantity) {
    const quote = quoteLine(unitCents, quantity);
    const line = `items=${quote.quantity}; totalCents=${quote.totalCents}`;
    write(line);
```

```
    return quote;
  };
}
// Expected output:
// (none)

// Time O(1), excluding the writer; additional space O(1) under the numeric bounds.
```

The factory performs no output when constructed. The returned command decides when the effect occurs:

```
// Runtime: Node.js ES module
import assert from 'node:assert/strict';
import { createReport } from './reporting.mjs';
const lines = [];
const report = createReport(line => lines.push(line));
assert.equal(lines.length, 0);
assert.equal(report(125, 4).totalCents, 500);
assert.deepEqual(lines, ['items=4; totalCents=500']);
assert.throws(() => report('125', 4), RangeError);
assert.equal(lines.length, 1);
assert.throws(() => createReport(null), TypeError);
const failure = new Error('writer unavailable');
const broken = createReport(() => { throw failure; });
assert.throws(() => broken(1, 1), error => error === failure);
console.log(lines[0]);
console.log('report assertions passed');
// Expected output:
// items=4; totalCents=500
// report assertions passed
```

Stored entry: `code/volume-1/chapter-11/example-02-report.mjs`. A promise-returning writer is outside this synchronous contract. An asynchronous version must await the writer and define rejection behavior; simply accepting an async callback would allow the command to report success too early.

Review the Boundary

The dependency graph is entry -> reporting -> pricing. Pricing depends on no application state. Reporting depends on pricing and a writer supplied at construction. Tests import the same implementations used by the entries, so an assertion cannot accidentally validate a separate copy of the library.

Keep configuration parsing at startup and pass validated values inward. Avoid reading changing environment variables at arbitrary import time. An explicit `start(config)` or factory call gives tests and applications control over initialization order and failure handling.

Interview Perspective

A Clear Opening Explanation

An ES module has its own scope and an explicit interface of exported bindings. Import declarations connect those bindings through a dependency graph. The host resolves paths; the language defines binding and evaluation behavior. Modules execute in strict mode, and importing a mutable object does not copy or freeze it.

Before predicting output, name the host, file format, package setting, and whether execution starts in a fresh process. A REPL, a browser classic script, a CommonJS file, and an ES module are different contexts.

Explain a Live Binding

```
// Runtime: Node.js ES module
import * as counter from './counter.mjs';
const { count: saved } = counter;
counter.increment();
console.log(saved, counter.count);
// Expected output:
// 0 1
```

The namespace property reflects the current exported binding. Destructuring into saved reads its value once. The result does not prove that primitive numbers are mutable; increment assigns a new primitive value to the source binding.

Does Import Create a New Instance?

Usually, repeating an import of the same resolved module identity in one loader context reuses that instance. Do not generalize this into a singleton across browser workers, Node processes, realms, or distinct resolved URLs. If each customer needs a separate counter, export a factory and allocate its state per invocation.

Is CommonJS Just Older Import Syntax?

No. CommonJS exposes an exported value through a runtime require call. ESM describes imported/exported bindings and links the graph. They also have different top-level environments and host rules. Node provides interoperability, but an interview answer should name the runtime version before promising a particular cross-format operation.

Why Can a Cyclic Import Fail?

An exported lexical binding can exist before its initializer has run. If another module in the cycle reads it at that point, the temporal dead zone produces a ReferenceError. Ask when the read occurs. Moving it into a later function call can alter the timing; removing an unnecessary cycle often makes the design easier to understand.

Production Follow-up

Suppose importing a report utility starts a server. Tests importing the calculation may bind a port or keep the process alive. Separate the definition from a start function and call start only from the application entry. This also puts startup failures at a boundary that can report them meaningfully.

Common Interview Mistakes

- Describing imported names as copied values in every case.
- Saying an imported object is immutable because its binding is read-only.
- Assuming a relative module specifier starts at the shell's current directory.
- Claiming every module runs in a separate thread.
- Treating dynamic import as a synchronous function returning the exported value.
- Explaining loader behavior without specifying browser, Node, or bundler.

A strong answer connects syntax to one observable consequence, then states the boundary of the claim. For example: the counter is shared by these two importers in this process; a new process starts a new counter.

Exercises and Coding Challenges

Run ES module solutions as `.mjs` files. Import-based solutions belong beside the supplied chapter libraries. The companion `example-03-module-challenges.mjs` adds failure and identity assertions for all six exercises.

Exercise 1: Expose a Bounded Calculation

Write an exported `doubleScore` function accepting integer scores from 0 through 50. Reject strings and out-of-range values with RangeError. Return the doubled number without logging inside the function.

Hint: An export does not change the function's validation obligations.

```
// Runtime: Node.js ES module
export function doubleScore(score) {
  if (!Number.isInteger(score) || score < 0 || score > 50) {
```

```
    throw new RangeError('score must be an integer from 0 through 50');
  }
  return score * 2;
}
console.log(doubleScore(21));
// Expected output:
// 42

// Time O(1); additional space O(1).
```

Alternative: Keep the function local if no other module needs it. Exporting every helper increases the interface that future changes must preserve.

Exercise 2: Alias an Import

Import the production quote function as `calculate`. Compute the total for two items at 175 cents each. Do not duplicate the validation logic in the caller.

Hint: Rename the local binding with `as` inside a named import.

```
// Runtime: Node.js ES module
import { quoteLine as calculate } from './pricing.mjs';
console.log(calculate(175, 2).totalCents);
// Expected output:
// 350

// Time O(1); additional space O(1).
```

Alternative: Import a namespace when grouping several related exports improves readability. An alias does not change the exported interface name.

Exercise 3: Give Callers Independent State

Write an exported factory `createSequence` that returns a function yielding 1, then 2, and so on. Each factory call must own its state. Throw RangeError when the next value would exceed Number.MAX_SAFE_INTEGER.

Hint: Put the counter inside the factory body, not at module top level.

```
// Runtime: Node.js ES module
export function createSequence() {
  let last = 0;
  return function next() {
    if (last === Number.MAX_SAFE_INTEGER) throw new RangeError('sequence exhausted');
    last += 1;
    return last;
  };
}
const first = createSequence();
const second = createSequence();
console.log(first(), first(), second());
```

```
// Expected output:
// 1 2 1

// Time O(1) per call; retained space O(1) per sequence.
```

Alternative: A deliberately shared module counter fits a different contract. This example is process-local and does not guarantee unique IDs across machines or restarts. Testing every value to exhaustion is impractical; review the bound or parameterize a limit when exhaustive boundary testing is needed.

Exercise 4: Separate Construction From Effects

Write `createAnnouncer(write)`. It validates a synchronous writer and returns a function accepting a non-empty string of at most 40 characters. Calling that function writes `ready:` plus the name once and returns the line. Construction and invalid calls must not write anything.

Hint: Keep write inside the returned function, after validation.

```
// Runtime: Node.js ES module
export function createAnnouncer(write) {
  if (typeof write !== 'function') throw new TypeError('writer required');
  return function announce(name) {
    if (typeof name !== 'string' || name.length === 0 || name.length > 40) {
      throw new TypeError('name must have 1 through 40 characters');
    }
    const line = `ready:${name}`;
    write(line);
    return line;
  };
}
const lines = [];
const announce = createAnnouncer(line => lines.push(line));
console.log(lines.length);
announce('worker');
console.log(lines[0]);
// Expected output:
// 0
// ready:worker

// Time and additional space O(1) under the name bound, excluding the writer.
```

Alternative: Return a line from a pure formatter and let the caller write it. Choose a command factory only when retaining the writer simplifies repeated calls.

Exercise 5: Preserve an Export Alias

Model a CommonJS export with a record containing an `exports` property. A second variable initially aliases that property value. Add a ready flag through the alias, then replace the exported value with a function. Show that replacing the record's property does not redirect the old alias.

Hint: Assignment changes one binding or property, not all references to the previous object.

```
'use strict';
const record = { exports: {} };
const alias = record.exports;
alias.ready = true;
record.exports = () => 'started';
console.log(alias.ready);
console.log(record.exports());
console.log(alias === record.exports);
// Expected output:
// true
// started
// false

// Time O(1); additional space O(1).
```

Alternative: Mutating properties of the initial export object preserves its identity. Replacing module.exports is appropriate when the whole public value should be a function or another object.

Exercise 6: Treat Dynamic Loading as Fallible

Implement `loadBasename` as an async exported function. It dynamically imports Node's path built-in and returns the POSIX basename of `/logs/run.txt`. Await the result at the caller. No filesystem read is needed.

Hint: Import resolves to a namespace, not directly to a basename string.

```
// Runtime: Node.js ES module
export async function loadBasename() {
  const path = await import('node:path');
  return path.posix.basename('/logs/run.txt');
}
console.log(await loadBasename());
// Expected output:
// run.txt

// Fixed input: O(1) application work; module loading cost belongs to the host.
```

Alternative: Use a static import when the dependency is always needed. Dynamic loading is justified by a conditional boundary, not by a belief that it automatically improves throughput. The companion also awaits a deliberately missing module and checks the resulting failure.

Review Your Solutions

Check the interface name, execution mode, initial state, number of effects, and failure behavior separately. A function body can be correct while its caller imports the wrong name. Conversely, successful linking does not prove that its argument validation works.

MCQs

Assume a fresh Node process for every question and the explicitly stated mode. Select one answer before reading the explanation.

1. Which filename explicitly selects Node ES modules regardless of the surrounding package type?

A. `worker.txt` B. `worker.mjs` C. `worker.cjs` D. `worker.json`

Answer: B. The `.mjs` extension identifies an ES module. `.cjs` identifies CommonJS; `.js` requires attention to package and runtime rules.

2. What is printed in an ES module?

```
// Runtime: Node.js ES module
console.log(this === undefined);
// Expected output:
// true
```

A. false, because this is always the global object B. false, because this is module.exports C. A ReferenceError D. true

Answer: D. ES module top-level this is undefined. CommonJS has a different top-level environment.

3. A library exports `quoteLine` as a named export only. Which interface choice matches it?

A. A named import selecting quoteLine, optionally with a local alias B. A default import without changing the library C. Calling require in every browser module D. Reading globalThis.quoteLine

Answer: A. The caller must select the exported name. A module does not automatically publish declarations on the global object or create a default export.

4. What is printed after a counter import?

```
// Runtime: Node.js ES module
import { count, increment } from './counter.mjs';
const saved = count;
increment();
console.log(saved, count);
// Expected output:
// 0 1
```

A. 1 1 B. 0 0 C. 0 1 D. 1 0

Answer: C. saved contains the earlier primitive value. The import remains connected to the exported binding.

5. What does read-only import binding mean for an imported object?

A. Every nested property is frozen B. Its properties may still be mutable even though the importer cannot reassign the binding C. The object is cloned for every importer D. The object can only be read through a getter

Answer: B. Binding assignment and object mutation are separate operations. Ownership requires its own interface contract.

6. Native Node ESM resolves `./pricing.mjs` relative to what?

A. The importing module's location B. The operating system root C. Always the shell's current directory D. The nearest file named pricing in node_modules

Answer: A. A relative module specifier starts from the importing module. A filesystem API receiving an ordinary relative path can follow different rules.

7. What is printed by this export-alias model?

```
'use strict';
const record = { exports: {} };
let alias = record.exports;
alias = { enabled: true };
console.log(Object.hasOwn(record.exports, 'enabled'));
// Expected output:
// false
```

A. true B. undefined C. A SyntaxError D. false

Answer: D. Reassigning alias does not change record.exports. This is the reason replacing exports alone is not the CommonJS replacement operation.

8. What is the best reason to keep a server's start call out of a reusable library's top level?

A. Functions cannot be exported after startup B. All top-level function calls are syntax errors C. Importing the library should not unexpectedly bind a port or start work D. Every imported file runs in another process

Answer: C. Explicit startup makes initialization and effects controllable. Modules can execute top-level calls, so the design must choose which effects belong there.

9. What does dynamic import return immediately?

A. A promise for a module namespace B. Always the module's default export C. A filesystem path D. A new worker thread

Answer: A. Await the promise or attach handlers to obtain the namespace and handle a loading failure.

10. A cyclic dependency reads an exported const before its initializer executes. What is the relevant failure?

A. Every cycle is rejected by the parser B. The const silently becomes null C. The const is initialized to zero D. The read can throw ReferenceError because the binding is uninitialized

Answer: D. Cycles are not universally invalid. The timing of the read relative to initialization matters.

11. What is printed in strict mode?

```
'use strict';
const settings = Object.freeze({ size: 2 });
try {
  settings.size = 3;
} catch (error) {
  console.log(error.name);
}
// Expected output:
// TypeError
```

A. ReferenceError B. TypeError C. SyntaxError D. 3

Answer: B. The property exists, but the frozen object's data property cannot be written. Strict mode turns this invalid write into a thrown failure.

12. Which statement about a shared module counter is accurate?

A. It survives every process restart B. All machines importing the package share it C. Its sharing is limited by resolved identity and the loader context D. Importing the counter always copies its current state

Answer: C. A new process has a new instance. Distinct module identities can also create distinct instances within a runtime.

13. What is printed by two calls to a state factory?

```
'use strict';
function createCounter() {
  let value = 0;
  return () => ++value;
}
const a = createCounter();
const b = createCounter();
console.log(a(), b(), a());
// Expected output:
// 1 1 2
```

A. 1 2 3 B. 0 0 1 C. 1 1 1 D. 1 1 2

Answer: D. Each factory call creates a separate local state. Exporting the factory would preserve this per-call behavior.

14. Which import is appropriate for a dependency that is always required at startup?

A. Usually a static import, making the dependency explicit B. Always dynamic import because it executes synchronously C. An import constructed from unvalidated user input D. A new Function containing the dependency source

Answer: A. Prefer a simple explicit graph unless a real conditional-loading requirement justifies dynamic import. Loading source from input changes the trust boundary.

15. What does strict mode guarantee?

A. Imported packages cannot access the filesystem B. Objects returned by every function are immutable C. Certain invalid operations throw and several ambiguous language behaviors are restricted D. All input values have the declared application type

Answer: C. Strict mode is a language execution rule. It does not provide process isolation, deep freezing, or schema validation.

16. What is printed when a namespace value is copied to a local?

```
// Runtime: Node.js ES module
import * as source from './counter.mjs';
const { count } = source;
source.increment();
source.increment();
console.log(count, source.count);
```

```
// Expected output:
// 0 2
```

A. 2 2 B. 0 2 C. 0 0 D. A SyntaxError because namespaces cannot be destructured

Answer: B. Ordinary destructuring reads a value at that moment. It does not create the special connection established by a named import declaration.

Revision Sheet and Chapter Summary

Revision Sheet

Question	Working rule
Which Node format?	Use explicit `.mjs` or `.cjs`, or declare the intended package type for `.js`.
Named or default?	Match the exported interface; a local alias is a separate choice.
Is a named import a copied value?	It is a read-only live binding connection.
Does read-only mean immutable?	No; a referenced object's properties can remain mutable.
Where does a relative import start?	At the importing module's location.
What makes startup predictable?	Keep library definitions separate from explicit start calls.
How do callers get independent state?	Call a factory that allocates state per invocation.
What does dynamic import produce?	A promise for a namespace, with a possible rejection.
Why can a cycle fail?	Evaluation can read a lexical binding before initialization.
Is strict mode isolation?	No; it restricts language behavior, not runtime authority.

Trace Before Running

```
// Runtime: Node.js ES module
import { quoteLine } from './pricing.mjs';
const a = quoteLine(50, 2);
const b = quoteLine(50, 2);
a.totalCents = 0;
console.log(a === b, b.totalCents);
// Expected output:
// false 100
```

The imported function value is reused, while its implementation creates a new result record on every call. Module caching does not force every function result to be shared.

Summary

Modules define interfaces and dependency relationships. Execution mode determines parsing and top-level behavior; the host determines how a request finds a module. A useful library minimizes surprising startup effects and documents ownership of exported state. Successful loading is only the beginning of correctness: callers still need valid arguments, controlled effects, and meaningful failure handling.

References

- Node.js 20.19 ESM documentation: version-specific resolution and interoperability rules.
- Node.js CommonJS documentation: the wrapper, exports, and caching.
- Node.js package rules: explicit package format and entry points.
- MDN import and export: interface syntax and bindings.
- MDN strict mode: language changes and error behavior.

Further Reading

Continue with Lexical Scope and Closures to explain retained factory state in depth. Revisit Errors and Debugging when distinguishing loading failures from application failures. Volume 3 will develop promise scheduling and asynchronous initialization; those chapters remain planned.

Edge Cases, Debugging, and Failure Modes

Identify the Stage of Failure

Symptom	Likely boundary	First check
Import syntax rejected	Wrong execution mode or invalid syntax	File extension, package type, parser context
Module cannot be found	Resolution/loading	Full filename, case, relative base, installed package
Requested export missing	Linking/interface mismatch	Named versus default export and exact interface name
ReferenceError in a cycle	Evaluation before initialization	Dependency graph and top-level reads

Symptom	Likely boundary	First check
Tests print or start services on import	Startup side effects	Top-level calls in imported libraries
Two tests share a counter	Cached module state	State allocation location and test isolation

Do not fix a missing-file error by changing unrelated arithmetic or adding catch blocks around every function. Reproduce the failure with the smallest entry and its immediate dependency.

A Namespace Is Not a Writable Configuration Object

```
// Runtime: Node.js ES module
import * as counter from './counter.mjs';
try {
  counter.count = 10;
} catch (error) {
  console.log(error.name);
}
console.log(counter.count);
// Expected output:
// TypeError
// 0
```

Pass configuration into a factory or export an intentional update operation. Do not expect assigning a namespace property to reconfigure the exporting module.

Catch a Dynamic Loading Failure Where It Is Awaited

```
// Runtime: Node.js ES module
try {
  await import('./intentionally-missing-module.mjs');
} catch (error) {
  console.log(error.code === 'ERR_MODULE_NOT_FOUND');
}
// Expected output:
// true
```

The error code here is a Node host detail. This local missing file is deliberate and must remain absent. Exact error messages contain paths and vary across environments, so the assertion checks a stable classification instead of a whole message.

Case and Path Portability

A case-insensitive development filesystem can hide a filename-case mismatch that fails in another environment. Match the committed filename exactly. Use explicit relative extensions and avoid hard-coded absolute machine paths. Paths containing spaces belong in quotes in shell commands; they do not

require hand-built string transformations inside a module URL.

CommonJS and ESM Interoperability

Do not repair every interop failure by renaming a package or changing a root package type. Check the Node version, package's supported entry points, and whether the dependency performs top-level await. The chapter's `.mjs` and `.cjs` examples intentionally remain explicit, avoiding version-dependent assumptions about requiring ESM synchronously.

Browser Loading

A browser module is loaded as a resource with URL and server rules. Opening an HTML file directly through a file URL is a poor substitute for serving a project locally. Inspect the network request, HTTP status, content type, and console error. A 200 response containing an HTML fallback page is not a successful JavaScript module load. Browser module scripts also have loading behavior distinct from classic scripts; verify it against the HTML script element specification.

Avoid Test Order Dependence

If importing a module creates mutable state, a second test in the same process may inherit changes from the first. Prefer a fresh factory result per test. Reloading through artificial query strings changes module identity and can create additional instances; it is not a general resource cleanup policy.

Performance and Security Notes

Measure Startup and Work Separately

Importing a dependency graph has costs: locating files, reading or fetching source, parsing, and evaluating top-level work. Calling an already loaded calculation measures something different. A benchmark that repeatedly calls quoteLine cannot establish how quickly a cold application starts.

Move unnecessary work out of module top level when it can be performed explicitly at the correct lifecycle stage. This is especially useful for services, database connections, timers, and large precomputed data. Deferring essential work only moves its latency to a later request; measure the user-visible boundary that matters.

Dynamic Import Is a Trade-off

Conditional loading can avoid loading an unused feature. It also introduces a promise boundary and a new failure path. In a browser, a build system may split the dependency into another resource request. In Node, dynamic import does not imply a browser-style chunk or a separate thread. Use static imports for core dependencies and dynamic loading where the application can tolerate and handle the delay.

Shared State Can Retain Data

A module-level Map can keep request records reachable for as long as the module remains reachable. A cache needs an entry bound, an eviction policy, and a lifecycle owner. Moving an unbounded array from globalThis into a module hides its name but does not bound its memory growth.

Factory-created state makes ownership clearer, but returned functions can retain it. Close external resources explicitly; garbage collection of a JavaScript object is not a substitute for a documented resource lifecycle.

Imports Execute Code With Runtime Authority

An imported dependency can perform effects at evaluation time. Review dependency provenance and updates, and keep privileged application boundaries explicit. Strict mode does not stop a Node module from using permitted filesystem or process APIs. Module scope is encapsulation, not a sandbox for hostile code.

Do not concatenate an untrusted request parameter into an import specifier. A finite mapping from allowed feature names to literal import functions makes the permitted surface reviewable. Validate the selector before loading and still handle rejection. Avoid eval or Function as an alternative loader for untrusted strings.

Keep Secrets Out of Browser Source

Exporting a constant or placing it in an unexported helper does not hide it from someone who can download the code. Credentials requiring secrecy belong on a trusted server with an appropriate protocol. Bundling and minification do not change that trust boundary.

When a New Module Is Unhelpful

A file containing one trivial expression can add navigation overhead without defining a useful boundary. Split around ownership, dependency direction, independent tests, and stable interfaces. Avoid a barrel module that imports large unrelated subsystems merely to re-export one small helper; inspect the generated build and runtime behavior before making performance claims.

Professional Field Guide

Record the Runtime Contract

A runnable example should name its host and mode. Record the minimum supported Node version, use explicit file formats where useful, and make package settings intentional. When a snippet fails in a console, first reproduce it as the documented file before changing the lesson's semantics.

Case Study: Importing a Helper Starts a Worker

A test imports a reporting helper and unexpectedly opens a connection. Its containing module called start at top level. Split definitions into a library and call start from an entry. Inject the connection or writer into the library so the test can assert behavior without creating the real resource.

Verify three separate moments: importing performs no application work, constructing validates dependencies without invoking them, and calling the returned operation performs exactly the documented effect. The report companion demonstrates this separation.

Case Study: Customers Share a Counter

A module exports a mutable sequence and several request handlers import it. The author expected each import to create fresh state. The loader instead reused one module instance. If per-request independence is required, export a factory and create state at the request boundary. If shared sequencing is intended, define concurrency, persistence, and exhaustion requirements separately; an in-memory counter alone is insufficient for distributed IDs.

Case Study: A Deployment Fails Before Startup Logging

The entry imports a named export that was renamed in a library. Linking fails before the entry's first log call. Compare the exported interface and import syntax, then run the entry directly. Adding a try/catch inside a function that never executes cannot repair this interface mismatch.

Review Checklist

- Does each file run under the intended format in the supported runtime?
- Are relative filenames explicit and case-correct?
- Does the module export a focused interface with documented ownership?
- Are calculation, configuration parsing, and startup effects separated?
- Does a shared module variable have an intentional scope and lifetime?
- Do cycles introduce early reads or confusing initialization order?
- Are asynchronous loads awaited and failures handled at a useful boundary?
- Do tests import the actual implementation and check invalid calls as well as success?
- Does every resource have an explicit owner responsible for closing it?

Volume 1 Mastery Check

Build a small application from three modules: a parser, a calculation, and an entry that reports a result. Validate data at the boundary, use integer units where exactness matters, return fresh records when ownership requires it, keep iteration bounded, preserve error context, and make startup explicit. Explain the program's execution and shared references before running it.

You now have the vocabulary required by Volume 2: Lexical Scope and Closures. Use the execution-model companion to connect bindings, function calls, and memory before moving to advanced scope and object behavior.

JavaScript Execution Model: A Companion

This companion revisits the introduction after Variables and Data Types and Functions and Callbacks. Its stable URL also serves readers entering Volume 2. It develops execution contexts and lexical environments; it does not replace the main Chapter 1 reading sequence.

The assertion programs in `code/chapter-01` exercise call initialization, call-stack order, lexical lookup, and the owned audit recorder. Run them with the repository's `npm run examples:test` command.

Objectives and Prerequisites

Know declarations, function calls, parameters, return values, and basic object/array syntax. Run the independent examples in Node.js 20 or later. By the end, distinguish a binding from a call frame, resolve an identifier from its definition environment, trace initialization order, and explain why a returned array copy may still expose internal records.

Execution Contexts Are a Semantic Model

An execution context tracks the evaluation of code, including its lexical environment and other specification state. An environment record associates names with bindings and can link to an outer environment. These concepts define behavior; engines do not have to allocate a literal heap object for every box drawn in a teaching diagram. See ECMAScript execution contexts and environment records.

The call stack answers which synchronous computation resumes when a call returns. Lexical lookup answers which binding an identifier denotes. These are different questions: a function does not generally obtain free variables from whichever function happens to call it.

Top-level code also depends on execution mode. A classic script, an ECMAScript module, and a Node CommonJS file do not have identical top-level binding behavior. This companion uses CommonJS-compatible files and avoids relying on top-level `this` or global-object properties.

Trace a Call

```
'use strict';

const centsPerCopy = 1200;
function quote(copies) {
  const totalCents = copies * centsPerCopy;
  return totalCents;
}
```

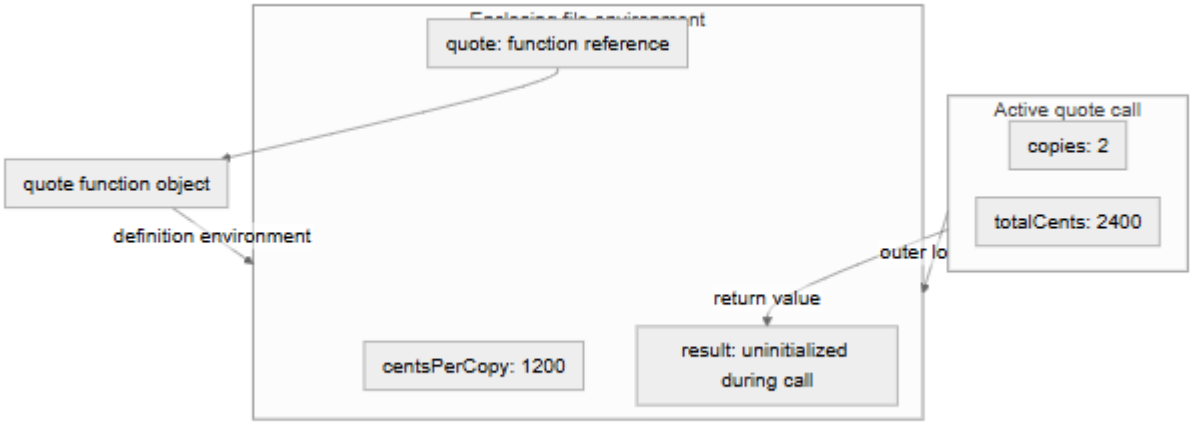
```
const result = quote(2);
console.log(result);

// Expected output:
// 2400
```

Before the surrounding statements run, declaration instantiation makes the function declaration callable. The lexical bindings for `centsPerCopy` and `result` initially remain uninitialized. Evaluating the first declaration initializes `centsPerCopy` to 1200. Calling `quote` supplies 2 to the parameter `copies`. The function resolves `centsPerCopy` from its enclosing environment, computes 2400, initializes its local `totalCents`, and returns that value. Only then is `result` initialized.

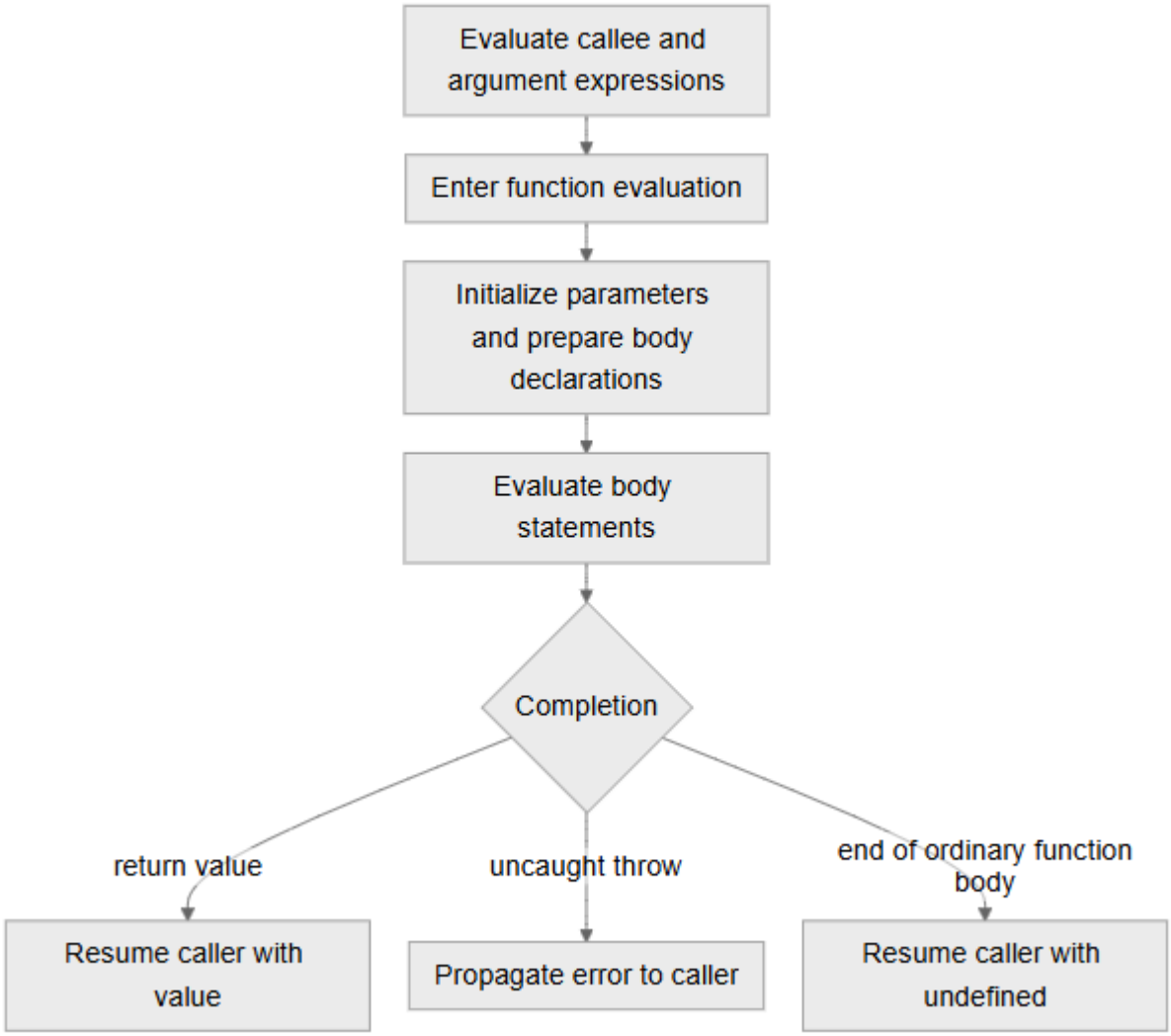
Calling this "creation, then execution" can be a useful simplification. It is not a claim that source lines move or that all declarations receive undefined. Different declaration forms have different initialization rules.

Memory and Lookup Diagram



Source: `diagrams/volume-1-chapter-01-companion-memory.mmd`. This is a semantic snapshot immediately before the return; it makes no claim about physical stack or heap allocation.

Call and Return Flow



Source: `diagrams/volume-1-chapter-01-companion-flow.mmd`. This diagram covers ordinary synchronous functions. Generators and asynchronous functions add suspension and completion behavior developed later.

Definition Location Wins Over Call Location

```
'use strict';

const region = 'APAC';
function label(id) {
  return `${region}:${id}`;
}
function runInAnotherScope() {
  const region = 'EU';
  return label('A-7');
}
```

```
console.log(runInAnotherScope());

// Expected output:
// APAC:A-7
```

The local `region` inside `runInAnotherScope` does not become an outer environment of `label`. The called function retains the lexical relationship established where it was created. A debugger's caller frame is not an extra lexical scope for the callee.

A Returned Function Retains a Binding

```
'use strict';

function createCounter(start) {
  if (!Number.isSafeInteger(start) || start < 0 || start >= 1000) {
    throw new RangeError('start must be an integer from 0 through 999');
  }
  let count = start;
  return function next() {
    if (count === 1000) throw new RangeError('counter limit reached');
    count += 1;
    return count;
  };
}
const first = createCounter(3);
const second = createCounter(10);
console.log(first(), first(), second());

// Expected output:
// 4 5 11
```

Each factory call creates its own `count` binding. Returning does not make that reachable state vanish: `first` and `second` retain access to separate environments. The active factory calls have finished, but their bindings remain useful to the returned functions. This distinction is the foundation of lexical scope and closures.

Initialization Errors Are About Timing

```
'use strict';

function readTooSoon() {
  return total;
  const total = 42;
}
try {
  readTooSoon();
} catch (error) {
  console.log(error.name);
}
function readAfterInitialization() {
  function read() { return total; }
  const total = 42;
```

```
  return read();
}
console.log(readAfterInitialization());

// Expected output:
// ReferenceError
// 42
```

The first function's declaration still creates the lexical binding even though its initializer is never reached. Reading it throws. In the second function the nested function is defined before `total`, but called after initialization. Textual position alone does not determine whether a read occurs in the temporal dead zone.

Production Exercise: An Audit Recorder With Owned Records

Requirements: Create an isolated recorder per request or workflow. Accept ordinary data records with nonempty string `id` and `action` fields. Copy only those primitive fields into internal storage. Return a history whose array and entry objects can be changed by the caller without affecting later results. No clocks, global state, or nested-data ownership claims are allowed in this contract.

Solution: Copy the record on ingress and copy each retained record on egress. A new outer array alone is insufficient.

```
'use strict';
const assert = require('node:assert/strict');

function createAuditRecorder() {
  const entries = [];
  return function record(event) {
    if (event === null || typeof event !== 'object' || Array.isArray(event)) {
      throw new TypeError('event object required');
    }
    const { id, action } = event;
    if (typeof id !== 'string' || id.trim() === '' ||
        typeof action !== 'string' || action.trim() === '') {
      throw new TypeError('nonempty id and action required');
    }
    entries.push({ id: id.trim(), action: action.trim() });
    return entries.map(entry => ({ id: entry.id, action: entry.action }));
  };
}
const record = createAuditRecorder();
const input = { id: 'A-1', action: 'created' };
const snapshot = record(input);
input.action = 'tampered';
snapshot[0].id = 'changed';
snapshot.push({ id: 'injected', action: 'fake' });
const next = record({ id: 'A-2', action: 'approved' });
assert.deepEqual(next, [
  { id: 'A-1', action: 'created' },
  { id: 'A-2', action: 'approved' }
]);
assert.throws(() => record({ id: 'A-3', action: {} }), TypeError);
assert.equal(createAuditRecorder()({ id: 'B-1', action: 'created' }).length, 1);
console.log(next.map(entry => `${entry.id}:${entry.action}`).join('|'));
```

```
// Expected output:
// A-1:created|A-2:approved
```

After `n` retained events, producing a complete history takes O(n) record-copy work plus the relevant string processing. The recorder keeps O(n) records; the returned history also contains O(n) records. Repeated full snapshots after each event incur quadratic cumulative record-copy work. For a long-lived audit log, append to durable storage and return one accepted record or a paginated query instead.

This implementation deliberately retains only primitive strings. If the schema later includes an object-valued `details`, decide who owns that graph. Adding `{ ...event }` would make only a shallow copy and would reintroduce shared nested state. The object ownership chapter develops that contract.

Debugging, Performance, and Security

When a free identifier has an unexpected value, inspect where its function was defined and the binding's current value, not just the immediate caller. When data changes after a function returns, inspect aliases to the object as well as retained closures. A stack frame finishing is not evidence that every value it touched has become unreachable.

A retained callback can keep a large graph reachable through its enclosing state. Garbage collection reclaims unreachable data at an implementation-chosen time; it does not impose an application retention policy. Keep histories bounded and release references when the owning feature is finished.

A closure limits ordinary access paths but is not a complete security boundary. Returning a mutable internal record grants an access path to that record. Likewise, exposing a method capable of writing arbitrary paths grants that capability even if its implementation is hidden in a closure. Validate and restrict the operations you expose.

V8's Ignition documentation describes one execution tier. Do not infer a fixed physical allocation or a guaranteed optimization from this chapter's semantic diagrams. Measure a real workload before rewriting clear function boundaries for speculative engine gains.

Interview Answer and Revision

An ordinary call evaluates the callee and arguments, enters function evaluation with parameter bindings, runs the body, and resumes its caller with a return value or propagated error. Identifier lookup follows lexical environment relationships. Returned functions can retain access to bindings after the creating call finishes. Copying a container does not automatically copy its elements.

Practice by drawing the two separate counter environments and by changing only one snapshot record in the audit example. Explain which bindings change, which objects are shared, and why the next history remains intact after the fix.

References and Further Reading

- ECMAScript environment records: formal binding operations and outer environments.
- ECMAScript function calls: ordinary function call semantics.
- MDN closures: lexical retention with practical examples.
- MDN memory management: reachability and collection limits.
- Volume 2: Lexical Scope and Closures: deeper lifetime, loop-binding, and callback behavior.